Phenix Model Building

8 documentation pages
Generated from local Phenix documentation

Contents

Model Building

xray-model-building.html

Why

Determining the structure of a macromolecule typically requires building an atomic model by fitting to maps obtained from experimental phasing or molecular replacement. For many cases, automated methods can be used to build models for both proteins and nucleic acids.

The accuracy and completeness of the model building is a function of the quality of the starting electron density map, and less critically the resolution of the data. At low resolution (3.0 Å or worse), partial models are usually built, which require manual completion.

How

In Phenix, several programs aid in automated model building; the main one is phenix.autobuild. The model building process integrates phase improvement using density modification methods with automated map interpretation in order to build and refine models. This minimally requires an experimental density map, experimental diffraction data, and the sequence of the molecule(s) in the crystal. The program outputs a model file with the automatically built model and the map coefficients for the best electron density map created during the model building process.

How to use the phenix.autobuild GUI: Click here

Phenix reference manual for phenix.autobuild

Common issues

- Frequently asked questions about model building

Related programs

- phenix.find_helices_strands: This program very rapidly locates protein secondary structure elements in an electron density map. This is extremely useful for quickly confirming that a map contains macromolecular features, or for interpreting low-resolution (4 Å or worse) maps. Optionally, you can provide the amino acid sequence and the program will fit residues where density permits.
- phenix.fit_loops: This program docks missing loops into protein structures given the current incomplete model, the sequence, and an electron density map. Model building procedures are sometimes unable to fit models to poor density in loop regions, especially in the initial stages of map interpretation. As the model becomes more complete and maps improve, phenix.fit_loops can perform simple real-space refinement to clean up the structure.
- phenix.map_to_model: This program is designed for lower-resolution model building. It is suitable for both crystallographic maps and cryo-EM maps. Taking into account symmetry used to create a map, it builds only the unique part of the model and expands to the remaining NCS-related copies.

AlphaFold and Phenix

alphafold.html

You can use the predicted models from AlphaFold and other prediction software in Phenix. Using these models can be very helpful in structure determination because the models can be very accurate over much of their length and the models come with accuracy estimates that allow removal of poorly-predicted regions.

Overall procedure for using AlphaFold models in Phenix

A typical procedure for structure determination with AlphaFold models is:

- Analyze your data (Xtriage for X-ray data or Mtriage for cryo-EM)
- Identify whether most parts of most of your structure can be predicted or modeled (PredictModel)
- Automatically generate an interpretation of your data (PredictAndBuild)

This tutorial video is available on the Phenix YouTube channel and explains how to determine your structure with AlphaFold in Phenix.

1. Analyzing your data

It is a good idea to evaluate your data to have an idea of its resolution and overall quality before working with it. You can use phenix.xtriage to do this for X-ray data and phenix.mtriage for cryo-EM data. For X-ray data in particular you will need to know the space group of your structure (or its enantiomer, as in P61 or P65).

2. Identify whether most parts of most of your structure can be predicted

You might want to go ahead and predict models for all the chains in your structure (even before getting the data) to get an idea of how much information you are going to get from structure prediction. You can use phenix.predict_model to do this for protein chains. For nucleic acid chains, normally you are going to have to either use an external procedure to predict their structures or to build them separately (i.e., with phenix.autobuild).

In general terms, if most parts of most of the components of your structure can be predicted with high confidence (i.e., a value of the plDDT metric of 70 or higher), then predicted models are likely to be very useful.

3. Automatically generate an interpretation of your data

If you have either cryo-EM or X-ray data with a resolution of about 4.5 A or better, and if you have useful predicted models for most of the components of your structure, you may be able to use Predict and build to automatically generate an initial interpretation of your data.

The Predict and build procedure (phenix.predict_and_build) just requires a sequence file (one sequence for each chain in the model to be built) and either X-ray data (an mtz file) or cryo-EM data (two half-maps). This procedure carries out iterative prediction and structure determination, yielding a final rebuilt model as well as a final morphed model that contains the final set of predicted models.

The tutorial video PredictAndBuild X-ray is available on the Phenix YouTube channel and explains how run PredictAndBuild with X-ray diffraction data.

The tutorial video PredictAndBuild cryo-EM is available on the Phenix YouTube channel and explains how run PredictAndBuild with cryo-EM data.

Other procedures for using AlphaFold models in Phenix

1. Get an AlphaFold model (or a model from the PDB) for each chain in your structure. You can use the phenix.predict_model method in the Phenix GUI to do this using the Phenix server.

The tutorial video AlphaFold prediction is available on the Phenix YouTube channel and explains how run predict a model with AlphaFold in Phenix.

2. Trim your model and break it into rigid domains. You can use phenix.process_predicted_model to do this.

3. Dock models (cryo-EM) or carry out molecular replacement (crystallography) to place your models in the right places in your map or unit cell. For cryo-EM structures, you can supply models directly to phenix.predict_and_build if you want. Alternatively you can dock your models with phenix.dock_predicted_model or phenix.em_placement. For X-ray structures you can also use phenix.predict_and_build or you can use phenix.phaser (crystallography) to find the locations of your models in the crystal.

4. Fill in missing parts of your models with loop fitting or iterative model-building. You can do this with phenix.predict_and_build if you want. Alternatively for crystallography you can use phenix.autobuild .

5. Refine the rebuilt predicted models that you obtain. You can use phenix.real_space_refine for cryo-EM models and phenix.refine for crytstallographic models.

6. Examine your resulting model in detail, using the validation tools that are part of phenix.real_space_refine and phenix.refine to help you identify problem areas and using manual model-building tools to fix them.

Advanced steps

Iteration of AlphaFold and model-building with experimental data

You may be able to improve your rebuilt AlphaFold model by iterating the AlphaFold step and including your rebuilt model as a template (and skipping any other templates). This process is what the phenix.predict_and_build Phenix tool in the GUI does for you automatically.

Structures with multiple chains

The phenix.predict_and_build tool can work with a structure with multiple chains, but you may wish to run one chain at a time for a cryo-EM structure. You can use the whole map for each chain, or if you have some idea of what chain goes where, you can mask out or box the map so that it shows only one chain and use that as your map. Boxing or masking the map can speed up the process and improve the result considerably.

Boxing maps before rebuilding

For complex structures with many chains or with chains that contain domains with long linkers, docking can be very complicated and take a long time. In these cases it may be especially helpful to box or mask the map if you have an idea where each chain is located. If you cannot, you might want to run the docking step individually with each domain that you get from phenix.process_predicted_model and then examine where they went in the map. If the domains seem to correspond to different molecules, you might want to mask out the part of the density that corresponds to the molecule you don't want to fit and re-try. You can also try

running phenix.dock_in_map with one domain at a time and ask to find multiple copies; then you can choose the one that matches up with the other domains you have placed.

Background

Structure prediction software is now capable of generating models that are highly accurate over some or all parts of the models. Importantly, these predictions often come with reliable residue-by-residue estimates of uncertainty.

Compact domains in these predicted models in which all the residues have high confidence often will be very accurate over the entire domains. However, separate domains that each have high confidence but are connected by lower confidence residues sometimes have relative positions and orientations that differ between predicted and experimentally-determined structures.

When using predicted models as a starting point for experimental structure determination, it can be helpful to:

```
Remove low-confidence residues entirely

Break up the model into domains and allow the domains to have
different orientations
```

For a high-confidence predicted model, you might try using the predicted model as-is first. For most predicted models, you may want to try removing low-confidence residues, then additionally try breaking the model into domains and placing the domains one at a time.

An important feature of recent predicted models is that they generally have very accurate sequence alignment. That means that the assignment of the sequence to the high-confidence parts of the model is usually correct. This can make a very big difference in completion of the remainder of the structure (the parts that were not predicted with high confidence) because you know exactly what residues go in the gaps. This means that model-building of the remainder of the structure can often be completed with loop-fitting tools instead of trying to rebuild everything.

What do do after applying Phenix tools to predicted models

While AlphaFold and other predicted models can be quite accurate overall, some details in otherwise-accurate regions and some whole regions can be incompatible with your experimental data. The Phenix procedures for producing models based on your experimental data and using predicted models as starting points are designed to try and keep the accurate parts of the predicted models and to replace the inaccurate parts. Depending on the resolution of your data, the automatically-produced models may be quite accurate or may themselves need a lot of trimming and rebuilding.

Once you have used Phenix tools to modify a predicted model based on experimental data you will want to carefully analyze the resulting model, comparing every detail to the experimental map or data. You will generally want to use manual model-building tools such as Coot or Isolde to fix small and large errors in the models that remain.

You can also iterate the AlphaFold prediction using your rebuilt model as an input to a new round of AlphaFold (See also the documentation for running AlphaFold predictions).

Acknowledgements

The Phenix AlphaFold server uses AlphaFold (Jumper et al., 2021), the mmseqs2 MSA server (Steinegger, and Söding, 2017, Mirdita et al., 2019), and scripts derived from ColabFold (Mirdita et al., 2022).

Literature

Accelerating crystal structure determination with iterative AlphaFold prediction. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner, RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. Acta Cryst D 79, 234-244 (2023). Improved AlphaFold modeling with implicit experimental information. T.C. Terwilliger, B.K. Poon, P.V. Afonine, C.J. Schlicksup, T.I. Croll, C. Millán, J.S. Richardson, R.J. Read, and P.D. Adams. Nature Methods 19, 1376-1382 (2022). AlphaFold predictions: great hypotheses but no match for experiment. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner, RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. BiorXiv (2023). Making protein folding accessible to all. M. Mirdita, K. Schütze, Y. Moriwaki, L. Heo, S. Ovchinnikov, and M. Steinegger. Nature Methods 19, 679–682 (2022). Highly accurate protein structure prediction with AlphaFold. J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S.A.A. Kohl, A.J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A.W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Nature 596, 583-589 (2021). MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. M. Steinegger, and J. Söding. Nature biotechnology 35, 1026-1028 (2017). MMseqs2 desktop and local web server app for fast interactive sequence searches. M. Mirdita, M. Steinegger, and J. Söding. Bioinformatics 35, 2856-2858 (2019).

- Accelerating crystal structure determination with iterative AlphaFold prediction. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner, RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. Acta Cryst D 79, 234-244 (2023).
- Improved AlphaFold modeling with implicit experimental information. T.C. Terwilliger, B.K. Poon, P.V. Afonine, C.J. Schlicksup, T.I. Croll, C. Millán, J.S. Richardson, R.J. Read, and P.D. Adams. Nature Methods 19, 1376-1382 (2022).
- AlphaFold predictions: great hypotheses but no match for experiment. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner, RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. BiorXiv (2023).
- Making protein folding accessible to all. M. Mirdita, K. Schütze, Y. Moriwaki, L. Heo, S. Ovchinnikov, and M. Steinegger. Nature Methods 19, 679–682 (2022).
- Highly accurate protein structure prediction with AlphaFold. J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S.A.A. Kohl, A.J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A.W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Nature 596, 583-589 (2021).
- MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. M. Steinegger, and J. Söding. Nature biotechnology 35, 1026-1028 (2017).
- MMseqs2 desktop and local web server app for fast interactive sequence searches. M. Mirdita, M. Steinegger, and J. Söding. Bioinformatics 35, 2856-2858 (2019).

Automated Model Building and Rebuilding using AutoBuild

autobuild.html

Contents

- Skip NCS in model_building and refinement
- Make a SA-omit map around atoms in target.pdb
- Make a simple composite omit map
- Make a SA composite omit map
- Combine composite OMIT files from a set of parallel runs on different computers
- Make an iterative-build omit map around atoms in target.pdb
- Make a sa-omit map around residues 3 and 4 in chain A of coords.pdb
- Create one very good rebuilt model
- Touch up a model
- Remove the worst-fitting residues from a model
- Create 20 very good rebuilt models that are as different as possible
- Combining files from a nearly-complete autobuild run with rebuild-in-place=true
- Build starting from a very accurate but very small part of a model
- Morph an MR model and rebuild it
- Build an RNA chain
- Build a DNA chain
- Density-modify with or without a model and make maps
- Density-modify starting with your map coefficients and make maps
- Calculate a prime-and-switch map
- Possible Problems
- General Limitations
- Specific limitations and problems
- Literature
- List of all available keywords

Author(s)

- AutoBuild Wizard: Tom Terwilliger
- PHENIX GUI: Nathaniel Echols
- phenix.refine: Ralf W. Grosse-Kunstleve, Peter Zwart and Paul D. Adams
- RESOLVE: Tom Terwilliger
- phenix.xtriage: Peter Zwart

Purpose of the AutoBuild Wizard

The purpose of the AutoBuild Wizard is to provide a highly automated system for model rebuilding and completion using X-ray crystallographic data. The Wizard design allows the user to specify data files and parameters through an interactive GUI, or alternatively through a parameters file. The AutoBuild Wizard begins with datafiles with structure factor amplitudes and uncertainties, along with either experimental phase information or a starting model, carries out cycles of model-building and refinement alternating with model-based density modification, and producing a relatively complete atomic model.

The AutoBuild Wizard uses RESOLVE, xtriage and phenix.refine to build an atomic model, refine it, and improve it with iterative density modification, refinement, and model-building

The Wizard begins with either experimental phases (i.e., from AutoSol) or with an atomic model that can be used to generate calculated phases. The AutoBuild Wizard produces a refined model that can be nearly complete if the data are strong and the resolution is about 2.5 A or better. At lower resolutions (2.5 - 3 A) the model may be less complete and at resolutions > 3A the model may be quite incomplete and not well refined.

The AutoBuild Wizard can be used to generate OMIT maps (simple omit, SA-omit, iterative-build omit) that can cover the entire unit cell or specific residues in a PDB file.

The AutoBuild Wizard can generate a set of models compatible with experimental data (multiple_models)

Usage

The AutoBuild Wizard can be run from the PHENIX GUI, from the command-line, and from parameters files. All three versions are identical except in the way that they take commands from the user. See Using the PHENIX Wizards for details of how to run a Wizard. The command-line version will be described here.

How the AutoBuild Wizard works

The AutoBuild Wizard begins with experimental structure factor amplitudes, along with either experimental or model-based estimates of crystallographic phases. The phase information is improved by using statistical density modification to improve the correlation of NCS-related density in the map (if present) and to improve the match of the distribution of electron densities in the map with those expected from a model map. This improved map is then used to build and refine an atomic model.

In subsequent cycles, the models from previous cycles are used as a source of phase information in statistical density modification, iteratively improving the quality of the map used for model-building.

Additionally, during the first few cycles additional phase information is obtained by detecting and enhancing (1) the presence of commonly-found local patterns of density in the map, and (2) the presence of density in the shape of helices and strands. The final model obtained is analyzed for residue-based map correlation and density at the coordinates of individual atoms, and an analysis including a summary of atoms and residues that are in strong, moderate, or weak density and out of density is provided.

Automation and user control

The AutoBuild Wizard has been designed for ease of use combined with maximal user control, with as many parameters set automatically by the Wizard as possible, but maintaining parameters accessible to the user through a GUI and through parameters files. The Wizard uses the input/output routines of the cctbx library, allowing data files of many different formats so that the user does not have to convert their data to any particular format before using the Wizard. Use of the phenix.refine refinement package in the AutoBuild Wizard allows a high degree of automation of refinement so that the neither user nor Wizard is required to specify parameters for refinement. The phenix.refine package automatically includes a bulk solvent model and automatically places solvent molecules.

Core modules in the AutoBuild Wizard

The five core modules in the AutoBuild Wizard are

- building a new model into an electron density map

- rebuilding an existing model
- refinement
- iterative model-building beginning from experimental phase information, and
- iterative model-building beginning from a model.

The standard procedures available in the AutoBuild Wizard that are based on these modules include:

- model-building and completion starting from experimental phases,
- rebuilding a model from scratch, with or without experimental phase information, and
- rebuilding a model in place, maintaining connectivity and sequence register.

Starting from a set of experimental phases and structure factor amplitudes, normally model-building and completion starting from experimental phases is carried out, and then the resulting model is rebuilt from scratch.

Starting from a model (e.g., from molecular replacement) and experimental structure factor amplitudes, rebuilding a model in place is by default carried out if the starting model differs less than about 50% in sequence from the desired model, and otherwise the resulting model is rebuilt from scratch. It is generally a good idea to specify which you want to happen using the keyword "rebuild_in_place=True" (to keep the basic input model) or "rebuild_in_place=False" (to build a new model).

What the AutoBuild wizard needs to run

- a data file, optionally with phases and HL coeffs and freeR flag (w1.sca or data=w1.sca)
- a sequence file (seq.dat or seq_file=seq.dat) or a model (coords.pdb or model=coords.pdb)
- NOTE: you need to supply either both phases and HL coeffs or neither. You can specify neither with the keyword use_hl_if_present=False.

...and optional files

- coefficients for a starting map (map_file=resolve.mtz)
- a file for refinement (refinement_file=exptl_fobs_freeR_flags.mtz)
- a high-resolution datafile (hires_file=high_res.sca)

Anisotropy correction and B-factor sharpening

The AutoBuild wizard will apply an anistropy correction and B-factor sharpening to all the raw experimental data by default (controlled by they keyword remove_aniso=True). The target overall Wilson B factor can be set with the keyword b_iso, as in b_iso=25. By default the target Wilson B will be 10 times the resolution of the data (e.g., if the resolution is 3 A then b_iso=30.), or the actual Wilson B of the data, whichever is lower.

If an anisotropy correction is applied then the entire AutoBuild run will be carried out with anisotropy-corrected and sharpened data. At the very end of the run the final model will be re-refined against the uncorrected refinement data and this re-refined model and the uncorrected refinement data (with freeR flags) will be written out as overall_best.pdb and overall_best_refine_data.mtz.

Specifying which columns of data to use from input data files

If one or more of your data files has column names that the Wizard cannot identify automatically, you can specify them yourself. You will need to provide one column "name" for each expected column of data, with

"None" for anything that is missing.

For example, if your data file ref.mtz has columns FP SIGFP and FreeR then you might specify

```
refinement_file=ref.mtz
input_refinement_labels="FP SIGFP None None None None None None FreeR"
```

The keywords for labels and anticipated input labels (program labels) are:

```
input_labels (for data file): FP SIGFP PHIB FOM HLA HLB HLC HLD FreeR_flag
input_refinement_labels: FP SIGFP FreeR_flag
input_map_labels: FP PHIB FOM
input_hires_labels: FP SIGFP FreeR_flag
```

You can find out all the possible label strings in a data file that you might use by typing:

```
phenix.autosol display_labels=w1.mtz # display all labels for w1.mtz
```

NOTES: if your data files contain a mixture of amplitude and intensity data then only the amplitude data is available. If you have only intensity data in a data file and want to select specific columns, then you need to specify the column names as they are after importing the data and conversion to amplitudes (see below under General Limitations for details).

Specifying other general parameters

You can specify many more parameters as well. See the list of keywords, defaults and descriptions at the end of this page and also general information about running Wizards at Using the PHENIX Wizards for how to do this. Some of the most common parameters are:

```
data=w1.sca # data file
model=coords.pdb # starting model
rebuild_in_place=true # rebuild input model in place
rebuild_in_place=false # build a new model; add or subtract residues
 # from input model as necessary
seq_file=seq.dat # sequence file
map_file=map_coeffs.mtz # coefficients for a starting map for building
resolution=3 # dmin of 3 A
s_annealing=True # use simulated annealing refinement at start of each cycle
n_cycle_build_max=5 # max number of build cycles (starting from experimental phases)
n_cycle_rebuild_max=5 # max number of rebuild cycles (starting from a model)
```

Running from a parameters file

You can run phenix.autobuild from a parameters file. This is often convenient because you can generate a default one with:

```
phenix.autobuild --show_defaults &gt; my_autobuild.eff
```

and then you can just edit this file to match your needs and run it with:

```
phenix.autobuild my_autobuild.eff
```

What to do after running AutoBuild

The next step after AutoBuild is usually refinement with PhenixRefine.

After you run AutoBuild, have a close look at the output files and summary statistics. You are hoping to find that AutoBuild has built a mostly complete model with reasonable values of R and Rfree. At high resolution (better than 2 A) you are expecting a very good model with waters placed. At lower resolution (lower than 3 A) you are expecting a moderately complete model.

Notice whether non-crystallographic symmetry (NCS) has been found. If so, notice whether the NCS-related copies are very similar (normally they should be quite similar.)

Note the R-factor for density modification. These R values are not the same as those for refinement. A good R-factor for density modification will be in the range of 0.3-0.4, and an ok one from 0.4-0.5.

If you have high-resolution data (better than 2 A), you should obtain a very complete model containing waters, with R/Rfree values near 0.2-0.25. For lower-resolution data (2-3 A), expect a partial model with R/Rfree values in the range of 0.25-0.3, and for low resolution data (worse that 3 A), expect a partial model without waters and with R/Rfree values in the range of 0.3-0.35. Have a look at your model and the overall best density-modified map. These should agree closely, with perhaps some parts of the map that show density not yet accounted for by the model (either not built or representing ligands).

If your results are in expected ranges and your map looks good, the next step is usually to run refinement and validation, iterating with rebuilding any poorly-modeled regions in Coot or Isolde. Then once you have obtained a model that agrees well with your map and data, you may want to find ligands with LigandFit.

If your data extend only to 3 A or worse, the model obtained after AutoBuild will generally be rather incomplete and you will need to do a lot of manual rebuilding. For high-resolution data, you will need to do much less by yourself.

Picking waters in AutoBuild

By default AutoBuild will instruct phenix.refine to pick waters using its standard procedure. This means that if the resolution of the data is high enough (typically 3 A) then waters are placed.

You can tell AutoBuild not to have phenix.refine pick waters with the command:

```
place_waters=False
```

If you want to place waters at a lower resolution, you will need to reset the low-resolution cutoff for placing waters in phenix.refine. You would do that in a "refinement_params.eff" file containing lines like these (see below for passing parameters to phenix.refine with an ".eff" file):

```
refinement {
 ordered_solvent {
 low_resolution = 2.8
 }
}
```

Keeping waters from your input file in AutoBuild

You can tell AutoBuild to keep the waters in your input file when you are using rebuild_in_place (the default is to toss them and replace them with new ones). You can say,

```
keep_input_waters=True
place_waters=No
```

NOTE: If you specify keep_input_waters=True you should also specify either "place_waters=No" or "keep_pdb_atoms=No" . This is because if place_waters=Yes and keep_pdb_atoms=Yes then phenix.refine will add waters and then the wizard will keep the new waters from the new PDB file created by phenix.refine preferentially over the ones in your input file.

Twinning and AutoBuild

AutoBuild does not know about twinning, but you can incorporate a twin law into the refinement steps in the AutoBuild procedure if your crystal is twinned. Use phenix.xtriage to identify twinning and the twin law. Then

just paste the twin law in the twin_law field in the GUI.

You may also want to try turning on the parameter two_fofc_in_rebuild ("Use 2Fo-Fc map in rebuilding") which will use the 2Fo-Fc map from phenix.refine in model-building.

R-free flags and test set

In Phenix the parameter test_flag_value sets the value of the test set that is to be free. Normally Phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix and you do not need to do anything special. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.

Note that phenix.refine and AutoBuild write out PDB files that contain the test_flag_value. AutoBuild can read this test_flag_value and use it automatically. However if there is a conflict between this test_flag_value and the default value based on your data file, you may have to specify which to use.

Special note on anomalous data and AutoBuild: Autobuild does not support anomalous test sets. If you have a data file with anomalous data that has Rfree flags such as Rfree(+),Rfree(-) then you will need to merge these Rfree flags before running Autobuild. Here is how:

Go to the reflection file editor, read in your refine_data.mtz (or whatever it is called) file with anomalous data. Copy all the data and Rfree flags to the output file, but select "Edit arrays" and in the window that comes up do the following:

- Change the names of the output data arrays from I+ SigI+ I- SigI- to I SigI (or equivalent)

Change the names of the output data arrays from

I+ SigI+ I- SigI- to I SigI (or equivalent)

- Specify "merge if present" for "anomalous"
- Do the same for the Rfree Flags array.
- Run the reflection editor.

Now you have a data file that is non-anomalous and that has the same test set as your original. You can use this in AutoBuild.

Specifying phenix.refine parameters

You can control phenix.refine parameters that are not specified directly by AutoBuild using a refinement parameters (.eff) file:

```
refine_eff_file=refinement_params.eff # set any phenix.refine params not set by AutoBuild
```

This file might contain a twin-law for refinement:

```
data_manager {
 fmodel {
 xray_data {
 twin_law="l,-k,h"
 }
 }
}
```

You can put any phenix.refine parameters in this file, but a few parameters that are set directly by AutoBuild override your inputs from the refine_eff_file. These parameters are listed below.

Refinement parameters that must be set using AutoBuild Wizard keywords (overwriting any values provided by user in input_eff_file)

The following parameters controlling phenix.refine output are set directly in AutoBuild and cannot be set by the user

- refinement.output.write_eff_file
- refinement.output.write_geo_file
- refinement.output.write_def_file
- refinement.output.write_maps
- refinement.output.write_map_coefficients

Specifying resolve/resolve_pattern parameters

Similarly, you can control resolve and resolve_pattern parameters. For these parameters, your inputs will not be overridden by AutoBuild. The format is a little tricky: you have to put two sets of quotes around the command like this:

```
resolve_command="'resolution 200 3'" # NOTE ' and " quotes
```

This will put the text

```
resolution 200 3
```

at the end of every temporary command file created to run resolve. (This is why it is not overridden by AutoBuild commands; they will all come before your commands in the resolve command file.) Note that some commands in resolve may be incompatible with this usage.

Including ligand coordinates in AutoBuild

If your input PDB file contains ligands (anything other than solvent that is not protein if your chain_type=PROTEIN, for example) then by default these ligands will be kept, used in refinement, and written out to your output PDB file. Any solvent molecules will by default be discarded. You can change this behavior by changing the keywords from these defaults:

```
keep_input_ligands=True
keep_input_waters=False
```

The AutoBuild Wizard will use phenix.elbow to generate geometries for any ligands that are not recognized.

You can also tell AutoBuild to add the contents of any PDB files that you wish to supply to the current version of the structure just before refinement, so all the refined models produced contain whatever AutoBuild has built, plus the contents of these PDB files. This can be done through the GUI, the command-line, or a parameters file. In the command-line version you do this with:

```
input_lig_file_list=my_ligand.pdb
```

NOTE: The files in input_lig_file_list will be edited to make them all HETATM records to tell AutoBuild to ignore these residues in rebuilding.

NOTE You may need to tell phenix.refine about the geometry of your ligands. You will get an error message if the ligand is not recognized and an automatic run of phenix.elbow does not succeed in generating your ligand. In that case you will want to run phenix.elbow to create a cif definition file for this ligand:

```
phenix.elbow my_ligand.pdb --id=LIG
```

where LIG is the 3-letter ID code that you use in my_ligand.pdb to identify your ligand. If the automatic run does not work you may need to give phenix.elbow additional information to generate your ligand.

Once phenix.elbow has generated your ligand you can use the keyword "cif_def_file_list" to tell AutoBuild about this ligand:

```
cif_def_file_list=elbow.LIG.my_ligand.pdb.cif
```

Specifying arbitrary commands and cif files for phenix.refine

You can tell AutoBuild to apply any set of cif definitions to the model during refinement by using a combination of specification files and the commands cif_def_file_list and refine_eff_file_list:

```
refine_eff_file_list=link.eff cif_def_file_list=link.cif
```

This example comes from the phenix.refine manual page in which a link is specified in a cif definition file link.cif:

```
data_mod_5pho
#
loop_
_chem_mod_atom.mod_id
_chem_mod_atom.function
_chem_mod_atom.atom_id
_chem_mod_atom.new_atom_id
_chem_mod_atom.new_type_symbol
_chem_mod_atom.new_type_energy
_chem_mod_atom.new_partial_charge
 5pho add . O5T O OH .
loop_
_chem_mod_bond.mod_id
_chem_mod_bond.function
_chem_mod_bond.atom_id_1
_chem_mod_bond.atom_id_2
_chem_mod_bond.new_type
_chem_mod_bond.new_value_dist
_chem_mod_bond.new_value_dist_esd
 5pho add O5T P coval 1.520 0.020
```

and this is applied with a parameters file link.eff:

```
refinement.pdb_interpretation.apply_cif_modification
{
 data_mod = 5pho
 residue_selection = resname GUA and name O5T
}
```

You can have any number of cif files and parameters files.

Output files from AutoBuild

When you run AutoBuild the output files will be in a subdirectory with your run number:

```
AutoBuild_run_1_/ # subdirectory with results
```

The key output files that are produced are:

- A summary file listing the results of the run and the other files produced:

```
AutoBuild_summary.dat # overall summary
```

- A log file describing the entire run: the other files produced:

```
AutoBuild_run_1_1.log # overall log file
```

- A warnings file listing any warnings about the run

```
AutoBuild_warnings.dat # any warnings
```

- Final refined model

```
overall_best.pdb
```

NOTE 1: The "working_best.pdb" file is the current working best model. If an anisotropy correction and sharpening are applied (remove_aniso=True) then working_best.pdb will be refined against the corrected data. At the end of the run the last working_best.pdb will be re-refined against the original data (overall B refined only) and written out as overall_best.pdb.

NOTE 2: If there are multiple chains or multiple ncs copies, each chain will be given its own chainID (A B C D...). Segments that are not assigned to a chain are given a separate chainID and are given a segid of "UNK" to indicate that their assignment is unknown. ChainID's for ligands are kept as input. The chainID for solvent molecules is normally S.

- Final map coefficients used to build refined model. Use FWT PHWT in maps. Normally this is a density-modified map from resolve.

```
overall_best_denmod_map_coeffs.mtz
```

- sigmaA-weighted 2mFo-DFc and Fo-Fc map coefficients from phenix.refine based on the last working_best.pdb model These map coefficients will be sharpened anisotropy-corrected if the remove_aniso=True. (The file working_best.pdb is the same as overall_best.pdb, except it is refined against sharpened, anisotropy-corrected data if remove_aniso=True). The map coefficients are 2FOFCWT PH2FOFCWT for the 2mFo-DFc map and FOFC and PHFOFC for the Fo-Fc difference map. These map coefficients are filled (missing reflections are given Fc values.)

```
overall_best_refine_map_coeffs.mtz
```

- MTZ file with FP, phases and HL coeffs if present, and freeR_flags for refinement

```
overall_best_refine_data.mtz
```

NOTE: The labels for this mtz file are typically:

```
FP SIGFP PHIM FOMM HLAM HLBM HLCM HLDM FreeR_flag
```

The file overall_best_refine_data.mtz (identical to the file exptl_fobs_phases_freeR_flags.mtz) has a copy of the (experimental) HL coefficients that were input to autobuild. The labels HLAM HLBM etc have the ending "M" because they were copied by resolve and it outputs these labels...but in fact they are not density modified phases from autobuild, just copied straight from the input data file.

- Final log file for model-building

```
overall_best.log
```

- Final log file for refinement

```
overall_best.log_refine
```

- Evaluation of fit of model to map

```
overall_best.log_eval
```

- Summary of NCS information

```
overall_best_ncs_info.ncs
```

Standard building, rebuild_in_place, and multiple-models

The AutoBuild Wizard has two overall methods for building a model.

The first method (standard build) is to build a model from scratch. This involves identification of where helices (and strands, for proteins) are located, extension using fragment libraries, connection of segments, identification of side-chains, and sequence alignment. These methods are augmented in the standard building procedure by loop-fitting and building model outside of the region that has already been built.

The second method (rebuild_in_place) takes an existing model and rebuilds it without adding or deleting any residues and without changing the connectivity of the chain. The way this works is a segment of the model is deleted and then is filled-in again by rebuilding from the remaining ends. This is repeated for overlapping segments covering the entire model. NOTE: If you are using rebuild_in_place then your model must be quite similar to your sequence file, and in particular the model must not extend in the N-terminal direction beyond your sequence file. Minor edits (amino acid replacements) will be done automatically. Also NOTE: rebuild_in_place is not designed for models that contain alternate conformations. It is designed for a model with a single conformation. If you supply a model with some residues or side-chains with a blank altloc, and some with an altloc of A and some with B, then all those with A or B will be ignored (only the first conformer is considered).

The multiple-models approach really has two levels of multiple models. At the first level, several (multiple_models_group_number, default is number_of_parallel_models) models are built (using rebuild_in_place) and are then recombined into a single good model. At the next level, this whole process may be done more than once (multiple_models_number times), yielding several very good models. By default, if you ask for rebuild_in_place, then you will get a single very good model, created by running rebuild_in_place several times and recombining the models.

Parallel jobs, nproc, nbatch, number_of_parallel_models and how AutoBuild works in parallel

The AutoBuild Wizard is set up to take advantage of multi-processor machines or batch queues by splitting the work into separate tasks. See Tutorial 4: Iterative model-building, density modification and refinement starting from experimental phases and Tutorial 6: Automatically rebuilding a structure solved by Molecular Replacement for a description of the method used by the AutoBuild Wizard to run build jobs as sub-processes and to combine the results into single models.

Here are the key factors that determine how splitting model-building into batches and running them on one or more processors works:

- nbatch is the number of batches of work. As long as nbatch is fixed then the results of running the Wizard will be the same, no matter how many processors are used. Normally you will not need to adjust it.
- nproc is the number of processors to split the work among
- number_of_parallel_models is the number of models to build at once. The default is to set number_of_parallel_models=nbatch. This affects both standard building (number_of_parallel_models sets how many initial models to build) and rebuild_in_place (number_of_parallel_models determines whether a single model is built or a set of models are built and recombined into a single model).

Phenix.autobuild is set up so that you can specify the number of processors (nproc). Here is how to choose how to set it:

- If you are using rebuild_in_place=False, then use nproc=4. (Any more will not make any difference.)
- If you are using rebuild_in_place=True, then use nproc=5. (Again, any more will not make any difference.)

- If you are calculating an omit map, then use nproc=5 * number of omit regions (i.e., up to 100 or more, depending on how many processors you have)

Additionally you will want to set two more parameters:

```
run_command ="command you use to submit a job to your system"
background=False # probably false if this is a cluster, true if this is a multiprocessor machine
```

If you have a queueing system with 20 nodes, then you probably submit jobs with something like "qsub -someflags myjob.sh" # where someflags are whatever flags you use (or just "qsub myjob.sh" if no flags) Then you might use

```
run_command="qsub -someflags" background=False nproc=20
```

```
run_command="qsub" background=False nproc=20
```

or If you have a 20-processor machine instead, then you might say

```
run_command=sh background=True nproc=20
```

so that it would run your jobs with sh on your machine, and run them all in the background (i.e., all at one time).

Resolution limits in AutoBuild

There are several resolution limits used in AutoBuild. You can leave them all to default, or you can set any of them individually. Here is a list of these limits and how their default values are set:

Phase extension in AutoBuild

If you supply a starting map file and a hires_file (with native data to higher resolution) and you do not supply a model,then autobuild will by default carry out phase extension (in increments of s (1/d_min) of s_step). If you do supply a model, or you do not supply a hires_file, or you do not supply a starting map file, then the resolution used will be the final resolution (no phase extension steps.)

Sample AutoBuild Commands

NOTE: Output files will be in subdirectories labelled "AutoBuild_run_1_" "AutoBuild_run_2_" etc.

Run AutoBuild beginning with experimental data

```
phenix.autobuild data=solve_1.mtz seq_file=seq.dat
input_ncs_file=ha.pdb
```

Here the data in solve_1.mtz (FP SIGFP PHIB FOM HLA HLB HLC HLD) will be used as the starting point for density modification. Then a model will be built and refined. In subsequent cycles the models that have been built will be used to improve the phases in density modification. If NCS can be found from the sites in ha.pdb or from any models that are built, then NCS will be used in density modification.

Run AutoBuild beginning with a model and rebuild in place

```
phenix.autobuild data=w1.sca seq.dat model=coords.pdb \
rebuild_in_place=True
```

Here "rebuild_in_place=True" tells AutoBuild to keep the overall model you have supplied, not to add or subtract residues from it, except that AutoBuild will try to edit the model to match the sequence in your sequence file. The AutoBuild Wizard will use your model and the data in w1.sca to generate starting phases,

then it will carry out density modification to improve those phases, and adjust your model, rebuilding the model to match the resulting map and refining the model. This will be done iteratively, with the new model from each cycle being used at the start of the next one. If NCS is found in your model then it will be used in the density modification process. Note: by default rebuild_in_place will not truncate side chains even if there is no density. You can change this with truncate_missing_side_chains=True.

Add more residues to a model or rebuild a model

```
phenix.autobuild data=solve_1.mtz seq_file=seq.dat \
 model=coords.pdb rebuild_in_place=False
```

Here "rebuild_in_place=False" tells AutoBuild to build a new model, adding or subtracting residues as necessary. The data in solve_1.mtz (FP SIGFP PHIB FOM HLA HLB HLC HLD) will be used along with your model as the starting point for density modification. Then a new model will be built and refined. In subsequent cycles the models that have been built will be used to improve the phases in density modification. If NCS is found in your model or any model that is built, then it will be used in density modification. Note: by default rebuild_in_place=False will truncate side chains if there is no density. You can change this with truncate_missing_side_chains=False.

Run AutoBuild automatically after AutoSol

```
phenix.autobuild after_autosol
```

AutoBuild will identify the AutoSol run (in your working directory) with the highest overall score, then it will take the experimental phases (solve_xx.mtz or phaser_xx.mtz, where xx is the solution number) from that run, along with the corresponding density-modified map (resolve_xx.mtz) and the heavy_atom file (ha_xx.pdb_formatted.pdb) as inputs. Additionally, data for refinement are read in from exptl_fobs_freeR_flags_xx.mtz.

AutoBuild will then build a model, refine it, use the refined model in density modification, then iterate the model-building, refinement, and density modification process until no further improvement in the model occurs.

Merge in hires data

```
phenix.autobuild data=solve_2.mtz hires_file=w1.sca seq_file=seq.dat
```

The high-resolution data in w1.sca will be used for FP and SIGFP. Other information from solve_2.mtz (PHIB FOM HLA HLB HLC HLD) will be kept.

Truncate density at heavy-atom sites

```
phenix.autobuild data=solve_2.mtz seq_file=seq.dat input_ha_file=ha.pdb truncate_ha_sites_in_resolve=True
```

The heavy-atom sites in ha.pdb will be used to mark locations where high density is to be ignored during initial cycles of density modification. This can be useful if the heavy-atom peaks are very pronounced in the experimental map. The sites in ha.pdb will also be included in the model for the structure if they do not overlap with any atoms that are built as part of the model.

Skip NCS in model_building and refinement

```
phenix.autobuild data=solve_2.mtz seq_file=seq.dat find_ncs=False refine_with_ncs=False
```

The keyword "find_ncs=False" disables the finding of NCS from the models that are built and its use in density modification and model-building. The keyword "refine_with_ncs=False" disables finding NCS and its use in the refinement process. Together they prevent all use of NCS.

Make a SA-omit map around atoms in target.pdb

```
phenix.autobuild data=data.mtz model=coords.pdb omit_box_pdb=target.pdb composite_omit_type=sa_omit
```

Coefficients for the output omit map will be in the file resolve_composite_map.mtz in the subdirectory OMIT/ . An additional map coefficients file omit_region.mtz will show you the region that has been omitted.

Make a simple composite omit map

```
phenix.autobuild data=data.mtz model=coords.pdb composite_omit_type=simple_omit
```

Coefficients for the output omit map will be in the file resolve_composite_map.mtz in the subdirectory OMIT/ .

Make a SA composite omit map

```
phenix.autobuild data=data.mtz model=coords.pdb composite_omit_type=sa_omit
```

Coefficients for the output simulated-annealing composite omit map will be in the file resolve_composite_map.mtz in the subdirectory OMIT/ .

Combine composite OMIT files from a set of parallel runs on different computers

If you run a composite OMIT job but it fails at the last step of combining files, or if you run all the individual omit boxes on different machines, you can still combine them all into one single composite omit map.

You can do this by copying all the individual mtz files with map coefficients for omit regions to a single directory.

Here is a script you can edit and use to combine omit maps representing different omit regions into one.

NOTE: you need to ensure that the OMIT regions are defined the same in the runs where you got your overall_best_denmod_map_coeffs.mtz_OMIT_REGION_1 etc files and this run. You ensure that with the n_xyz command that sets the grid. You can copy this from one of your resolve log files created when you ran your omit (i.e., AutoBuild_run_1_/TEMP0/AutoBuild_run_1_/TEMP0/resolve.log will have a line like "nu nv nw: 32 32 32 " and you copy those numbers).

```
-----------------------------------
#!/bin/csh -f
# COMBINE OMIT SCRIPT
phenix.resolve &lt;&lt; EOD
hklin exptl_fobs_phases_freeR_flags.mtz
labin FP=FP SIGFP=SIGFP
n_xyz 32 32 32 # YOU MUST SET THIS BASED ON THE nu nv nw in a resolve log
file.
solvent_content 0.85
no_build
ha_file NONE
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_1
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_2
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_3
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_4
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_5
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_6
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_7
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_8
```

```
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_9
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_10
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_11
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_12
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_13
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_14
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_15
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_16
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_17
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_18
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_19
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_20
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_21
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_22
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_23
combine_map overall_best_denmod_map_coeffs.mtz_OMIT_REGION_24
omit
EOD
# END OF COMBINE OMIT SCRIPT
```

Make an iterative-build omit map around atoms in target.pdb

```
phenix.autobuild data=w1.sca model=coords.pdb omit_box_pdb=target.pdb \
 composite_omit_type=iterative_build_omit
```

Coefficients for the output omit map will be in the file resolve_composite_map.mtz in the subdirectory OMIT/ . An additional map coefficients file omit_region.mtz will show you the region that has been omitted.

Make a sa-omit map around residues 3 and 4 in chain A of coords.pdb

```
phenix.autobuild data=w1.sca model=coords.pdb omit_box_pdb=coords.pdb \
 omit_res_start_list=3 omit_res_end_list=4 omit_chain_list=A \
 composite_omit_type=sa_omit
```

Coefficients for the output omit map will be in the file resolve_composite_map.mtz in the subdirectory OMIT/ . An additional map coefficients file omit_region.mtz will show you the region that has been omitted.

Create one very good rebuilt model

```
phenix.autobuild data=data.mtz model=coords.pdb multiple_models=True \
 include_input_model=True \
 multiple_models_number=1 n_cycle_rebuild_max=5
```

The final model will be in the subdirectory MULTIPLE_MODELS in the file all_models.pdb (this file will contain just one model).

Note that this procedure will keep the sequence that is present in coords.pdb. If you supply a sequence file it will edit the sequence of coords.pdb to match your sequence file and discard any residues that do not match. (If you want to input a sequence file but not edit the sequence in coords.pdb and not discard any non-matching residues, then specify also edit_pdb=False.)

Note also that if include_input_model=True then no randomization cycle will be carried out and multiple_models_starting_resolution is ignored.

Touch up a model

```
phenix.autobuild data=data.mtz model=coords.pdb \
touch_up=True worst_percent_res_rebuild=2 min_cc_res_rebuild=0.8
```

You can rebuild just the worst parts of your model by setting touch_up=True. You can decide what parts to rebuild based on a minimum model-map correlation (by residue). You can decide how much to rebuild using worst_percent_res_rebuild or with min_cc_res_rebuild, or both.

Remove the worst-fitting residues from a model

```
phenix.autobuild data=data.mtz model=coords.pdb \
 delete_bad_residues_only=True \
 input_map_file=map_coeffs.mtz \
 worst_percent_res_rebuild=2 min_cc_res_rebuild=0.8
```

The trimmed model will be in the file (the run number may vary):

```
AutoBuild_run_1_/starting_model_trimmed.pdb
```

and the removed residues will be in the file:

```
AutoBuild_run_1_/starting_model_removed_residues.pdb
```

You can delete just the worst parts of your model by setting delete_bad_residues_only=True. You can decide what parts to remove based on a minimum model-map correlation (by residue). You can decide how much to remove using worst_percent_res_rebuild or with min_cc_res_rebuild, or both. (these are the same parameters used to decide which residues to rebuild in touch_up=True).

Here the input_map_file is optional; if you do not provide it then a model- based density modified map will be used to evaluate your model.

Create 20 very good rebuilt models that are as different as possible

```
phenix.autobuild data=data.mtz model=coords.pdb multiple_models=True \
 multiple_models_number=20 n_cycle_rebuild_max=5
```

The 20 final models will be in the subdirectory MULTIPLE_MODELS in the file all_models.pdb. This procedure is useful for generating an ensemble of models that are each individually consistent with the data, and yet are diverse. The variation among these models is an indication of the uncertainty in each of the models. Note that the ensemble of models is not a representation of the ensemble of structures that is truly present in the crystal.

Combining files from a nearly-complete autobuild run with rebuild-in-place=true

If you have run autobuild with rebuild_in_place=True then the last step is combining the models that have been produced. If you ran the job in separate batches and want to combine the final models, you can use the script below.

Note that all the models must have exactly the same set of atoms (aside from any solvent).

Basically you run a dummy autobuild run to create a directory and database entries, then you copy your files there, then you run autobuild and tell it to carry on and do the combine step. You'll need a map_coeffs.mtz file that has map coefficients (they won't be used but have to be there just to make it run).

```
-------------------------------------------------------
#!/bin/csh -f
#COMBINE_MODELS SCRIPT

if (-d PDS || -d AutoBuild_run_1_) then
 echo "Please run in a directory without PDS or AutoBuild_run_1_"
 exit 1
endif
```

```
echo "Setting up combine models with a dummy run. NOTE:
multiple_models_group_number must be correct"

phenix.autobuild fobs.mtz multiple_models=true seq_file=seq.dat \
 combine_only=true multiple_models_group_number=2 \
input_map_file=map_coeffs.mtz \
multiple_models_number=1 &gt; dummy_autobuild.log

echo "Copying files to AutoBuild_run_1_/MULTIPLE_MODELS"
mkdir AutoBuild_run_1_/MULTIPLE_MODELS
cp coords1.pdb AutoBuild_run_1_/MULTIPLE_MODELS/initial_model.pdb_1_1
cp coords2.pdb AutoBuild_run_1_/MULTIPLE_MODELS/initial_model.pdb_1_2
cp map_coeffs_1.mtz AutoBuild_run_1_/MULTIPLE_MODELS/initial_model.mtz_1_1
cp map_coeffs_2.mtz AutoBuild_run_1_/MULTIPLE_MODELS/initial_model.mtz_1_2

ls AutoBuild_run_1_/MULTIPLE_MODELS/

echo "Running autobuild to combine files in
AutoBuild_run_1_/MULTIPLE_MODELS"

phenix.autobuild combine_only=true seq_file=seq.dat carry_on=true run=1 &gt; autobuild_combine.log

# END OF COMBINE_MODELS SCRIPT
-------------------------------------------------------
```

Build starting from a very accurate but very small part of a model

```
phenix.autobuild data=data.mtz model=MR.pdb \
rebuild_from_fragments=True\
seq_file=seq.dat \
i_ran_seed=124881 \
nproc=4
```

You can have autobuild try to start rebuilding from fragments of a model. Keyword is rebuild_from_fragments=True. This sets the parameters two_fofc_denmod_in_rebuild=True, all_maps_in_rebuild=True, rebuild_in_place=False, and sets consider_main_chain_list to include your input model. You might want to use this if you look for ideal helices using Phaser, then rebuild the resulting partial model, as in the Arcimboldo procedure. The special feature of finding helices is that they can be very accurately placed in some cases. This really helps the subsequent rebuilding. If you have enough computer time, then run it several or even many times with different values of i_ran_seed. Each time you'll get a slightly different result. Here two different types of density-modified maps are calculated and models are built with each. The starting phases and phase probabilities for one type are based on a sigmaA-weighted 2mFo-DFc map. Those for the other type come from density modification using a model-based map as a target map and finding phases that yield a map that is as close to this one as possible. In either case the starting phases and phase probabilities are used in a second cycle of density modification in which part of the density modification target is a calculated map and part is standard density modification (including solvent flattening, histogram matching, NCS).

Morph an MR model and rebuild it

```
phenix.autobuild data=data.mtz model=MR.pdb \
morph=True rebuild_in_place=False seq_file=seq.dat
```

You can have autobuild morph your input model, distorting it to match the density-modified map that is produced from your model and data. This can be used to make an improved starting model in cases where the MR model is very different than the structure that is to be solved. For the morphing to work, the two structures must be topologically similar and differ mostly by movements of domains or motifs such as a group of helices or a sheet.

The morphing process consists of identifying a coordinate shift to apply to each N (or P for nucleic acids) atom that maximizes the local density correlation between the model and the map. This is smoothed and applied to the structure to generate a morphed structure.

Build an RNA chain

```
phenix.autobuild data=solve_1.mtz seq_file=seq.dat chain_type=RNA
```

Build a DNA chain

```
phenix.autobuild data=solve_1.mtz seq_file=seq.dat chain_type=DNA
```

Density-modify with or without a model and make maps

You can use the AutoBuild Wizard as a convenient way to run resolve density modification with or without including model-based information. Just use a command like this:

```
phenix.autobuild data=data.mtz model=coords.pdb \
 maps_only=True seq_file=seq.dat

phenix.autobuild data=data.mtz \
 maps_only=True seq_file=seq.dat
```

The Wizard will calculate the same map that it would normally calculate given these data, and then it will write the map out and stop.

Density-modify starting with your map coefficients and make maps

You can use the AutoBuild Wizard as a convenient way to run resolve density modification starting with map coefficients you define. Just use a command like this:

```
phenix.autobuild data=data.mtz \
 maps_only=True seq_file=seq.dat \
 map_file=starting_map.mtz map_labels="2FOFCWT PH2FOFCWT"
```

The Wizard will start with the phases in starting_map.mtz calculate the same map that it would normally calculate given these data, and then it will write the map out and stop.

Calculate a prime-and-switch map

```
phenix.autobuild data=data.mtz solvent_fraction=.6 \
 ps_in_rebuild=True model=coords.pdb maps_only=True
```

The output prime-and-switch map will be in the file prime_and_switch.mtz.

Possible Problems

General Limitations

- The AutoBuild wizard edits input PDB files to remove multiple conformations. It will also renumber residues if the file contains residues with insertion codes. All references to residue numbers (e.g. rebuild_res_start_list) refer to the edited, renumbered model. This model can be found in the AutoBuild_run_1_ (or appropriate) directory as "edited_pdb.pdb".
- If you are using rebuild_in_place then your model must be quite similar to your sequence file, and in particular the model must not extend in the N-terminal direction beyond your sequence file. Minor edits

(amino acid replacements) will be done automatically.

• The AutoBuild wizard expects residue numbers to not decrease along a chain. It will stop if residue 250 in chain B is found between residues 116 and 117 in the same chain, for example. To get around this, use insertion codes (make residue 250 residue 116A instead).

• The keywords "cell" and "sg" have been replaced with "unit_cell" and "space_group" to make the keywords the same as in other phenix applications.

• The AutoBuild model-building can only build one type of chain at a time (default chain_type='PROTEIN'; other choices are RNA and DNA). If you supply a PDB file containing more than one type of chain for rebuilding, then all the residues that are not that type of chain are treated as ligands and are (by default, keep_input_ligands=True) included in refinement but not in rebuilding. Any input solvent molecules are (by default, keep_input_waters=False) ignored.

You can include more than one type of chain in rebuilding by supplying one type of chains as ligands with input_lig_file_list and rebuilding another type:

```
chain_type=PROTEIN # build only protein
input_lig_file_list=MyDNA.pdb # just read in DNA coordinates and include in refinement
```

In this case only protein chains will be built, but the DNA coordinates in MyDNA.pdb will be included in all refinements and will be written out to the final coordinate file. You may wish to add the keyword:

```
keep_pdb_atoms=False #keep the ligand atoms if model (pdb) and ligand overlap
```

which will tell AutoBuild that the ligand (DNA) atoms are to be kept if the model that is being built (protein) overlaps with it. (The default is to keep the model that is being built and to discard any ligand atoms that overlap).

This whole process is likely to require substantial editing of the PDB files by hand because when you build DNA, a lot of chains are going to be built into the protein region, and when you build protein, it is going to be accidentally built into the DNA.

• Any file in input_lig_file_list containing ATOM records will have them replaced with HETATM records. This is so that the rebuild_in_place algorithm does not try to use them in rebuilding.

• The ligand generation routine in phenix.elbow will not generate heme groups at this point. Most other ligands can be automatically generated.

• If your input data file contains both intensity data and amplitude data, only the amplitude data is exposed in the AutoBuild Wizard. If you want to use the intensity data then you have to create a file that does not have amplitude data in it.

• If your input data file has only intensity data and you wish to specify which columns of data the AutoBuild Wizard is to use, then you have to specify the names that the columns will have AFTER importing the data and conversion to amplitudes, not the original column names.

These column names may not be obvious. Here is how to find out what they will be. Do a quick dummy run like this with XXX as labels:

```
phenix.autobuild w2.sca coords.pdb input_labels="XXX XXX"
```

The Wizard will print out a list of available labels like this:

```
Sorry, the label XXX does not exist as an amplitude array in
the input_data_file ImportRawData_run_8_/w2_PHX.mtz
...available labels are: ['w2', 'SIGw2', 'None']
```

Then you know that the correct command is:

```
phenix.autobuild w2.sca coords.pdb input_labels="w2 SIGw2"
```

• The AutoBuild Wizard cannot build modified residues. If you supply a model with modified residues, these will be taken out of the chain and treated as ligands, and the chain will be broken at that point. By default the modified residues will be added to your model just before refinement and a cif definitions file will be automatically generated for these residues. You can also add these residues with the input_lig_file_list procedure if you want.

• The AutoBuild Wizard will not build very short chains unless you set the variable group_ca_length (default=4 for building a model from scratch) to a smaller number. The shortest chain that will be built is group_ca_length. If you use rebuild_in_place, then the default shortest chain allowed is 1 residue, so any part of a model you supply is rebuilt.

Specific limitations and problems

• By default the AutoBuild Wizard splits jobs into one or more parts (determined by the parameter "nbatch") and runs them as sub-processes. These may run sequentially or in parallel, depending on the value of the parameter "nproc" . In some cases the running of sub-processes can lead to timing errors in which a file is not written fully before it is to be read by the next process. This appears more often when jobs are run on nfs-mounted disks than on a local disk. If this occurs, a solution is to set the parameter "nbatch=1" so that the jobs not be run as sub-processes. You can also specify "number_of_parallel_models=1" which will do much the same thing. Note that changing the value of "nbatch" will normally change the results of running the Wizard. (Changing the value of "nproc" does not change the results, it changes only how many jobs are run at once.)

• In many versions of the shell tcsh (and sh), the length of the shell variable PATH is limited (for example to 4096 characters). If your PATH is quite long then when AutoBuild runs a sub-process, it may accidentally increase the PATH to a value that is over the limit. The symptom is that you get a message like "Word too long". If this happens, try 'echo $PATH' to see if it is very long...and if so see if you can remove some entries in it. Or...you may want to shorten your path in PHENIX by specifying: remove_path_word_list='coot cns ccp4' (add as many paths that you have but do not need within PHENIX). Or...you may want to install a new version of tcsh which will allow a much longer path. You can get a new version from ftp://ftp.astron.com/pub/tcsh/

• The size of the asymmetric unit in the SOLVE/RESOLVE portion of the AutoBuild wizard is limited by the memory in your computer and the binaries used. The Wizard is supplied with regular-size ("", size=6), giant ("_giant", size=12), huge ("_huge", size=18) and extra_huge ("_extra_huge", size=36). Larger-size versions can be obtained on request.

• The AutoBuild Wizard can take most settings of most space groups, however it can only use the hexagonal setting of rhombohedral space groups (eg., #146 R3:H or #155 R32:H), and it cannot use space groups 114-119 (not found in macromolecular crystallography) even in the standard setting due to difficulties with the use of asuset in the version of ccp4 libraries used in PHENIX for these settings and space groups.

Literature

Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. Acta Cryst. D64, 61-69 (2008).

• Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. Acta Cryst. D64, 61-69 (2008).

Interpretation of ensembles created by multiple iterative rebuilding of macromolecular models. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, P.D. Adams, N.W. Moriarty, P.H. Zwart, R.J. Read, D. Turk, and L.-W. Hung. Acta Cryst. D63, 597-610 (2007).

• Interpretation of ensembles created by multiple iterative rebuilding of macromolecular models. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, P.D. Adams, N.W. Moriarty, P.H. Zwart, R.J. Read, D. Turk, and L.-W. Hung. Acta Cryst. D63, 597-610 (2007).

Improving macromolecular atomic models at moderate resolution by automated iterative model building, statistical density modification and refinement. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 1174-82 (2003).

• Improving macromolecular atomic models at moderate resolution by automated iterative model building, statistical density modification and refinement. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 1174-82 (2003).

Using prime-and-switch phasing to reduce model bias in molecular replacement. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 60, 2144-9 (2004).

• Using prime-and-switch phasing to reduce model bias in molecular replacement. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 60, 2144-9 (2004).

Rapid automatic NCS identification using heavy-atom substructures. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2213-5 (2002).

• Rapid automatic NCS identification using heavy-atom substructures. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2213-5 (2002).

Maximum-likelihood density modification. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 56, 965-72 (2000).

• Maximum-likelihood density modification. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 56, 965-72 (2000).

Statistical density modification with non-crystallographic symmetry. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2082-6 (2002).

• Statistical density modification with non-crystallographic symmetry. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 58, 2082-6 (2002).

Statistical density modification using local pattern matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 1688-701 (2003).

• Statistical density modification using local pattern matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 1688-701 (2003).

Maximum-likelihood density modification using pattern recognition of structural motifs. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 57, 1755-62 (2001).

• Maximum-likelihood density modification using pattern recognition of structural motifs. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 57, 1755-62 (2001).

Map-likelihood phasing. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 57, 1763-75 (2001).

• Map-likelihood phasing. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 57, 1763-75 (2001).

Automated side-chain model building and sequence assignment by template matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 45-9 (2003).

• Automated side-chain model building and sequence assignment by template matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 45-9 (2003).

Automated main-chain model building by template matching and iterative fragment extension. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 38-44 (2003).

• Automated main-chain model building by template matching and iterative fragment extension. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 38-44 (2003).

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command lineautobuilddata = None Datafile. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_labels='FP SIGFP PHIB FOM HLA HLB HLC HLD FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP you can just say input_labels='myFP mysigFP'. If you have free R flags, phase information or HL coefficients that you want to use then an mtz file is required. If this file contains phase information, this phase information should be experimental (i.e., MAD/SAD/MIR etc), and should not be density-modified phases (enter any files with density-modified phases as input_map_file instead). NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. If you also specify a hires data file, then FP and SIGFP will come from that data file (and not this one) If an input_refinement_file is specified, then F, Sigma, FreeR_flag (if present) from that file will be used for refinement instead of this one. model = None PDB file with starting model. NOTE: If your PDB file has been previously refined, then please make sure that you provide the free R flags that were used in that refinement. These can come from the data file or from the refinement_file. seq_file = Auto Text file with 1-letter code of protein sequence. Separate chains with a blank line or line starting with >. Normally you should include one copy of each unique chain. NOTE: if 1 copy of each unique chain is provided it is assumed that there are ncs_copies (could be 1) of each unique chain. If more than one copy of any chain is provided it is assumed that the asymmetric unit contains the number of copies of each chain that are given, multiplied by ncs_copies. So if the sequence file has two copies of the sequence for chain A and one of chain B, the cell contents are assumed to be ncs_copies*2 of chain A and ncs_copies of chain B. If there are unequal numbers of copies of chains, be sure to set solvent_fraction. ADDITIONAL NOTES: 1. lines starting with > are ignored and separate chains 2. FASTA format is fine 3. If you enter a PDB file for rebuilding and it has the sequence you want, then the sequence file is not necessary. NOTE: You can also enter the name of a PDB file that contains SEQRES records, and the sequence from the SEQRES records will be read, written to seq_from_seqres_records.dat, and used as your input sequence. If you have a duplex DNA, enter each strand as a separate chain. NOTE 1: for AutoBuild you can specify start_chains_list on the first line of your sequence file: >> start_chains_list 23 11 5 NOTE 2. Characters such as numbers and non-printing characters in the sequence file are ignored. NOTE 3. Be sure that your sequence file does not have any blank lines in the middle of your sequence, as these are interpreted as the beginning of another chain.map_file = Auto MTZ file containing starting map. This file must be a mtz file. The Wizard will guess the column identification. You can specify the column labels to use with: input_map_labels='FP PHIB FOM' Substitute any labels you do not have with None. If you only have myFP and myPHIB you can just say input_map_labels='myFP myPHIB'. This map will be used in the first cycle of model-building. NOTE 1: If use_map_file_as_hklstart=True then this file will be used instead to start density modification. NOTE 2: default for this keyword is Auto, which means "carry out normal process to guess this keyword". This means if you specify "after_autosol" in AutoBuild, AutoBuild will automatically take the value from AutoSol. If you do not want this to happen, you can specify None which means "No file&quot...

• job_title = None Job title in PHENIX GUI, not used on command line

• autobuilddata = None Datafile. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_labels='FP SIGFP PHIB FOM HLA HLB HLC HLD FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP you can just say input_labels='myFP mysigFP'. If you have free R flags, phase information or HL coefficients that you want to use then an mtz file is required. If this file contains phase information, this phase information should be experimental (i.e., MAD/SAD/MIR etc), and should not be density-modified phases (enter any files with density-modified phases as input_map_file instead). NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. If you also specify a hires data file, then FP and SIGFP will come from that data file (and not this one) If an input_refinement_file is specified, then F, Sigma, FreeR_flag (if present) from that file will be used for refinement instead of this one. model = None PDB file with starting model. NOTE: If your PDB file has been previously refined, then please make sure that you provide the free R flags that were used in that refinement. These can come from the data file or from the refinement_file. seq_file = Auto Text file with 1-letter code of protein sequence. Separate chains with a blank line or line starting with >. Normally you should include one copy of each unique chain. NOTE: if 1 copy of each unique chain is provided it is assumed that there are ncs_copies (could be 1) of each unique chain. If more than one copy of any chain is provided it is assumed that the asymmetric unit contains the number of copies of each chain that are given, multiplied by ncs_copies. So if the sequence file has two copies of the sequence for chain A and one of chain B, the cell contents are assumed to be ncs_copies*2 of chain A and ncs_copies of chain B. If there are unequal numbers of copies of chains, be sure to set solvent_fraction. ADDITIONAL NOTES: 1. lines starting with > are ignored and separate chains 2. FASTA format is fine 3. If you enter a PDB file for rebuilding and it has the sequence you want, then the sequence file is not necessary. NOTE: You can also enter the name of a PDB file that contains SEQRES records, and the sequence from the SEQRES records will be read, written to seq_from_seqres_records.dat, and used as your input sequence. If you have a duplex DNA, enter each strand as a separate chain. NOTE 1: for AutoBuild you can specify start_chains_list on the first line of your sequence file: >> start_chains_list 23 11 5 NOTE 2. Characters such as numbers and non-printing characters in the sequence file are ignored. NOTE 3. Be sure that your sequence file does not have any blank lines in the middle of your sequence, as these are interpreted as the beginning of another chain.map_file = Auto MTZ file containing starting map. This file must be a mtz file. The Wizard will guess the column identification. You can specify the column labels to use with: input_map_labels='FP PHIB FOM' Substitute any labels you do not have with None. If you only have myFP and myPHIB you can just say input_map_labels='myFP myPHIB'. This map will be used in the first cycle of model-building. NOTE 1: If use_map_file_as_hklstart=True then this file will be used instead to start density modification. NOTE 2: default for this keyword is Auto, which means "carry out normal process to guess this keyword". This means if you specify "after_autosol" in AutoBuild, AutoBuild will automatically take the value from AutoSol. If you do not want this to happen, you can specify None which means "No file" refinement_file = Auto File for refinement. This file can be a ....

• data = None Datafile. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_labels='FP SIGFP PHIB FOM HLA HLB HLC HLD FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP you can just say input_labels='myFP mysigFP'. If you have free R flags, phase information or HL coefficients that you want to use then an mtz file is required. If this file contains phase information, this phase information should be experimental (i.e., MAD/SAD/MIR etc), and should not be density-modified phases (enter any files with density-modified phases as input_map_file instead). NOTE:

If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. If you also specify a hires data file, then FP and SIGFP will come from that data file (and not this one) If an input_refinement_file is specified, then F, Sigma, FreeR_flag (if present) from that file will be used for refinement instead of this one.

• model = None PDB file with starting model. NOTE: If your PDB file has been previously refined, then please make sure that you provide the free R flags that were used in that refinement. These can come from the data file or from the refinement_file.

• seq_file = Auto Text file with 1-letter code of protein sequence. Separate chains with a blank line or line starting with >. Normally you should include one copy of each unique chain. NOTE: if 1 copy of each unique chain is provided it is assumed that there are ncs_copies (could be 1) of each unique chain. If more than one copy of any chain is provided it is assumed that the asymmetric unit contains the number of copies of each chain that are given, multiplied by ncs_copies. So if the sequence file has two copies of the sequence for chain A and one of chain B, the cell contents are assumed to be ncs_copies*2 of chain A and ncs_copies of chain B. If there are unequal numbers of copies of chains, be sure to set solvent_fraction. ADDITIONAL NOTES: 1. lines starting with > are ignored and separate chains 2. FASTA format is fine 3. If you enter a PDB file for rebuilding and it has the sequence you want, then the sequence file is not necessary. NOTE: You can also enter the name of a PDB file that contains SEQRES records, and the sequence from the SEQRES records will be read, written to seq_from_seqres_records.dat, and used as your input sequence. If you have a duplex DNA, enter each strand as a separate chain. NOTE 1: for AutoBuild you can specify start_chains_list on the first line of your sequence file: >> start_chains_list 23 11 5 NOTE 2. Characters such as numbers and non-printing characters in the sequence file are ignored. NOTE 3. Be sure that your sequence file does not have any blank lines in the middle of your sequence, as these are interpreted as the beginning of another chain.

• map_file = Auto MTZ file containing starting map. This file must be a mtz file. The Wizard will guess the column identification. You can specify the column labels to use with: input_map_labels='FP PHIB FOM' Substitute any labels you do not have with None. If you only have myFP and myPHIB you can just say input_map_labels='myFP myPHIB'. This map will be used in the first cycle of model-building. NOTE 1: If use_map_file_as_hklstart=True then this file will be used instead to start density modification. NOTE 2: default for this keyword is Auto, which means "carry out normal process to guess this keyword". This means if you specify "after_autosol" in AutoBuild, AutoBuild will automatically take the value from AutoSol. If you do not want this to happen, you can specify None which means "No file"

• refinement_file = Auto File for refinement. This file can be a .sca or mtz or other standard file. This file will be merged with your data file, with any phase information coming from your data file. If this file has free R flags, they will be used, otherwise if the data file has them, those will be used, otherwise they will be generated. The Wizard will guess the column identification. You can specify the column labels to use with: input_refinement_labels='FP SIGFP FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP you can just say input_refinement_labels='myFP mysigFP'. Data file to use for refinement. The data in this file should not be corrected for anisotropy. It will be combined with experimental phase information (if any) from input_data_file for refinement. If you leave this blank, then the data in the input_data_file will be used in refinement. If no anisotropy correction is applied to the data you do not need to specify a datafile for refinement. If an anisotropy correction is applied to the data files, then you should enter an uncorrected datafile for refinement. Any standard format is fine; normally only F and sigF will be used. Bijvoet pairs and duplicates will be averaged. If an mtz file is provided then a free R flag can be read in as well. Any HL coeffs and phase information in this file is ignored. NOTE: default for

this keyword is Auto, which means "carry out normal process to guess this keyword". This means if you specify "after_autosol" in AutoBuild, AutoBuild will automatically take the value from AutoSol. If you do not want this to happen, you can specify None which means "No file"

• hires_file = Auto File with high-resolution data. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_hires_labels='FP SIGFP'.

• crystal_infounit_cell = None Enter cell parameter (a b c alpha beta gamma)space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)solvent_fraction = None Solvent fraction in crystals (0 to 1). This is normally set automatically from the number of NCS copies and the sequence. If your has unequal numbers of different chains, then be sure to set the solvent fraction.chain_type = *Auto PROTEIN DNA RNA You can specify whether to build protein, DNA, or RNA chains. At present you can only build one of these in a single run. If you have both DNA and protein, build one first, then run AutoBuild again, supplying the prebuilt model in the "input_lig_file_list" and build the other. NOTE: default for this keyword is Auto, which means "carry out normal process to guess this keyword". The process is to look at the sequence file and/or input pdb file to see what the chain type is. If there are more than one type, the type with the larger number of residues is guessed. If you want to force the chain_type, then set it to PROTEIN RNA or DNA. resolution = 0 High-resolution limit. Used as resolution limit for density modification and as general default high-resolution limit. If resolution_build or refinement_resolution are set then they override this for model-building or refinement. If overall_resolution is set then data beyond that resolution is ignored completely. Zero means keep everything.dmax = 500 Low-resolution limit overall_resolution = 0 If overall_resolution is set, then all data beyond this is ignored. NOTE: this is only suggested if you have a very big cell and need to truncate the data to allow the wizard to run at all. Normally you should use 'resolution' and 'resolution_build' and 'refinement_resolution' to set the high-resolution limit sequence = None Plain text containing 1-letter code of protein sequence Same as seq_file except the sequence is read directly, not from a file. If both are given, seq_file is ignored.

• unit_cell = None Enter cell parameter (a b c alpha beta gamma)

• space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)

• solvent_fraction = None Solvent fraction in crystals (0 to 1). This is normally set automatically from the number of NCS copies and the sequence. If your has unequal numbers of different chains, then be sure to set the solvent fraction.

• chain_type = *Auto PROTEIN DNA RNA You can specify whether to build protein, DNA, or RNA chains. At present you can only build one of these in a single run. If you have both DNA and protein, build one first, then run AutoBuild again, supplying the prebuilt model in the "input_lig_file_list" and build the other. NOTE: default for this keyword is Auto, which means "carry out normal process to guess this keyword". The process is to look at the sequence file and/or input pdb file to see what the chain type is. If there are more than one type, the type with the larger number of residues is guessed. If you want to force the chain_type, then set it to PROTEIN RNA or DNA.

• resolution = 0 High-resolution limit. Used as resolution limit for density modification and as general default high-resolution limit. If resolution_build or refinement_resolution are set then they override this for model-building or refinement. If overall_resolution is set then data beyond that resolution is ignored completely. Zero means keep everything.

• dmax = 500 Low-resolution limit

• overall_resolution = 0 If overall_resolution is set, then all data beyond this is ignored. NOTE: this is only suggested if you have a very big cell and need to truncate the data to allow the wizard to run at all.

Normally you should use 'resolution' and 'resolution_build' and 'refinement_resolution' to set the high-resolution limit

• sequence = None Plain text containing 1-letter code of protein sequence Same as seq_file except the sequence is read directly, not from a file. If both are given, seq_file is ignored.

• output_filestarget_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.

• target_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.

• input_filesinput_labels = None Labels for input data columns input_hires_labels = None Labels for input hires file (FP SIGFP FreeR_flag) input_map_labels = None Labels for input map coefficient columns (FP PHIB FOM) NOTE: FOM is optional (set to None if you wish) input_refinement_labels = None Labels for input refinement file columns (FP SIGFP FreeR_flag) input_ha_file = None If the flag "truncate_ha_sites_in_resolve" is set then density at sites specified with input_ha_file is truncated to improve the density modification procedure. Additionally these sites are added to input_lig_file_list. NOTE: if the chain ID for atoms in this file is A then the chain ID will be renamed to Z so that the residue number and chain ID combination does not duplicate residues to be built. NOTE: if a hires_file is supplied, this input_ha_file and also any atoms in chain Z of input PDB file will be ignored unless force_input_ha is set to True.force_input_ha = False If a hires_file is supplied, input_ha_file and any atoms in chain Z of input pdb file will be ignored unless force_input_ha is set to True.include_ha_in_model = True Add contents of input_ha_file to the working model just before refinement (by adding it to input_lig_file_list).cif_def_file_list = None You can enter any number of CIF definition files. These are normally used to tell phenix.refine about the geometry of a ligand or unusual residue. You usually will use these in combination with "PDB file with metals/ligands" (keyword "input_lig_file_list") which allows you to attach the contents of any PDB file you like to your model just before it gets refined. You can use phenix.elbow to generate these if you do not have a CIF file and one is requested by phenix.refine input_lig_file_list = None This script adds the contents of these PDB files to each model just prior to refinement. Normally you might use this to put in any heavy-atoms that are in the refined structure (for example the heavy atoms that were used in phasing), or to add a ligand to your model. (By default if you supply input_ha_file this will be added to your input_lig_file_list.) If the atoms in this PDB file are not recognized by phenix.refine, then you can specify their geometries with a cif definitions file using the keyword "cif_def_files_list". You can easily generate cif definitions for many ligands using phenix.elbow in PHENIX. You can put anything you like in the files in input_lig_file_list, but any atoms that fall within 1.5 A of any atom in the current model will be tossed (not written to the model). keep_input_ligands = True You can choose whether to (by default) let the wizard keep ligands by separating them out from the rest of your model and adding them back to your rebuilt model, or alternatively to remove all ligands from your input pdb file before rebuild_in_place. keep_input_waters = False You can choose whether to keep input waters (solvent) when using rebuild_in_place. If you keep them, then you should specify either "place_waters=No" or "keep_pdb_atoms=No" because if place_waters=True and keep_pdb_atoms=True then phenix.refine will add waters and then the wizard will keep the new waters from the new PDB file created by phenix.refine preferentially over the ones in your input file. keep_pdb_atoms = True If true, keep the model coordinates when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current model. If false, keep instead the ligand coordinates. remove_residues_on_special_positions = False If true, remove any residues with atoms on special positions just before refinement.refine_eff_file_list = None You can enter any number of refinement

parameter files. These are normally used to tell phenix.refine defaults to apply, as well as creating specialized definitions such as unusual amino acid residues and linkages. These para...

• input_labels = None Labels for input data columns

• input_hires_labels = None Labels for input hires file (FP SIGFP FreeR_flag)

• input_map_labels = None Labels for input map coefficient columns (FP PHIB FOM) NOTE: FOM is optional (set to None if you wish)

• input_refinement_labels = None Labels for input refinement file columns (FP SIGFP FreeR_flag)

• input_ha_file = None If the flag "truncate_ha_sites_in_resolve" is set then density at sites specified with input_ha_file is truncated to improve the density modification procedure. Additionally these sites are added to input_lig_file_list. NOTE: if the chain ID for atoms in this file is A then the chain ID will be renamed to Z so that the residue number and chain ID combination does not duplicate residues to be built. NOTE: if a hires_file is supplied, this input_ha_file and also any atoms in chain Z of input PDB file will be ignored unless force_input_ha is set to True.

• force_input_ha = False If a hires_file is supplied, input_ha_file and any atoms in chain Z of input pdb file will be ignored unless force_input_ha is set to True.

• include_ha_in_model = True Add contents of input_ha_file to the working model just before refinement (by adding it to input_lig_file_list).

• cif_def_file_list = None You can enter any number of CIF definition files. These are normally used to tell phenix.refine about the geometry of a ligand or unusual residue. You usually will use these in combination with "PDB file with metals/ligands" (keyword "input_lig_file_list") which allows you to attach the contents of any PDB file you like to your model just before it gets refined. You can use phenix.elbow to generate these if you do not have a CIF file and one is requested by phenix.refine

• input_lig_file_list = None This script adds the contents of these PDB files to each model just prior to refinement. Normally you might use this to put in any heavy-atoms that are in the refined structure (for example the heavy atoms that were used in phasing), or to add a ligand to your model. (By default if you supply input_ha_file this will be added to your input_lig_file_list.) If the atoms in this PDB file are not recognized by phenix.refine, then you can specify their geometries with a cif definitions file using the keyword "cif_def_files_list". You can easily generate cif definitions for many ligands using phenix.elbow in PHENIX. You can put anything you like in the files in input_lig_file_list, but any atoms that fall within 1.5 A of any atom in the current model will be tossed (not written to the model).

• keep_input_ligands = True You can choose whether to (by default) let the wizard keep ligands by separating them out from the rest of your model and adding them back to your rebuilt model, or alternatively to remove all ligands from your input pdb file before rebuild_in_place.

• keep_input_waters = False You can choose whether to keep input waters (solvent) when using rebuild_in_place. If you keep them, then you should specify either "place_waters=No" or "keep_pdb_atoms=No" because if place_waters=True and keep_pdb_atoms=True then phenix.refine will add waters and then the wizard will keep the new waters from the new PDB file created by phenix.refine preferentially over the ones in your input file.

• keep_pdb_atoms = True If true, keep the model coordinates when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current model. If false, keep instead the ligand coordinates.

• remove_residues_on_special_positions = False If true, remove any residues with atoms on special positions just before refinement.

• refine_eff_file_list = None You can enter any number of refinement parameter files. These are normally used to tell phenix.refine defaults to apply, as well as creating specialized definitions such as unusual amino acid residues and linkages. These parameters override the normal phenix.refine defaults. They themselves can be overridden by parameters set by the Wizard and by you, controlling the Wizard. NOTE: Any parameters set by AutoBuild directly (such as number_of_macro_cycles, high_resolution, etc...) will not be taken from this parameters file. This is useful only for adding extra parameters not normally set by AutoBuild.

• map_file_is_density_modified = True You can specify that the input_map_file has been density modified. (This changes the assumptions on statistics of the map.)

• map_file_fom = None You can specify the FOM of the input map file (useful in cases where the map file has only FWT PHFWT and no FOM column). This FOM is used to set the default smoothing radius for the density modification solvent boundary and also to decide whether extreme density modification is to be applied

• use_constant_input_map = False You can use the input map file for all model-building

• use_map_file_as_hklstart = None You can specify that the file named as input_map_file will be used as starting coefficients for density modification in the first cycle. NOTE: if maps_only=True and input_map_file is set, then use_map_file_as_hklstart will be set to True

• use_map_in_resolve_with_model = False You can specify that the current map file be used as hklstart in density modification with a model.

• identity_from_remark = True You can use sequence identity from remark like this one: REMARK PHASER ENSEMBLE MODEL 0 ID 100.0 Here the identity is 100 percent. Ignored if there is no remark of this kind in the input model.

• input_data_type = None You can specify the data type for shelx files. The options are: amplitudes (same as hklf3) or intensities (same as hklf4)

• anisoremove_aniso = True Remove anisotropy from data files before use Note: map files are assumed to be already corrected and are not affected by this. Also the input refinement file is not affected by this.b_iso = None Target overall B value for anisotropy correction. Ignored if remove_aniso = False. If None, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)max_b_iso = 40. Default maximum overall B value for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)target_b_ratio = 10. Default ratio of target B value to resolution for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• remove_aniso = True Remove anisotropy from data files before use Note: map files are assumed to be already corrected and are not affected by this. Also the input refinement file is not affected by this.

• b_iso = None Target overall B value for anisotropy correction. Ignored if remove_aniso = False. If None, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• max_b_iso = 40. Default maximum overall B value for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• target_b_ratio = 10. Default ratio of target B value to resolution for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• decision_makingacceptable_r = 0.25 Used to decide whether the model is acceptable enough to quit if it is not improving much. A good value is 0.25 r_switch = 0.4 R-value criteria for deciding whether to use R-value or map correlation as a criteria for model quality. A good value is 0.40 semi_acceptable_r = 0.3 Used to decide whether the model is acceptable enough to skip rebuilding the model from scratch and focus on adding loops and extending it. A good value is 0.3 reject_weak = False You can rebuild or remove just the residues in weak density This will reject residues with density < 0.5 * mean - SD where the density, mean and SD are for either main-chain or all atoms in residues. If set, overrides min_cc_res_rebuild and worst_percent_res_rebuild.min_weak_z = 0.2 Minimum number of sd of rho above 0.5*mean of all residues for keeping weak residues if reject_weak=Truemin_cc_res_rebuild = 0.4 You can rebuild just the worst parts of your model by setting touch_up=True. You can decide what parts to rebuild based on a minimum model-map correlation (by residue). You can decide how much to rebuild using worst_percent_res_rebuild or with min_cc_res_rebuild, or both. min_seq_identity_percent = 50 The sequence in your input PDB file will be adjusted to match the sequence in your sequence file (if any). If there are insertions/deletions in your model and the wizard does not seem to identify them, you can split up your PDB file by adding records like this: BREAK You can specify the minimum sequence identity between your sequence file and a segment from your input PDB file to consider the sequences to be matched. Default is 50.0%. You might want a higher number to make sure that deletions in the sequence are noticed. dist_close = None If main-chain atom rmsd is less than dist_close then crossover between chains in different models is allowed at this point. If you input a negative number the defaults will be used dist_close_overlap = 1.5 Model or ligand coordinates but not both are kept when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current model. NOTE: you might want to decrease this if your ligand atoms get removed by the wizard. Default=1.5 A loop_cc_min = 0.4 You can specify the minimum correlation of density from a loop with the map. group_ca_length = 4 In resolve building you can specify how short a fragment to keep. Normally 4 or 5 residues should be the minimum.group_length = 2 In resolve building you can specify how many fragments must be joined to make a connected group that is kept. Normally 2 fragments should be the minimum. include_molprobity = False This command is currently disabled. You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero. ok_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score (the threshold defined by ok_molp_score) then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero. scale_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• acceptable_r = 0.25 Used to decide whether the model is acceptable enough to quit if it is not improving much. A good value is 0.25

• r_switch = 0.4 R-value criteria for deciding whether to use R-value or map correlation as a criteria for model quality. A good value is 0.40

• semi_acceptable_r = 0.3 Used to decide whether the model is acceptable enough to skip rebuilding the model from scratch and focus on adding loops and extending it. A good value is 0.3

• reject_weak = False You can rebuild or remove just the residues in weak density This will reject residues with density < 0.5 * mean - SD where the density, mean and SD are for either main-chain or all

atoms in residues. If set, overrides min_cc_res_rebuild and worst_percent_res_rebuild.

• min_weak_z = 0.2 Minimum number of sd of rho above 0.5*mean of all residues for keeping weak residues if reject_weak=True

• min_cc_res_rebuild = 0.4 You can rebuild just the worst parts of your model by setting touch_up=True. You can decide what parts to rebuild based on a minimum model-map correlation (by residue). You can decide how much to rebuild using worst_percent_res_rebuild or with min_cc_res_rebuild, or both.

• min_seq_identity_percent = 50 The sequence in your input PDB file will be adjusted to match the sequence in your sequence file (if any). If there are insertions/deletions in your model and the wizard does not seem to identify them, you can split up your PDB file by adding records like this: BREAK You can specify the minimum sequence identity between your sequence file and a segment from your input PDB file to consider the sequences to be matched. Default is 50.0%. You might want a higher number to make sure that deletions in the sequence are noticed.

• dist_close = None If main-chain atom rmsd is less than dist_close then crossover between chains in different models is allowed at this point. If you input a negative number the defaults will be used

• dist_close_overlap = 1.5 Model or ligand coordinates but not both are kept when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current model. NOTE: you might want to decrease this if your ligand atoms get removed by the wizard. Default=1.5 A

• loop_cc_min = 0.4 You can specify the minimum correlation of density from a loop with the map.

• group_ca_length = 4 In resolve building you can specify how short a fragment to keep. Normally 4 or 5 residues should be the minimum.

• group_length = 2 In resolve building you can specify how many fragments must be joined to make a connected group that is kept. Normally 2 fragments should be the minimum.

• include_molprobity = False This command is currently disabled. You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• ok_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score (the threshold defined by ok_molp_score) then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• scale_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• density_modificationadd_classic_denmod = None You can run classic density modification with solvent flipping after any other kind of density modification. Note: this keyword cannot be used with an omit map. Note also: requires experimental phases (HL coeffs). Default is False unless extreme_dm=True and FOM is less than fom_for_extreme_dm.skip_classic_if_worse_fom = True Skip results of add_classic_denmod if FOM gets worse during density modificationskip_ncs_in_add_classic = True Skip using NCS in add_classic_denmod (speeds it up)thorough_denmod = *Auto True False Choose whether you want to go for thorough density modification when no model is used ("False" speeds it up and for a terrible map is sometimes better) hl = False You can choose whether to calculate hl coeffs when doing

density modification (True) or not to do so (False). Default is No. mask_type = *histograms probability wang classic Choose method for obtaining probability that a point is in the protein vs solvent region. Default is "histograms". If you have a SAD dataset with a heavy atom such as Pt or Au then you may wish to choose "wang" because the histogram method is sensitive to very high peaks. Options are: histograms: compare local rms of map and local skew of map to values from a model map and estimate probabilities. This one is usually the best. probability: compare local rms of map to distribution for all points in this map and estimate probabilities. In a few cases this one is much better than histograms. wang: take points with highest local rms and define as protein. Classic runs classical density modification with solvent flipping.mask_from_pdb = None You can specify a PDB file to define a mask for the macromolecule in density modification (i.e., the solvent boundary). All points within rad_mask_from_pdb of an atom in the PDB file defined by mask_from_pdb will be considered to be within the macromoleculemask_type_extreme_dm = histograms probability *wang classic If FOM of phasing is less up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dmmask_cycles_extreme_dm = 1 Mask cycles in extreme density modificationminor_cycles_extreme_dm = 4 Minor cycles in extreme density modificationwang_radius_extreme_dm = 20. Wang radius in extreme density modificationprecondition = False Precondition density before modificationminimum_ncs_cc = 0.30 Minimum NCS correlation to keep, except in case of extreme_dmextreme_dm = False Turns on extreme density modification if True or if Auto and FOM is up to fom_for_extreme_dm Use extreme_dm=True if your phasing is really weak and density modification is not working well. Note: requires input phase information (HL coeffs). Also note: not compatible with ps_in_rebuild, two_fofc_in_rebuild, or two_fofc_denmod_in_rebuild.fom_for_extreme_dm_rebuild = 0.10 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=mask_type_extreme_dm wang_radius=wang_radius_extreme_dm mask_cycles=mask_cycles_extreme_dm minor_cycles=minor_cycles_extreme_dm Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dmfom_for_extreme_dm = 0.35 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to build stages of autobuild. For rebuild use instead fom_for_extreme_dm_rebuildrad_mask_from_pdb = 2 You can define the radius for calculation of the protein mask Applies only to mask_from_pdbmodify_outside_delta_solvent = 0.05 You can set the initial solvent content to be a little lower than calculated when you are running modify_outside_model Usually 0.05 is fine. modify_outside_model = False You can choose whether to modify ...

• add_classic_denmod = None You can run classic density modification with solvent flipping after any other kind of density modification. Note: this keyword cannot be used with an omit map. Note also: requires experimental phases (HL coeffs). Default is False unless extreme_dm=True and FOM is less than fom_for_extreme_dm.

• skip_classic_if_worse_fom = True Skip results of add_classic_denmod if FOM gets worse during density modification

• skip_ncs_in_add_classic = True Skip using NCS in add_classic_denmod (speeds it up)

• thorough_denmod = *Auto True False Choose whether you want to go for thorough density modification when no model is used ("False" speeds it up and for a terrible map is sometimes better)

• hl = False You can choose whether to calculate hl coeffs when doing density modification (True) or not to do so (False). Default is No.

• mask_type = *histograms probability wang classic Choose method for obtaining probability that a point is in the protein vs solvent region. Default is "histograms". If you have a SAD dataset with a heavy atom such as Pt or Au then you may wish to choose "wang" because the histogram method is sensitive to very high peaks. Options are: histograms: compare local rms of map and local skew of map to values from a model map and estimate probabilities. This one is usually the best. probability: compare local rms of map to distribution for all points in this map and estimate probabilities. In a few cases this one is much better than histograms. wang: take points with highest local rms and define as protein. Classic runs classical density modification with solvent flipping.

• mask_from_pdb = None You can specify a PDB file to define a mask for the macromolecule in density modification (i.e., the solvent boundary). All points within rad_mask_from_pdb of an atom in the PDB file defined by mask_from_pdb will be considered to be within the macromolecule

• mask_type_extreme_dm = histograms probability *wang classic If FOM of phasing is less up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dm

• mask_cycles_extreme_dm = 1 Mask cycles in extreme density modification

• minor_cycles_extreme_dm = 4 Minor cycles in extreme density modification

• wang_radius_extreme_dm = 20. Wang radius in extreme density modification

• precondition = False Precondition density before modification

• minimum_ncs_cc = 0.30 Minimum NCS correlation to keep, except in case of extreme_dm

• extreme_dm = False Turns on extreme density modification if True or if Auto and FOM is up to fom_for_extreme_dm Use extreme_dm=True if your phasing is really weak and density modification is not working well. Note: requires input phase information (HL coeffs). Also note: not compatible with ps_in_rebuild, two_fofc_in_rebuild, or two_fofc_denmod_in_rebuild.

• fom_for_extreme_dm_rebuild = 0.10 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=mask_type_extreme_dm wang_radius=wang_radius_extreme_dm mask_cycles=mask_cycles_extreme_dm minor_cycles=minor_cycles_extreme_dm Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dm

• fom_for_extreme_dm = 0.35 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to build stages of autobuild. For rebuild use instead fom_for_extreme_dm_rebuild

• rad_mask_from_pdb = 2 You can define the radius for calculation of the protein mask Applies only to mask_from_pdb

• modify_outside_delta_solvent = 0.05 You can set the initial solvent content to be a little lower than calculated when you are running modify_outside_model Usually 0.05 is fine.

• modify_outside_model = False You can choose whether to modify the density in the "protein" region outside the region specified in your current model by matching histograms with the region that is specified by that model. This can help by raising the density in this protein region up to a value similar to that where atoms are already placed.

• truncate_ha_sites_in_resolve = *Auto True False You can choose to truncate the density near heavy-atom sites at a maximum of 2.5 sigma. This is useful in cases where the heavy-atom sites are very strong, and rarely hurts in cases where they are not. The heavy-atom sites are specified with

"input_ha_file" and the radius is rad_mask

• rad_mask = None You can define the radius for calculation of the protein mask Applies only to truncate_ha_sites_in_resolve. Default is resolution of data.

• s_step = None You can define the increment in s (1/resolution) for phase extension in resolve density modification. Default is 0.005

• res_start = None You can define the resolution for starting phase extension resolve density modification. Default is resolution to which phase information is available. Please note res_start and map_dmin_start have different effects. The starting resolution within RESOLVE is set by res_start and the step within RESOLVE is set by s_step. These can be used with any density modification procedure. The keyword map_dmin_start creates a set of macro-cycles of RESOLVE density modification where the output map for each cycle is the input_map_file for the next. The map_dmin_start keyword only applies if maps_only=True and only applies to density modification without a model.

• map_dmin_start = None Starting resolution for step-wise macro-cycle density modification. Only applies if maps_only=True and no model is supplied. Please note res_start and map_dmin_start have different effects. The starting resolution within RESOLVE is set by res_start and the step within RESOLVE is set by s_step. These can be used with any density modification procedure. The keyword map_dmin_start creates a set of macro-cycles of RESOLVE density modification where the output map for each cycle is the input_map_file for the next. The map_dmin_start keyword only applies if maps_only=True and only applies to density modification without a model.

• map_dmin_incr = 0.25 Step size (A) for changes in resolution for macro-cycle density modification

• use_resolve_fragments = True This script normally uses information from fragment identification as part of density modification for the first few cycles of model-building. Fragments are identified during model-building. The fragments are used, with weighting according to the confidence in their placement, in density modification as targets for density values.

• use_resolve_pattern = True Local pattern identification is normally used as part of density modification during the first few cycles of model building.

• use_hl_anom_in_denmod = False Default is False (use HL coefficients in density modification) NOTE: if True, you must supply HLanom coefficients Allows you to specify that HL coefficients including only the phase information from the imaginary (anomalous difference) contribution from the anomalous scatterers are to be used in density modification. Two sets of HL coefficients are produced by Phaser. HLA HLB etc are HL coefficients including the contribution of both the real scattering and the anomalous differences. HLanomA HLanomB etc are HL coefficients including the contribution of the anomalous differences alone. These HL coefficients for anomalous differences alone are the ones that you will want to use in cases where you are bringing in model information that includes the real scattering from the model used in Phaser, such as when you are carrying out density modification with a model or refinement of a model If use_hl_anom_in_denmod=True then the HLanom HL coefficients from Phaser are used in density modification

• use_hl_anom_in_denmod_with_model = False See use_hl_anom_in_denmod If use_hl_anom_in_denmod=True then the HLanom HL coefficients from Phaser are used in density modification with a model

• mask_as_mtz = False Defines how omit_output_mask_file ncs_output_mask_file and protein_output_mask_file are written out. If mask_as_mtz=False it will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with map coefficients FP PHIM FOMM (all three required)

• protein_output_mask_file = None Name of map to be written out representing your protein (non-solvent) region. If mask_as_mtz=False the map will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with

map coefficients FP PHIM FOMM (all three required)

• ncs_output_mask_file = None Name of map to be written out representing your ncs asymmetric unit. If mask_as_mtz=False the map will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with map coefficients FP PHIM FOMM (all three required)

• omit_output_mask_file = None Name of map to be written out representing your omit region. If mask_as_mtz=False the map will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with map coefficients FP PHIM FOMM (all three required)

• mapsmaps_only = False You can choose whether to skip all model-building and just calculate maps and write out the results. This also runs just 1 cycle and turns on HL coefficients. n_xyz_list = None You can specify the grid to use for map calculations.

• maps_only = False You can choose whether to skip all model-building and just calculate maps and write out the results. This also runs just 1 cycle and turns on HL coefficients.

• n_xyz_list = None You can specify the grid to use for map calculations.

• model_buildingbuild_type = *RESOLVE RESOLVE_AND_BUCCANEER You can choose to build models with RESOLVE or with RESOLVE and BUCCANEER #and TEXTAL and how many different models to build with RESOLVE. The more you build, the more likely to get a complete model. Note that rebuild_in_place can only be carried out with RESOLVE model-building. For BUCCANEER model building you need CCP4 version 6.1.2 or higher and BUCCANEER version 1.3.0 or higherallow_negative_residues = False Normally the wizard does not allow negative residue numbers, and all residues with negative numbers are rejected when they are read in. You can allow them if you wish. highest_resno = None Highest residue number to be considered "placed" in sequence for rebuild_in_place semet = False You can specify that the dataset that is used for refinement is a selenomethionine dataset, and that the model should be the SeMet version of the protein, with all SD of MET replaced with Se of MSE. use_met_in_align = *Auto True False You can use the heavy-atom positions in input_ha_file as markers for Met SD positions. base_model = None You can enter a PDB file with coordinates to be used as a starting point for model-building. These coordinates will be included in the same way as fragments placed by searching for helices and strand in initial model-building. Note the difference from the use of models in consider_main_chain_list, which are merged with models after they are built. NOTE: Only use this if you want to keep the input model and just add to it. consider_main_chain_list = None This keyword lets you name any number of PDB files to consider as templates for model-building. Every time models are built, the contents of these files will be merged with them and the best parts will be kept. NOTE: this only uses the main-chain atoms of your PDB files. dist_connect_max_helices = None Set maximum distance between ends of helices and other ends to try and connect them in insert_helices. edit_pdb = True You can choose to edit the input PDB file in rebuild_in_place to match the input sequence (default=True). NOTE: residues with residue numbers higher than 'highest_resno' are assumed to not have a known sequence and will not be edited. By default the value of 'highest_resno' is the highest residue number from the sequence file, after adding it to the starting residue number from start_chains_list. You can also set it directly helices_strands_only = False You can choose to use a quick model-building method that only builds secondary structure. At low resolution this may be both quicker and more accurate than trying to build the entire structure resolution_helices_strands = 3.1 Resolution to switch to helices_strands_onlyhelices_strands_start = False You can choose to use a quick model-building method that builds secondary structure as a way to get started...then model completion is done as usual. (Contrast with helices_strands_only which only does secondary structure) cc_helix_min = None Minimum CC of helical density to map at low resolution when using helices_strands_onlycc_strand_min = None Minimum CC of strand density to map when using helices_strands_only trim_ends_if_needed = True If a single loop is specified and no loop is found,

try trimming back the ends.loop_backup_residues = None Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial gap Default is 4 (1 if quick)split_by_gap = True Split up work by gaps and run individually (normally True)loop_lib = False Use loop library to fit loops Only applicable for chain_type=PROTEINstandard_loops = True Use standard loop fitting trace_loops = False Use loop tracing to fit loops Only applicable for chain_type=PROTEINrefine_trace_loops = True Refine loops (real-space) after trace_loopsdensity_of_points = None Packing density of points to consider as as possible CA atoms in trace_loops. Try 1.0 for a quick run, up to 5 for much more thorough run If None, try ...

• build_type = *RESOLVE RESOLVE_AND_BUCCANEER You can choose to build models with RESOLVE or with RESOLVE and BUCCANEER #and TEXTAL and how many different models to build with RESOLVE. The more you build, the more likely to get a complete model. Note that rebuild_in_place can only be carried out with RESOLVE model-building. For BUCCANEER model building you need CCP4 version 6.1.2 or higher and BUCCANEER version 1.3.0 or higher

• allow_negative_residues = False Normally the wizard does not allow negative residue numbers, and all residues with negative numbers are rejected when they are read in. You can allow them if you wish.

• highest_resno = None Highest residue number to be considered "placed" in sequence for rebuild_in_place

• semet = False You can specify that the dataset that is used for refinement is a selenomethionine dataset, and that the model should be the SeMet version of the protein, with all SD of MET replaced with Se of MSE.

• use_met_in_align = *Auto True False You can use the heavy-atom positions in input_ha_file as markers for Met SD positions.

• base_model = None You can enter a PDB file with coordinates to be used as a starting point for model-building. These coordinates will be included in the same way as fragments placed by searching for helices and strand in initial model-building. Note the difference from the use of models in consider_main_chain_list, which are merged with models after they are built. NOTE: Only use this if you want to keep the input model and just add to it.

• consider_main_chain_list = None This keyword lets you name any number of PDB files to consider as templates for model-building. Every time models are built, the contents of these files will be merged with them and the best parts will be kept. NOTE: this only uses the main-chain atoms of your PDB files.

• dist_connect_max_helices = None Set maximum distance between ends of helices and other ends to try and connect them in insert_helices.

• edit_pdb = True You can choose to edit the input PDB file in rebuild_in_place to match the input sequence (default=True). NOTE: residues with residue numbers higher than 'highest_resno' are assumed to not have a known sequence and will not be edited. By default the value of 'highest_resno' is the highest residue number from the sequence file, after adding it to the starting residue number from start_chains_list. You can also set it directly

• helices_strands_only = False You can choose to use a quick model-building method that only builds secondary structure. At low resolution this may be both quicker and more accurate than trying to build the entire structure

• resolution_helices_strands = 3.1 Resolution to switch to helices_strands_only

• helices_strands_start = False You can choose to use a quick model-building method that builds secondary structure as a way to get started...then model completion is done as usual. (Contrast with helices_strands_only which only does secondary structure)

• cc_helix_min = None Minimum CC of helical density to map at low resolution when using helices_strands_only

• cc_strand_min = None Minimum CC of strand density to map when using helices_strands_only

• trim_ends_if_needed = True If a single loop is specified and no loop is found, try trimming back the ends.

• loop_backup_residues = None Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial gap Default is 4 (1 if quick)

• split_by_gap = True Split up work by gaps and run individually (normally True)

• loop_lib = False Use loop library to fit loops Only applicable for chain_type=PROTEIN

• standard_loops = True Use standard loop fitting

• trace_loops = False Use loop tracing to fit loops Only applicable for chain_type=PROTEIN

• refine_trace_loops = True Refine loops (real-space) after trace_loops

• density_of_points = None Packing density of points to consider as as possible CA atoms in trace_loops. Try 1.0 for a quick run, up to 5 for much more thorough run If None, try value depending on value of quick.

• a_cut_min = None Minimum density (relative to SD of map, normalized for solvent content) for trace_loop points

• max_density_of_points = None Maximum packing density of points to consider as as possible CA atoms in trace_loops.

• cutout_model_radius = None Radius to cut out density for trace_loops If None, guess based on length of loop

• max_cutout_model_radius = 20. Maximum value of cutout_model_radius to try

• padding = 1. Padding for cut out density in trace_loops

• cut_out_density = True Cut out density for trace_loops

• max_span = 30 Maximum length of a gap to try to fill

• max_c_ca_dist = None Maximum C-CA distance for a connection

• max_overlap_rmsd = 2. Maximum rmsd for 3 residues on each end of loop-lib fit

• max_overlap = None Maximum number of residues from ends to start with. (1=use existing ends, 2=one in from ends etc) If None, set based on value of quick.

• min_overlap = None Minimum number of residues from ends to start with. (1=use existing ends, 2=one in from ends etc)

• include_input_model = True The keyword include_input_model defines whether the input model (if any) is to be crossed with models that are derived from it, and the best parts of each kept. It also defines whether the input model is to be included in combination steps during initial model-building. Note that if multiple_models=True and include_input_model=True then no initial cycle of randomization will be carried out and the keyword multiple_models_starting_resolution is ignored. In most cases you should use include_input_model=True If you want to generate maximum diversity with multiple-models then you may wish to use include_input_model=False. Also if you want to decrease the amount of bias from your starting model you may wish to use include_input_model=False.

• input_compare_file = None If you are rebuilding a model or already think you know what the model should be, you can include a comparison file in rebuilding. The model is not used for anything except to write out information on coordinate differences in the output log files. NOTE: this feature does not always

work correctly.

• merge_models = False You can choose to only merge any input models and write out the resulting model. The best parts of each model will be kept based on model-map correlation. Normally used along with number_of_parallel_models=1

• morph = False You can choose whether to distort your input model in order to match the current working map. This may be useful for MR models that are quite distant from the correct structure.

• morph_main = False You can choose whether to use only main-chain atoms plus c-beta atoms in calculation of shifts in morphing. Default is morph_main=False; use all atoms including side-chain atoms.

• dist_cut_base = 3.0 Tolerance for base pairing (A) for RNA/DNA)

• morph_cycles = 2 Number of iterations of morphing each time it is run.

• morph_rad = 7 Smoothing radius for morphing. The density from your model and from the map are calculated with the radius rad_morph, then they are adjusted to overlap optimally

• n_ca_enough_helices = None Set maximum number of CA to add to ends of helices and other ends to try and connect them in insert_helices.

• delta_phi = 20 Approximate angular sampling for search for regular secondary structure in building

• offsets_list = 53 7 23 You can specify an offset for the orientation of the helix and strand templates in building. This is used in generating different starting models.

• all_maps_in_rebuild = False If two_fofc_in_rebuild or two_fofc_denmod_in_rebuild are set you can choose to try both density-modified and two_fofc-based maps in building. Note: Set to True if you specify rebuild_from_fragments=True. Note: not compatible with map_phasing=True.

• ps_in_rebuild = False You can choose to use a prime-and-switch resolve map in all cycles of rebuilding instead of a density-modified map. This is normally used in combination with maps_only to generate a prime-and-switch map. The map coeffs will be in prime_and_switch.mtz

• use_ncs_in_ps = False You can choose to use NCS in prime-and-switch

• remove_outlier_segments_z_cut = 3.0 You can remove any segments that are not assigned to sequence during model-building if the mean density at atomic positions are more than remove_outlier_segments_z_cut sd lower than the mean for the structure.

• refine = True This script normally refines the model during building. Say False to skip refinement

• refine_final_model_vs_orig_data = True This script normally refines the model at the end against the original (non-aniso-corrected) data and writes out a CIF version of the model as well

• reference_model = None You can specify a reference model for refinement

• resolution_build = None Enter the high-resolution limit for model-building. If None, the value of resolution (or 2 A if better than 2 A) is used as a default.

• restart_cycle_after_morph = 5 Morphing (if morph=True) will go only up to this cycle, and then the morphed PDB file will be used as a starting PDB file from then on, removing all previous models. If restart_cycle_after_morph=0 then the model will be morphed and not rebuilt

• retrace_before_build = False You can choose to retrace your model n_mini times and use a map based on these retraced models to start off model-building. This is the default for rebuilding models if you are not using rebuild_in_place. You can also specify n_iter_rebuild, the number of cycles of retrace-density-modify-build before starting the main build.

• reuse_chain_prev_cycle = True You can choose to allow model-building to include atoms from each cycle in the model the next cycle or not This must be true if you use retrace_before_build

• richardson_rotamers = *Auto True False You can choose to use the rotamer library from SC Lovell, JM Word, JS Richardson and DC Richardson (2000) " The Penultimate Rotamer Library" Proteins: Structure Function and Genetics 40 389-408. if you wish. Typically this works well in RESOLVE model-building for nearly-final models but not as well earlier in the process . Default (Auto) is to use these rotamers for rebuild_in_place but not otherwise.

• rms_random_frag = None Rms random position change added to residues on ends of fragments when extending them If you enter a negative number, defaults will be used.

• rms_random_loop = None Rms random position change added to residues on ends of loops in tries for building loops If you enter a negative number, defaults will be used.

• start_chains_list = None You can specify the starting residue number for each of the unique chains in your structure. If you use a sequence file then the unique chains are extracted and the order must match the order of your starting residue numbers. For example, if your sequence file has chains A and B (identical) and chains C and D (identical to each other, but different than A and B) then you can enter 2 numbers, the starting residues for chains A and C. NOTE: you need to specify an input sequence file for start_chains_list to be applied.

• trace_as_lig = False You can specify that in building steps the ends of chains are to be extended using the LigandFit algorithm. This is default for nucleic acid model-building.

• track_libs = False You can keep track of what libraries each atom in a built structure comes from.

• two_fofc_denmod_in_rebuild = False You can choose to use a density-modified sigmaa-weighted 2Fo-Fc map in all cycles of rebuilding instead of a density-modified map. In density modification the density in the region defined by the current model will be truncated at +2sigma to reduce the dominance of parts of the map with model defined. Additionaly only 2 mask cycles of 3 minor cycles will be done. Additionally place_waters will be turned off. If the model is highly incomplete this can sometimes allow model-building to work even when it will not for density-modified maps. The map coeffs will be in two_fofc_denmod_map.mtz. You might consider turning on all_maps_in_rebuild as well. Note: Setting two_fofc_denmod_in_rebuild=True will by default set place_waters=False. Set to True if you specify rebuild_from_fragments=True.

• rebuild_from_fragments = False You can use rebuild_from_fragments=True as a shortcut to turn on two_fofc_denmod_in_rebuild and all_maps_in_rebuild and to use your model in each cycle with consider_main_chain_list. If you use rebuild_from_fragments=True you might also want to set i_ran_seed=xxxxx for some integer xxxxx and run the job 10 or 20 times to have a higher chance of success. This approach is designed for cases where you have a small part of your model very accurately placed and want to build the rest of the model.

• two_fofc_in_rebuild = False You can choose to use a sigmaa-weighted 2Fo-Fc map in all cycles of rebuilding instead of a density-modified map. If the model is poor this can sometimes allow model-building in place to work even when it will not for density-modified maps. This is also useful if you specify a twin law.

• refine_map_coeff_labels = "2FOFCWT PH2FOFCWT" You can pick which map coefficients from phenix.refine to use if two_fofc_in_rebuild=True

• filled_2fofc_maps = True You can choose to use filled 2Fo-Fc maps when two_fofc_in_rebuild is used. Default is True

• map_phasing = False You can choose to use statistical density modification starting with a 2mFo-DFc map, including model information instead of a standard density-modified map with model information. This density modification will include NCS if present. Note: not compatible with all_maps_in_rebuild=True

• use_any_side = True You can choose to have resolve model-building place the best-fitting side chain at each position, even if the sequence is not matched to the map.

• truncate_missing_side_chains = None You can choose to truncate side chains with missing density at CB or equivalent. Missing density means average density at the side chain atoms is less than zero. Default is False for rebuild_in_place=True and True for rebuild_in_place=False.

• use_cc_in_combine_extend = False You can choose to use the correlation of density rather than density at atomic positions to score models in combine_extend

• sort_hetatms = False Waters are automatically named with the chain of the closest macromolecule if you set sort_hetatms=True This is for the final model only.

• map_to_object = None you can supply a target position for your model with map_to_object=my_target.pdb. Then at the very end your molecule will be placed as close to this as possible. The center of mass of the autobuild model will be superimposed on the center of mass of my_target.pdb using space group symmetry, taking any match closer than 15 A within 3 unit cells of the original position. The new file will be overall_best_mapped.pdb

• multiple_modelscombine_only = False Once you have created a set of initial models you can merge them together into a final set. This option is useful if you have split up the creation of multiple models into different directories, and then you have copied all the initial models to one directory for combining. multiple_models = False You can build a set of models, all compatible with your data. You can specify how many models with multiple_models_number. If you are using rebuild_in_place you can specify whether to generate starting models or not with multiple_models_starting. multiple_models_first = 1 Specify which model to build first multiple_models_group_number = 5 You can build several initial models and merge them. Normally 5 initial models is fine. multiple_models_last = 20 Specify which model to end with multiple_models_number = 20 Specify how many models to build. multiple_models_starting = True You can specify how to generate starting models for multiple models. If you are using rebuild_in_place and you specify "True" then the Wizard will rebuild your starting model at the resolution specified in multiple_models_starting_resolution. If you are not using rebuild_in_place the Wizard will always build a starting model at the current resolution. multiple_models_starting_resolution = 4 You can set the resolution for rebuilding an initial model. A value of 0.0 will use the resolution of the dataset. place_waters_in_combine = None You can choose whether phenix.refine automatically places ordered solvent (waters) during the last cycle of multiple-model generation. This is separate from place_waters, which applies to all other cycles. If None, then value of place_waters will be used.

• combine_only = False Once you have created a set of initial models you can merge them together into a final set. This option is useful if you have split up the creation of multiple models into different directories, and then you have copied all the initial models to one directory for combining.

• multiple_models = False You can build a set of models, all compatible with your data. You can specify how many models with multiple_models_number. If you are using rebuild_in_place you can specify whether to generate starting models or not with multiple_models_starting.

• multiple_models_first = 1 Specify which model to build first

• multiple_models_group_number = 5 You can build several initial models and merge them. Normally 5 initial models is fine.

• multiple_models_last = 20 Specify which model to end with

• multiple_models_number = 20 Specify how many models to build.

• multiple_models_starting = True You can specify how to generate starting models for multiple models. If you are using rebuild_in_place and you specify "True" then the Wizard will rebuild your starting model at the resolution specified in multiple_models_starting_resolution. If you are not using

rebuild_in_place the Wizard will always build a starting model at the current resolution.

• multiple_models_starting_resolution = 4 You can set the resolution for rebuilding an initial model. A value of 0.0 will use the resolution of the dataset.

• place_waters_in_combine = None You can choose whether phenix.refine automatically places ordered solvent (waters) during the last cycle of multiple-model generation. This is separate from place_waters, which applies to all other cycles. If None, then value of place_waters will be used.

• ncsfind_ncs = *Auto True False This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. The update is done each cycle in which an improved model is obtained. Say False to skip this. See also "input_ncs_file" which can be used to specify NCS at the start of the process. If find_ncs="No" then only this starting NCS will be used and it will not be updated. You can use find_ncs "No" to specify exactly what residues will be used in NCS refinement and exactly what NCS operators to use in density modification. You can use the function $PHENIX/phenix/phenix/command_line/simple_ncs_from_pdb.py to help you set up an input_ncs_file that has your specifications in it. NOTE: if an input map_file is provided then if no ncs is found from a model, ncs will be searched for in the density of that map.input_ncs_file = None You can enter NCS information in 3 ways: (1) an ncs_spec file produced by AutoSol or AutoBuild with NCS information (2) a heavy-atom PDB file that contains ncs in the heavy-atom sites (3) a PDB file with a model that contains chains with NCS The wizard will derive NCS information from any of these if specified. See also "find_ncs" which determines whether the wizard will update NCS from models that are built during iterative building. ncs_copies = None Number of copies of the molecule in the au (note: only one type of molecule allowed at present) ncs_refine_coord_sigma_from_rmsd = False You can choose to use the current NCS rmsd as the value of the sigma for NCS restraints. See also ncs_refine_coord_sigma_from_rmsd_ratio ncs_refine_coord_sigma_from_rmsd_ratio = 1 You can choose to multiply the current NCS rmsd by this value before using it as the sigma for NCS restraints See also ncs_refine_coord_sigma_from_rmsd no_merge_ncs_copies = False Normally False (do merge NCS copies). If True, then do not use each NCS copy to try to build the others. optimize_ncs = True This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. Optimize NCS adds a step to try and make the molecule formed by NCS as compact as possible, without losing any point-group symmetry. use_ncs_in_build = True Use NCS information in the model assembly stage of model-building. Also if no_merge_ncs_copies is not set, then use each NCS copy to try to build the others. ncs_in_refinement = *torsion cartesian None Use torsion_angle refinement of NCS. Alternative is cartesian or None (None will use phenix.refine default)

• find_ncs = *Auto True False This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. The update is done each cycle in which an improved model is obtained. Say False to skip this. See also "input_ncs_file" which can be used to specify NCS at the start of the process. If find_ncs="No" then only this starting NCS will be used and it will not be updated. You can use find_ncs "No" to specify exactly what residues will be used in NCS refinement and exactly what NCS operators to use in density modification. You can use the function $PHENIX/phenix/phenix/command_line/simple_ncs_from_pdb.py to help you set up an input_ncs_file that has your specifications in it. NOTE: if an input map_file is provided then if no ncs is found from a model, ncs will be searched for in the density of that map.

• input_ncs_file = None You can enter NCS information in 3 ways: (1) an ncs_spec file produced by AutoSol or AutoBuild with NCS information (2) a heavy-atom PDB file that contains ncs in the heavy-atom sites (3) a PDB file with a model that contains chains with NCS The wizard will derive NCS information from any of these if specified. See also "find_ncs" which determines whether the wizard will update NCS from models that are built during iterative building.

• ncs_copies = None Number of copies of the molecule in the au (note: only one type of molecule allowed at present)

• ncs_refine_coord_sigma_from_rmsd = False You can choose to use the current NCS rmsd as the value of the sigma for NCS restraints. See also ncs_refine_coord_sigma_from_rmsd_ratio

• ncs_refine_coord_sigma_from_rmsd_ratio = 1 You can choose to multiply the current NCS rmsd by this value before using it as the sigma for NCS restraints See also ncs_refine_coord_sigma_from_rmsd

• no_merge_ncs_copies = False Normally False (do merge NCS copies). If True, then do not use each NCS copy to try to build the others.

• optimize_ncs = True This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. Optimize NCS adds a step to try and make the molecule formed by NCS as compact as possible, without losing any point-group symmetry.

• use_ncs_in_build = True Use NCS information in the model assembly stage of model-building. Also if no_merge_ncs_copies is not set, then use each NCS copy to try to build the others.

• ncs_in_refinement = *torsion cartesian None Use torsion_angle refinement of NCS. Alternative is cartesian or None (None will use phenix.refine default)

• omitcomposite_omit_type = *None simple_omit refine_omit sa_omit iterative_build_omit Your choices of types of OMIT maps are: None - normal operation, no omit simple_omit - omit the atoms in OMIT region in calculating a sigmaA-weighted 2mFo-DFc map with no refinement. refine_omit - as simple_omit, but refine with standard refinement. sa_omit - omit the atoms in OMIT region, carry out simulated-annealing refinement, then calculate a sigmaA-weighted 2mFo-DFc map. iterative_build_omit - set occupancy of atoms in OMIT region to 0 throughout an entire iterative model-building, density modification and refinement process (takes a long time). All these omit map types are available as composite omit maps (default) or as omit maps around a region defined by a PDB file (using omit_box_pdb_list) The resulting OMIT map will be in the directory OMIT with file name resolve_composite_map.mtz . This mtz file contains the map coefficients to create the OMIT map. The file "omit_region.mtz" contains the coefficients for a map showing the boundaries of the OMIT region. n_box_target = None You can tell the Wizard how many omit boxes to try and set up (but it will not necessarily choose your number because it has to be nicely divisible into boxes that fit your asymmetric unit). A suitable number is 24. The larger the number of boxes, the better the map will be, but the longer it will take to calculate the map. n_cycle_image_min = 3 Pattern recognition (resolve_pattern) and fragment identification ("image based density modification") are used as part of the density modification process. These are normally only useful in the first few cycles of iterative model-building. This script tries model-building both with and without including image information, and proceeds with the most complete model. Once at least n_cycle_image_min cycles have been carried out with image information, if the image-based map results in a less-complete model than the one without image information, image information is no longer included. n_cycle_rebuild_omit = 10 Model-building is normally carried out using the "best" available map. If omit_on_rebuild is True, then every n_cycle_rebuild_omit cycle of model rebuilding, a composite omit map is used instead. If you specify 0 and omit_on_rebuild is True, omit maps will be used every cycle. Normally every 10th cycle is optimal. offset_boundary = 2. Specify the boundary in A around atoms in omit_box_pdb for definition of omit region. Contrast with omit_boundary which applies for composite omitomit_boundary = 2. Specify the boundary in A around atoms in omit_boxes for definition of omit region. Contrast with offset_boundary which applies for omit_box_pdbomit_box_start = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end omit_box_end = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end omit_box_pdb_list = None This keyword applies if you have set OMIT region specification to "omit_around_pdb". To automatically set an OMIT region specify a PDB file(s) with omit_box_pdb_list. The omit region

boundaries will be the limits in x y z of the atoms in this file, plus a border of offset_boundary. To use only some of the atoms in the file, specify values for starting, ending and chain to omit (omit_res_start_list and omit_res_end_list and omit_chain_list) If you specify more than one file (or if you specify more than one segment of a file with omit_chain_list or omit_res_start_list and omit_res_end_list) then a set of omit runs will be carried out and combined into one composite omit. omit_chain_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use "" to select all chains) The residues from 3 to 4 of chain1 will be omitted. You can specify more than one region by listing them separated by spaces ...

• composite_omit_type = *None simple_omit refine_omit sa_omit iterative_build_omit Your choices of types of OMIT maps are: None - normal operation, no omit simple_omit - omit the atoms in OMIT region in calculating a sigmaA-weighted 2mFo-DFc map with no refinement. refine_omit - as simple_omit, but refine with standard refinement. sa_omit - omit the atoms in OMIT region, carry out simulated-annealing refinement, then calculate a sigmaA-weighted 2mFo-DFc map. iterative_build_omit - set occupancy of atoms in OMIT region to 0 throughout an entire iterative model-building, density modification and refinement process (takes a long time). All these omit map types are available as composite omit maps (default) or as omit maps around a region defined by a PDB file (using omit_box_pdb_list) The resulting OMIT map will be in the directory OMIT with file name resolve_composite_map.mtz . This mtz file contains the map coefficients to create the OMIT map. The file "omit_region.mtz" contains the coefficients for a map showing the boundaries of the OMIT region.

• n_box_target = None You can tell the Wizard how many omit boxes to try and set up (but it will not necessarily choose your number because it has to be nicely divisible into boxes that fit your asymmetric unit). A suitable number is 24. The larger the number of boxes, the better the map will be, but the longer it will take to calculate the map.

• n_cycle_image_min = 3 Pattern recognition (resolve_pattern) and fragment identification ("image based density modification") are used as part of the density modification process. These are normally only useful in the first few cycles of iterative model-building. This script tries model-building both with and without including image information, and proceeds with the most complete model. Once at least n_cycle_image_min cycles have been carried out with image information, if the image-based map results in a less-complete model than the one without image information, image information is no longer included.

• n_cycle_rebuild_omit = 10 Model-building is normally carried out using the "best" available map. If omit_on_rebuild is True, then every n_cycle_rebuild_omit cycle of model rebuilding, a composite omit map is used instead. If you specify 0 and omit_on_rebuild is True, omit maps will be used every cycle. Normally every 10th cycle is optimal.

• offset_boundary = 2. Specify the boundary in A around atoms in omit_box_pdb for definition of omit region. Contrast with omit_boundary which applies for composite omit

• omit_boundary = 2. Specify the boundary in A around atoms in omit_boxes for definition of omit region. Contrast with offset_boundary which applies for omit_box_pdb

• omit_box_start = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end

• omit_box_end = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end

• omit_box_pdb_list = None This keyword applies if you have set OMIT region specification to "omit_around_pdb". To automatically set an OMIT region specify a PDB file(s) with omit_box_pdb_list. The omit region boundaries will be the limits in x y z of the atoms in this file, plus a

border of offset_boundary. To use only some of the atoms in the file, specify values for starting, ending and chain to omit (omit_res_start_list and omit_res_end_list and omit_chain_list) If you specify more than one file (or if you specify more than one segment of a file with omit_chain_list or omit_res_start_list and omit_res_end_list) then a set of omit runs will be carried out and combined into one composite omit.

• omit_chain_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use "" to select all chains) The residues from 3 to 4 of chain1 will be omitted. You can specify more than one region by listing them separated by spaces If you specify more than one region, a separate omit run will be carried out for each one and then the maps will be put together afterwards. If there are more than one chains in the input PDB file then only the chain defined by omit_chain will be omitted NOTE: Zero for start and end and "" for chain is the same as choosing everything

• omit_offset_list = 0 0 0 0 0 0 To carry out one iterative build omit, with a region defined in grid units, enter nxs,nxe,nys,nye,nzs,nze in omit_offset_list.

• omit_on_rebuild = False You can specify whether to use an omit map for building the model on rebuild cycles. Default is True if you start with a model, False if you are building a model from scratch. The omit map is calculated every n_cycle_rebuild_omit cycles

• omit_selection = None Selection string defining atoms in input pdb to be used to define the OMIT region. For use with omit_region_specification=omit_selection

• omit_region_specification = *composite_omit omit_around_pdb omit_selection You can specify what region an omit (simple/sa-omit/iterative-build-omit) map is to be calculated for. Composite omit will create a map over the entire asymmetric unit by dividing the asymmetric unit into overlapping boxes, calculating omit maps for each, and splicing all the results together into a single composite omit map. You can tell the Wizard how many omit boxes to try and set up with the keyword "n_box_target" (but it will not necessarily choose your number because it has to be nicely divisible into boxes that fit your asymmetric unit). Omit around PDB will omit around the region defined by the PDB file(s) you enter for omit_box_pdb (or around the residues in that PDB file that you specify). If you specify omit_around_pdb then you must enter a pdb file to omit around. If you specify omit_selection you must enter a selection string in omit_selection

• omit_res_start_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be omitted. You can specify more more than one region by listing them separated by spaces If you specify more than one region, a separate omit run will be carried out for each one and then the maps will be put together afterwards. If there are more than one chains in the input PDB file then only the chain defined by omit_chain will be rebuilt. NOTE: Zero for start and end and "" for chain is the same as choosing everything

• omit_res_end_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be omitted. You can specify more more than one region by listing them separated by spaces If you specify more than one region, a separate omit run will be carried out for each one and then the maps will be put together afterwards. If there are more than one chains in the input PDB file then only the chain defined by omit_chain will be omitted NOTE: Zero for start and end and "" for chain is the same as choosing everything

• rebuild_in_placemin_seq_identity_percent_rebuild_in_place = 95 Minimum sequence identity to use rebuild_in_place by defaultn_cycle_rebuild_in_place = None Number of cycles for rebuild_in_place for multiple models only n_rebuild_in_place = 1 You can choose how many times to rebuild your model in place with rebuild_in_place rebuild_chain_list = None You can choose to rebuild just a portion of your

model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.rebuild_in_place = *Auto True False You can choose to rebuild your model while fixing the sequence alignment by iteratively rebuilding segments within the model. This is done n_rebuild_in_place times, then the models are recombined, taking the best-fitting parts of each. Crossovers allowed where main-chain atom rmsd is less than dist_close. Note that the sequence of the input model must match the supplied sequence closely enough to allow a clear alignment. Also this method does not build any new chain, it just moves the existing model around. Normally this procedure is useful if the model is greater than 95% identical with the target sequence. You can include information directly from the starting model if you want with the keyword include_input_model. Then this model will be recombined with the models that are built based on it. Note that this requires that the input model have a sequence that is identical to the model to be rebuilt. You can also rebuild just a portion of the model with the keywords keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank) The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines NOTE: if a region cannot be rebuilt the original coordinates will be preserved for that region. rebuild_near_chain = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist. rebuild_near_dist = 7.5 You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist. rebuild_near_res = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist. rebuild_res_end_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.rebuild_res_start_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.rebuild_side_chains = False You can choose to replace side chains (with extend_only) before rebuilding the model (...

• min_seq_identity_percent_rebuild_in_place = 95 Minimum sequence identity to use rebuild_in_place by default

• n_cycle_rebuild_in_place = None Number of cycles for rebuild_in_place for multiple models only

• n_rebuild_in_place = 1 You can choose how many times to rebuild your model in place with rebuild_in_place

• rebuild_chain_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.

• rebuild_in_place = *Auto True False You can choose to rebuild your model while fixing the sequence alignment by iteratively rebuilding segments within the model. This is done n_rebuild_in_place times, then the models are recombined, taking the best-fitting parts of each. Crossovers allowed where main-chain atom rmsd is less than dist_close. Note that the sequence of the input model must match the supplied sequence closely enough to allow a clear alignment. Also this method does not build any new chain, it just moves the existing model around. Normally this procedure is useful if the model is greater than 95% identical with the target sequence. You can include information directly from the starting model if you want with the keyword include_input_model. Then this model will be recombined with the models that are built based on it. Note that this requires that the input model have a sequence that is identical to the model to be rebuilt. You can also rebuild just a portion of the model with the keywords keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank) The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines NOTE: if a region cannot be rebuilt the original coordinates will be preserved for that region.

• rebuild_near_chain = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist.

• rebuild_near_dist = 7.5 You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist.

• rebuild_near_res = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist.

• rebuild_res_end_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.

• rebuild_res_start_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.

• rebuild_side_chains = False You can choose to replace side chains (with extend_only) before rebuilding the model (not normally used)

• redo_side_chains = True You can chooses to have AutoBuild choose whether to replace all your side chains in rebuild_in_place, taking new ones if they fit the density better. If True, this is applied to all side chains, not only those that are rebuilt.

• replace_existing = True In rebuild_in_place the usual default is to force the replacement of all residues, even if the rebuilt ones are not as good a fit as the original. The rebuilt model is then crossed with the original model (if include_input_model=True) and the better parts of each are then kept. You can override the replacement of all residues in the initial model-building by saying "False" (do not force replacement of residues, keep whatever is better). Additionally if you set the "touch_up" flag then the default is "True": keep whatever is better.

• delete_bad_residues_only = False You can simply delete the worst parts of your model and write out the resulting model with delete_bad_residues_only=True The criteria used are the ones set with touch_up. Any residues that would be rebuild by touch_up=True will be deleted by delete_bad_residues_only. NOTE: delete_bad_residues_only ignores ligands, waters etc. so you may need to put them back in afterwards.

• touch_up = False You can rebuild just the worst parts of your model by setting touch_up=True. You can decide what parts to rebuild based on an minimum model-map correlation (by residue). This is set with min_cc_residue_rebuild=0.82 Alternatively you can rebuild the worst percentage of these: worst_percent_res_rebuild=6. If a value is set for both of these then residues qualifying in either way are rebuilt. NOTE: touch_up is only available with rebuild_in_place.

• touch_up_extra_residues = None Number of residues on each side of the residues identified in touch_up that you want to rebuild. Normally you will want to rebuild one or more on each side.

• worst_percent_res_rebuild = 2 You can rebuild just the worst parts of your model by setting touch_up=True. You can decide how much to rebuild using worst_percent_res_rebuild or with min_cc_res_rebuild, or both.

• smooth_range = None You can specify what number of residues to smooth in making choices for touch_up and delete_bad_residues_only Typically use 3 or 5.

• smooth_minimum_length = None If specified, then any segments remaining after smoothing that are shorter than smooth_mininum_length will be removed.

• refinementrefine_b = True You can choose whether phenix.refine is to refine individual atomic displacement parameters (B values) refine_se_occ = True You can choose to refine the occupancy of SE atoms in a SEMET structure (default=True). This only applies if semet=true skip_clash_guard = True Skip refinement check for atoms that clash Also skips check for side chains crossing special position (allow_polymer_cross_special_position=True in refinement)correct_special_position_tolerance = None Adjust tolerance for special position check. If 0., then check for clashes near special positions is not carried out. This sometimes allows phenix.refine to continue even if an atom is near a special position. If 1., then checks within 1 A of special positions. If None, then uses phenix.refine default. (1)use_mlhl = True This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to only refine on Fobs generate_hl_if_missing = False This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to not generate HL coeffs from input phases.place_waters = True You can choose whether phenix.refine automatically places ordered solvent (waters) during the refinement process. refinement_resolution = 0 Enter the high-resolution limit for refinement only. This high-resolution limit can be different than the high-resolution limit for other steps. The default ("None" or 0.0) is to use the overall high-resolution limit for this run (as set by resolution) ordered_solvent_low_resolution = None You can choose what resolution cutoff to use fo placing ordered solvent in phenix.refine. If the resolution of refinement is greater than this cutoff, then no ordered solvent will be placed, even if refinement.main.ordered_solvent=True. link_distance_cutoff = 3 You can specify the maximum bond distance for linking residues in phenix.refine called from the wizards. r_free_flags_fraction = 0.1 Maximum fraction of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters. r_free_flags_max_free = 2000 Maximum number of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters. r_free_flags_use_lattice_symmetry = True When generating r_free_flags you can decide whether to include lattice symmetry (good in general, necessary if there is twinning).

r_free_flags_lattice_symmetry_max_delta = 5 You can set the maximum deviation of distances in the lattice that are to be considered the same for purposes of generating a lattice-symmetry-unique set of free R flags. allow_overlapping = None Default is None (set automatically, normally False unless S or Se atoms are the anomalously-scattering atoms). You can allow atoms in your ligand files to overlap atoms in your protein/nucleic acid model. This overrides 'keep_pdb_atoms' Useful in early stages of model-building and refinement The ligand atoms get the altloc indicator 'L' NOTE: The ligand occupancy will be refined by default if you set allow_overlapping=True (because of the altloc indicator) You can turn this off with fix_ligand_occupancy=Truefix_ligand_occupancy = None If allow_overlapping=True then ligand occupancies are refined as a group. You can turn this off with fix_ligand_occupancy=true NOTE: has no effect if allow_overlapping=Falseremove_outlier_segments = True You can remove any segments that are not assigned to sequence if their mean B values are more than remove_outlier_segments_z_cut sd higher than the mean for the structure. NOTE: this is done after refinement, so the R/Rfree are no longer applicable; the remarks...

• refine_b = True You can choose whether phenix.refine is to refine individual atomic displacement parameters (B values)

• refine_se_occ = True You can choose to refine the occupancy of SE atoms in a SEMET structure (default=True). This only applies if semet=true

• skip_clash_guard = True Skip refinement check for atoms that clash Also skips check for side chains crossing special position (allow_polymer_cross_special_position=True in refinement)

• correct_special_position_tolerance = None Adjust tolerance for special position check. If 0., then check for clashes near special positions is not carried out. This sometimes allows phenix.refine to continue even if an atom is near a special position. If 1., then checks within 1 A of special positions. If None, then uses phenix.refine default. (1)

• use_mlhl = True This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to only refine on Fobs

• generate_hl_if_missing = False This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to not generate HL coeffs from input phases.

• place_waters = True You can choose whether phenix.refine automatically places ordered solvent (waters) during the refinement process.

• refinement_resolution = 0 Enter the high-resolution limit for refinement only. This high-resolution limit can be different than the high-resolution limit for other steps. The default ("None" or 0.0) is to use the overall high-resolution limit for this run (as set by resolution)

• ordered_solvent_low_resolution = None You can choose what resolution cutoff to use fo placing ordered solvent in phenix.refine. If the resolution of refinement is greater than this cutoff, then no ordered solvent will be placed, even if refinement.main.ordered_solvent=True.

• link_distance_cutoff = 3 You can specify the maximum bond distance for linking residues in phenix.refine called from the wizards.

• r_free_flags_fraction = 0.1 Maximum fraction of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters.

• r_free_flags_max_free = 2000 Maximum number of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters.

• r_free_flags_use_lattice_symmetry = True When generating r_free_flags you can decide whether to include lattice symmetry (good in general, necessary if there is twinning).

• r_free_flags_lattice_symmetry_max_delta = 5 You can set the maximum deviation of distances in the lattice that are to be considered the same for purposes of generating a lattice-symmetry-unique set of free R flags.

• allow_overlapping = None Default is None (set automatically, normally False unless S or Se atoms are the anomalously-scattering atoms). You can allow atoms in your ligand files to overlap atoms in your protein/nucleic acid model. This overrides 'keep_pdb_atoms' Useful in early stages of model-building and refinement The ligand atoms get the altloc indicator 'L' NOTE: The ligand occupancy will be refined by default if you set allow_overlapping=True (because of the altloc indicator) You can turn this off with fix_ligand_occupancy=True

• fix_ligand_occupancy = None If allow_overlapping=True then ligand occupancies are refined as a group. You can turn this off with fix_ligand_occupancy=true NOTE: has no effect if allow_overlapping=False

• remove_outlier_segments = True You can remove any segments that are not assigned to sequence if their mean B values are more than remove_outlier_segments_z_cut sd higher than the mean for the structure. NOTE: this is done after refinement, so the R/Rfree are no longer applicable; the remarks in the PDB file are removed

• twin_law = None You can specify a twin law for refinement like this: twin_law='-h,k,-l'

• max_occ = None You can choose to set the maximum value of occupancy for atoms that have their occupancies refined. Default is None (use default value of 1.0 from phenix.refine)

• refine_before_rebuild = True You can choose to refine the input model before rebuilding it

• refine_with_ncs = True This script can allow phenix.refine to automatically identify NCS and use it in refinement. NOTE: ncs refinement and placing waters automatically are mutually exclusive at present.

• refine_xyz = True You can choose whether phenix.refine is to refine coordinates

• s_annealing = False You can choose to carry out simulated annealing during the first refinement after initial model-building

• skip_hexdigest = False You may wish to ignore the hexdigest of the free R flags in your input PDB file if (1) the dataset you provide is not identical to the one that you refined with (but has the same free R flags), or (2) you are providing both an input_data_file and an input_refinement_file or input_hires_file and. In the second case, the resulting composite file may not have the same hexdigest even though the free R flags are copied over. The default is to set skip_hexdigest=True for case #2. For case #1 you have to tell the Wizard to skip the hexdigest (because it cannot know about this).

• use_hl_anom_in_refinement = False See use_hl_anom_in_denmod. If use_hl_anom_in_refinement=True then the HLanom HL coefficients from Phaser are used in refinement

• use_hl_if_present = True You can turn off use of HL coeffs in refinement if you want. Default is to use them if present. Requires PHIB as well.

• thoroughnessbuild_outside = True Define whether to use the BuildOutside module in model_buildingconnect = True Define whether to use the connect module in model_building. This module tries to connect nearby chains with loops, without using the sequence. This is different than fit_loops (which uses the sequence to identify the exact number of residues in the loop). extensive_build = False You can choose whether to build a new model on every cycle and carry out extra model-building steps every cycle. Default is False (build a new model on first cycle, after that carry out extra steps). fit_loops = True You can fit loops automatically if sequence alignment has been done. insert_helices = True Define

whether to use the insert_helices module in model_building. This module tries to insert helices identified with find_helices_strands into the current working model. This can be useful as the standard build sometimes builds strands into helical density at low resolution. n_cycle_build = None Choose number of cycles of building and chain extension during each cycle of model-building. (default of 1). n_cycle_build_max = 6 Maximum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is not found, a maximum of n_cycle_build_max cycles will be carried out. n_cycle_build_min = 1 Minimum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is found, n_cycle_build_min cycles will be carried out. n_cycle_rebuild_max = 15 Maximum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is not found, a maximum of n_cycle_rebuild_max cycles will be carried out. n_cycle_rebuild_min = 1 Mininum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is found, n_cycle_rebuild_min cycles will be carried out. n_mini = 10 You can choose how many times to retrace your model in "retrace_before_build" n_random_frag = 0 In resolve building you can randomize each fragment slightly so as to generate more possibilities for tracing based on extending it. n_random_loop = 3 Number of randomized tries from each end for building loops If 0, then one try. If N, then N additional tries with randomization based on rms_random_loop. n_try_rebuild = 2 Number of attempts to build each segment of chain ncycle_refine = 3 Choose number of refinement cycles (3) number_of_models = None This parameter lets you choose how many initial models to build with RESOLVE within a single build cycle. This parameter is now superseded by number_of_parallel_models, which sets the number of models (but now entire build cycles) to carry out in parallel. None or zero means set it automatically. That is what you normally should use. The number_of_models is by default set to 1 and number_of_parallel_models is set to the value of nbatch (typically 4). number_of_parallel_models = 0 This parameter lets you choose how many models to build in parallel. None or 0 means set it automatically. That is what you normally should use. You can set this to 1 to prevent the wizard from running multiple jobs in parallelskip_combine_extend = False You can choose whether to skip the combine-extend step in model-building if only one model is availablefully_skip_combine_extend = False You can choose whether to skip the combine-extend step in model-building in all casesthorough_loop_fit = True Try many conformations and accept them even if the fit is not perfect? If you say True the parameters for thorough loop fitting are: n_random_loop=100 rms_random_loop=0.3 rho_min_main=0.5 while if you say False those for quick loop fitting are: n_random_loop=20 rms_random_loop=0.3 rho_min_main=1.0

• build_outside = True Define whether to use the BuildOutside module in model_building

• connect = True Define whether to use the connect module in model_building. This module tries to connect nearby chains with loops, without using the sequence. This is different than fit_loops (which uses the sequence to identify the exact number of residues in the loop).

• extensive_build = False You can choose whether to build a new model on every cycle and carry out extra model-building steps every cycle. Default is False (build a new model on first cycle, after that carry out extra steps).

• fit_loops = True You can fit loops automatically if sequence alignment has been done.

• insert_helices = True Define whether to use the insert_helices module in model_building. This module tries to insert helices identified with find_helices_strands into the current working model. This can be useful as the standard build sometimes builds strands into helical density at low resolution.

• n_cycle_build = None Choose number of cycles of building and chain extension during each cycle of model-building. (default of 1).

• n_cycle_build_max = 6 Maximum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is not found, a maximum of n_cycle_build_max cycles will be carried out.

• n_cycle_build_min = 1 Minimum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is found, n_cycle_build_min cycles will be carried out.

• n_cycle_rebuild_max = 15 Maximum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is not found, a maximum of n_cycle_rebuild_max cycles will be carried out.

• n_cycle_rebuild_min = 1 Mininum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is found, n_cycle_rebuild_min cycles will be carried out.

• n_mini = 10 You can choose how many times to retrace your model in "retrace_before_build"

• n_random_frag = 0 In resolve building you can randomize each fragment slightly so as to generate more possibilities for tracing based on extending it.

• n_random_loop = 3 Number of randomized tries from each end for building loops If 0, then one try. If N, then N additional tries with randomization based on rms_random_loop.

• n_try_rebuild = 2 Number of attempts to build each segment of chain

• ncycle_refine = 3 Choose number of refinement cycles (3)

• number_of_models = None This parameter lets you choose how many initial models to build with RESOLVE within a single build cycle. This parameter is now superseded by number_of_parallel_models, which sets the number of models (but now entire build cycles) to carry out in parallel. None or zero means set it automatically. That is what you normally should use. The number_of_models is by default set to 1 and number_of_parallel_models is set to the value of nbatch (typically 4).

• number_of_parallel_models = 0 This parameter lets you choose how many models to build in parallel. None or 0 means set it automatically. That is what you normally should use. You can set this to 1 to prevent the wizard from running multiple jobs in parallel

• skip_combine_extend = False You can choose whether to skip the combine-extend step in model-building if only one model is available

• fully_skip_combine_extend = False You can choose whether to skip the combine-extend step in model-building in all cases

• thorough_loop_fit = True Try many conformations and accept them even if the fit is not perfect? If you say True the parameters for thorough loop fitting are: n_random_loop=100 rms_random_loop=0.3 rho_min_main=0.5 while if you say False those for quick loop fitting are: n_random_loop=20 rms_random_loop=0.3 rho_min_main=1.0

• generalcoot_name = "coot" If your version of coot is called something else, then you can specify that here. i_ran_seed = 72432 Random seed (positive integer) for model-building and simulated annealing refinement raise_sorry = False You can have any failure end with a Sorry instead of simply printout to the screenbackground = True When you specify nproc=nn, you can run the jobs in background (default if nproc is greater than 1) or foreground (default if nproc=1). If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If you use run_command='sh ' (or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously. check_wait_time = 1.0 You can specify the

length of time (seconds) to wait between checking for subprocesses to endmax_wait_time = 1.0 You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....cache_resolve_libs = True Use caching of resolve libraries to speed up resolveresolve_size = 12 Size for solve/resolve ("","_giant", "_huge","_extra_huge" or a number where 12=giant 18=hugecheck_run_command = False You can have the wizard check your run command at startuprun_command = "sh " When you specify nproc=nn, you can run the subprocesses as jobs in background with sh (default) or submit them to a queue with the command of your choice (i.e., qsub). If you have a multi-processor machine, use sh. If you have a cluster, use qsub or the equivalent command for your system. NOTE: If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If nproc is greater than 1 and you use run_command='sh '(or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously. queue_commands = None You can add any commands that need to be run for your queueing system. These are written before any other commands in the file that is submitted to your queueing system. For example on a PBS system you might say: queue_commands='#PBS -N mr_rosetta' queue_commands='#PBS -j oe' queue_commands='#PBS -l walltime=03:00:00' queue_commands='#PBS -l nodes=1:ppn=1' NOTE: you can put in the characters '<path>' in any queue_commands line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_',up to 15 characters. For example 'test_autobuild/WORK_5/AutoBuild_run_1_/TEMP0/RUN_1' would be represented by: 'tld_W_5_A1__TP0_1' condor_universe = vanilla The universe for condor is usually vanilla. However you might need to set it to local for your clusteradd_double_quotes_in_condor = True You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.condor = None Specifies if the group_run_command is submitting a job to a condor cluster. Set by default to True if group_run_command=condor_submit, otherwise False. For condor job submission mr_rosetta uses a customized script wit...

• coot_name = "coot" If your version of coot is called something else, then you can specify that here.

• i_ran_seed = 72432 Random seed (positive integer) for model-building and simulated annealing refinement

• raise_sorry = False You can have any failure end with a Sorry instead of simply printout to the screen

• background = True When you specify nproc=nn, you can run the jobs in background (default if nproc is greater than 1) or foreground (default if nproc=1). If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If you use run_command='sh ' (or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously.

• check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end

• max_wait_time = 1.0 You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too

short a wait time is 'File not found'

• wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....

• cache_resolve_libs = True Use caching of resolve libraries to speed up resolve

• resolve_size = 12 Size for solve/resolve ("","_giant", "_huge","_extra_huge" or a number where 12=giant 18=huge

• check_run_command = False You can have the wizard check your run command at startup

• run_command = "sh " When you specify nproc=nn, you can run the subprocesses as jobs in background with sh (default) or submit them to a queue with the command of your choice (i.e., qsub). If you have a multi-processor machine, use sh. If you have a cluster, use qsub or the equivalent command for your system. NOTE: If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If nproc is greater than 1 and you use run_command='sh '(or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously.

• queue_commands = None You can add any commands that need to be run for your queueing system. These are written before any other commands in the file that is submitted to your queueing system. For example on a PBS system you might say: queue_commands='#PBS -N mr_rosetta' queue_commands='#PBS -j oe' queue_commands='#PBS -l walltime=03:00:00' queue_commands='#PBS -l nodes=1:ppn=1' NOTE: you can put in the characters '<path>' in any queue_commands line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_',up to 15 characters. For example 'test_autobuild/WORK_5/AutoBuild_run_1_/TEMP0/RUN_1' would be represented by: 'tld_W_5_A1__TP0_1'

• condor_universe = vanilla The universe for condor is usually vanilla. However you might need to set it to local for your cluster

• add_double_quotes_in_condor = True You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.

• condor = None Specifies if the group_run_command is submitting a job to a condor cluster. Set by default to True if group_run_command=condor_submit, otherwise False. For condor job submission mr_rosetta uses a customized script with condor commands. Also uses one_subprocess_level=True

• last_process_is_local = True If true, run the last process in a group in background with sh as part of the job that is submitting jobs. This prevents having the job that is submitting jobs sit and wait for all the others while doing nothing

• skip_r_factor = False You can skip R-factor calculation if refinement is not done and maps_only=True

• test_flag_value = Auto Normally leave this at Auto (default). This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.

• skip_xtriage = False You can bypass xtriage if you want. This will prevent you from applying anisotropy corrections, however.

• base_path = None You can specify the base path for files (default is current working directory)

• temp_dir = None Define a temporary directory

• local_temp_directory = None Write all temporary files to this directory and copy files specified by keep_files back to TEMP directory in working directory at the end of run. Used to speed up read/write operations. NOTE: Deletes all files except those specified by keep_files just like clean_up=True.

• clean_up = None At the end of the entire run the TEMP directories will be removed if clean_up is True. Also all files except for overall_best_xxx (final) files and .eff files In the AutoSol_run_xx_ AutoBuild_run_xx_ etc directory will be removed Files listed in keep_files will not be deleted. If you want to remove files after your run is finished use a command like "phenix.autobuild run=1 clean_up=True"

• print_citations = True Print citations at end of run

• solution_output_pickle_file = None At end of run, write solutions to this file in output directory if defined

• job_title = None Job title in PHENIX GUI, not used on command line

• top_output_dir = None This is used in subprocess calls of wizards and to tell the Wizard where to look for the STOPWIZARD file.

• wizard_directory_number = None This is used by the GUI to define the run number for Wizards. It is the same as desired_run_number NOTE: this value can only be specified on the command line, as the directory number is set before parameters files are read.

• verbose = False Command files and other verbose output will be printed

• extra_verbose = False Facts and possible commands will be printed every cycle if True

• debug = False You can have the wizard stop with error messages about the code if you use debug. Additionally the output goes to the terminal if you specify "debug=True"

• require_nonzero = True Require non-zero values in data columns to consider reading in.

• remove_path_word_list = None List of words identifying paths to remove from PATH These can be used to shorten your PATH. For example... cns ccp4 coot would remove all paths containing these words except those also containing phenix. Capitalization is ignored.

• fill = False Fill in all missing reflections to resolution res_fill. Applies to density modified maps. See also filled_2fofc_maps in autobuild.

• res_fill = None Resolution for filling in missing data (default = highest resolution of any datafile). Only applies to density modified maps. Default is fill to high resolution of data. Ignored if fill=False

• check_only = False Just read in and check initial parameters. Not for general use

• keep_files = overall_best* AutoBuild_run_*.log List of files that are not to be cleaned up. wildcards permitted

• after_autosol = False You can specify that you want to continue on starting with the highest-scoring run of AutoSol in your working directory.

• nbatch = 3 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch.

• nproc = 1 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch. If you set nproc=Auto and your machine has n processors, then it will use n-1 processors, or 1 if only 1 is available

• quick = False Run everything quickly (number_of_parallel_models=1 n_cycle_build_max=1 n_cycle_rebuild_max=1)

• resolve_command_list = None Commands for resolve. One per line in the form: keyword value value can be optional Examples: coarse_grid resolution 200 2.0 hklin test.mtz NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "

• resolve_pattern_command_list = None Commands for resolve_pattern. One per line in the form: keyword value value can be optional Examples: resolution 200 2.0 hklin test.mtz NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_pattern_command_list="'resolution 200 20' 'hklin test.mtz' "

• ignore_errors_in_subprocess = False Try to ignore errors in sub-processes This is useful in cases where a very rare crash occurs and you want to just ignore that step and go on.

• send_notification = False

• notify_email = None

• special_keywordswrite_run_directory_to_file = None Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.

• write_run_directory_to_file = None Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.

• run_controlcoot = None Set coot to True and optionally run=[run-number] to run Coot with the current model and map for run run-number. In some wizards (AutoBuild) you can edit the model and give it back to PHENIX to use as part of the model-building process. If you just say coot then the facts for the highest-numbered existing run will be shown. ignore_blanks = None ignore_blanks allows you to have a command-line keyword with a blank value like "input_lig_file_list=" stop = None You can stop the current wizard with "stopwizard" or "stop". If you type "phenix.autobuild run=3 stop" then this will stop run 3 of autobuild. display_facts = None Set display_facts to True and optionally run=[run-number] to display the facts for run run-number. If you just say display_facts then the facts for the highest-numbered existing run will be shown. display_summary = None Set display_summary to True and optionally run=[run-number] to show the summary for run run-number. If you just say display_summary then the summary for the highest-numbered existing run will be shown. carry_on = None Set carry_on to True to carry on with highest-numbered run from where you left off. run = None Set run to n to continue with run n where you left off. copy_run = None Set copy_run to n to copy run n to a new run and continue where you left off. display_runs = None List all runs for this wizard. delete_runs = None List runs to delete: 1 2 3-5 9:12 display_labels = None display_labels=test.mtz will list all the labels that identify data in test.mtz. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: peak.data="F+ SIGF+ F- SIGF-". The entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol or AutoBuild like this: input_refinement_labels="FP SIGFP

FreeR_flags" # each individual label counts dry_run = False Just read in and check parameter namesparams_only = False Just read in and return parameter defaults. Not for general usedisplay_all = False Just read in and display parameter defaults

• coot = None Set coot to True and optionally run=[run-number] to run Coot with the current model and map for run run-number. In some wizards (AutoBuild) you can edit the model and give it back to PHENIX to use as part of the model-building process. If you just say coot then the facts for the highest-numbered existing run will be shown.

• ignore_blanks = None ignore_blanks allows you to have a command-line keyword with a blank value like "input_lig_file_list="

• stop = None You can stop the current wizard with "stopwizard" or "stop". If you type "phenix.autobuild run=3 stop" then this will stop run 3 of autobuild.

• display_facts = None Set display_facts to True and optionally run=[run-number] to display the facts for run run-number. If you just say display_facts then the facts for the highest-numbered existing run will be shown.

• display_summary = None Set display_summary to True and optionally run=[run-number] to show the summary for run run-number. If you just say display_summary then the summary for the highest-numbered existing run will be shown.

• carry_on = None Set carry_on to True to carry on with highest-numbered run from where you left off.

• run = None Set run to n to continue with run n where you left off.

• copy_run = None Set copy_run to n to copy run n to a new run and continue where you left off.

• display_runs = None List all runs for this wizard.

• delete_runs = None List runs to delete: 1 2 3-5 9:12

• display_labels = None display_labels=test.mtz will list all the labels that identify data in test.mtz. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: peak.data="F+ SIGF+ F- SIGF-". The entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol or AutoBuild like this: input_refinement_labels="FP SIGFP FreeR_flags" # each individual label counts

• dry_run = False Just read in and check parameter names

• params_only = False Just read in and return parameter defaults. Not for general use

• display_all = False Just read in and display parameter defaults

• non_user_parameters These are obsolete parameters and parameters that the wizards use to communicate among themselves. Not normally for general use.gui_output_dir = None Used only by the GUIbackground_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in an omit map that are not part of any omitted region. (Default="") boundary_background_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in the boundary map that are not part of any omitted region. (Default="") extend_try_list = True You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with nbatch. force_combine_extend = False You can choose whether to force the combine-extend step in model-building model_list = None This keyword lets you name any number of PDB files to consider as starting models for model-building. NOTE: This differs from consider_main_chain_list which will try to add your PDB files EVERY cycle of merging models. In contrast model_list will only do it on the first cycle. NOTE: this only uses the main-chain atoms of your PDB files. oasis_cnos = None Enter number of C N O and S atoms here if you have OASIS and want to

run it before resolve density modification like this: "C 250 N 121 O 85 S 3" offset_boundary_background_map = None You can set the offset of the boundary_background_map. skip_refine = False Skip refinement (used in get_connections/assign_sequence)sg = None Obsolete. Use space_group insteadinput_data_file = None Not normally used (same as "data=").input_map_file = Auto Not normally used. (Same as map_file).input_refinement_file = Auto Not normally used. Same as refinement_fileinput_pdb_file = None Not normally used. Same as "model="input_seq_file = Auto Not normally used. Same as seq_filesuper_quick = None Shortcut for very quick run of autobuild. Same as : number_of_parallel_models=1 refine=false n_cycle_rebuild_max=1 remove_aniso=False skip_xtriage=True ncs_copies=1 find_ncs=false fully_skip_combine_extend=True fit_loops=False redo_side_chains=False insert_helices=False build_outside=False connect=Falserequire_test_set = False Require that input data file have a test set

• gui_output_dir = None Used only by the GUI

• background_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in an omit map that are not part of any omitted region. (Default="")

• boundary_background_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in the boundary map that are not part of any omitted region. (Default="")

• extend_try_list = True You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with nbatch.

• force_combine_extend = False You can choose whether to force the combine-extend step in model-building

• model_list = None This keyword lets you name any number of PDB files to consider as starting models for model-building. NOTE: This differs from consider_main_chain_list which will try to add your PDB files EVERY cycle of merging models. In contrast model_list will only do it on the first cycle. NOTE: this only uses the main-chain atoms of your PDB files.

• oasis_cnos = None Enter number of C N O and S atoms here if you have OASIS and want to run it before resolve density modification like this: "C 250 N 121 O 85 S 3"

• offset_boundary_background_map = None You can set the offset of the boundary_background_map.

• skip_refine = False Skip refinement (used in get_connections/assign_sequence)

• sg = None Obsolete. Use space_group instead

• input_data_file = None Not normally used (same as "data=").

• input_map_file = Auto Not normally used. (Same as map_file).

• input_refinement_file = Auto Not normally used. Same as refinement_file

• input_pdb_file = None Not normally used. Same as "model="

• input_seq_file = Auto Not normally used. Same as seq_file

• super_quick = None Shortcut for very quick run of autobuild. Same as : number_of_parallel_models=1 refine=false n_cycle_rebuild_max=1 remove_aniso=False skip_xtriage=True ncs_copies=1 find_ncs=false fully_skip_combine_extend=True fit_loops=False redo_side_chains=False insert_helices=False build_outside=False connect=False

• require_test_set = False Require that input data file have a test set

Model building (and more) in the AutoBuild GUI

autobuild_gui.html

Contents

Overview

NOTE: this document is mainly an introduction for users who are not familiar with AutoBuild (or the GUI), rather than a comprehensive reference for the program. The documentation for the command-line version covers the full range of options and functionality.

AutoBuild is the main model-building program in PHENIX; it combines density modification and chain tracing in RESOLVE with phenix.refine to generate a high-quality model. It is optimized to be very thorough by default, and as a result is one of the most processor-intensive programs in PHENIX. Most common protocols can be parallelized over an arbitary number of processors or cluster nodes, however. If you need a faster alternative, there are several simpler building programs in PHENIX:

phenix.find_helices_strands performs very fast secondary structure fitting and chain-tracing. It will generate a partial protein (or nucleic acid) backbone in a few minutes (depending on processor speed) but performs no refinement or sequence docking. This method tends to work well for low-resolution maps, and AutoBuild has a similar mode. phenix.fit_loops docks small protein loops missing from an incomplete model into electron density. It can perform simple real-space refinement to clean up the structure. phenix.phase_and_build is essentially the model-building component of AutoSol, and like AutoBuild, it performs iterative density modification, model-building, and refinement. It is still substantially faster than AutoBuild, usually at the cost of a few percent higher R-factors. Currently only available as a command-line program.

- phenix.find_helices_strands performs very fast secondary structure fitting and chain-tracing. It will generate a partial protein (or nucleic acid) backbone in a few minutes (depending on processor speed) but performs no refinement or sequence docking. This method tends to work well for low-resolution maps, and AutoBuild has a similar mode.
- phenix.fit_loops docks small protein loops missing from an incomplete model into electron density. It can perform simple real-space refinement to clean up the structure.
- phenix.phase_and_build is essentially the model-building component of AutoSol, and like AutoBuild, it performs iterative density modification, model-building, and refinement. It is still substantially faster than AutoBuild, usually at the cost of a few percent higher R-factors. Currently only available as a command-line program.

Several other programs have features similar to functions in AutoBuild; in particular, phenix.refine includes simple local rebuilding for sidechain rotamers. Ligand placement is done separately in the LigandFit wizard.

Video Tutorial

A tutorial video is available on the Phenix YouTube channel and covers the following topics:

- basic overview
- how to run AutoBuild via the GUI
- how to interpret the output

Configuration

Like several other programs in PHENIX, the AutoBuild GUI can accept a variety of files in bulk; it is also designed to be started directly from other programs (e.g. the AutoSol and AutoMR wizards) with files pre-loaded. At a minimum, it requires experimental amplitudes and a source of phases (PHI/FOM columns in the MTZ file, or a partial model), and works best when supplied with the sequence(s) to be built. If Hendrickson-Lattman coefficients or R-free flags are present in the reflections file(s), these will be used as well. Additional files for starting map coefficients and high-resolution data are optional. (An example of the latter would be building into an isomorphous native dataset after solving a SeMet SAD/MAD structure in AutoSol.)

AutoBuild is limited to building one chain type at a time (protein, DNA, or RNA), and performs no ligand fitting (the LigandFit wizard should be used for this). However, additional chains may be included as a "ligand file", which contains any existing parts of the model that you want AutoBuild to leave untouched (it will still be used in refinement). For non-standard ligands, you will need to include CIF files. Custom settings for phenix.refine (i.e. the ".eff file") may also be supplied.

If you are running AutoBuild directly from experimental phasing, the PDB file containing heavy atoms should be included. For selenomethionine-derivatized proteins, the heavy atom sites will be used to anchor the methionines in the sequence. AutoBuild can also be directed to build actual SeMet residues in place of Met.

AutoBuild has two different modes for using an PDB file supplied as a starting model, set by the "Rebuild in place" control. By default, if the sequence identity versus the supplied sequence file is at least 50%, the model will be rebuilt as necessary and refined without adding or removing atoms ("Auto" or "True"). If False (or if sequence identity is low), the model will be used as a source of phasing information but a new model will be built into the density-modified map. You can still incorporate parts of the initial model by checking the "Include input model" box. Currently, if you want to extend an existing model without removing any of the input atoms, we recommend using phenix.fit_loops instead.

General options:

Twin law determines whether twinned refinement should be performed. If you have a noticeably twinned structure this is essential to obtain interpretable maps. Include input model is only relevant when rebuild-in-place mode is not used; if true, the input model will be combined with the autobuilt model using the best segments of each. Build outside model will only add new residues outside of the existing model. Refine model during building runs phenix.refine between each cycle of model-building; this adds to the runtime but is strongly recommended as it will always lead to a better final model. Build helices and strands only will only place secondary structure elements. This is most useful at lower resolutions. Build SeMet residues places SeMet instead of Met residues (as dictated by the sequence file). Place waters in refinement uses the ordered_solvent option in phenix.refine; this is highly recommended at medium-to-high resolution (better than

3.0A). Morph input model into density tries to move the model to match the density-modified map before rebuilding and refining. This is often an effective method to improve poor MR solutions. Refine input model before rebuilding will run phenix.refine first; this should be unchecked if you have just come from refinement. Use simulated annealing turns on the Cartesian-space simulated annealing during refinement. This adds significantly to runtime, but can often improve convergence and fix building mistakes.

- Twin law determines whether twinned refinement should be performed. If you have a noticeably twinned structure this is essential to obtain interpretable maps.
- Include input model is only relevant when rebuild-in-place mode is not used; if true, the input model will be combined with the autobuilt model using the best segments of each.
- Build outside model will only add new residues outside of the existing model.
- Refine model during building runs phenix.refine between each cycle of model-building; this adds to the runtime but is strongly recommended as it will always lead to a better final model.
- Build helices and strands only will only place secondary structure elements. This is most useful at lower resolutions.
- Build SeMet residues places SeMet instead of Met residues (as dictated by the sequence file).
- Place waters in refinement uses the ordered_solvent option in phenix.refine; this is highly recommended at medium-to-high resolution (better than 3.0A).
- Morph input model into density tries to move the model to match the density-modified map before rebuilding and refining. This is often an effective method to improve poor MR solutions.
- Refine input model before rebuilding will run phenix.refine first; this should be unchecked if you have just come from refinement.
- Use simulated annealing turns on the Cartesian-space simulated annealing during refinement. This adds significantly to runtime, but can often improve convergence and fix building mistakes.

Multiprocessing and queueing systems

To build as thorough a model as possible, AutoBuild will spawn many similar processes, each of which will produce a slightly different result (depending on build options and random number generation). This can be disabled by checking the "Quick mode" box, but the parallel builds typically result in more residues placed and a few percent lower R-free. As many processors as are available may be utilized, although for building only three or four processes will be run at a time. The GUI will automatically set the maximum number of processors to one less than are present in the system (or just one for older, single-core machines).

This type of parallelism is very well suited for execution across a cluster, and AutoBuild supports spawning of processes using a queuing system such as Sun Grid Engine (SGE). To enable this method, open the Preferences, switch to the "Processes" section, and click the box labeled "Activate queuing system support", then re-launch the AutoBuild GUI. Another control will now appear labeled "Use queuing system to distribute tasks". If this is checked, all child processes will be submitted to the queue instead of being run locally. Note that this is independent of the option to run the entire AutoBuild process on the cluster, which only appears when the "Run" button is clicked. Both options may be combined, but only if the cluster setup allows submitting jobs from the individual nodes.

For composite omit map generation (described below), the parallelism is even more effective, since each omitted box can be processed separately.

Output

As soon as AutoBuild starts running, a new tab will appear for the current job. This will be divided into sub-tabs, starting with job status and console output. A second sub-tab will display a list of output files and additional results as they appear; these will include the Xtriage report on the experimental data, which can be loaded into the GUI by clicking the "Results and graphs" button. Once maps and a model are available, clicking the Coot or PyMOL buttons will load these into the desired viewer.

The map file will be named overall_best_denmod_map_coeffs.mtz, which contains only an mFo map (FOM-weighted F(obs), and possibly anisotropy- corrected). Users who want difference maps like those produced by phenix.refine should run the separate phenix.maps GUI. AutoBuild will output two PDB files, one containing all built residues (overall_best.pdb), and one with only those residues for which the known sequence could be docked (overall_best_placed.pdb). The final tab, "Structure status", will display a schematic of the sequence and secondary structure for each chain.

While the wizard is running in parallel build mode, the "Model-building" tab will show the progress of the individual jobs. Although the output of these should not be used since a higher-quality final consensus model will be assembled by AutoBuild, you can view the current model for any job by selecting from the list and clicking the Coot button.

Omit maps

For clarity, the omit map functionality has been moved to a separate GUI, with similar behavior but different configuration options.

Omit maps may be generated for either a specific (usually small) region of the structure, such as a ligand or problematic loop, or a composite of the entire structure. In the latter mode, the program will iterate over boxes covering the unit cell, calculate maps for each box with the atoms inside left out, and assemble a complete map at the end. (Note that an omit map for a specific region could also be generated manually by deleting part of a model or setting the occupancies to zero, then running phenix.refine.)

Because of the interdependency of all reflections and all atoms, re-calculating maps immediately after deleting atoms does not reliably remove phase bias. The "Omit map type" control offers three choices for handling the partial models ("Simple", "Simulated annealing", and "Iterative build"); the first will run phenix.refine with simple minimization and ADP refinement alone. A much more thorough method of removing bias is a simulated annealing omit map, which allows the structure to escape local minima. (Currently this only supports the Cartesian dynamics; torsion angle SA will be available in a future version.) A third, even more robust option is the iterative build omit map, which rebuilds the partial structure. In addition to thoroughly removing bias, the resulting models provide an estimate of uncertainty for atomic coordinates.

The main disadvantage of these methods is the long running time especially for the iterative build omit maps. However, because AutoBuild is highly parallel, we recommend always using the simulated annealing or iterative build options. If you only want to generate figures for publication, specifying a region of the structure to leave out will be more efficient than running the full composite omit map calculation.

Standalone density modification

A third variant of the AutoBuild GUI is dedicated to generating density modified maps with no additional building. This version of the GUI is single-processor only, but runs in much less time than the more advanced versions. Users who wish to create a prime-and-switch map after molecular replacement should use this interface. Note that if an input model is supplied, the default behavior is still to run refinement first, which considerably increases runtime. (This can be disabled by checking the "Refine model" box.)

Density modification video tutorial

The ` density modification tutorial video <https://www.youtube.com/watch?v=_BK2ETHuZhU>`_ is available on the Overall Phenix YouTube channel and covers the following topics:

- basic overview
- how to run phenix.map_to_model via the GUI

References

Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. Acta Cryst. D64, 61-69 (2008).

- Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. Acta Cryst. D64, 61-69 (2008).

Iterative-build OMIT maps: map improvement by iterative model building and refinement without model bias. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.D. Adams, R.J. Read, P.H. Zwart, and L.-W. Hung. Acta Cryst. D64, 515-524 (2008).

- Iterative-build OMIT maps: map improvement by iterative model building and refinement without model bias. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.D. Adams, R.J. Read, P.H. Zwart, and L.-W. Hung. Acta Cryst. D64, 515-524 (2008).

Using prime-and-switch phasing to reduce model bias in molecular replacement. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 60, 2144-9 (2004).

- Using prime-and-switch phasing to reduce model bias in molecular replacement. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 60, 2144-9 (2004).

Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. Acta Crystallogr D Biol Crystallogr 68, 352-67 (2012).

- Towards automated crystallographic structure refinement with phenix.refine. P.V. Afonine, R.W. Grosse-Kunstleve, N. Echols, J.J. Headd, N.W. Moriarty, M. Mustyakimov, T.C. Terwilliger, A. Urzhumtsev, P.H. Zwart, and P.D. Adams. Acta Crystallogr D Biol Crystallogr 68, 352-67 (2012).

Quick model-building of a single model with resolve using build_one_model

build_one_model.html

Author(s)

- build_one_model: Tom Terwilliger

Purpose

If you have an mtz map coefficients file and a sequence file, you can use build_one_model to just build a single model with resolve. You can also extend an existing model.

Usage

How build_one_model works:

build_one_model runs resolve model-building, by default in superquick_build mode. If you supply a model, then resolve will try to extend the ends of each chain in your model.

Output files from build_one_model

build_one_model.pdb: A PDB file with the resulting model

Examples

Standard run of build_one_model:

Running build_one_model is easy. From the command-line you can type:

```
phenix.build_one_model map_coeffs.mtz \
 sequence.dat free_in=exptl_fobs_phases_freeR_flags.mtz
```

where exptl_fobs_phases_freeR_flags.mtz has your free R flags for refinement. If you want to supply a PDB file to extend instead you can do that:

```
phenix.build_one_model map_coeffs.mtz \
 sequence.dat free_in=exptl_fobs_phases_freeR_flags.mtz \
 model.pdb
```

Possible Problems

Specific limitations and problems:

Literature

Additional information

List of all available keywords

input_filesseq_file = None File with 1-letter code sequence of molecule. Chains separated by blank line or greater-than signpdb_in = None Optional starting PDB file (ends will be extended if present)map_coeffs = None MTZ file with coefficients for a map map_coeffs_labels = None Map coeffs labels (like: FWT,PHFWT). Alternative to labin_map_coeffslabin_map_coeffs = None Labin line for MTZ file with map coefficients. This is optional if build_one_model can guess the correct coefficients for FP PHI . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP ncs_info_file = None ncs_spec file with NCS information (written by simple_ncs_from_pdb or find_ncs)remove_freefree_in = None MTZ file containing FreeR_flags NOTE free_in only used in build_one_model. Ignored by phase_and_build Used as source of freeR information for real_space refinement. Note other columns of data may be present and can be used in reciprocal-space refinement. A suitable file is exptf_fobs_phases_freeR_flags.mtz from autosol/autobuild or my_model_refine_data.mtz from phenix.refinelabin_free = None Labin line for MTZ file with FreeR_flags. This is optional if build_one_model can guess the correct coefficients for FreeR_flags.Otherwise specify: FreeR_flags==myFreeR_flagsmap_coeffs_no_free = None Optional MTZ file with coefficients for a map with freeR set removed. Use instead of free_in. This map will be used for real-space refinement Not needed if use_all_in_refine is Truemap_coeffs_labels_no_free = None Map coeffs labels (like: FWT,PHFWT). Alternative to labin_map_coeffs_no_freelabin_no_free = None Labin line for MTZ file with map coefficients and freeR set removed. This is optional if build_one_model can guess the correct coefficients for FP PHI . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP test_flag_value = None This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.output_filestarget_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.pdb_out = build_one_model.pdb Output PDB filelog = build_one_model.log Output log fileparams_out = build_one_model_params.eff Parameters file to rerun build_one_modelmodel_building_log = model_building.log Log file for model-buildingrefinementrs_refine = True You can run real-space refinement after model-building NOTE: real_space refinement requires a source of FreeR_flag and standard requires Fobs SigFobs and a source of FreeR_flag For real-space refinement you can supply either an mtz file with a FreeR_flag column or an mtz map file that has all the FreeR reflections removeduse_all_in_refine = False You can use all your data in refinement (i.e., for cryo-EM data) if you want to. Not suitable for X-ray data.macro_cycles = None Macro cycles in real-space refinement (None=default)rs_refine_adp = False Refine with ADP in real-space refinementreal_space_refine_before_merge = False Run real-space refinement on partial models before mergingmacro_cycles_in_refine_before_merge = 3 Macro cycles in real_space_refine_before_mergemodel_buildingquick = False You can run quickly (superquick_build/delta_phi=30.) or more thoroughly (default, thorough_build/delta_phi=20.)extend_only = True If input model is supplied, just extend it (do not build a new model from scratch).extend = True If input model is supplied, extend it. (Set to False to not extend input model or helices_strands_only model).trace_chain_only = None You can find just a trace_chain model.target_time_for_trace_...

- input_filesseq_file = None File with 1-letter code sequence of molecule. Chains separated by blank line or greater-than signpdb_in = None Optional starting PDB file (ends will be extended if present)map_coeffs = None MTZ file with coefficients for a map map_coeffs_labels = None Map coeffs labels (like: FWT,PHFWT). Alternative to labin_map_coeffslabin_map_coeffs = None Labin line for MTZ file with map coefficients. This is optional if build_one_model can guess the correct coefficients for FP PHI . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP ncs_info_file = None ncs_spec file with NCS information (written by simple_ncs_from_pdb or

find_ncs)remove_freefree_in = None MTZ file containing FreeR_flags NOTE free_in only used in build_one_model. Ignored by phase_and_build Used as source of freeR information for real_space refinement. Note other columns of data may be present and can be used in reciprocal-space refinement. A suitable file is exptf_fobs_phases_freeR_flags.mtz from autosol/autobuild or my_model_refine_data.mtz from phenix.refinelabin_free = None Labin line for MTZ file with FreeR_flags. This is optional if build_one_model can guess the correct coefficients for FreeR_flags.Otherwise specify: FreeR_flags==myFreeR_flagsmap_coeffs_no_free = None Optional MTZ file with coefficients for a map with freeR set removed. Use instead of free_in. This map will be used for real-space refinement Not needed if use_all_in_refine is Truemap_coeffs_labels_no_free = None Map coeffs labels (like: FWT,PHFWT). Alternative to labin_map_coeffs_no_freelabin_no_free = None Labin line for MTZ file with map coefficients and freeR set removed. This is optional if build_one_model can guess the correct coefficients for FP PHI . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP test_flag_value = None This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.

• seq_file = None File with 1-letter code sequence of molecule. Chains separated by blank line or greater-than sign

• pdb_in = None Optional starting PDB file (ends will be extended if present)

• map_coeffs = None MTZ file with coefficients for a map

• map_coeffs_labels = None Map coeffs labels (like: FWT,PHFWT). Alternative to labin_map_coeffs

• labin_map_coeffs = None Labin line for MTZ file with map coefficients. This is optional if build_one_model can guess the correct coefficients for FP PHI . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP

• ncs_info_file = None ncs_spec file with NCS information (written by simple_ncs_from_pdb or find_ncs)

• remove_freefree_in = None MTZ file containing FreeR_flags NOTE free_in only used in build_one_model. Ignored by phase_and_build Used as source of freeR information for real_space refinement. Note other columns of data may be present and can be used in reciprocal-space refinement. A suitable file is exptf_fobs_phases_freeR_flags.mtz from autosol/autobuild or my_model_refine_data.mtz from phenix.refinelabin_free = None Labin line for MTZ file with FreeR_flags. This is optional if build_one_model can guess the correct coefficients for FreeR_flags.Otherwise specify: FreeR_flags==myFreeR_flagsmap_coeffs_no_free = None Optional MTZ file with coefficients for a map with freeR set removed. Use instead of free_in. This map will be used for real-space refinement Not needed if use_all_in_refine is Truemap_coeffs_labels_no_free = None Map coeffs labels (like: FWT,PHFWT). Alternative to labin_map_coeffs_no_freelabin_no_free = None Labin line for MTZ file with map coefficients and freeR set removed. This is optional if build_one_model can guess the correct coefficients for FP PHI . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP test_flag_value = None This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.

• free_in = None MTZ file containing FreeR_flags NOTE free_in only used in build_one_model. Ignored by phase_and_build Used as source of freeR information for real_space refinement. Note other columns of data may be present and can be used in reciprocal-space refinement. A suitable file is

exptf_fobs_phases_freeR_flags.mtz from autosol/autobuild or my_model_refine_data.mtz from phenix.refine

• labin_free = None Labin line for MTZ file with FreeR_flags. This is optional if build_one_model can guess the correct coefficients for FreeR_flags.Otherwise specify: FreeR_flags==myFreeR_flags

• map_coeffs_no_free = None Optional MTZ file with coefficients for a map with freeR set removed. Use instead of free_in. This map will be used for real-space refinement Not needed if use_all_in_refine is True

• map_coeffs_labels_no_free = None Map coeffs labels (like: FWT,PHFWT). Alternative to labin_map_coeffs_no_free

• labin_no_free = None Labin line for MTZ file with map coefficients and freeR set removed. This is optional if build_one_model can guess the correct coefficients for FP PHI . Otherwise specify: LABIN FP=myFP PHIB=myPHI where myFP is your column label for FP

• test_flag_value = None This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.

• output_filestarget_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.pdb_out = build_one_model.pdb Output PDB filelog = build_one_model.log Output log fileparams_out = build_one_model_params.eff Parameters file to rerun build_one_modelmodel_building_log = model_building.log Log file for model-building

• target_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.

• pdb_out = build_one_model.pdb Output PDB file

• log = build_one_model.log Output log file

• params_out = build_one_model_params.eff Parameters file to rerun build_one_model

• model_building_log = model_building.log Log file for model-building

• refinementrs_refine = True You can run real-space refinement after model-building NOTE: real_space refinement requires a source of FreeR_flag and standard requires Fobs SigFobs and a source of FreeR_flag For real-space refinement you can supply either an mtz file with a FreeR_flag column or an mtz map file that has all the FreeR reflections removeduse_all_in_refine = False You can use all your data in refinement (i.e., for cryo-EM data) if you want to. Not suitable for X-ray data.macro_cycles = None Macro cycles in real-space refinement (None=default)rs_refine_adp = False Refine with ADP in real-space refinementreal_space_refine_before_merge = False Run real-space refinement on partial models before mergingmacro_cycles_in_refine_before_merge = 3 Macro cycles in real_space_refine_before_merge

• rs_refine = True You can run real-space refinement after model-building NOTE: real_space refinement requires a source of FreeR_flag and standard requires Fobs SigFobs and a source of FreeR_flag For real-space refinement you can supply either an mtz file with a FreeR_flag column or an mtz map file that has all the FreeR reflections removed

• use_all_in_refine = False You can use all your data in refinement (i.e., for cryo-EM data) if you want to. Not suitable for X-ray data.

• macro_cycles = None Macro cycles in real-space refinement (None=default)

• rs_refine_adp = False Refine with ADP in real-space refinement

• real_space_refine_before_merge = False Run real-space refinement on partial models before merging

• macro_cycles_in_refine_before_merge = 3 Macro cycles in real_space_refine_before_merge

• model_buildingquick = False You can run quickly (superquick_build/delta_phi=30.) or more thoroughly (default, thorough_build/delta_phi=20.)extend_only = True If input model is supplied, just extend it (do not build a new model from scratch).extend = True If input model is supplied, extend it. (Set to False to not extend input model or helices_strands_only model).trace_chain_only = None You can find just a trace_chain model.target_time_for_trace_chain = 15 Target length of time to work on trace_chain (sec)helices_strands_only = None You can find just helices and strands and extend them. Same as insert_helices=True and include_strands=True. This is useful at resolutions of 3. A and worse.insert_helices = None You can find helices and use them as a starting point for model-building. This is useful if your resolution is worse than 3 A. Same as helices_strands_only=True include_strands=False.include_strands = None You can find strands and use them as a starting point for model-building. This is useful if your resolution is worse than 3 A. Same as helices_strands_only=True insert_helices=False.build_rna_helices = None Use build_rna_helices tool to build RNA if chain_type=RNAn_build_repeat = 0 Iterations of buildingn_random_frag = None Tries of fragment extensiondelta_phi = None Rotation angle for secondary structure search (set with quick/thorough to 30/20 degrees by default)rna_rho_min_main_base = -1. starting minimum density at main-chain atoms for RNA buildingrna_rho_min_main_low = -1. final minimum density at main-chain atoms for RNA buildingrna_rho_min_main_seg = -1. minimum density at main-chain atoms in segment ID in RNA buildingrna_rho_min_side_seg = -1. minimum density at side-chain atoms in segment ID in RNA buildingcc_min = 0.40 Minimum map-model correlation to keep a segment after refinementcc_min_rna = 0.25 Minimum map-model correlation (RNA building)merge_with_combine_models = True Merge using parallel combine_models method. Can include standard merge. Alternative is merging with resolve.merge_by_segment_correlation = True Merge using segment correlation if merge_with_combine_models is selected. Can be used along with merge_remainder=True/False and standard_merge=True/Falsemerge_remainder = True Merge remainder in merge_by_segment_correlationstandard_merge = True If merge_with_combine_models is set, merge by reading all chains and running resolve merging.use_cc_in_combine_extend = False You can choose to use the correlation of density rather than density at atomic positions to score models in the merge_second_model or merge_both_models step. This may be useful at lower resolution (> 3 A)

• quick = False You can run quickly (superquick_build/delta_phi=30.) or more thoroughly (default, thorough_build/delta_phi=20.)

• extend_only = True If input model is supplied, just extend it (do not build a new model from scratch).

• extend = True If input model is supplied, extend it. (Set to False to not extend input model or helices_strands_only model).

• trace_chain_only = None You can find just a trace_chain model.

• target_time_for_trace_chain = 15 Target length of time to work on trace_chain (sec)

• helices_strands_only = None You can find just helices and strands and extend them. Same as insert_helices=True and include_strands=True. This is useful at resolutions of 3. A and worse.

• insert_helices = None You can find helices and use them as a starting point for model-building. This is useful if your resolution is worse than 3 A. Same as helices_strands_only=True include_strands=False.

• include_strands = None You can find strands and use them as a starting point for model-building. This is useful if your resolution is worse than 3 A. Same as helices_strands_only=True insert_helices=False.

• build_rna_helices = None Use build_rna_helices tool to build RNA if chain_type=RNA

• n_build_repeat = 0 Iterations of building

• n_random_frag = None Tries of fragment extension

• delta_phi = None Rotation angle for secondary structure search (set with quick/thorough to 30/20 degrees by default)

• rna_rho_min_main_base = -1. starting minimum density at main-chain atoms for RNA building

• rna_rho_min_main_low = -1. final minimum density at main-chain atoms for RNA building

• rna_rho_min_main_seg = -1. minimum density at main-chain atoms in segment ID in RNA building

• rna_rho_min_side_seg = -1. minimum density at side-chain atoms in segment ID in RNA building

• cc_min = 0.40 Minimum map-model correlation to keep a segment after refinement

• cc_min_rna = 0.25 Minimum map-model correlation (RNA building)

• merge_with_combine_models = True Merge using parallel combine_models method. Can include standard merge. Alternative is merging with resolve.

• merge_by_segment_correlation = True Merge using segment correlation if merge_with_combine_models is selected. Can be used along with merge_remainder=True/False and standard_merge=True/False

• merge_remainder = True Merge remainder in merge_by_segment_correlation

• standard_merge = True If merge_with_combine_models is set, merge by reading all chains and running resolve merging.

• use_cc_in_combine_extend = False You can choose to use the correlation of density rather than density at atomic positions to score models in the merge_second_model or merge_both_models step. This may be useful at lower resolution (> 3 A)

• directoriestemp_dir = "temp_dir" Optional temporary work directory output_dir = "" Output directory where files are to be writtentop_output_dir = "" Top output directory for control files

• temp_dir = "temp_dir" Optional temporary work directory

• output_dir = "" Output directory where files are to be written

• top_output_dir = "" Top output directory for control files

• crystal_inforesolution = 0. high-resolution limit for map calculationsolvent_fraction = None You can specify the solvent fraction Normally it is set automaticallychain_type = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms)scattering_table = *n_gaussian wk1995 it1992 electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.semet = False You can specify that your protein contains selenomethionine

• resolution = 0. high-resolution limit for map calculation

• solvent_fraction = None You can specify the solvent fraction Normally it is set automatically

• chain_type = *PROTEIN DNA RNA Chain type (for identifying main-chain and side-chain atoms)

• scattering_table = *n_gaussian wk1995 it1992 electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.

• semet = False You can specify that your protein contains selenomethionine

• controlverbose = False Verbose outputraise_sorry = False Raise sorry if problemsdebug = False Debugging outputcarry_on = False Try to continue from previous run. For phase_and_build onlydry_run = False Just read in and check parameter namesdo_only_one_thing = False Just do one thingwrite_run_directory_to_file = None The working directory name is written to this filemultiprocessing =

*multiprocessing sge lsf pbs condor pbspro slurm Choices are multiprocessing (single machine) or queuing systemsqueue_run_command = None run command for queue jobs. For example qsub.parallel_step = *final_assembly assembly peak_search all Last step to carry out in parallel in model-building. peak_search means find the secondary structure in parallel. assembly means find ss and extend in parallel. final_assembly means find ss, extend and create final model. all means do entire model-building nproc times.nproc = 1 Number of processors to use (None is do not use multiprocessing)i_ran_seed = 712341 Random seed for model-buildingremove_pdb_block = None Clear out any .pdb_block files written by previous runs of map_to_model. Normally only set to False when you are running jobs with split_up_job. Default is True unless do_only_one_thing is set. Used in phase_and_build only.model_start = None You can use this to specify skipping model-building of model numbers below this (used to start in the middle). Used in phase_and_build only.model_end = None You can use this to specify skipping model-building of model numbers above this (used to start in the middle). Used in phase_and_build only.resolve_size = 12 Size for resolveresolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "

• verbose = False Verbose output

• raise_sorry = False Raise sorry if problems

• debug = False Debugging output

• carry_on = False Try to continue from previous run. For phase_and_build only

• dry_run = False Just read in and check parameter names

• do_only_one_thing = False Just do one thing

• write_run_directory_to_file = None The working directory name is written to this file

• multiprocessing = *multiprocessing sge lsf pbs condor pbspro slurm Choices are multiprocessing (single machine) or queuing systems

• queue_run_command = None run command for queue jobs. For example qsub.

• parallel_step = *final_assembly assembly peak_search all Last step to carry out in parallel in model-building. peak_search means find the secondary structure in parallel. assembly means find ss and extend in parallel. final_assembly means find ss, extend and create final model. all means do entire model-building nproc times.

• nproc = 1 Number of processors to use (None is do not use multiprocessing)

• i_ran_seed = 712341 Random seed for model-building

• remove_pdb_block = None Clear out any .pdb_block files written by previous runs of map_to_model. Normally only set to False when you are running jobs with split_up_job. Default is True unless do_only_one_thing is set. Used in phase_and_build only.

• model_start = None You can use this to specify skipping model-building of model numbers below this (used to start in the middle). Used in phase_and_build only.

• model_end = None You can use this to specify skipping model-building of model numbers above this (used to start in the middle). Used in phase_and_build only.

• resolve_size = 12 Size for resolve

• resolve_command_list = None You can supply any resolve command here NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "

Building starting with a very poor map with parallel_autobuild

parallel_autobuild.html

Authors

parallel_autobuild: Tom Terwilliger

Purpose

The goal of parallel_autobuild is to build models starting from very poor maps. In this situation most of the models created by autobuild are not very good, but some will be a little better than others. The strategy is to find these slightly-better models and iterate model-building based on them.

The working hypothesis for parallel_autobuild is that the quality of the starting map is a key determinant of how good the final model will be. The strategy is then to take a starting map and data, get the best model from this starting point, and use it to calculate a new starting map that is hopefully a little better than the first one. Iterating this process may finally lead to a starting map that is good enough to yield a complete model.

How parallel autobuild works

parallel_autobuild is a tool to carry out the process of picking good models from many autobuild runs automatically. It will run parallel jobs of phenix.autobuild, identify the best model and the corresponding map, then take the best map as the starting point for another cycle of building.

The choice of what is the best model is not always obvious. Parallel_autobuild uses the R-values of the models to make this choice if the R is less than 0.50 (by default), and if all R-values are higher, the map correlation of each model to its corresponding density-modified map is used to make the choice.

Inputs for parallel autobuild

The inputs for parallel autobuild are the same as for autobuild, with the addition of a few control parameters. Also parallel autobuild requires you to specify what each input file is (it won't guess). The main parameters are the number of processors to use, the number of overall cycles, and the number of parallel autobuild jobs to be run in each cycle. The number of processors available for each autobuild job then determines how many models are built in each autobuild cycle.

You can also specify whether the model obtained on each cycle is to be used in the next cycle (by default, if you supply a starting model the model obtained each cycle is used, and if you do not, it is tossed and only the new map is used.)

A run_command keyword allows you to specify how jobs are to be run. Parallel autobuild can be run on a queueing system (with a command such as qsub) or on a multiprocessor machine (e.g., with sh).

Possible Problems

Please note that for parallel_autobuild the input data file must be an MTZ file and it must have columns for FP SIGFP FreeR_flag. A convenient way to create the right file is to run phenix.autobuild with your data (you don't need to let it finish, just get started), then use the exptl_fobs_freeR_flags.mtz (or overall_best.refine_data.mtz) it creates as your data file for phenix.parallel_autobuild.

Specific limitations:

Parallel_autobuild does not recombine models from different parallel runs. It is possible that the use of phenix.combine_models with parallel_autobuild could further improve its performance.

References:

Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. Acta Cryst. D64, 61-69 (2008).

- Iterative model building, structure refinement and density modification with the PHENIX AutoBuild wizard. T.C. Terwilliger, R.W. Grosse-Kunstleve, P.V. Afonine, N.W. Moriarty, P.H. Zwart, L.-W. Hung, R.J. Read, and P.D. Adams. Acta Cryst. D64, 61-69 (2008).

Keywords:

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command lineparallel_autobuilditerations = 3 Total number of iterations of autobuilding n_parallel = Auto Total number of autobuild runs in each iteration Default is nproc/4parallel_r_switch = 0.5 If R is less than parallel_r_switch, use R to choose best model; otherwise use map correlationnproc = 1 Number of processors to usebackground = True run jobs in background or not (if nproc is greater than 1)run_command = "sh " Command for running jobs (e.g., sh or qsub)verbose = False verbose outputraise_sorry = False Raise sorry if problemsdebug = False debugging outputtemp_dir = Auto Optional temporary work directorywork_dir = "" Work directory (by default, PARALLEL_AUTOBUILD_RUN_xxxx/)output_dir = "" Output directory where files are to be writtendry_run = False Just read in and check parameter namescheck_wait_time = 10.0 You can specify the length of time (seconds) to wait between checking for subprocesses to endwait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....average_map_coeffs_each_cycle = None You can use the average map coeffs from all runs in a cycle as input to the next cycle (as opposed to just the map coeffs for the best model). Default is True unless one model has an R at least big_difference_in_r better than the average model.big_difference_in_r = 0.10 Used to decide whether average_map_coeffs_each_cycle is used. Default average_map_coeffs_each_cycle is True unless one model has an R at least big_difference_in_r better than the average modeldensity_modify_average_map_coeffs = True You can density modify the average map coeffs (if used).update_model_each_cycle = None You can specify whether the model obtained each cycle is to be used in the next cycle. Default is True if a model is supplied at the beginning and False otherwise.sufficient_ratio_finished = 0.75 You can reduce the possibility of the whole parallel set of jobs hanging by allowing parallel_autobuild to ignore any jobs that run longer than run_time_ratio times the longest successful jobs once the number of successful jobs is sufficient_ratio_finished of the total number of jobsrun_time_ratio = 1.5 You can reduce the possibility of the whole parallel set of jobs hanging by allowing parallel_autobuild to ignore any jobs that run longer than run_time_ratio times the longest successful jobs once the number of successful jobs is sufficient_ratio_finished of the total number of jobsautobuilddata = None Datafile. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_labels='FP SIGFP PHIB FOM HLA HLB HLC HLD FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP

you can just say input_labels='myFP mysigFP'. If you have free R flags, phase information or HL coefficients that you want to use then an mtz file is required. If this file contains phase information, this phase information should be experimental (i.e., MAD/SAD/MIR etc), and should not be density-modified phases (enter any files with density-modified phases as input_map_file instead). NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. If you also specify a hires data file, then FP and SIGFP will come from that data file (and not this one) If an input_refinement_file is specified, then F, Sigma, FreeR_flag (if present) from that file will be used for refinement instead of this one. model = None PDB file with starting model. NOTE: If your PDB file has been previously refin...

- job_title = None Job title in PHENIX GUI, not used on command line
- parallel_autobuilditerations = 3 Total number of iterations of autobuilding n_parallel = Auto Total number of autobuild runs in each iteration Default is nproc/4parallel_r_switch = 0.5 If R is less than parallel_r_switch, use R to choose best model; otherwise use map correlationnproc = 1 Number of processors to usebackground = True run jobs in background or not (if nproc is greater than 1)run_command = "sh " Command for running jobs (e.g., sh or qsub)verbose = False verbose outputraise_sorry = False Raise sorry if problemsdebug = False debugging outputtemp_dir = Auto Optional temporary work directorywork_dir = "" Work directory (by default, PARALLEL_AUTOBUILD_RUN_xxxx/)output_dir = "" Output directory where files are to be writtendry_run = False Just read in and check parameter namescheck_wait_time = 10.0 You can specify the length of time (seconds) to wait between checking for subprocesses to endwait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....average_map_coeffs_each_cycle = None You can use the average map coeffs from all runs in a cycle as input to the next cycle (as opposed to just the map coeffs for the best model). Default is True unless one model has an R at least big_difference_in_r better than the average model.big_difference_in_r = 0.10 Used to decide whether average_map_coeffs_each_cycle is used. Default average_map_coeffs_each_cycle is True unless one model has an R at least big_difference_in_r better than the average modeldensity_modify_average_map_coeffs = True You can density modify the average map coeffs (if used).update_model_each_cycle = None You can specify whether the model obtained each cycle is to be used in the next cycle. Default is True if a model is supplied at the beginning and False otherwise.sufficient_ratio_finished = 0.75 You can reduce the possibility of the whole parallel set of jobs hanging by allowing parallel_autobuild to ignore any jobs that run longer than run_time_ratio times the longest successful jobs once the number of successful jobs is sufficient_ratio_finished of the total number of jobsrun_time_ratio = 1.5 You can reduce the possibility of the whole parallel set of jobs hanging by allowing parallel_autobuild to ignore any jobs that run longer than run_time_ratio times the longest successful jobs once the number of successful jobs is sufficient_ratio_finished of the total number of jobs
- iterations = 3 Total number of iterations of autobuilding
- n_parallel = Auto Total number of autobuild runs in each iteration Default is nproc/4
- parallel_r_switch = 0.5 If R is less than parallel_r_switch, use R to choose best model; otherwise use map correlation
- nproc = 1 Number of processors to use
- background = True run jobs in background or not (if nproc is greater than 1)
- run_command = "sh " Command for running jobs (e.g., sh or qsub)

• verbose = False verbose output

• raise_sorry = False Raise sorry if problems

• debug = False debugging output

• temp_dir = Auto Optional temporary work directory

• work_dir = "" Work directory (by default, PARALLEL_AUTOBUILD_RUN_xxxx/)

• output_dir = "" Output directory where files are to be written

• dry_run = False Just read in and check parameter names

• check_wait_time = 10.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end

• wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....

• average_map_coeffs_each_cycle = None You can use the average map coeffs from all runs in a cycle as input to the next cycle (as opposed to just the map coeffs for the best model). Default is True unless one model has an R at least big_difference_in_r better than the average model.

• big_difference_in_r = 0.10 Used to decide whether average_map_coeffs_each_cycle is used. Default average_map_coeffs_each_cycle is True unless one model has an R at least big_difference_in_r better than the average model

• density_modify_average_map_coeffs = True You can density modify the average map coeffs (if used).

• update_model_each_cycle = None You can specify whether the model obtained each cycle is to be used in the next cycle. Default is True if a model is supplied at the beginning and False otherwise.

• sufficient_ratio_finished = 0.75 You can reduce the possibility of the whole parallel set of jobs hanging by allowing parallel_autobuild to ignore any jobs that run longer than run_time_ratio times the longest successful jobs once the number of successful jobs is sufficient_ratio_finished of the total number of jobs

• run_time_ratio = 1.5 You can reduce the possibility of the whole parallel set of jobs hanging by allowing parallel_autobuild to ignore any jobs that run longer than run_time_ratio times the longest successful jobs once the number of successful jobs is sufficient_ratio_finished of the total number of jobs

• autobuilddata = None Datafile. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_labels='FP SIGFP PHIB FOM HLA HLB HLC HLD FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP you can just say input_labels='myFP mysigFP'. If you have free R flags, phase information or HL coefficients that you want to use then an mtz file is required. If this file contains phase information, this phase information should be experimental (i.e., MAD/SAD/MIR etc), and should not be density-modified phases (enter any files with density-modified phases as input_map_file instead). NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. If you also specify a hires data file, then FP and SIGFP will come from that data file (and not this one) If an input_refinement_file is specified, then F, Sigma, FreeR_flag (if present) from that file will be used for refinement instead of this one. model = None PDB file with starting model. NOTE: If your PDB file has been previously refined, then please make sure that you provide the free R flags that were used in that refinement. These can come from the data file or from the refinement_file. seq_file = Auto Text file with 1-letter code of protein sequence. Separate chains with a blank line or line starting with >. Normally you should include one copy of each unique chain. NOTE: if 1 copy of each unique chain

is provided it is assumed that there are ncs_copies (could be 1) of each unique chain. If more than one copy of any chain is provided it is assumed that the asymmetric unit contains the number of copies of each chain that are given, multiplied by ncs_copies. So if the sequence file has two copies of the sequence for chain A and one of chain B, the cell contents are assumed to be ncs_copies*2 of chain A and ncs_copies of chain B. If there are unequal numbers of copies of chains, be sure to set solvent_fraction. ADDITIONAL NOTES: 1. lines starting with > are ignored and separate chains 2. FASTA format is fine 3. If you enter a PDB file for rebuilding and it has the sequence you want, then the sequence file is not necessary. NOTE: You can also enter the name of a PDB file that contains SEQRES records, and the sequence from the SEQRES records will be read, written to seq_from_seqres_records.dat, and used as your input sequence. If you have a duplex DNA, enter each strand as a separate chain. NOTE 1: for AutoBuild you can specify start_chains_list on the first line of your sequence file: >> start_chains_list 23 11 5 NOTE 2. Characters such as numbers and non-printing characters in the sequence file are ignored. NOTE 3. Be sure that your sequence file does not have any blank lines in the middle of your sequence, as these are interpreted as the beginning of another chain.map_file = Auto MTZ file containing starting map. This file must be a mtz file. The Wizard will guess the column identification. You can specify the column labels to use with: input_map_labels='FP PHIB FOM' Substitute any labels you do not have with None. If you only have myFP and myPHIB you can just say input_map_labels='myFP myPHIB'. This map will be used in the first cycle of model-building. NOTE 1: If use_map_file_as_hklstart=True then this file will be used instead to start density modification. NOTE 2: default for this keyword is Auto, which means "carry out normal process to guess this keyword". This means if you specify "after_autosol" in AutoBuild, AutoBuild will automatically take the value from AutoSol. If you do not want this to happen, you can specify None which means "No file" refinement_file = Auto File for refinement. This file can be a ....

• data = None Datafile. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_labels='FP SIGFP PHIB FOM HLA HLB HLC HLD FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP you can just say input_labels='myFP mysigFP'. If you have free R flags, phase information or HL coefficients that you want to use then an mtz file is required. If this file contains phase information, this phase information should be experimental (i.e., MAD/SAD/MIR etc), and should not be density-modified phases (enter any files with density-modified phases as input_map_file instead). NOTE: If you supply HL coefficients they will be used in phase recombination. If you supply PHIB or PHIB and FOM and not HL coefficients, then HL coefficients will be derived from your PHIB and FOM and used in phase recombination. If you also specify a hires data file, then FP and SIGFP will come from that data file (and not this one) If an input_refinement_file is specified, then F, Sigma, FreeR_flag (if present) from that file will be used for refinement instead of this one.

• model = None PDB file with starting model. NOTE: If your PDB file has been previously refined, then please make sure that you provide the free R flags that were used in that refinement. These can come from the data file or from the refinement_file.

• seq_file = Auto Text file with 1-letter code of protein sequence. Separate chains with a blank line or line starting with >. Normally you should include one copy of each unique chain. NOTE: if 1 copy of each unique chain is provided it is assumed that there are ncs_copies (could be 1) of each unique chain. If more than one copy of any chain is provided it is assumed that the asymmetric unit contains the number of copies of each chain that are given, multiplied by ncs_copies. So if the sequence file has two copies of the sequence for chain A and one of chain B, the cell contents are assumed to be ncs_copies*2 of chain A and ncs_copies of chain B. If there are unequal numbers of copies of chains, be sure to set solvent_fraction. ADDITIONAL NOTES: 1. lines starting with > are ignored and separate chains 2. FASTA format is fine 3. If you enter a PDB file for rebuilding and it has the sequence you want, then the

sequence file is not necessary. NOTE: You can also enter the name of a PDB file that contains SEQRES records, and the sequence from the SEQRES records will be read, written to seq_from_seqres_records.dat, and used as your input sequence. If you have a duplex DNA, enter each strand as a separate chain. NOTE 1: for AutoBuild you can specify start_chains_list on the first line of your sequence file: >> start_chains_list 23 11 5 NOTE 2. Characters such as numbers and non-printing characters in the sequence file are ignored. NOTE 3. Be sure that your sequence file does not have any blank lines in the middle of your sequence, as these are interpreted as the beginning of another chain.

• map_file = Auto MTZ file containing starting map. This file must be a mtz file. The Wizard will guess the column identification. You can specify the column labels to use with: input_map_labels='FP PHIB FOM' Substitute any labels you do not have with None. If you only have myFP and myPHIB you can just say input_map_labels='myFP myPHIB'. This map will be used in the first cycle of model-building. NOTE 1: If use_map_file_as_hklstart=True then this file will be used instead to start density modification. NOTE 2: default for this keyword is Auto, which means "carry out normal process to guess this keyword". This means if you specify "after_autosol" in AutoBuild, AutoBuild will automatically take the value from AutoSol. If you do not want this to happen, you can specify None which means "No file"

• refinement_file = Auto File for refinement. This file can be a .sca or mtz or other standard file. This file will be merged with your data file, with any phase information coming from your data file. If this file has free R flags, they will be used, otherwise if the data file has them, those will be used, otherwise they will be generated. The Wizard will guess the column identification. You can specify the column labels to use with: input_refinement_labels='FP SIGFP FreeR_flag' Substitute any labels you do not have with None. If you only have myFP and mysigFP you can just say input_refinement_labels='myFP mysigFP'. Data file to use for refinement. The data in this file should not be corrected for anisotropy. It will be combined with experimental phase information (if any) from input_data_file for refinement. If you leave this blank, then the data in the input_data_file will be used in refinement. If no anisotropy correction is applied to the data you do not need to specify a datafile for refinement. If an anisotropy correction is applied to the data files, then you should enter an uncorrected datafile for refinement. Any standard format is fine; normally only F and sigF will be used. Bijvoet pairs and duplicates will be averaged. If an mtz file is provided then a free R flag can be read in as well. Any HL coeffs and phase information in this file is ignored. NOTE: default for this keyword is Auto, which means "carry out normal process to guess this keyword". This means if you specify "after_autosol" in AutoBuild, AutoBuild will automatically take the value from AutoSol. If you do not want this to happen, you can specify None which means "No file"

• hires_file = Auto File with high-resolution data. This file can be a .sca or mtz or other standard file. The Wizard will guess the column identification. You can specify the column labels to use with: input_hires_labels='FP SIGFP'.

• crystal_infounit_cell = None Enter cell parameter (a b c alpha beta gamma)space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)solvent_fraction = None Solvent fraction in crystals (0 to 1). This is normally set automatically from the number of NCS copies and the sequence. If your has unequal numbers of different chains, then be sure to set the solvent fraction.chain_type = *Auto PROTEIN DNA RNA You can specify whether to build protein, DNA, or RNA chains. At present you can only build one of these in a single run. If you have both DNA and protein, build one first, then run AutoBuild again, supplying the prebuilt model in the "input_lig_file_list" and build the other. NOTE: default for this keyword is Auto, which means "carry out normal process to guess this keyword". The process is to look at the sequence file and/or input pdb file to see what the chain type is. If there are more than one type, the type with the larger number of residues is guessed. If you want to force the chain_type, then set it to PROTEIN RNA or DNA. resolution = 0 High-resolution limit. Used as resolution limit for

density modification and as general default high-resolution limit. If resolution_build or refinement_resolution are set then they override this for model-building or refinement. If overall_resolution is set then data beyond that resolution is ignored completely. Zero means keep everything.dmax = 500 Low-resolution limit overall_resolution = 0 If overall_resolution is set, then all data beyond this is ignored. NOTE: this is only suggested if you have a very big cell and need to truncate the data to allow the wizard to run at all. Normally you should use 'resolution' and 'resolution_build' and 'refinement_resolution' to set the high-resolution limit sequence = None Plain text containing 1-letter code of protein sequence Same as seq_file except the sequence is read directly, not from a file. If both are given, seq_file is ignored.

• unit_cell = None Enter cell parameter (a b c alpha beta gamma)

• space_group = None Space Group symbol (i.e., C2221 or C 2 2 21)

• solvent_fraction = None Solvent fraction in crystals (0 to 1). This is normally set automatically from the number of NCS copies and the sequence. If your has unequal numbers of different chains, then be sure to set the solvent fraction.

• chain_type = *Auto PROTEIN DNA RNA You can specify whether to build protein, DNA, or RNA chains. At present you can only build one of these in a single run. If you have both DNA and protein, build one first, then run AutoBuild again, supplying the prebuilt model in the "input_lig_file_list" and build the other. NOTE: default for this keyword is Auto, which means "carry out normal process to guess this keyword". The process is to look at the sequence file and/or input pdb file to see what the chain type is. If there are more than one type, the type with the larger number of residues is guessed. If you want to force the chain_type, then set it to PROTEIN RNA or DNA.

• resolution = 0 High-resolution limit. Used as resolution limit for density modification and as general default high-resolution limit. If resolution_build or refinement_resolution are set then they override this for model-building or refinement. If overall_resolution is set then data beyond that resolution is ignored completely. Zero means keep everything.

• dmax = 500 Low-resolution limit

• overall_resolution = 0 If overall_resolution is set, then all data beyond this is ignored. NOTE: this is only suggested if you have a very big cell and need to truncate the data to allow the wizard to run at all. Normally you should use 'resolution' and 'resolution_build' and 'refinement_resolution' to set the high-resolution limit

• sequence = None Plain text containing 1-letter code of protein sequence Same as seq_file except the sequence is read directly, not from a file. If both are given, seq_file is ignored.

• output_filestarget_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.

• target_output_format = *None pdb mmcif Desired output format (if possible). Choices are None (try to use input format), pdb, mmcif. If output model does not fit in pdb format, mmcif will be used. Default is pdb.

• input_filesinput_labels = None Labels for input data columns input_hires_labels = None Labels for input hires file (FP SIGFP FreeR_flag) input_map_labels = None Labels for input map coefficient columns (FP PHIB FOM) NOTE: FOM is optional (set to None if you wish) input_refinement_labels = None Labels for input refinement file columns (FP SIGFP FreeR_flag) input_ha_file = None If the flag "truncate_ha_sites_in_resolve" is set then density at sites specified with input_ha_file is truncated to improve the density modification procedure. Additionally these sites are added to input_lig_file_list. NOTE: if the chain ID for atoms in this file is A then the chain ID will be renamed to Z so

that the residue number and chain ID combination does not duplicate residues to be built. NOTE: if a hires_file is supplied, this input_ha_file and also any atoms in chain Z of input PDB file will be ignored unless force_input_ha is set to True.force_input_ha = False If a hires_file is supplied, input_ha_file and any atoms in chain Z of input pdb file will be ignored unless force_input_ha is set to True.include_ha_in_model = True Add contents of input_ha_file to the working model just before refinement (by adding it to input_lig_file_list).cif_def_file_list = None You can enter any number of CIF definition files. These are normally used to tell phenix.refine about the geometry of a ligand or unusual residue. You usually will use these in combination with "PDB file with metals/ligands" (keyword "input_lig_file_list") which allows you to attach the contents of any PDB file you like to your model just before it gets refined. You can use phenix.elbow to generate these if you do not have a CIF file and one is requested by phenix.refine input_lig_file_list = None This script adds the contents of these PDB files to each model just prior to refinement. Normally you might use this to put in any heavy-atoms that are in the refined structure (for example the heavy atoms that were used in phasing), or to add a ligand to your model. (By default if you supply input_ha_file this will be added to your input_lig_file_list.) If the atoms in this PDB file are not recognized by phenix.refine, then you can specify their geometries with a cif definitions file using the keyword "cif_def_files_list". You can easily generate cif definitions for many ligands using phenix.elbow in PHENIX. You can put anything you like in the files in input_lig_file_list, but any atoms that fall within 1.5 A of any atom in the current model will be tossed (not written to the model). keep_input_ligands = True You can choose whether to (by default) let the wizard keep ligands by separating them out from the rest of your model and adding them back to your rebuilt model, or alternatively to remove all ligands from your input pdb file before rebuild_in_place. keep_input_waters = False You can choose whether to keep input waters (solvent) when using rebuild_in_place. If you keep them, then you should specify either "place_waters=No" or "keep_pdb_atoms=No" because if place_waters=True and keep_pdb_atoms=True then phenix.refine will add waters and then the wizard will keep the new waters from the new PDB file created by phenix.refine preferentially over the ones in your input file. keep_pdb_atoms = True If true, keep the model coordinates when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current model. If false, keep instead the ligand coordinates. remove_residues_on_special_positions = False If true, remove any residues with atoms on special positions just before refinement.refine_eff_file_list = None You can enter any number of refinement parameter files. These are normally used to tell phenix.refine defaults to apply, as well as creating specialized definitions such as unusual amino acid residues and linkages. These para...

• input_labels = None Labels for input data columns

• input_hires_labels = None Labels for input hires file (FP SIGFP FreeR_flag)

• input_map_labels = None Labels for input map coefficient columns (FP PHIB FOM) NOTE: FOM is optional (set to None if you wish)

• input_refinement_labels = None Labels for input refinement file columns (FP SIGFP FreeR_flag)

• input_ha_file = None If the flag "truncate_ha_sites_in_resolve" is set then density at sites specified with input_ha_file is truncated to improve the density modification procedure. Additionally these sites are added to input_lig_file_list. NOTE: if the chain ID for atoms in this file is A then the chain ID will be renamed to Z so that the residue number and chain ID combination does not duplicate residues to be built. NOTE: if a hires_file is supplied, this input_ha_file and also any atoms in chain Z of input PDB file will be ignored unless force_input_ha is set to True.

• force_input_ha = False If a hires_file is supplied, input_ha_file and any atoms in chain Z of input pdb file will be ignored unless force_input_ha is set to True.

• include_ha_in_model = True Add contents of input_ha_file to the working model just before refinement (by adding it to input_lig_file_list).

• cif_def_file_list = None You can enter any number of CIF definition files. These are normally used to tell phenix.refine about the geometry of a ligand or unusual residue. You usually will use these in combination with "PDB file with metals/ligands" (keyword "input_lig_file_list") which allows you to attach the contents of any PDB file you like to your model just before it gets refined. You can use phenix.elbow to generate these if you do not have a CIF file and one is requested by phenix.refine

• input_lig_file_list = None This script adds the contents of these PDB files to each model just prior to refinement. Normally you might use this to put in any heavy-atoms that are in the refined structure (for example the heavy atoms that were used in phasing), or to add a ligand to your model. (By default if you supply input_ha_file this will be added to your input_lig_file_list.) If the atoms in this PDB file are not recognized by phenix.refine, then you can specify their geometries with a cif definitions file using the keyword "cif_def_files_list". You can easily generate cif definitions for many ligands using phenix.elbow in PHENIX. You can put anything you like in the files in input_lig_file_list, but any atoms that fall within 1.5 A of any atom in the current model will be tossed (not written to the model).

• keep_input_ligands = True You can choose whether to (by default) let the wizard keep ligands by separating them out from the rest of your model and adding them back to your rebuilt model, or alternatively to remove all ligands from your input pdb file before rebuild_in_place.

• keep_input_waters = False You can choose whether to keep input waters (solvent) when using rebuild_in_place. If you keep them, then you should specify either "place_waters=No" or "keep_pdb_atoms=No" because if place_waters=True and keep_pdb_atoms=True then phenix.refine will add waters and then the wizard will keep the new waters from the new PDB file created by phenix.refine preferentially over the ones in your input file.

• keep_pdb_atoms = True If true, keep the model coordinates when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current model. If false, keep instead the ligand coordinates.

• remove_residues_on_special_positions = False If true, remove any residues with atoms on special positions just before refinement.

• refine_eff_file_list = None You can enter any number of refinement parameter files. These are normally used to tell phenix.refine defaults to apply, as well as creating specialized definitions such as unusual amino acid residues and linkages. These parameters override the normal phenix.refine defaults. They themselves can be overridden by parameters set by the Wizard and by you, controlling the Wizard. NOTE: Any parameters set by AutoBuild directly (such as number_of_macro_cycles, high_resolution, etc...) will not be taken from this parameters file. This is useful only for adding extra parameters not normally set by AutoBuild.

• map_file_is_density_modified = True You can specify that the input_map_file has been density modified. (This changes the assumptions on statistics of the map.)

• map_file_fom = None You can specify the FOM of the input map file (useful in cases where the map file has only FWT PHFWT and no FOM column). This FOM is used to set the default smoothing radius for the density modification solvent boundary and also to decide whether extreme density modification is to be applied

• use_constant_input_map = False You can use the input map file for all model-building

• use_map_file_as_hklstart = None You can specify that the file named as input_map_file will be used as starting coefficients for density modification in the first cycle. NOTE: if maps_only=True and

input_map_file is set, then use_map_file_as_hklstart will be set to True

• use_map_in_resolve_with_model = False You can specify that the current map file be used as hklstart in density modification with a model.

• identity_from_remark = True You can use sequence identity from remark like this one: REMARK PHASER ENSEMBLE MODEL 0 ID 100.0 Here the identity is 100 percent. Ignored if there is no remark of this kind in the input model.

• input_data_type = None You can specify the data type for shelx files. The options are: amplitudes (same as hklf3) or intensities (same as hklf4)

• anisoremove_aniso = True Remove anisotropy from data files before use Note: map files are assumed to be already corrected and are not affected by this. Also the input refinement file is not affected by this.b_iso = None Target overall B value for anisotropy correction. Ignored if remove_aniso = False. If None, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)max_b_iso = 40. Default maximum overall B value for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)target_b_ratio = 10. Default ratio of target B value to resolution for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• remove_aniso = True Remove anisotropy from data files before use Note: map files are assumed to be already corrected and are not affected by this. Also the input refinement file is not affected by this.

• b_iso = None Target overall B value for anisotropy correction. Ignored if remove_aniso = False. If None, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• max_b_iso = 40. Default maximum overall B value for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• target_b_ratio = 10. Default ratio of target B value to resolution for anisotropy correction. Ignored if remove_aniso = False. Ignored if b_iso is set. If used, default is minimum of (max_b_iso, lowest B of datasets, target_b_ratio*resolution)

• decision_makingacceptable_r = 0.25 Used to decide whether the model is acceptable enough to quit if it is not improving much. A good value is 0.25 r_switch = 0.4 R-value criteria for deciding whether to use R-value or map correlation as a criteria for model quality. A good value is 0.40 semi_acceptable_r = 0.3 Used to decide whether the model is acceptable enough to skip rebuilding the model from scratch and focus on adding loops and extending it. A good value is 0.3 reject_weak = False You can rebuild or remove just the residues in weak density This will reject residues with density < 0.5 * mean - SD where the density, mean and SD are for either main-chain or all atoms in residues. If set, overrides min_cc_res_rebuild and worst_percent_res_rebuild.min_weak_z = 0.2 Minimum number of sd of rho above 0.5*mean of all residues for keeping weak residues if reject_weak=Truemin_cc_res_rebuild = 0.4 You can rebuild just the worst parts of your model by setting touch_up=True. You can decide what parts to rebuild based on a minimum model-map correlation (by residue). You can decide how much to rebuild using worst_percent_res_rebuild or with min_cc_res_rebuild, or both. min_seq_identity_percent = 50 The sequence in your input PDB file will be adjusted to match the sequence in your sequence file (if any). If there are insertions/deletions in your model and the wizard does not seem to identify them, you can split up your PDB file by adding records like this: BREAK You can specify the minimum sequence identity between your sequence file and a segment from your input PDB file to consider the sequences to be matched. Default is 50.0%. You might want a higher number to make sure that deletions in the sequence are noticed. dist_close = None If main-chain atom rmsd is less than dist_close then crossover between

chains in different models is allowed at this point. If you input a negative number the defaults will be used dist_close_overlap = 1.5 Model or ligand coordinates but not both are kept when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current model. NOTE: you might want to decrease this if your ligand atoms get removed by the wizard. Default=1.5 A loop_cc_min = 0.4 You can specify the minimum correlation of density from a loop with the map. group_ca_length = 4 In resolve building you can specify how short a fragment to keep. Normally 4 or 5 residues should be the minimum.group_length = 2 In resolve building you can specify how many fragments must be joined to make a connected group that is kept. Normally 2 fragments should be the minimum. include_molprobity = False This command is currently disabled. You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero. ok_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score (the threshold defined by ok_molp_score) then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero. scale_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• acceptable_r = 0.25 Used to decide whether the model is acceptable enough to quit if it is not improving much. A good value is 0.25

• r_switch = 0.4 R-value criteria for deciding whether to use R-value or map correlation as a criteria for model quality. A good value is 0.40

• semi_acceptable_r = 0.3 Used to decide whether the model is acceptable enough to skip rebuilding the model from scratch and focus on adding loops and extending it. A good value is 0.3

• reject_weak = False You can rebuild or remove just the residues in weak density This will reject residues with density < 0.5 * mean - SD where the density, mean and SD are for either main-chain or all atoms in residues. If set, overrides min_cc_res_rebuild and worst_percent_res_rebuild.

• min_weak_z = 0.2 Minimum number of sd of rho above 0.5*mean of all residues for keeping weak residues if reject_weak=True

• min_cc_res_rebuild = 0.4 You can rebuild just the worst parts of your model by setting touch_up=True. You can decide what parts to rebuild based on a minimum model-map correlation (by residue). You can decide how much to rebuild using worst_percent_res_rebuild or with min_cc_res_rebuild, or both.

• min_seq_identity_percent = 50 The sequence in your input PDB file will be adjusted to match the sequence in your sequence file (if any). If there are insertions/deletions in your model and the wizard does not seem to identify them, you can split up your PDB file by adding records like this: BREAK You can specify the minimum sequence identity between your sequence file and a segment from your input PDB file to consider the sequences to be matched. Default is 50.0%. You might want a higher number to make sure that deletions in the sequence are noticed.

• dist_close = None If main-chain atom rmsd is less than dist_close then crossover between chains in different models is allowed at this point. If you input a negative number the defaults will be used

• dist_close_overlap = 1.5 Model or ligand coordinates but not both are kept when model and ligand coordinates are within dist_close_overlap and ligands in input_lig_file_list are being added to the current

model. NOTE: you might want to decrease this if your ligand atoms get removed by the wizard. Default=1.5 A

• loop_cc_min = 0.4 You can specify the minimum correlation of density from a loop with the map.

• group_ca_length = 4 In resolve building you can specify how short a fragment to keep. Normally 4 or 5 residues should be the minimum.

• group_length = 2 In resolve building you can specify how many fragments must be joined to make a connected group that is kept. Normally 2 fragments should be the minimum.

• include_molprobity = False This command is currently disabled. You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• ok_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score (the threshold defined by ok_molp_score) then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• scale_molp_score = None You can choose to include the clash score from MolProbity as one of the scoring criteria in comparing and merging models. The score is combined with the model-map correlation CC by summing in a weighted clashscore. If clashscore for a residue has a value < ok_molp_score then its value is (clashscore-ok_molp_score)*scale_molp_score, otherwise its value is zero.

• density_modificationadd_classic_denmod = None You can run classic density modification with solvent flipping after any other kind of density modification. Note: this keyword cannot be used with an omit map. Note also: requires experimental phases (HL coeffs). Default is False unless extreme_dm=True and FOM is less than fom_for_extreme_dm.skip_classic_if_worse_fom = True Skip results of add_classic_denmod if FOM gets worse during density modificationskip_ncs_in_add_classic = True Skip using NCS in add_classic_denmod (speeds it up)thorough_denmod = *Auto True False Choose whether you want to go for thorough density modification when no model is used ("False" speeds it up and for a terrible map is sometimes better) hl = False You can choose whether to calculate hl coeffs when doing density modification (True) or not to do so (False). Default is No. mask_type = *histograms probability wang classic Choose method for obtaining probability that a point is in the protein vs solvent region. Default is "histograms". If you have a SAD dataset with a heavy atom such as Pt or Au then you may wish to choose "wang" because the histogram method is sensitive to very high peaks. Options are: histograms: compare local rms of map and local skew of map to values from a model map and estimate probabilities. This one is usually the best. probability: compare local rms of map to distribution for all points in this map and estimate probabilities. In a few cases this one is much better than histograms. wang: take points with highest local rms and define as protein. Classic runs classical density modification with solvent flipping.mask_from_pdb = None You can specify a PDB file to define a mask for the macromolecule in density modification (i.e., the solvent boundary). All points within rad_mask_from_pdb of an atom in the PDB file defined by mask_from_pdb will be considered to be within the macromoleculemask_type_extreme_dm = histograms probability *wang classic If FOM of phasing is less up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dmmask_cycles_extreme_dm = 1 Mask cycles in extreme density modificationminor_cycles_extreme_dm = 4 Minor cycles in extreme density modificationwang_radius_extreme_dm = 20. Wang radius in extreme density modificationprecondition =

False Precondition density before modificationminimum_ncs_cc = 0.30 Minimum NCS correlation to keep, except in case of extreme_dmextreme_dm = False Turns on extreme density modification if True or if Auto and FOM is up to fom_for_extreme_dm Use extreme_dm=True if your phasing is really weak and density modification is not working well. Note: requires input phase information (HL coeffs). Also note: not compatible with ps_in_rebuild, two_fofc_in_rebuild, or two_fofc_denmod_in_rebuild.fom_for_extreme_dm_rebuild = 0.10 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=mask_type_extreme_dm wang_radius=wang_radius_extreme_dm mask_cycles=mask_cycles_extreme_dm minor_cycles=minor_cycles_extreme_dm Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dmfom_for_extreme_dm = 0.35 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to build stages of autobuild. For rebuild use instead fom_for_extreme_dm_rebuildrad_mask_from_pdb = 2 You can define the radius for calculation of the protein mask Applies only to mask_from_pdbmodify_outside_delta_solvent = 0.05 You can set the initial solvent content to be a little lower than calculated when you are running modify_outside_model Usually 0.05 is fine. modify_outside_model = False You can choose whether to modify ...

• add_classic_denmod = None You can run classic density modification with solvent flipping after any other kind of density modification. Note: this keyword cannot be used with an omit map. Note also: requires experimental phases (HL coeffs). Default is False unless extreme_dm=True and FOM is less than fom_for_extreme_dm.

• skip_classic_if_worse_fom = True Skip results of add_classic_denmod if FOM gets worse during density modification

• skip_ncs_in_add_classic = True Skip using NCS in add_classic_denmod (speeds it up)

• thorough_denmod = *Auto True False Choose whether you want to go for thorough density modification when no model is used ("False" speeds it up and for a terrible map is sometimes better)

• hl = False You can choose whether to calculate hl coeffs when doing density modification (True) or not to do so (False). Default is No.

• mask_type = *histograms probability wang classic Choose method for obtaining probability that a point is in the protein vs solvent region. Default is "histograms". If you have a SAD dataset with a heavy atom such as Pt or Au then you may wish to choose "wang" because the histogram method is sensitive to very high peaks. Options are: histograms: compare local rms of map and local skew of map to values from a model map and estimate probabilities. This one is usually the best. probability: compare local rms of map to distribution for all points in this map and estimate probabilities. In a few cases this one is much better than histograms. wang: take points with highest local rms and define as protein. Classic runs classical density modification with solvent flipping.

• mask_from_pdb = None You can specify a PDB file to define a mask for the macromolecule in density modification (i.e., the solvent boundary). All points within rad_mask_from_pdb of an atom in the PDB file defined by mask_from_pdb will be considered to be within the macromolecule

• mask_type_extreme_dm = histograms probability *wang classic If FOM of phasing is less up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dm

• mask_cycles_extreme_dm = 1 Mask cycles in extreme density modification

• minor_cycles_extreme_dm = 4 Minor cycles in extreme density modification

• wang_radius_extreme_dm = 20. Wang radius in extreme density modification

• precondition = False Precondition density before modification

• minimum_ncs_cc = 0.30 Minimum NCS correlation to keep, except in case of extreme_dm

• extreme_dm = False Turns on extreme density modification if True or if Auto and FOM is up to fom_for_extreme_dm Use extreme_dm=True if your phasing is really weak and density modification is not working well. Note: requires input phase information (HL coeffs). Also note: not compatible with ps_in_rebuild, two_fofc_in_rebuild, or two_fofc_denmod_in_rebuild.

• fom_for_extreme_dm_rebuild = 0.10 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm_rebuild then defaults for density modification become: mask_type=mask_type_extreme_dm wang_radius=wang_radius_extreme_dm mask_cycles=mask_cycles_extreme_dm minor_cycles=minor_cycles_extreme_dm Applies to rebuild stages of autobuild. For build use instead fom_for_extreme_dm

• fom_for_extreme_dm = 0.35 If extreme_dm is on and FOM of phasing is up to fom_for_extreme_dm then defaults for density modification become: mask_type=wang wang_radius=20 mask_cycles=1 minor_cycles=4. Applies to build stages of autobuild. For rebuild use instead fom_for_extreme_dm_rebuild

• rad_mask_from_pdb = 2 You can define the radius for calculation of the protein mask Applies only to mask_from_pdb

• modify_outside_delta_solvent = 0.05 You can set the initial solvent content to be a little lower than calculated when you are running modify_outside_model Usually 0.05 is fine.

• modify_outside_model = False You can choose whether to modify the density in the "protein" region outside the region specified in your current model by matching histograms with the region that is specified by that model. This can help by raising the density in this protein region up to a value similar to that where atoms are already placed.

• truncate_ha_sites_in_resolve = *Auto True False You can choose to truncate the density near heavy-atom sites at a maximum of 2.5 sigma. This is useful in cases where the heavy-atom sites are very strong, and rarely hurts in cases where they are not. The heavy-atom sites are specified with "input_ha_file" and the radius is rad_mask

• rad_mask = None You can define the radius for calculation of the protein mask Applies only to truncate_ha_sites_in_resolve. Default is resolution of data.

• s_step = None You can define the increment in s (1/resolution) for phase extension in resolve density modification. Default is 0.005

• res_start = None You can define the resolution for starting phase extension resolve density modification. Default is resolution to which phase information is available. Please note res_start and map_dmin_start have different effects. The starting resolution within RESOLVE is set by res_start and the step within RESOLVE is set by s_step. These can be used with any density modification procedure. The keyword map_dmin_start creates a set of macro-cycles of RESOLVE density modification where the output map for each cycle is the input_map_file for the next. The map_dmin_start keyword only applies if maps_only=True and only applies to density modification without a model.

• map_dmin_start = None Starting resolution for step-wise macro-cycle density modification. Only applies if maps_only=True and no model is supplied. Please note res_start and map_dmin_start have different effects. The starting resolution within RESOLVE is set by res_start and the step within RESOLVE is set by s_step. These can be used with any density modification procedure. The keyword map_dmin_start creates a set of macro-cycles of RESOLVE density modification where the output map for each cycle is

the input_map_file for the next. The map_dmin_start keyword only applies if maps_only=True and only applies to density modification without a model.

• map_dmin_incr = 0.25 Step size (A) for changes in resolution for macro-cycle density modification

• use_resolve_fragments = True This script normally uses information from fragment identification as part of density modification for the first few cycles of model-building. Fragments are identified during model-building. The fragments are used, with weighting according to the confidence in their placement, in density modification as targets for density values.

• use_resolve_pattern = True Local pattern identification is normally used as part of density modification during the first few cycles of model building.

• use_hl_anom_in_denmod = False Default is False (use HL coefficients in density modification) NOTE: if True, you must supply HLanom coefficients Allows you to specify that HL coefficients including only the phase information from the imaginary (anomalous difference) contribution from the anomalous scatterers are to be used in density modification. Two sets of HL coefficients are produced by Phaser. HLA HLB etc are HL coefficients including the contribution of both the real scattering and the anomalous differences. HLanomA HLanomB etc are HL coefficients including the contribution of the anomalous differences alone. These HL coefficients for anomalous differences alone are the ones that you will want to use in cases where you are bringing in model information that includes the real scattering from the model used in Phaser, such as when you are carrying out density modification with a model or refinement of a model If use_hl_anom_in_denmod=True then the HLanom HL coefficients from Phaser are used in density modification

• use_hl_anom_in_denmod_with_model = False See use_hl_anom_in_denmod If use_hl_anom_in_denmod=True then the HLanom HL coefficients from Phaser are used in density modification with a model

• mask_as_mtz = False Defines how omit_output_mask_file ncs_output_mask_file and protein_output_mask_file are written out. If mask_as_mtz=False it will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with map coefficients FP PHIM FOMM (all three required)

• protein_output_mask_file = None Name of map to be written out representing your protein (non-solvent) region. If mask_as_mtz=False the map will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with map coefficients FP PHIM FOMM (all three required)

• ncs_output_mask_file = None Name of map to be written out representing your ncs asymmetric unit. If mask_as_mtz=False the map will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with map coefficients FP PHIM FOMM (all three required)

• omit_output_mask_file = None Name of map to be written out representing your omit region. If mask_as_mtz=False the map will be a ccp4 map. If mask_as_mtz=True it will be an mtz file with map coefficients FP PHIM FOMM (all three required)

• mapsmaps_only = False You can choose whether to skip all model-building and just calculate maps and write out the results. This also runs just 1 cycle and turns on HL coefficients. n_xyz_list = None You can specify the grid to use for map calculations.

• maps_only = False You can choose whether to skip all model-building and just calculate maps and write out the results. This also runs just 1 cycle and turns on HL coefficients.

• n_xyz_list = None You can specify the grid to use for map calculations.

• model_buildingbuild_type = *RESOLVE RESOLVE_AND_BUCCANEER You can choose to build models with RESOLVE or with RESOLVE and BUCCANEER #and TEXTAL and how many different models to build with RESOLVE. The more you build, the more likely to get a complete model. Note that

rebuild_in_place can only be carried out with RESOLVE model-building. For BUCCANEER model building you need CCP4 version 6.1.2 or higher and BUCCANEER version 1.3.0 or higherallow_negative_residues = False Normally the wizard does not allow negative residue numbers, and all residues with negative numbers are rejected when they are read in. You can allow them if you wish. highest_resno = None Highest residue number to be considered "placed" in sequence for rebuild_in_place semet = False You can specify that the dataset that is used for refinement is a selenomethionine dataset, and that the model should be the SeMet version of the protein, with all SD of MET replaced with Se of MSE. use_met_in_align = *Auto True False You can use the heavy-atom positions in input_ha_file as markers for Met SD positions. base_model = None You can enter a PDB file with coordinates to be used as a starting point for model-building. These coordinates will be included in the same way as fragments placed by searching for helices and strand in initial model-building. Note the difference from the use of models in consider_main_chain_list, which are merged with models after they are built. NOTE: Only use this if you want to keep the input model and just add to it. consider_main_chain_list = None This keyword lets you name any number of PDB files to consider as templates for model-building. Every time models are built, the contents of these files will be merged with them and the best parts will be kept. NOTE: this only uses the main-chain atoms of your PDB files. dist_connect_max_helices = None Set maximum distance between ends of helices and other ends to try and connect them in insert_helices. edit_pdb = True You can choose to edit the input PDB file in rebuild_in_place to match the input sequence (default=True). NOTE: residues with residue numbers higher than 'highest_resno' are assumed to not have a known sequence and will not be edited. By default the value of 'highest_resno' is the highest residue number from the sequence file, after adding it to the starting residue number from start_chains_list. You can also set it directly helices_strands_only = False You can choose to use a quick model-building method that only builds secondary structure. At low resolution this may be both quicker and more accurate than trying to build the entire structure resolution_helices_strands = 3.1 Resolution to switch to helices_strands_onlyhelices_strands_start = False You can choose to use a quick model-building method that builds secondary structure as a way to get started...then model completion is done as usual. (Contrast with helices_strands_only which only does secondary structure) cc_helix_min = None Minimum CC of helical density to map at low resolution when using helices_strands_onlycc_strand_min = None Minimum CC of strand density to map when using helices_strands_only trim_ends_if_needed = True If a single loop is specified and no loop is found, try trimming back the ends.loop_backup_residues = None Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial gap Default is 4 (1 if quick)split_by_gap = True Split up work by gaps and run individually (normally True)loop_lib = False Use loop library to fit loops Only applicable for chain_type=PROTEINstandard_loops = True Use standard loop fitting trace_loops = False Use loop tracing to fit loops Only applicable for chain_type=PROTEINrefine_trace_loops = True Refine loops (real-space) after trace_loopsdensity_of_points = None Packing density of points to consider as as possible CA atoms in trace_loops. Try 1.0 for a quick run, up to 5 for much more thorough run If None, try ...

• build_type = *RESOLVE RESOLVE_AND_BUCCANEER You can choose to build models with RESOLVE or with RESOLVE and BUCCANEER #and TEXTAL and how many different models to build with RESOLVE. The more you build, the more likely to get a complete model. Note that rebuild_in_place can only be carried out with RESOLVE model-building. For BUCCANEER model building you need CCP4 version 6.1.2 or higher and BUCCANEER version 1.3.0 or higher

• allow_negative_residues = False Normally the wizard does not allow negative residue numbers, and all residues with negative numbers are rejected when they are read in. You can allow them if you wish.

• highest_resno = None Highest residue number to be considered "placed" in sequence for rebuild_in_place

• semet = False You can specify that the dataset that is used for refinement is a selenomethionine dataset, and that the model should be the SeMet version of the protein, with all SD of MET replaced with Se of MSE.

• use_met_in_align = *Auto True False You can use the heavy-atom positions in input_ha_file as markers for Met SD positions.

• base_model = None You can enter a PDB file with coordinates to be used as a starting point for model-building. These coordinates will be included in the same way as fragments placed by searching for helices and strand in initial model-building. Note the difference from the use of models in consider_main_chain_list, which are merged with models after they are built. NOTE: Only use this if you want to keep the input model and just add to it.

• consider_main_chain_list = None This keyword lets you name any number of PDB files to consider as templates for model-building. Every time models are built, the contents of these files will be merged with them and the best parts will be kept. NOTE: this only uses the main-chain atoms of your PDB files.

• dist_connect_max_helices = None Set maximum distance between ends of helices and other ends to try and connect them in insert_helices.

• edit_pdb = True You can choose to edit the input PDB file in rebuild_in_place to match the input sequence (default=True). NOTE: residues with residue numbers higher than 'highest_resno' are assumed to not have a known sequence and will not be edited. By default the value of 'highest_resno' is the highest residue number from the sequence file, after adding it to the starting residue number from start_chains_list. You can also set it directly

• helices_strands_only = False You can choose to use a quick model-building method that only builds secondary structure. At low resolution this may be both quicker and more accurate than trying to build the entire structure

• resolution_helices_strands = 3.1 Resolution to switch to helices_strands_only

• helices_strands_start = False You can choose to use a quick model-building method that builds secondary structure as a way to get started...then model completion is done as usual. (Contrast with helices_strands_only which only does secondary structure)

• cc_helix_min = None Minimum CC of helical density to map at low resolution when using helices_strands_only

• cc_strand_min = None Minimum CC of strand density to map when using helices_strands_only

• trim_ends_if_needed = True If a single loop is specified and no loop is found, try trimming back the ends.

• loop_backup_residues = None Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial gap Default is 4 (1 if quick)

• split_by_gap = True Split up work by gaps and run individually (normally True)

• loop_lib = False Use loop library to fit loops Only applicable for chain_type=PROTEIN

• standard_loops = True Use standard loop fitting

• trace_loops = False Use loop tracing to fit loops Only applicable for chain_type=PROTEIN

• refine_trace_loops = True Refine loops (real-space) after trace_loops

• density_of_points = None Packing density of points to consider as as possible CA atoms in trace_loops. Try 1.0 for a quick run, up to 5 for much more thorough run If None, try value depending on value of quick.

• a_cut_min = None Minimum density (relative to SD of map, normalized for solvent content) for trace_loop points

• max_density_of_points = None Maximum packing density of points to consider as as possible CA atoms in trace_loops.

• cutout_model_radius = None Radius to cut out density for trace_loops If None, guess based on length of loop

• max_cutout_model_radius = 20. Maximum value of cutout_model_radius to try

• padding = 1. Padding for cut out density in trace_loops

• cut_out_density = True Cut out density for trace_loops

• max_span = 30 Maximum length of a gap to try to fill

• max_c_ca_dist = None Maximum C-CA distance for a connection

• max_overlap_rmsd = 2. Maximum rmsd for 3 residues on each end of loop-lib fit

• max_overlap = None Maximum number of residues from ends to start with. (1=use existing ends, 2=one in from ends etc) If None, set based on value of quick.

• min_overlap = None Minimum number of residues from ends to start with. (1=use existing ends, 2=one in from ends etc)

• include_input_model = True The keyword include_input_model defines whether the input model (if any) is to be crossed with models that are derived from it, and the best parts of each kept. It also defines whether the input model is to be included in combination steps during initial model-building. Note that if multiple_models=True and include_input_model=True then no initial cycle of randomization will be carried out and the keyword multiple_models_starting_resolution is ignored. In most cases you should use include_input_model=True If you want to generate maximum diversity with multiple-models then you may wish to use include_input_model=False. Also if you want to decrease the amount of bias from your starting model you may wish to use include_input_model=False.

• input_compare_file = None If you are rebuilding a model or already think you know what the model should be, you can include a comparison file in rebuilding. The model is not used for anything except to write out information on coordinate differences in the output log files. NOTE: this feature does not always work correctly.

• merge_models = False You can choose to only merge any input models and write out the resulting model. The best parts of each model will be kept based on model-map correlation. Normally used along with number_of_parallel_models=1

• morph = False You can choose whether to distort your input model in order to match the current working map. This may be useful for MR models that are quite distant from the correct structure.

• morph_main = False You can choose whether to use only main-chain atoms plus c-beta atoms in calculation of shifts in morphing. Default is morph_main=False; use all atoms including side-chain atoms.

• dist_cut_base = 3.0 Tolerance for base pairing (A) for RNA/DNA)

• morph_cycles = 2 Number of iterations of morphing each time it is run.

• morph_rad = 7 Smoothing radius for morphing. The density from your model and from the map are calculated with the radius rad_morph, then they are adjusted to overlap optimally

• n_ca_enough_helices = None Set maximum number of CA to add to ends of helices and other ends to try and connect them in insert_helices.

• delta_phi = 20 Approximate angular sampling for search for regular secondary structure in building

• offsets_list = 53 7 23 You can specify an offset for the orientation of the helix and strand templates in building. This is used in generating different starting models.

• all_maps_in_rebuild = False If two_fofc_in_rebuild or two_fofc_denmod_in_rebuild are set you can choose to try both density-modified and two_fofc-based maps in building. Note: Set to True if you specify rebuild_from_fragments=True. Note: not compatible with map_phasing=True.

• ps_in_rebuild = False You can choose to use a prime-and-switch resolve map in all cycles of rebuilding instead of a density-modified map. This is normally used in combination with maps_only to generate a prime-and-switch map. The map coeffs will be in prime_and_switch.mtz

• use_ncs_in_ps = False You can choose to use NCS in prime-and-switch

• remove_outlier_segments_z_cut = 3.0 You can remove any segments that are not assigned to sequence during model-building if the mean density at atomic positions are more than remove_outlier_segments_z_cut sd lower than the mean for the structure.

• refine = True This script normally refines the model during building. Say False to skip refinement

• refine_final_model_vs_orig_data = True This script normally refines the model at the end against the original (non-aniso-corrected) data and writes out a CIF version of the model as well

• reference_model = None You can specify a reference model for refinement

• resolution_build = None Enter the high-resolution limit for model-building. If None, the value of resolution (or 2 A if better than 2 A) is used as a default.

• restart_cycle_after_morph = 5 Morphing (if morph=True) will go only up to this cycle, and then the morphed PDB file will be used as a starting PDB file from then on, removing all previous models. If restart_cycle_after_morph=0 then the model will be morphed and not rebuilt

• retrace_before_build = False You can choose to retrace your model n_mini times and use a map based on these retraced models to start off model-building. This is the default for rebuilding models if you are not using rebuild_in_place. You can also specify n_iter_rebuild, the number of cycles of retrace-density-modify-build before starting the main build.

• reuse_chain_prev_cycle = True You can choose to allow model-building to include atoms from each cycle in the model the next cycle or not This must be true if you use retrace_before_build

• richardson_rotamers = *Auto True False You can choose to use the rotamer library from SC Lovell, JM Word, JS Richardson and DC Richardson (2000) " The Penultimate Rotamer Library" Proteins: Structure Function and Genetics 40 389-408. if you wish. Typically this works well in RESOLVE model-building for nearly-final models but not as well earlier in the process . Default (Auto) is to use these rotamers for rebuild_in_place but not otherwise.

• rms_random_frag = None Rms random position change added to residues on ends of fragments when extending them If you enter a negative number, defaults will be used.

• rms_random_loop = None Rms random position change added to residues on ends of loops in tries for building loops If you enter a negative number, defaults will be used.

• start_chains_list = None You can specify the starting residue number for each of the unique chains in your structure. If you use a sequence file then the unique chains are extracted and the order must match the order of your starting residue numbers. For example, if your sequence file has chains A and B (identical) and chains C and D (identical to each other, but different than A and B) then you can enter 2 numbers, the starting residues for chains A and C. NOTE: you need to specify an input sequence file for start_chains_list to be applied.

• trace_as_lig = False You can specify that in building steps the ends of chains are to be extended using the LigandFit algorithm. This is default for nucleic acid model-building.

• track_libs = False You can keep track of what libraries each atom in a built structure comes from.

• two_fofc_denmod_in_rebuild = False You can choose to use a density-modified sigmaa-weighted 2Fo-Fc map in all cycles of rebuilding instead of a density-modified map. In density modification the density in the region defined by the current model will be truncated at +2sigma to reduce the dominance of parts of the map with model defined. Additionaly only 2 mask cycles of 3 minor cycles will be done. Additionally place_waters will be turned off. If the model is highly incomplete this can sometimes allow model-building to work even when it will not for density-modified maps. The map coeffs will be in two_fofc_denmod_map.mtz. You might consider turning on all_maps_in_rebuild as well. Note: Setting two_fofc_denmod_in_rebuild=True will by default set place_waters=False. Set to True if you specify rebuild_from_fragments=True.

• rebuild_from_fragments = False You can use rebuild_from_fragments=True as a shortcut to turn on two_fofc_denmod_in_rebuild and all_maps_in_rebuild and to use your model in each cycle with consider_main_chain_list. If you use rebuild_from_fragments=True you might also want to set i_ran_seed=xxxxx for some integer xxxxx and run the job 10 or 20 times to have a higher chance of success. This approach is designed for cases where you have a small part of your model very accurately placed and want to build the rest of the model.

• two_fofc_in_rebuild = False You can choose to use a sigmaa-weighted 2Fo-Fc map in all cycles of rebuilding instead of a density-modified map. If the model is poor this can sometimes allow model-building in place to work even when it will not for density-modified maps. This is also useful if you specify a twin law.

• refine_map_coeff_labels = "2FOFCWT PH2FOFCWT" You can pick which map coefficients from phenix.refine to use if two_fofc_in_rebuild=True

• filled_2fofc_maps = True You can choose to use filled 2Fo-Fc maps when two_fofc_in_rebuild is used. Default is True

• map_phasing = False You can choose to use statistical density modification starting with a 2mFo-DFc map, including model information instead of a standard density-modified map with model information. This density modification will include NCS if present. Note: not compatible with all_maps_in_rebuild=True

• use_any_side = True You can choose to have resolve model-building place the best-fitting side chain at each position, even if the sequence is not matched to the map.

• truncate_missing_side_chains = None You can choose to truncate side chains with missing density at CB or equivalent. Missing density means average density at the side chain atoms is less than zero. Default is False for rebuild_in_place=True and True for rebuild_in_place=False.

• use_cc_in_combine_extend = False You can choose to use the correlation of density rather than density at atomic positions to score models in combine_extend

• sort_hetatms = False Waters are automatically named with the chain of the closest macromolecule if you set sort_hetatms=True This is for the final model only.

• map_to_object = None you can supply a target position for your model with map_to_object=my_target.pdb. Then at the very end your molecule will be placed as close to this as possible. The center of mass of the autobuild model will be superimposed on the center of mass of my_target.pdb using space group symmetry, taking any match closer than 15 A within 3 unit cells of the original position. The new file will be overall_best_mapped.pdb

• multiple_modelscombine_only = False Once you have created a set of initial models you can merge them together into a final set. This option is useful if you have split up the creation of multiple models into different directories, and then you have copied all the initial models to one directory for combining. multiple_models = False You can build a set of models, all compatible with your data. You can specify how many models with multiple_models_number. If you are using rebuild_in_place you can specify whether to generate starting models or not with multiple_models_starting. multiple_models_first = 1 Specify which model to build first multiple_models_group_number = 5 You can build several initial models and merge them. Normally 5 initial models is fine. multiple_models_last = 20 Specify which model to end with multiple_models_number = 20 Specify how many models to build. multiple_models_starting = True You can specify how to generate starting models for multiple models. If you are using rebuild_in_place and you specify "True" then the Wizard will rebuild your starting model at the resolution specified in multiple_models_starting_resolution. If you are not using rebuild_in_place the Wizard will always build a starting model at the current resolution. multiple_models_starting_resolution = 4 You can set the resolution for rebuilding an initial model. A value of 0.0 will use the resolution of the dataset. place_waters_in_combine = None You can choose whether phenix.refine automatically places ordered solvent (waters) during the last cycle of multiple-model generation. This is separate from place_waters, which applies to all other cycles. If None, then value of place_waters will be used.

• combine_only = False Once you have created a set of initial models you can merge them together into a final set. This option is useful if you have split up the creation of multiple models into different directories, and then you have copied all the initial models to one directory for combining.

• multiple_models = False You can build a set of models, all compatible with your data. You can specify how many models with multiple_models_number. If you are using rebuild_in_place you can specify whether to generate starting models or not with multiple_models_starting.

• multiple_models_first = 1 Specify which model to build first

• multiple_models_group_number = 5 You can build several initial models and merge them. Normally 5 initial models is fine.

• multiple_models_last = 20 Specify which model to end with

• multiple_models_number = 20 Specify how many models to build.

• multiple_models_starting = True You can specify how to generate starting models for multiple models. If you are using rebuild_in_place and you specify "True" then the Wizard will rebuild your starting model at the resolution specified in multiple_models_starting_resolution. If you are not using rebuild_in_place the Wizard will always build a starting model at the current resolution.

• multiple_models_starting_resolution = 4 You can set the resolution for rebuilding an initial model. A value of 0.0 will use the resolution of the dataset.

• place_waters_in_combine = None You can choose whether phenix.refine automatically places ordered solvent (waters) during the last cycle of multiple-model generation. This is separate from place_waters, which applies to all other cycles. If None, then value of place_waters will be used.

• ncsfind_ncs = *Auto True False This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. The update is done each cycle in which an improved model is obtained. Say False to skip this. See also "input_ncs_file" which can be used to specify NCS at the start of the process. If find_ncs="No" then only this starting NCS will be used and it will not be updated. You can use find_ncs "No" to specify exactly what residues will be used in NCS refinement and exactly what NCS operators to use in density modification. You can use the function $PHENIX/phenix/phenix/command_line/simple_ncs_from_pdb.py to help you set up an input_ncs_file that has your specifications in it. NOTE: if an input map_file is provided then if no ncs is

found from a model, ncs will be searched for in the density of that map.input_ncs_file = None You can enter NCS information in 3 ways: (1) an ncs_spec file produced by AutoSol or AutoBuild with NCS information (2) a heavy-atom PDB file that contains ncs in the heavy-atom sites (3) a PDB file with a model that contains chains with NCS The wizard will derive NCS information from any of these if specified. See also "find_ncs" which determines whether the wizard will update NCS from models that are built during iterative building. ncs_copies = None Number of copies of the molecule in the au (note: only one type of molecule allowed at present) ncs_refine_coord_sigma_from_rmsd = False You can choose to use the current NCS rmsd as the value of the sigma for NCS restraints. See also ncs_refine_coord_sigma_from_rmsd_ratio ncs_refine_coord_sigma_from_rmsd_ratio = 1 You can choose to multiply the current NCS rmsd by this value before using it as the sigma for NCS restraints See also ncs_refine_coord_sigma_from_rmsd no_merge_ncs_copies = False Normally False (do merge NCS copies). If True, then do not use each NCS copy to try to build the others. optimize_ncs = True This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. Optimize NCS adds a step to try and make the molecule formed by NCS as compact as possible, without losing any point-group symmetry. use_ncs_in_build = True Use NCS information in the model assembly stage of model-building. Also if no_merge_ncs_copies is not set, then use each NCS copy to try to build the others. ncs_in_refinement = *torsion cartesian None Use torsion_angle refinement of NCS. Alternative is cartesian or None (None will use phenix.refine default)

• find_ncs = *Auto True False This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. The update is done each cycle in which an improved model is obtained. Say False to skip this. See also "input_ncs_file" which can be used to specify NCS at the start of the process. If find_ncs="No" then only this starting NCS will be used and it will not be updated. You can use find_ncs "No" to specify exactly what residues will be used in NCS refinement and exactly what NCS operators to use in density modification. You can use the function $PHENIX/phenix/phenix/command_line/simple_ncs_from_pdb.py to help you set up an input_ncs_file that has your specifications in it. NOTE: if an input map_file is provided then if no ncs is found from a model, ncs will be searched for in the density of that map.

• input_ncs_file = None You can enter NCS information in 3 ways: (1) an ncs_spec file produced by AutoSol or AutoBuild with NCS information (2) a heavy-atom PDB file that contains ncs in the heavy-atom sites (3) a PDB file with a model that contains chains with NCS The wizard will derive NCS information from any of these if specified. See also "find_ncs" which determines whether the wizard will update NCS from models that are built during iterative building.

• ncs_copies = None Number of copies of the molecule in the au (note: only one type of molecule allowed at present)

• ncs_refine_coord_sigma_from_rmsd = False You can choose to use the current NCS rmsd as the value of the sigma for NCS restraints. See also ncs_refine_coord_sigma_from_rmsd_ratio

• ncs_refine_coord_sigma_from_rmsd_ratio = 1 You can choose to multiply the current NCS rmsd by this value before using it as the sigma for NCS restraints See also ncs_refine_coord_sigma_from_rmsd

• no_merge_ncs_copies = False Normally False (do merge NCS copies). If True, then do not use each NCS copy to try to build the others.

• optimize_ncs = True This script normally deduces ncs information from the NCS in chains of models that are built during iterative model-building. Optimize NCS adds a step to try and make the molecule formed by NCS as compact as possible, without losing any point-group symmetry.

• use_ncs_in_build = True Use NCS information in the model assembly stage of model-building. Also if no_merge_ncs_copies is not set, then use each NCS copy to try to build the others.

• ncs_in_refinement = *torsion cartesian None Use torsion_angle refinement of NCS. Alternative is cartesian or None (None will use phenix.refine default)

• omitcomposite_omit_type = *None simple_omit refine_omit sa_omit iterative_build_omit Your choices of types of OMIT maps are: None - normal operation, no omit simple_omit - omit the atoms in OMIT region in calculating a sigmaA-weighted 2mFo-DFc map with no refinement. refine_omit - as simple_omit, but refine with standard refinement. sa_omit - omit the atoms in OMIT region, carry out simulated-annealing refinement, then calculate a sigmaA-weighted 2mFo-DFc map. iterative_build_omit - set occupancy of atoms in OMIT region to 0 throughout an entire iterative model-building, density modification and refinement process (takes a long time). All these omit map types are available as composite omit maps (default) or as omit maps around a region defined by a PDB file (using omit_box_pdb_list) The resulting OMIT map will be in the directory OMIT with file name resolve_composite_map.mtz . This mtz file contains the map coefficients to create the OMIT map. The file "omit_region.mtz" contains the coefficients for a map showing the boundaries of the OMIT region. n_box_target = None You can tell the Wizard how many omit boxes to try and set up (but it will not necessarily choose your number because it has to be nicely divisible into boxes that fit your asymmetric unit). A suitable number is 24. The larger the number of boxes, the better the map will be, but the longer it will take to calculate the map. n_cycle_image_min = 3 Pattern recognition (resolve_pattern) and fragment identification ("image based density modification") are used as part of the density modification process. These are normally only useful in the first few cycles of iterative model-building. This script tries model-building both with and without including image information, and proceeds with the most complete model. Once at least n_cycle_image_min cycles have been carried out with image information, if the image-based map results in a less-complete model than the one without image information, image information is no longer included. n_cycle_rebuild_omit = 10 Model-building is normally carried out using the "best" available map. If omit_on_rebuild is True, then every n_cycle_rebuild_omit cycle of model rebuilding, a composite omit map is used instead. If you specify 0 and omit_on_rebuild is True, omit maps will be used every cycle. Normally every 10th cycle is optimal. offset_boundary = 2. Specify the boundary in A around atoms in omit_box_pdb for definition of omit region. Contrast with omit_boundary which applies for composite omitomit_boundary = 2. Specify the boundary in A around atoms in omit_boxes for definition of omit region. Contrast with offset_boundary which applies for omit_box_pdbomit_box_start = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end omit_box_end = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end omit_box_pdb_list = None This keyword applies if you have set OMIT region specification to "omit_around_pdb". To automatically set an OMIT region specify a PDB file(s) with omit_box_pdb_list. The omit region boundaries will be the limits in x y z of the atoms in this file, plus a border of offset_boundary. To use only some of the atoms in the file, specify values for starting, ending and chain to omit (omit_res_start_list and omit_res_end_list and omit_chain_list) If you specify more than one file (or if you specify more than one segment of a file with omit_chain_list or omit_res_start_list and omit_res_end_list) then a set of omit runs will be carried out and combined into one composite omit. omit_chain_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use "" to select all chains) The residues from 3 to 4 of chain1 will be omitted. You can specify more than one region by listing them separated by spaces ...

• composite_omit_type = *None simple_omit refine_omit sa_omit iterative_build_omit Your choices of types of OMIT maps are: None - normal operation, no omit simple_omit - omit the atoms in OMIT region in calculating a sigmaA-weighted 2mFo-DFc map with no refinement. refine_omit - as simple_omit, but refine with standard refinement. sa_omit - omit the atoms in OMIT region, carry out simulated-annealing refinement, then calculate a sigmaA-weighted 2mFo-DFc map. iterative_build_omit - set occupancy of atoms in OMIT region to 0 throughout an entire iterative model-building, density modification and

refinement process (takes a long time). All these omit map types are available as composite omit maps (default) or as omit maps around a region defined by a PDB file (using omit_box_pdb_list) The resulting OMIT map will be in the directory OMIT with file name resolve_composite_map.mtz . This mtz file contains the map coefficients to create the OMIT map. The file "omit_region.mtz" contains the coefficients for a map showing the boundaries of the OMIT region.

• n_box_target = None You can tell the Wizard how many omit boxes to try and set up (but it will not necessarily choose your number because it has to be nicely divisible into boxes that fit your asymmetric unit). A suitable number is 24. The larger the number of boxes, the better the map will be, but the longer it will take to calculate the map.

• n_cycle_image_min = 3 Pattern recognition (resolve_pattern) and fragment identification ("image based density modification") are used as part of the density modification process. These are normally only useful in the first few cycles of iterative model-building. This script tries model-building both with and without including image information, and proceeds with the most complete model. Once at least n_cycle_image_min cycles have been carried out with image information, if the image-based map results in a less-complete model than the one without image information, image information is no longer included.

• n_cycle_rebuild_omit = 10 Model-building is normally carried out using the "best" available map. If omit_on_rebuild is True, then every n_cycle_rebuild_omit cycle of model rebuilding, a composite omit map is used instead. If you specify 0 and omit_on_rebuild is True, omit maps will be used every cycle. Normally every 10th cycle is optimal.

• offset_boundary = 2. Specify the boundary in A around atoms in omit_box_pdb for definition of omit region. Contrast with omit_boundary which applies for composite omit

• omit_boundary = 2. Specify the boundary in A around atoms in omit_boxes for definition of omit region. Contrast with offset_boundary which applies for omit_box_pdb

• omit_box_start = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end

• omit_box_end = 0 To only carry out omit in some of the omit boxes, use omit_box_start and omit_box_end

• omit_box_pdb_list = None This keyword applies if you have set OMIT region specification to "omit_around_pdb". To automatically set an OMIT region specify a PDB file(s) with omit_box_pdb_list. The omit region boundaries will be the limits in x y z of the atoms in this file, plus a border of offset_boundary. To use only some of the atoms in the file, specify values for starting, ending and chain to omit (omit_res_start_list and omit_res_end_list and omit_chain_list) If you specify more than one file (or if you specify more than one segment of a file with omit_chain_list or omit_res_start_list and omit_res_end_list) then a set of omit runs will be carried out and combined into one composite omit.

• omit_chain_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use "" to select all chains) The residues from 3 to 4 of chain1 will be omitted. You can specify more than one region by listing them separated by spaces If you specify more than one region, a separate omit run will be carried out for each one and then the maps will be put together afterwards. If there are more than one chains in the input PDB file then only the chain defined by omit_chain will be omitted NOTE: Zero for start and end and "" for chain is the same as choosing everything

• omit_offset_list = 0 0 0 0 0 0 To carry out one iterative build omit, with a region defined in grid units, enter nxs,nxe,nys,nye,nzs,nze in omit_offset_list.

• omit_on_rebuild = False You can specify whether to use an omit map for building the model on rebuild cycles. Default is True if you start with a model, False if you are building a model from scratch. The omit map is calculated every n_cycle_rebuild_omit cycles

• omit_selection = None Selection string defining atoms in input pdb to be used to define the OMIT region. For use with omit_region_specification=omit_selection

• omit_region_specification = *composite_omit omit_around_pdb omit_selection You can specify what region an omit (simple/sa-omit/iterative-build-omit) map is to be calculated for. Composite omit will create a map over the entire asymmetric unit by dividing the asymmetric unit into overlapping boxes, calculating omit maps for each, and splicing all the results together into a single composite omit map. You can tell the Wizard how many omit boxes to try and set up with the keyword "n_box_target" (but it will not necessarily choose your number because it has to be nicely divisible into boxes that fit your asymmetric unit). Omit around PDB will omit around the region defined by the PDB file(s) you enter for omit_box_pdb (or around the residues in that PDB file that you specify). If you specify omit_around_pdb then you must enter a pdb file to omit around. If you specify omit_selection you must enter a selection string in omit_selection

• omit_res_start_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be omitted. You can specify more more than one region by listing them separated by spaces If you specify more than one region, a separate omit run will be carried out for each one and then the maps will be put together afterwards. If there are more than one chains in the input PDB file then only the chain defined by omit_chain will be rebuilt. NOTE: Zero for start and end and "" for chain is the same as choosing everything

• omit_res_end_list = None You can choose to omit just a portion of your model keywords omit_res_start_list 3 omit_res_end_list 4 omit_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be omitted. You can specify more more than one region by listing them separated by spaces If you specify more than one region, a separate omit run will be carried out for each one and then the maps will be put together afterwards. If there are more than one chains in the input PDB file then only the chain defined by omit_chain will be omitted NOTE: Zero for start and end and "" for chain is the same as choosing everything

• rebuild_in_placemin_seq_identity_percent_rebuild_in_place = 95 Minimum sequence identity to use rebuild_in_place by defaultn_cycle_rebuild_in_place = None Number of cycles for rebuild_in_place for multiple models only n_rebuild_in_place = 1 You can choose how many times to rebuild your model in place with rebuild_in_place rebuild_chain_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.rebuild_in_place = *Auto True False You can choose to rebuild your model while fixing the sequence alignment by iteratively rebuilding segments within the model. This is done n_rebuild_in_place times, then the models are recombined, taking the best-fitting parts of each. Crossovers allowed where main-chain atom rmsd is less than dist_close. Note that the sequence of the input model must match the supplied sequence closely enough to allow a clear alignment. Also this method does not build any new chain, it just moves the existing model around. Normally this procedure is useful if the model is greater than 95% identical with the target sequence. You can include information directly from the starting model if you want with the keyword include_input_model. Then this model will be recombined with the models that are built based on it. Note that this requires that the input model have a sequence that is identical to the model to be rebuilt. You can also rebuild just a portion of the model with

the keywords keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank) The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines NOTE: if a region cannot be rebuilt the original coordinates will be preserved for that region. rebuild_near_chain = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist. rebuild_near_dist = 7.5 You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist. rebuild_near_res = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist. rebuild_res_end_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.rebuild_res_start_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.rebuild_side_chains = False You can choose to replace side chains (with extend_only) before rebuilding the model (...

• min_seq_identity_percent_rebuild_in_place = 95 Minimum sequence identity to use rebuild_in_place by default

• n_cycle_rebuild_in_place = None Number of cycles for rebuild_in_place for multiple models only

• n_rebuild_in_place = 1 You can choose how many times to rebuild your model in place with rebuild_in_place

• rebuild_chain_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.

• rebuild_in_place = *Auto True False You can choose to rebuild your model while fixing the sequence alignment by iteratively rebuilding segments within the model. This is done n_rebuild_in_place times, then the models are recombined, taking the best-fitting parts of each. Crossovers allowed where main-chain atom rmsd is less than dist_close. Note that the sequence of the input model must match the supplied sequence closely enough to allow a clear alignment. Also this method does not build any new chain, it just moves the existing model around. Normally this procedure is useful if the model is greater than 95% identical with the target sequence. You can include information directly from the starting model if you want with the keyword include_input_model. Then this model will be recombined with the models that are built based on it. Note that this requires that the input model have a sequence that is identical to the model to be rebuilt. You can also rebuild just a portion of the model with the keywords keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank) The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines NOTE: if a region cannot be rebuilt the original coordinates will be preserved for that region.

• rebuild_near_chain = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist.

• rebuild_near_dist = 7.5 You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist.

• rebuild_near_res = None You can specify where to rebuild either with rebuild_res_start_list rebuild_res_end_list rebuild_chain_list or with rebuild_near_res and rebuild_near_chain and rebuild_near_dist.

• rebuild_res_end_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.

• rebuild_res_start_list = None You can choose to rebuild just a portion of your model keywords rebuild_res_start_list 3 rebuild_res_end_list 4 rebuild_chain_list chain1 (use " " for blank). The residues from 3 to 4 of chain1 will be rebuilt. You can specify more than one region by using the Parameter Group Options button to add lines. If there are more than one chains in the input PDB file then only the chain defined by rebuild_chain will be rebuilt. The smallest region that can be rebuilt is 4 residues.

• rebuild_side_chains = False You can choose to replace side chains (with extend_only) before rebuilding the model (not normally used)

• redo_side_chains = True You can chooses to have AutoBuild choose whether to replace all your side chains in rebuild_in_place, taking new ones if they fit the density better. If True, this is applied to all side chains, not only those that are rebuilt.

• replace_existing = True In rebuild_in_place the usual default is to force the replacement of all residues, even if the rebuilt ones are not as good a fit as the original. The rebuilt model is then crossed with the original model (if include_input_model=True) and the better parts of each are then kept. You can override the replacement of all residues in the initial model-building by saying "False" (do not force replacement of residues, keep whatever is better). Additionally if you set the "touch_up" flag then the default is "True": keep whatever is better.

• delete_bad_residues_only = False You can simply delete the worst parts of your model and write out the resulting model with delete_bad_residues_only=True The criteria used are the ones set with touch_up. Any residues that would be rebuild by touch_up=True will be deleted by delete_bad_residues_only. NOTE: delete_bad_residues_only ignores ligands, waters etc. so you may need to put them back in afterwards.

• touch_up = False You can rebuild just the worst parts of your model by setting touch_up=True. You can decide what parts to rebuild based on an minimum model-map correlation (by residue). This is set with min_cc_residue_rebuild=0.82 Alternatively you can rebuild the worst percentage of these: worst_percent_res_rebuild=6. If a value is set for both of these then residues qualifying in either way are rebuilt. NOTE: touch_up is only available with rebuild_in_place.

• touch_up_extra_residues = None Number of residues on each side of the residues identified in touch_up that you want to rebuild. Normally you will want to rebuild one or more on each side.

• worst_percent_res_rebuild = 2 You can rebuild just the worst parts of your model by setting touch_up=True. You can decide how much to rebuild using worst_percent_res_rebuild or with min_cc_res_rebuild, or both.

• smooth_range = None You can specify what number of residues to smooth in making choices for touch_up and delete_bad_residues_only Typically use 3 or 5.

• smooth_minimum_length = None If specified, then any segments remaining after smoothing that are shorter than smooth_mininum_length will be removed.

• refinementrefine_b = True You can choose whether phenix.refine is to refine individual atomic displacement parameters (B values) refine_se_occ = True You can choose to refine the occupancy of SE atoms in a SEMET structure (default=True). This only applies if semet=true skip_clash_guard = True Skip refinement check for atoms that clash Also skips check for side chains crossing special position (allow_polymer_cross_special_position=True in refinement)correct_special_position_tolerance = None Adjust tolerance for special position check. If 0., then check for clashes near special positions is not carried out. This sometimes allows phenix.refine to continue even if an atom is near a special position. If 1., then checks within 1 A of special positions. If None, then uses phenix.refine default. (1)use_mlhl = True This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to only refine on Fobs generate_hl_if_missing = False This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to not generate HL coeffs from input phases.place_waters = True You can choose whether phenix.refine automatically places ordered solvent (waters) during the refinement process. refinement_resolution = 0 Enter the high-resolution limit for refinement only. This high-resolution limit can be different than the high-resolution limit for other steps. The default ("None" or 0.0) is to use the overall high-resolution limit for this run (as set by resolution) ordered_solvent_low_resolution = None You can choose what resolution cutoff to use fo placing ordered solvent in phenix.refine. If the resolution of refinement is greater than this cutoff, then no ordered solvent will be placed, even if refinement.main.ordered_solvent=True. link_distance_cutoff = 3 You can specify the maximum bond distance for linking residues in phenix.refine called from the wizards. r_free_flags_fraction = 0.1 Maximum fraction of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters. r_free_flags_max_free = 2000 Maximum number of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters. r_free_flags_use_lattice_symmetry = True When generating r_free_flags you can decide whether to include lattice symmetry (good in general, necessary if there is twinning). r_free_flags_lattice_symmetry_max_delta = 5 You can set the maximum deviation of distances in the lattice that are to be considered the same for purposes of generating a lattice-symmetry-unique set of free R flags. allow_overlapping = None Default is None (set automatically, normally False unless S or Se atoms are the anomalously-scattering atoms). You can allow atoms in your ligand files to overlap atoms in your protein/nucleic acid model. This overrides 'keep_pdb_atoms' Useful in early stages of model-building and refinement The ligand atoms get the altloc indicator 'L' NOTE: The ligand occupancy will be refined by default if you set allow_overlapping=True (because of the altloc indicator) You can turn this off with fix_ligand_occupancy=Truefix_ligand_occupancy = None If allow_overlapping=True then ligand occupancies are refined as a group. You can turn this off with fix_ligand_occupancy=true NOTE: has no effect if allow_overlapping=Falseremove_outlier_segments = True You can remove any segments that are not assigned to sequence if their mean B values are more than remove_outlier_segments_z_cut sd higher than the mean for the structure. NOTE: this is done after refinement, so the R/Rfree are no longer applicable; the remarks...

• refine_b = True You can choose whether phenix.refine is to refine individual atomic displacement parameters (B values)

• refine_se_occ = True You can choose to refine the occupancy of SE atoms in a SEMET structure (default=True). This only applies if semet=true

• skip_clash_guard = True Skip refinement check for atoms that clash Also skips check for side chains crossing special position (allow_polymer_cross_special_position=True in refinement)

• correct_special_position_tolerance = None Adjust tolerance for special position check. If 0., then check for clashes near special positions is not carried out. This sometimes allows phenix.refine to continue even if an atom is near a special position. If 1., then checks within 1 A of special positions. If None, then uses phenix.refine default. (1)

• use_mlhl = True This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to only refine on Fobs

• generate_hl_if_missing = False This script normally uses information from the input file (HLA HLB HLC HLD) in refinement. Say No to not generate HL coeffs from input phases.

• place_waters = True You can choose whether phenix.refine automatically places ordered solvent (waters) during the refinement process.

• refinement_resolution = 0 Enter the high-resolution limit for refinement only. This high-resolution limit can be different than the high-resolution limit for other steps. The default ("None" or 0.0) is to use the overall high-resolution limit for this run (as set by resolution)

• ordered_solvent_low_resolution = None You can choose what resolution cutoff to use fo placing ordered solvent in phenix.refine. If the resolution of refinement is greater than this cutoff, then no ordered solvent will be placed, even if refinement.main.ordered_solvent=True.

• link_distance_cutoff = 3 You can specify the maximum bond distance for linking residues in phenix.refine called from the wizards.

• r_free_flags_fraction = 0.1 Maximum fraction of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters.

• r_free_flags_max_free = 2000 Maximum number of reflections in the free R set. You can choose the maximum fraction of reflections in the free R set and the maximum number of reflections in the free R set. The number of reflections in the free R set will be up the lower of the values defined by these two parameters.

• r_free_flags_use_lattice_symmetry = True When generating r_free_flags you can decide whether to include lattice symmetry (good in general, necessary if there is twinning).

• r_free_flags_lattice_symmetry_max_delta = 5 You can set the maximum deviation of distances in the lattice that are to be considered the same for purposes of generating a lattice-symmetry-unique set of free R flags.

• allow_overlapping = None Default is None (set automatically, normally False unless S or Se atoms are the anomalously-scattering atoms). You can allow atoms in your ligand files to overlap atoms in your protein/nucleic acid model. This overrides 'keep_pdb_atoms' Useful in early stages of model-building and refinement The ligand atoms get the altloc indicator 'L' NOTE: The ligand occupancy will be refined by default if you set allow_overlapping=True (because of the altloc indicator) You can turn this off with fix_ligand_occupancy=True

• fix_ligand_occupancy = None If allow_overlapping=True then ligand occupancies are refined as a group. You can turn this off with fix_ligand_occupancy=true NOTE: has no effect if allow_overlapping=False

• remove_outlier_segments = True You can remove any segments that are not assigned to sequence if their mean B values are more than remove_outlier_segments_z_cut sd higher than the mean for the structure. NOTE: this is done after refinement, so the R/Rfree are no longer applicable; the remarks in the PDB file are removed

• twin_law = None You can specify a twin law for refinement like this: twin_law='-h,k,-l'

• max_occ = None You can choose to set the maximum value of occupancy for atoms that have their occupancies refined. Default is None (use default value of 1.0 from phenix.refine)

• refine_before_rebuild = True You can choose to refine the input model before rebuilding it

• refine_with_ncs = True This script can allow phenix.refine to automatically identify NCS and use it in refinement. NOTE: ncs refinement and placing waters automatically are mutually exclusive at present.

• refine_xyz = True You can choose whether phenix.refine is to refine coordinates

• s_annealing = False You can choose to carry out simulated annealing during the first refinement after initial model-building

• skip_hexdigest = False You may wish to ignore the hexdigest of the free R flags in your input PDB file if (1) the dataset you provide is not identical to the one that you refined with (but has the same free R flags), or (2) you are providing both an input_data_file and an input_refinement_file or input_hires_file and. In the second case, the resulting composite file may not have the same hexdigest even though the free R flags are copied over. The default is to set skip_hexdigest=True for case #2. For case #1 you have to tell the Wizard to skip the hexdigest (because it cannot know about this).

• use_hl_anom_in_refinement = False See use_hl_anom_in_denmod. If use_hl_anom_in_refinement=True then the HLanom HL coefficients from Phaser are used in refinement

• use_hl_if_present = True You can turn off use of HL coeffs in refinement if you want. Default is to use them if present. Requires PHIB as well.

• thoroughnessbuild_outside = True Define whether to use the BuildOutside module in model_buildingconnect = True Define whether to use the connect module in model_building. This module tries to connect nearby chains with loops, without using the sequence. This is different than fit_loops (which uses the sequence to identify the exact number of residues in the loop). extensive_build = False You can choose whether to build a new model on every cycle and carry out extra model-building steps every cycle. Default is False (build a new model on first cycle, after that carry out extra steps). fit_loops = True You can fit loops automatically if sequence alignment has been done. insert_helices = True Define whether to use the insert_helices module in model_building. This module tries to insert helices identified with find_helices_strands into the current working model. This can be useful as the standard build sometimes builds strands into helical density at low resolution. n_cycle_build = None Choose number of cycles of building and chain extension during each cycle of model-building. (default of 1). n_cycle_build_max = 6 Maximum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is not found, a maximum of n_cycle_build_max cycles will be carried out. n_cycle_build_min = 1 Minimum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is found, n_cycle_build_min cycles will be carried out. n_cycle_rebuild_max = 15 Maximum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is not found, a maximum of n_cycle_rebuild_max cycles will be carried out. n_cycle_rebuild_min = 1 Mininum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is found,

n_cycle_rebuild_min cycles will be carried out. n_mini = 10 You can choose how many times to retrace your model in "retrace_before_build" n_random_frag = 0 In resolve building you can randomize each fragment slightly so as to generate more possibilities for tracing based on extending it. n_random_loop = 3 Number of randomized tries from each end for building loops If 0, then one try. If N, then N additional tries with randomization based on rms_random_loop. n_try_rebuild = 2 Number of attempts to build each segment of chain ncycle_refine = 3 Choose number of refinement cycles (3) number_of_models = None This parameter lets you choose how many initial models to build with RESOLVE within a single build cycle. This parameter is now superseded by number_of_parallel_models, which sets the number of models (but now entire build cycles) to carry out in parallel. None or zero means set it automatically. That is what you normally should use. The number_of_models is by default set to 1 and number_of_parallel_models is set to the value of nbatch (typically 4). number_of_parallel_models = 0 This parameter lets you choose how many models to build in parallel. None or 0 means set it automatically. That is what you normally should use. You can set this to 1 to prevent the wizard from running multiple jobs in parallelskip_combine_extend = False You can choose whether to skip the combine-extend step in model-building if only one model is availablefully_skip_combine_extend = False You can choose whether to skip the combine-extend step in model-building in all casesthorough_loop_fit = True Try many conformations and accept them even if the fit is not perfect? If you say True the parameters for thorough loop fitting are: n_random_loop=100 rms_random_loop=0.3 rho_min_main=0.5 while if you say False those for quick loop fitting are: n_random_loop=20 rms_random_loop=0.3 rho_min_main=1.0

• build_outside = True Define whether to use the BuildOutside module in model_building

• connect = True Define whether to use the connect module in model_building. This module tries to connect nearby chains with loops, without using the sequence. This is different than fit_loops (which uses the sequence to identify the exact number of residues in the loop).

• extensive_build = False You can choose whether to build a new model on every cycle and carry out extra model-building steps every cycle. Default is False (build a new model on first cycle, after that carry out extra steps).

• fit_loops = True You can fit loops automatically if sequence alignment has been done.

• insert_helices = True Define whether to use the insert_helices module in model_building. This module tries to insert helices identified with find_helices_strands into the current working model. This can be useful as the standard build sometimes builds strands into helical density at low resolution.

• n_cycle_build = None Choose number of cycles of building and chain extension during each cycle of model-building. (default of 1).

• n_cycle_build_max = 6 Maximum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is not found, a maximum of n_cycle_build_max cycles will be carried out.

• n_cycle_build_min = 1 Minimum number of cycles for iterative model-building, starting from experimental phases without a model. Even if a satisfactory model is found, n_cycle_build_min cycles will be carried out.

• n_cycle_rebuild_max = 15 Maximum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is not found, a maximum of n_cycle_rebuild_max cycles will be carried out.

• n_cycle_rebuild_min = 1 Mininum number of cycles for iterative model-rebuilding, starting from a model. Even if a satisfactory model is found, n_cycle_rebuild_min cycles will be carried out.

• n_mini = 10 You can choose how many times to retrace your model in "retrace_before_build"

• n_random_frag = 0 In resolve building you can randomize each fragment slightly so as to generate more possibilities for tracing based on extending it.

• n_random_loop = 3 Number of randomized tries from each end for building loops If 0, then one try. If N, then N additional tries with randomization based on rms_random_loop.

• n_try_rebuild = 2 Number of attempts to build each segment of chain

• ncycle_refine = 3 Choose number of refinement cycles (3)

• number_of_models = None This parameter lets you choose how many initial models to build with RESOLVE within a single build cycle. This parameter is now superseded by number_of_parallel_models, which sets the number of models (but now entire build cycles) to carry out in parallel. None or zero means set it automatically. That is what you normally should use. The number_of_models is by default set to 1 and number_of_parallel_models is set to the value of nbatch (typically 4).

• number_of_parallel_models = 0 This parameter lets you choose how many models to build in parallel. None or 0 means set it automatically. That is what you normally should use. You can set this to 1 to prevent the wizard from running multiple jobs in parallel

• skip_combine_extend = False You can choose whether to skip the combine-extend step in model-building if only one model is available

• fully_skip_combine_extend = False You can choose whether to skip the combine-extend step in model-building in all cases

• thorough_loop_fit = True Try many conformations and accept them even if the fit is not perfect? If you say True the parameters for thorough loop fitting are: n_random_loop=100 rms_random_loop=0.3 rho_min_main=0.5 while if you say False those for quick loop fitting are: n_random_loop=20 rms_random_loop=0.3 rho_min_main=1.0

• generalcoot_name = "coot" If your version of coot is called something else, then you can specify that here. i_ran_seed = 72432 Random seed (positive integer) for model-building and simulated annealing refinement raise_sorry = False You can have any failure end with a Sorry instead of simply printout to the screenbackground = True When you specify nproc=nn, you can run the jobs in background (default if nproc is greater than 1) or foreground (default if nproc=1). If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If you use run_command='sh ' (or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously. check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to endmax_wait_time = 1.0 You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....cache_resolve_libs = True Use caching of resolve libraries to speed up resolveresolve_size = 12 Size for solve/resolve ("","_giant", "_huge","_extra_huge" or a number where 12=giant 18=hugecheck_run_command = False You can have the wizard check your run command at startuprun_command = "sh " When you specify nproc=nn, you can run the subprocesses as jobs in background with sh (default) or submit them to a queue with the command of your choice (i.e., qsub). If

you have a multi-processor machine, use sh. If you have a cluster, use qsub or the equivalent command for your system. NOTE: If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If nproc is greater than 1 and you use run_command='sh '(or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously. queue_commands = None You can add any commands that need to be run for your queueing system. These are written before any other commands in the file that is submitted to your queueing system. For example on a PBS system you might say: queue_commands='#PBS -N mr_rosetta' queue_commands='#PBS -j oe' queue_commands='#PBS -l walltime=03:00:00' queue_commands='#PBS -l nodes=1:ppn=1' NOTE: you can put in the characters '<path>' in any queue_commands line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_',up to 15 characters. For example 'test_autobuild/WORK_5/AutoBuild_run_1_/TEMP0/RUN_1' would be represented by: 'tld_W_5_A1__TP0_1' condor_universe = vanilla The universe for condor is usually vanilla. However you might need to set it to local for your clusteradd_double_quotes_in_condor = True You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.condor = None Specifies if the group_run_command is submitting a job to a condor cluster. Set by default to True if group_run_command=condor_submit, otherwise False. For condor job submission mr_rosetta uses a customized script wit...

• coot_name = "coot" If your version of coot is called something else, then you can specify that here.

• i_ran_seed = 72432 Random seed (positive integer) for model-building and simulated annealing refinement

• raise_sorry = False You can have any failure end with a Sorry instead of simply printout to the screen

• background = True When you specify nproc=nn, you can run the jobs in background (default if nproc is greater than 1) or foreground (default if nproc=1). If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If you use run_command='sh ' (or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously.

• check_wait_time = 1.0 You can specify the length of time (seconds) to wait between checking for subprocesses to end

• max_wait_time = 1.0 You can specify the length of time (seconds) to wait when looking for a file. If you have a cluster where jobs do not start right away you may need a longer time to wait. The symptom of too short a wait time is 'File not found'

• wait_between_submit_time = 1.0 You can specify the length of time (seconds) to wait between each job that is submitted when running sub-processes. This can be helpful on NFS-mounted systems when running with multiple processors to avoid file conflicts. The symptom of too short a wait_between_submit_time is File exists:....

• cache_resolve_libs = True Use caching of resolve libraries to speed up resolve

• resolve_size = 12 Size for solve/resolve ("","_giant", "_huge","_extra_huge" or a number where 12=giant 18=huge

• check_run_command = False You can have the wizard check your run command at startup

• run_command = "sh " When you specify nproc=nn, you can run the subprocesses as jobs in background with sh (default) or submit them to a queue with the command of your choice (i.e., qsub). If

you have a multi-processor machine, use sh. If you have a cluster, use qsub or the equivalent command for your system. NOTE: If you set run_command=qsub (or otherwise submit to a batch queue), then you should set background=False, so that the batch queue can keep track of your runs. There is no need to use background=True in this case because all the runs go as controlled by your batch system. If nproc is greater than 1 and you use run_command='sh '(or similar, sh is default) then normally you will use background=True so that all the jobs run simultaneously.

• queue_commands = None You can add any commands that need to be run for your queueing system. These are written before any other commands in the file that is submitted to your queueing system. For example on a PBS system you might say: queue_commands='#PBS -N mr_rosetta' queue_commands='#PBS -j oe' queue_commands='#PBS -l walltime=03:00:00' queue_commands='#PBS -l nodes=1:ppn=1' NOTE: you can put in the characters '<path>' in any queue_commands line and this will be replaced by a string of characters based on the path to the run directory. The first character and last two characters of each part of the path will be included, separated by '_',up to 15 characters. For example 'test_autobuild/WORK_5/AutoBuild_run_1_/TEMP0/RUN_1' would be represented by: 'tld_W_5_A1__TP0_1'

• condor_universe = vanilla The universe for condor is usually vanilla. However you might need to set it to local for your cluster

• add_double_quotes_in_condor = True You might need to turn on or off double quotes in condor job submission scripts. These are already default elsewhere but may interfere with condor paths.

• condor = None Specifies if the group_run_command is submitting a job to a condor cluster. Set by default to True if group_run_command=condor_submit, otherwise False. For condor job submission mr_rosetta uses a customized script with condor commands. Also uses one_subprocess_level=True

• last_process_is_local = True If true, run the last process in a group in background with sh as part of the job that is submitting jobs. This prevents having the job that is submitting jobs sit and wait for all the others while doing nothing

• skip_r_factor = False You can skip R-factor calculation if refinement is not done and maps_only=True

• test_flag_value = Auto Normally leave this at Auto (default). This parameter sets the value of the test set that is to be free. Normally phenix sets up test sets with values of 0 and 1 with 1 as the free set. The CCP4 convention is values of 0 through 19 with 0 as the free set. Either of these is recognized by default in Phenix. If you have any other convention (for example values of 0 to 19 and test set is 1) then you can specify this with test_flag_value.

• skip_xtriage = False You can bypass xtriage if you want. This will prevent you from applying anisotropy corrections, however.

• base_path = None You can specify the base path for files (default is current working directory)

• temp_dir = None Define a temporary directory

• local_temp_directory = None Write all temporary files to this directory and copy files specified by keep_files back to TEMP directory in working directory at the end of run. Used to speed up read/write operations. NOTE: Deletes all files except those specified by keep_files just like clean_up=True.

• clean_up = None At the end of the entire run the TEMP directories will be removed if clean_up is True. Also all files except for overall_best_xxx (final) files and .eff files In the AutoSol_run_xx_ AutoBuild_run_xx_ etc directory will be removed Files listed in keep_files will not be deleted. If you want to remove files after your run is finished use a command like "phenix.autobuild run=1 clean_up=True"

• print_citations = True Print citations at end of run

• solution_output_pickle_file = None At end of run, write solutions to this file in output directory if defined

• job_title = None Job title in PHENIX GUI, not used on command line

• top_output_dir = None This is used in subprocess calls of wizards and to tell the Wizard where to look for the STOPWIZARD file.

• wizard_directory_number = None This is used by the GUI to define the run number for Wizards. It is the same as desired_run_number NOTE: this value can only be specified on the command line, as the directory number is set before parameters files are read.

• verbose = False Command files and other verbose output will be printed

• extra_verbose = False Facts and possible commands will be printed every cycle if True

• debug = False You can have the wizard stop with error messages about the code if you use debug. Additionally the output goes to the terminal if you specify "debug=True"

• require_nonzero = True Require non-zero values in data columns to consider reading in.

• remove_path_word_list = None List of words identifying paths to remove from PATH These can be used to shorten your PATH. For example... cns ccp4 coot would remove all paths containing these words except those also containing phenix. Capitalization is ignored.

• fill = False Fill in all missing reflections to resolution res_fill. Applies to density modified maps. See also filled_2fofc_maps in autobuild.

• res_fill = None Resolution for filling in missing data (default = highest resolution of any datafile). Only applies to density modified maps. Default is fill to high resolution of data. Ignored if fill=False

• check_only = False Just read in and check initial parameters. Not for general use

• keep_files = overall_best* AutoBuild_run_*.log List of files that are not to be cleaned up. wildcards permitted

• after_autosol = False You can specify that you want to continue on starting with the highest-scoring run of AutoSol in your working directory.

• nbatch = 3 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch.

• nproc = 1 You can specify the number of processors to use (nproc) and the number of batches to divide the data into for parallel jobs. Normally you will set nproc to the number of processors available and leave nbatch alone. If you leave nbatch as None it will be set automatically, with a value depending on the Wizard. This is recommended. The value of nbatch can affect the results that you get, as the jobs are not split into exact replicates, but are rather run with different random numbers. If you want to get the same results, keep the same value of nbatch. If you set nproc=Auto and your machine has n processors, then it will use n-1 processors, or 1 if only 1 is available

• quick = False Run everything quickly (number_of_parallel_models=1 n_cycle_build_max=1 n_cycle_rebuild_max=1)

• resolve_command_list = None Commands for resolve. One per line in the form: keyword value value can be optional Examples: coarse_grid resolution 200 2.0 hklin test.mtz NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_command_list="'no_build' 'b_overall 23' "

• resolve_pattern_command_list = None Commands for resolve_pattern. One per line in the form: keyword value value can be optional Examples: resolution 200 2.0 hklin test.mtz NOTE: for command-line usage you need to enclose the whole set of commands in double quotes (") and each individual command in single quotes (') like this: resolve_pattern_command_list="'resolution 200 20' 'hklin test.mtz' "

• ignore_errors_in_subprocess = False Try to ignore errors in sub-processes This is useful in cases where a very rare crash occurs and you want to just ignore that step and go on.

• send_notification = False

• notify_email = None

• special_keywordswrite_run_directory_to_file = None Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.

• write_run_directory_to_file = None Writes the full name of a run directory to the specified file. This can be used as a call-back to tell a script where the output is going to go.

• run_controlcoot = None Set coot to True and optionally run=[run-number] to run Coot with the current model and map for run run-number. In some wizards (AutoBuild) you can edit the model and give it back to PHENIX to use as part of the model-building process. If you just say coot then the facts for the highest-numbered existing run will be shown. ignore_blanks = None ignore_blanks allows you to have a command-line keyword with a blank value like "input_lig_file_list=" stop = None You can stop the current wizard with "stopwizard" or "stop". If you type "phenix.autobuild run=3 stop" then this will stop run 3 of autobuild. display_facts = None Set display_facts to True and optionally run=[run-number] to display the facts for run run-number. If you just say display_facts then the facts for the highest-numbered existing run will be shown. display_summary = None Set display_summary to True and optionally run=[run-number] to show the summary for run run-number. If you just say display_summary then the summary for the highest-numbered existing run will be shown. carry_on = None Set carry_on to True to carry on with highest-numbered run from where you left off. run = None Set run to n to continue with run n where you left off. copy_run = None Set copy_run to n to copy run n to a new run and continue where you left off. display_runs = None List all runs for this wizard. delete_runs = None List runs to delete: 1 2 3-5 9:12 display_labels = None display_labels=test.mtz will list all the labels that identify data in test.mtz. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: peak.data="F+ SIGF+ F- SIGF-". The entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol or AutoBuild like this: input_refinement_labels="FP SIGFP FreeR_flags" # each individual label counts dry_run = False Just read in and check parameter namesparams_only = False Just read in and return parameter defaults. Not for general usedisplay_all = False Just read in and display parameter defaults

• coot = None Set coot to True and optionally run=[run-number] to run Coot with the current model and map for run run-number. In some wizards (AutoBuild) you can edit the model and give it back to PHENIX to use as part of the model-building process. If you just say coot then the facts for the highest-numbered existing run will be shown.

• ignore_blanks = None ignore_blanks allows you to have a command-line keyword with a blank value like "input_lig_file_list="

• stop = None You can stop the current wizard with "stopwizard" or "stop". If you type "phenix.autobuild run=3 stop" then this will stop run 3 of autobuild.

• display_facts = None Set display_facts to True and optionally run=[run-number] to display the facts for run run-number. If you just say display_facts then the facts for the highest-numbered existing run will be

shown.

• display_summary = None Set display_summary to True and optionally run=[run-number] to show the summary for run run-number. If you just say display_summary then the summary for the highest-numbered existing run will be shown.

• carry_on = None Set carry_on to True to carry on with highest-numbered run from where you left off.

• run = None Set run to n to continue with run n where you left off.

• copy_run = None Set copy_run to n to copy run n to a new run and continue where you left off.

• display_runs = None List all runs for this wizard.

• delete_runs = None List runs to delete: 1 2 3-5 9:12

• display_labels = None display_labels=test.mtz will list all the labels that identify data in test.mtz. You can use the label strings that are produced in AutoSol to identify which data to use from a datafile like this: peak.data="F+ SIGF+ F- SIGF-". The entire string in quotes counts here You can use the individual labels from these strings as identifiers for data columns in AutoSol or AutoBuild like this: input_refinement_labels="FP SIGFP FreeR_flags" # each individual label counts

• dry_run = False Just read in and check parameter names

• params_only = False Just read in and return parameter defaults. Not for general use

• display_all = False Just read in and display parameter defaults

• non_user_parameters These are obsolete parameters and parameters that the wizards use to communicate among themselves. Not normally for general use.gui_output_dir = None Used only by the GUIbackground_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in an omit map that are not part of any omitted region. (Default="") boundary_background_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in the boundary map that are not part of any omitted region. (Default="") extend_try_list = True You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with nbatch. force_combine_extend = False You can choose whether to force the combine-extend step in model-building model_list = None This keyword lets you name any number of PDB files to consider as starting models for model-building. NOTE: This differs from consider_main_chain_list which will try to add your PDB files EVERY cycle of merging models. In contrast model_list will only do it on the first cycle. NOTE: this only uses the main-chain atoms of your PDB files. oasis_cnos = None Enter number of C N O and S atoms here if you have OASIS and want to run it before resolve density modification like this: "C 250 N 121 O 85 S 3" offset_boundary_background_map = None You can set the offset of the boundary_background_map. skip_refine = False Skip refinement (used in get_connections/assign_sequence)sg = None Obsolete. Use space_group insteadinput_data_file = None Not normally used (same as "data=").input_map_file = Auto Not normally used. (Same as map_file).input_refinement_file = Auto Not normally used. Same as refinement_fileinput_pdb_file = None Not normally used. Same as "model="input_seq_file = Auto Not normally used. Same as seq_filesuper_quick = None Shortcut for very quick run of autobuild. Same as : number_of_parallel_models=1 refine=false n_cycle_rebuild_max=1 remove_aniso=False skip_xtriage=True ncs_copies=1 find_ncs=false fully_skip_combine_extend=True fit_loops=False redo_side_chains=False insert_helices=False build_outside=False connect=Falserequire_test_set = False Require that input data file have a test set

• gui_output_dir = None Used only by the GUI

• background_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in an omit map that are not part of any omitted region. (Default="")

• boundary_background_map = None You can supply an mtz file (REQUIRED LABELS: FP PHIM FOMM) to use as map coefficients to calculate the electron density in all points in the boundary map that are not part of any omitted region. (Default="")

• extend_try_list = True You can fill out the list of parallel jobs to match the number of jobs you want to run at one time, as specified with nbatch.

• force_combine_extend = False You can choose whether to force the combine-extend step in model-building

• model_list = None This keyword lets you name any number of PDB files to consider as starting models for model-building. NOTE: This differs from consider_main_chain_list which will try to add your PDB files EVERY cycle of merging models. In contrast model_list will only do it on the first cycle. NOTE: this only uses the main-chain atoms of your PDB files.

• oasis_cnos = None Enter number of C N O and S atoms here if you have OASIS and want to run it before resolve density modification like this: "C 250 N 121 O 85 S 3"

• offset_boundary_background_map = None You can set the offset of the boundary_background_map.

• skip_refine = False Skip refinement (used in get_connections/assign_sequence)

• sg = None Obsolete. Use space_group instead

• input_data_file = None Not normally used (same as "data=").

• input_map_file = Auto Not normally used. (Same as map_file).

• input_refinement_file = Auto Not normally used. Same as refinement_file

• input_pdb_file = None Not normally used. Same as "model="

• input_seq_file = Auto Not normally used. Same as seq_file

• super_quick = None Shortcut for very quick run of autobuild. Same as : number_of_parallel_models=1 refine=false n_cycle_rebuild_max=1 remove_aniso=False skip_xtriage=True ncs_copies=1 find_ncs=false fully_skip_combine_extend=True fit_loops=False redo_side_chains=False insert_helices=False build_outside=False connect=False

• require_test_set = False Require that input data file have a test set

PredictAndBuild: Solving an X-ray structure or interpreting a cryo-EM map using predicted models

predict_and_build.html

Author(s)

- predict_and_build: Tom Terwilliger

Acknowledgements

The Phenix AlphaFold server uses AlphaFold (Jumper et al., 2021), the mmseqs2 MSA server (Steinegger, and Söding, 2017), and scripts derived from ColabFold (Mirdita et al., 2022).

Tutorial video - X-ray

Tutorial video - Cryo-EM

Tutorial videos for X-ray and cryo-EM are available on the Phenix YouTube channel and cover the following topics:

- basic overview of X-ray or cryo-EM structure determination with AlphaFold models
- how to run PredictAndBuild with X-ray or cryo-EM data via the GUI
- how to interpret the output

Purpose

Predict and build can be used to generate predicted models and to use them to solve an X-ray structure by molecular replacement or to interpret a cryo-EM map. Predict and build then carries out iterative model rebuilding and prediction to improve the models. The iterative procedure allows creation of more accurate predicted models than can be obtained with a simple prediction.

Normally Predict and build is used as a way to automatically generate a fairly accurate model starting from just a sequence file and either cryo-EM half-maps or X-ray data. Additionally Predict and build provides morphed versions of unrefined predicted models that can be useful as reference models for refinement.

Steps in predict_and_build

The principal steps in predict_and_build are:

```
Model prediction (e.g., with AlphaFold)

Model trimming

Structure solution by molecular replacement (X-ray) or map interpretation
by docking (cryo-EM) to create a scaffold model

Morphing of original predicted models onto the scaffold model

Model refinement and rebuilding

Iteration of model prediction using the rebuilt models as templates
```

Sequence file

The sequence file that you supply specifies what is going to be predicted and how many copies of each chain are present in the structure (one for every copy in the sequence file). All models that are created and input will be associated with one or more of the chains in your sequence file based on sequence identity (normally every model should match a chain in the sequence file exactly).

Input data (cryo-EM)

For cryo-EM maps, normally you will supply two full-size half-maps. These will be used in density modification to create a single full-size map. This map will then be boxed to extract just the part that contains the molecule (the part with high density). Alternatively, you can supply a single full-size or boxed map if you wish.

Input data (X-ray)

For a crystal structure, you will supply an mtz-style file containing data for Fobs and sigFobs. Optionally this file can contain a test set (FreeR_flags). The space group specified in this data file and its enantiomer, if any, will be used in structure determination. This means you should run phenix.xtriage first so that you have a good idea of the space group.

Model prediction

Predict and build normally uses the Phenix server to carry out AlphaFold prediction of one chain at a time. It can also use predictions that you supply or that you generate on demand and put in the same place on your computer as those that are created by one of the servers.

Prediction (see phenix.predict_model for details) is fully automated with the Phenix server through the Phenix GUI, so all you have to do is let it run. The Phenix GUI will use the Phenix server to carry out the prediction and put it in the working directory.

You can specify what inputs the AlphaFold prediction should use. These always include a sequence file, but it can include an optional multiple sequence alignment file, optional templates, keywords for model prediction such as the number of models to generate, random seed, and whether to use multiple sequence alignment.

When prediction is being carried out, the Phenix GUI waits for the predicted models to appear in the working directory, then it goes on to the next steps. If you want to create these models in some other way, you can just put them in the working directory (the expected file names are listed in the GUI when prediction starts) and they will be used. Note that all prediction files must have all the residues in the corresponding sequence present.

Model trimming

The predicted models are trimmed to remove low-confidence sections and to split into compact domains using phenix.process_predicted_model . This allows the molecular replacement (X-ray) or docking (cryo-EM) procedures to use the most accurate parts of the models, increasing the chance of finding the correct solution. Splitting into compact domains increases the chance of correctly placing parts of chains that have different orientations in the predicted model compared to the actual structure.

Predict and build uses phenix.process_predicted_model to estimate suitable VRMS values from the pLDDT values in a processed chain to be used in molecular replacement (the next step below). The VRMS values are estimates of the RMSD between the model and the true structure. Process predicted model first converts average pLDDT to estimated RMS error using an empirical formula from RoseTTAFold:

rmsd_est = 1.5 * exp(4*(0.7-pLDDT)) where pLDDT is fractional (0 to 1)

Then it uses a second empirical formula to suggest a value of VRMS based on the RMS error:

vrms = vrms_from_rmsd_slope * rms + vrms_from_rmsd_intercept where the slope is 1 and intercept is 0.25 A.

Structure determination by molecular replacement (X-ray data)

Predict and build uses phenix.phaser with all default parameters to solve a structure by molecular replacement, except that the value of VRMS (estimated RMSD between model and true structure) is estimated from the pLDDT values as described above. You can also set the value of VRMS directly with the keyword vrms_value. If your structure cannot be solved automatically in this way, then you will want to take your trimmed predicted models, solve your structure separately by molecular replacement (e.g., by using phenix.phaser but changing some parameters), then supplying the molecular replacement model to Predict and build as a "scaffold model". After molecular replacement, domains are rearranged if necessary to allow sequential parts to be connected, creating a scaffold model for use in the morphing step to follow.

The molecular replacement model is refined using the X-ray data, yielding a 2mFo-DFc map. This map is density-modified as well, and the map that has the higher correlation with the refined model is used as the working map.

Structure determination by docking (cryo-EM data)

Predict and build first carries out density modification based on the two half-maps that you provide. This creates an optimized map for interpretation.

Predict and build then sequentially docks all the domains from all the predicted models into the map. After docking, the domains are rearranged if necessary to allow sequential parts to be connected. This docking is carried out by phenix.dock_and_rebuild . The rearranged docked model is the scaffold model that will be used in the next step.

Morphing of original predicted models onto the scaffold model

The scaffold model obtained from predicted models and X-ray or cryo-EM maps usually contains one chain for each sequence in your sequence file. Your predicted models normally include one model matching each unique sequence in the sequence file. For each chain in the scaffold model, the predicted model that is most similar is then morphed (adjusted) to superimpose on the scaffold model.

In the simplest case, your predicted models all yielded a single domain in the trimming step. In this case the chains in your scaffold model will match your predicted models exactly and morphing consistes of simple superposition of the full predicted model on the docked chain. In a more complicated case, your trimmed model may have consisted of multiple domains for one chain, so that your scaffold model may have separately-placed domains for a particular chain. In this case, the morphing consists of superimposing the parts of the full predicted model that match the scaffold, and smoothly deforming the connecting segments.

Model refinement, rebuilding, and trimming

Model refinement and rebuilding is done by phenix.dock_and_rebuild . This procedure consists of identifying all the parts of each chain that are either predicted with low confidence or that do not match the density, rebuilding them in several different ways, and picking the one that matches the density the best. Then the individual chains are refined with real-space refinement.

At the end of rebuilding, residues in the model that are in poor density are identified and marked with zero occupancies. A new trimmed final model lacking these residues is also created

For X-ray data, the trimmed model is refined using the X-ray data, yielding a new working map (2mFo-DFc or density modified, as in the molecular replacement step).

Iteration of model prediction using the rebuilt models as templates

A key element of the Predict and build procedure is iteration of model prediction using chains from the rebuilt model as templates. This improves model prediction compared to a single prediction step. Normally the entire procedure is repeated until the change in predicted models between subsequent cycles is small.

Output models

The models produced by the Predict and build procedure are:

```
A final docked model
(PredictAndBuild_xxx_overall_best_superposed_predicted_models.pdb) This
model is the last scaffold model and is
normally suitable for use as a reference model in further refinement.
Normally this model will consist of pure predicted models,
placed appropriately to match the map.

A final trimmed rebuilt model (PredictAndBuild_xxx_overall_best.pdb).
This model is obtained by rebuilding and
refinement, followed by trimming residues that are in poor density.
It is normally suitable for use as a
starting point for further model rebuilding, refinement, and addition of
ligands and covalent modifications. A final untrimmed model is also
provided. This can be used as a hypothesis for the poorly-fitting regions.

In some cases (low-resolution data), the trimmed rebuilt model may have very
poor geometry and you might want to use the final docked model as the
starting point for refinement and continuation of building of your structure.
```

Output data file, map file, and and map coefficients files (X-ray data):

After a run of PredictAndBuild with X-ray data, the principal output data file is PredictAndBuild_xxx_overall_best_refinement.mtz. This is essentially the same as the input data, though it has a test set (R-free flags) even if the input file did not. This file can be used in further refinement along with overall_best.pdb. This overall_best_refinement.mtz file does not contain the results of refinement. It is a refinement data file.

The output file from PredictAndBuild with X-ray data containing map coefficients is PredictAndBuild_xxx_overall_best_map_coeffs.mtz. This file should be used to draw an electron density map, for example in Coot.

The output file from PredictAndBuild with X-ray data containing a map is PredictAndBuild_xxx_overall_best_map.ccp4. This file can also be used to draw a map. It is most suitable for ChimeraX.

Output map file (PredictAndBuild with cryo-EM data):

The output file from PredictAndBuild with cryo-EM data containing a map is PredictAndBuild_xxx_overall_best_map.ccp4. This is suitable for drawing a map in ChimeraX (or in Coot).

The Carry-on directory and restarting or re-running jobs

Predict and build saves the working files in a carry_on_directory. At the end of the log file (and in the GUI at the end of a run) the carry-on directory is listed. In the GUI it is normally a directory with the same number as the job and ending in "CarryOn": for PredictAndBuild_30 it is PredictAndBuild_30_CarryOn. If you have a previous run that you want to restart or continue, specify the name of this file in the GUI as the Carry-on directory, make sure the flag "carry_on=True" is set, and rerun the job with the same inputs (otherwise) as before. Then running the job will result in reading all the results that had been accumulated by the previous job, then continuing from where it left off.

The carry-on directory lets you restart after stopping or crashing. It also lets you change some parameters and carry on from where you left off.

The jobname also lets you move a project to another computer if you want. You can copy the carry-on directory and put it in any directory on any computer. Then you can specify that directory as the Carry-on directory on that computer and it will carry on from where you left off.

Note that the file names inside the Carry-on directory contain the jobname, so if you want to move individual files around, you might have to change their names.

The universal solution for (most) problems in PredictAndBuild

The solution to many problems that you may have with PredictAndBuild is simply to completely stop running your job, then restart, specifying the Carry-on directory from the job that stopped. This will pick up where you left off. It is suitable for cases where something crashed, where you accidentally stopped a job, where you lost connections to the server, or where something else went wrong.

In some cases to stop running your job you might have to stop all python processes on your computer and quit the GUI, but in most cases you can just hit Abort in the GUI and wait a little.

MSA calculation vs model prediction

If you use the Phenix server, the calculation of multiple sequence alignments (MSAs) is a separate step from model prediction. The Phenix GUI on your computer sends a request to the mmseqs2 server, which creates an MSA and sends it back to your computer. Then your computer uploads the MSA to the Phenix server, which uses it in an AlphaFold prediction.

If you want, you can supply your own MSAs. The key requirement is that the sequence of the first entry in your MSA must exactly match the sequence to be predicted.

You can also skip the use of MSAs. This can be useful if you supply a template and you want AlphaFold to rebuild your template instead of doing a new prediction.

Number of models

AlphaFold can carry out multiple predictions for a sequence. You can specify how many of these to carry out. The PredictModel tool will choose the one with the highest value of pLDDT (predicted local difference distance test).

Using templates from the PDB

You can request that templates from the PDB be used in prediction. If you use this feature, models will be predicted both with and without the templates, and the model with the highest pLDDT will be saved.

Using supplied templates

You can supply your own templates. As for templates from the PDB, if you use this feature, models will be predicted both with and without the templates, and the model with the highest pLDDT will be saved.

Using supplied predictions or RNA/DNA models

You can supply homology models or models that you have built yourself for any of the chains in your structure. If you are working with RNA or DNA, you have to do this. You just supply a model (does not have to be perfect) for each chain that you do not want to have AlphaFold predict, and you call it a predicted model (note the difference between a template -- a model to guide AlphaFold prediction, and a predicted model -- the prediction or an alternative to a prediction.)

For example, suppose you already have a model with 4 DNA chains and four protein chains called model_ABGH_CDEF.pdb with a sequence file model_ABGH_CDEF.seq, and you have copied the 4 DNA chains to model_C.pdb, model_D.pdb, model_E.pdb, and model_F.pdb. You can run PredictAndBuild, using the DNA chains as is, and predicting models for the 4 protein chains (A, B, G, H) like this if you have xray data in data.mtz:

```
phenix.predict_and_build seq_file=model_ABGH_CDEF.seq \
 scaffold_model=model_ABGH_CDEF.pdb data.mtz \
 control.nproc=4 crystal_info.resolution=2 \
 predicted_model=model_C.pdb \
 predicted_model=model_D.pdb \
 predicted_model=model_E.pdb \
 predicted_model=model_F.pdb
```

This command will use model_ABGH_CDEF.pdb as a scaffold, superposing all predicted protein models and the DNA models on it before rebuilding.

Using PredictAndBuild to change the sequences in a model

If you have docked (cryo-EM) or placed (X-ray) chains for a structure from one species using data from another species, you can use PredictAndBuild to fix all the sequences and create a plausible model for each chain.

There are several ways you can do this. If you first run AlphaFold to get a predicted model for each unique chain, you can use a command like this:

```
phenix.predict_and_build predicted_model=replacements.pdb \
input_files.b_value_field_is=b_value scaffold_model=existing.pdb \
seq_file=seq.dat cycles=1 refine_only=True refine_cycles=0 \
crystal_info.resolution=3 full_map=full_map.ccp4
```

Here existing.pdb is the model you have already created (wrong sequences). The model replacements.pdb contains your predicted models for each unique chain, where each predicted model has the right sequence, but can be in any orientation or position. The sequence file seq.dat has all the correct sequences. The map is required as it is used to adjust the predicted models. The keyword b_value_field_is=b_value tells PredictAndBuild that the predicted models contain B-values (atomic displacement parameters), not pLDDT values (predicted Local Difference Distance Test).

If you actually have no map, you can create a model-based map from your existing.pdb structure with the phenix.fmodel tool. The map type you want is called complex (structure factors) and you would want to make it a low-resolution map (like 10 A).

If you don't want to generate the predicted models yourself, you can skip that input and just let PredictAndBuild do the predictions. Of course if you have RNA or DNA chains, you will need to supply predicted models for those chains.

Rebuilding a model without prediction (morph and refine only):

You can use predict_and_build to just rebuild a model that is placed (in a cryo-EM map) like this:

```
In the GUI, select PredictAndBuild (cryo-EM data)

Supply: your map (call it “Map file”), your model (call it
“predicted model”), your sequence.

Just below the inputs there is a line that starts with
“Contents of b-value field”… Select “b_value” (not plddt),
as this model has B values in its b-value field.

In Prediction and building settings, go to All parameters,
the “Search parameters”, and search for “already”. Tick
the check box for “Models are already placed”.

In Prediction and building settings, select “Building” and check
the box for “Refine only”.

Hit Run. It should take your model, morph it to match
the map, and refine it.
```

Examples

Standard run of predict_and_build

Running predict_and_build is easy. From the command-line you can type:

```
phenix.predict_and_build jobname=myjob xray_data=my_xray_data.mtz seq_file=seq.dat
```

This will carry out all the steps of prediction, molecular replacement and iterative rebuilding and prediction to yield my_model_rebuilt.pdb

If you run again with the same command, Predict and build will read all the previous files and just give you the results.

Common questions

How long will it take to run?

Running predict_and_build can take anywhere from an hour to several days, depending largely on the size of the chains to be placed and the number of chains. Increasing the number of processors used (default of 4) will speed it up. If you are unsure if the program is running, have a look at the log file, where the status and current time are printed out periodically. Also you can just see if any Python processes are running on your machine.

R and Rfree values for density modification and for refinement for Xray data

If you are running predict_and_build with Xray data, you will obtain R values for both density modification and refinement. These are quite different. A good R value for density modification is in the range of 0.3-0.4, while a good value for refinement is in the range of 0.2-0.25 for high-resolution data (2 A or better) and 0.25-0.3 for

lower resolution data.

What to do next after predict_and_build?

What to do next after predict and build depends on whether the input data for your run was cryo-EM data (i.e., my_map.ccp4), or X-ray crystallography data (i.e, my_reflection_data.mtz). For cryo-EM data, you are normally working with a map file (xxx.ccp4) and for X-ray crystallography data, you are normally working with reflection data (xxx.mtz). With cryo-EM data, you will use real-space refinement (RealSpaceRefine), and with X-ray data you will use reciprocal-space refinement (phenix.refine)

For X-ray crystallography data, if predict_and_build yields a largely-complete model with reasonable R-value, the next step is usually refinement with phenix.refine. Inputs for crystallographic refinement will be your model (overall_best.pdb) and your refinement data (overall_best_refinement.mtz).

For cryo-EM data, if predict_and_build yields a largely-complete model with good fit to density, the next step is usually real-space refinement with phenix.real_space_refine. Inputs for real-space refinement will be your model (overall_best.pdb) and your optimized map (overall_best_map.ccp4).

For either X-ray or Cryo-EM data, another important step is validation with "Comprehensive Validation (Cryo EM)". Inputs for validation for X-ray data will be your model (overall_best.pdb) and your refinement data (overall_best_refinement.mtz.) Inputs for validation for cryo_EM data will be your model and your optimized map (overall_best_map.ccp4)

You may also want to use Coot or Isolde to rebuild parts of your model that do not fit the density well or that are incomplete. For X-ray data your inputs to Coot or Isolde will be your map coefficients (overall_best_map_coeffs.mtz) and your model (overall_best.pdb)

If your structure contains ligands, once you have a very good model you can use LigandFit to find the placement and orientation of your ligand. For X-ray data your inputs will be your map coefficients (overall_best_map_coeffs.mtz) and your model (overall_best.pdb) and the name of your ligand. For cryo_EM data your inputs will be your map filel (overall_best_map.ccp4) and your model (overall_best.pdb) and the name of your ligand.

Server problems

The most common problem in running Predict and build is that the Phenix server is not working as expected. Normally the first thing to try is just let the program retry (it will do this for a while normally).

Here is a test you can run to see if everything is ok: In the GUI, hit AlphaFold/AlphaFold model prediction/Prediction settings/ and check the box for “Run test job on server and stop”. Then hit Run. In about 10 sec it should say “Test job completed successfully” if everything is ok.

If the server is still not working, you can take the files in the packaged .tgz file (listed in the GUI output), use them to get your own prediction with any server, and put the resulting predicted models in the place specified in the GUI or program output.

Problems with low-confidence models

If your predicted models have low confidence, the entire procedure may not work.

If the resolution of your data is about 4.5 A or better, and most of most of your predicted models have high confidence (plDDT > 70), then the procedure has a good chance of working. Otherwise, the chances are lower.

Symmetry problems

If your cryo-EM map has pseudo-symmetry (like a proteasome) you might need to box one subunit or try ssm_search=False to use a more thorough search in docking.

Working on individual chains in large cryo-EM structures

If you are working with a large cryo-EM structure (or even a small one), you may find it most efficient to break up the work into small pieces. If you can identify where individual chains are in your map, you can box around each chain, creating individual boxes for each chain to work on. One way to create a box around a chain is to put a dummy molecule in the density you are interested (anything that more or less covers the region you want), then use the MapBox tool to create a little map surrounding that dummy molecule.

When you use the MapBox tool be sure to use the defaults and do not shift the origin (if you shift the origin then the models you get will not superimpose on the original map).

Then of course you have to guess which sequence goes with that density. You could try a few possibilites and run PredictAndBuild on your small map with each likely sequence.

When you are done with each box, you can just combine all the models because they will stay in their correct locations.

Inverted map:

If your map is inverted (left-handed), docking and model-building will not work properly. You can often tell if your map is inverted because any helices will be left-handed. If you are unsure, you can run MapBox with invert_hand=True to invert the map and then see if docking works. Note that if your map is inverted, you will want to invert all your maps and start everything from the beginning.

Specific limitations and problems:

AlphaFold 2 only predicts protein structures, not RNA or DNA, so you will have to either build those chains separately or supply predicted models (homology models, for example) for those chains. If you supply predicted models for some chains and not others, predict_and_build will try to predict the missing chains and build with all the available models.

The predict_and_build tool has two command-line parameters for nproc (prediction.nproc and control.nproc), two for resolution (prediction.resolution and crystal_info.resolution), and two for random_seed (prediction.random_seed and control.random_seed). Normally use the control.nproc, crystal_info.resolution and control.random_seed parameters. However if you want to change the number of processors (on the server, nproc only used for getting templates with structure_search and limited to 4 processors) you can set prediction.nproc. If you want to set the random seed used in prediction to a particular value you can set prediction.random_seed (and also usually set use_supplied_random_seed_in_prediction=True).

Literature

Accelerating crystal structure determination with iterative AlphaFold prediction. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner, RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. Acta Cryst D 79, 234-244 (2023). Improved AlphaFold modeling with implicit experimental information. T.C. Terwilliger, B.K. Poon, P.V. Afonine, C.J. Schlicksup, T.I. Croll, C. Millán, J.S. Richardson, R.J. Read, and P.D. Adams. Nature Methods 19, 1376-1382 (2022). AlphaFold predictions: great hypotheses but no match for experiment. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner,

RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. BiorXiv (2023). Making protein folding accessible to all. M. Mirdita, K. Schütze, Y. Moriwaki, L. Heo, S. Ovchinnikov, and M. Steinegger. Nature Methods 19, 679–682 (2022). Highly accurate protein structure prediction with AlphaFold. J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S.A.A. Kohl, A.J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A.W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Nature 596, 583-589 (2021). MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. M. Steinegger, and J. Söding. Nature biotechnology 35, 1026-1028 (2017). lDDT: a local superposition-free score for comparing protein structures and models using distance difference tests. V. Mariani, M. Biasini, A. Barbato, and T.. Schwede. Bioinformatics 29, 2722-2728 (2013).

- Accelerating crystal structure determination with iterative AlphaFold prediction. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner, RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. Acta Cryst D 79, 234-244 (2023).
- Improved AlphaFold modeling with implicit experimental information. T.C. Terwilliger, B.K. Poon, P.V. Afonine, C.J. Schlicksup, T.I. Croll, C. Millán, J.S. Richardson, R.J. Read, and P.D. Adams. Nature Methods 19, 1376-1382 (2022).
- AlphaFold predictions: great hypotheses but no match for experiment. T.C. Terwilliger, P.V. Afonine, Liebschner, D., T.I. Croll, McCoy, AJ, Oeffner, RD., Williams, CJ, Poon, BK, J.S. Richardson, R.J. Read, and P.D. Adams. BiorXiv (2023).
- Making protein folding accessible to all. M. Mirdita, K. Schütze, Y. Moriwaki, L. Heo, S. Ovchinnikov, and M. Steinegger. Nature Methods 19, 679–682 (2022).
- Highly accurate protein structure prediction with AlphaFold. J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S.A.A. Kohl, A.J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A.W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Nature 596, 583-589 (2021).
- MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. M. Steinegger, and J. Söding. Nature biotechnology 35, 1026-1028 (2017).
- lDDT: a local superposition-free score for comparing protein structures and models using distance difference tests. V. Mariani, M. Biasini, A. Barbato, and T.. Schwede. Bioinformatics 29, 2722-2728 (2013).

Additional information

List of all available keywords

jobname = '' Name of this job. Used to create a directory that will hold all the working files for the run. Normally set automatically by the GUI to match the directory where the job is run: directory: PredictAndBuild_30, jobname: PredictAndBuild_30. When run on command line you can specify any 4-or-more character jobname if you want.carry_on_directory = None Name of carry-on directory (stores intermediate and final results). You can continue with a previous run by specifying this directory from a previous run. To rerun or carry on from the job in /Users/Documents/PredictAndBuild_30, you specify carry_on_directory= /Users/Documents/PredictAndBuild_30/PredictAndBuild_30_CarryOn. Then if you also specify carry_on=True, files will be read from this directory and copied to a new carry_on_directory inside the working

directory. Normally set automatically by the GUI in the directory containing your job: directory: PredictAndBuild_30, jobname: PredictAndBuild_30 carry_on_directory: PredictAndBuild_30_CarryOn. NOTE: Cannot be the the current directory. NOTE: if you specify a CarryOn directory you must specify carry_on=True if you want it to be used.upload_download_directory = /Users/terwill/Downloads/ Directory to use for uploading and downloading files from Colab. Normally use your standard Downloads folder.alphafold_model_suffix = AlphaFold_cycle_1.pdb Name of AlphaFold model is <jobname>_<alphafold_model_suffix> when coming from Colab server.pae_suffix = PAE_cycle_1.jsn Name of PAE file is <jobname>_<pae_suffix> when coming from Colab server.carry_on = False Carry on with a job from where it left off. Read in all files that were created instead of making new ones. To use, run the job you want exactly as you did the first time except set carry_on=True and specify carry_on_directory=xxx where xxx is the carry_on_directory from the run you want to continue. If your previous run was /Users/Documents/PredictAndBuild_30, specify: carry_on_directory= /Users/Documents/PredictAndBuild_30/PredictAndBuild_30_CarryOn. NOTE: if you specify a CarryOn directory you must also specify carry_on=True (False is the default) if you want this to be used.prediction_server = *PhenixServer Colab ManualPrediction LocalServer Choose where prediction calculations will be done. Normally use the Phenix server (PhenixServer). As a backup you can use Colab (you will need a Google account.) Use ManualPrediction if you are going to do the prediction off-line and copy the result to the location specified by PredictAndBuild. LocalServer is used internally only.run_directly = False Run directly. Used internally onlymax_len_at_server = 1500 Maximum length of sequence at server. Used internally onlydummy_run = False Write dummy file. Used internally onlyrun_server_test = False Run test of server and stop (does nothing else and does not require any inputs)copy_of_final_model = None Copy final model here. Used internally and by predict_model. (Also copy map files to same directory). If None, copy to working directory. Use copy_of_final_model=null to not write these files at all.copy_of_msa = None Copy final msa here (only if there is just one). Used internallycopy_of_pae = None Copy final pae here (only if there is just one). Used internallypredicted_model_prefix = None Predicted models will have this prefix. Used by predict_model.job_title = None Job title in PHENIX GUI, not used on command linepredict_and_buildmax_rmsd_to_predict_together = 2 Two models are considered similar (and not worth predicting separately) if they have an rmsd after superposition of this value or less (A)template_file = None Template model to be used in prediction.get_msa_with_mmseqs2 = True Run the mmseqs2 server from this computer to get MSA if not already available. Alternatives are to supply and MSA or to run the mmseqs2 server from the Phenix server or Colab.combine_rebuilt_and_autobuild = True Combine rebuilt and autobuild models and use if better Rfree (X-ray only)....

- jobname = '' Name of this job. Used to create a directory that will hold all the working files for the run. Normally set automatically by the GUI to match the directory where the job is run: directory: PredictAndBuild_30, jobname: PredictAndBuild_30. When run on command line you can specify any 4-or-more character jobname if you want.
- carry_on_directory = None Name of carry-on directory (stores intermediate and final results). You can continue with a previous run by specifying this directory from a previous run. To rerun or carry on from the job in /Users/Documents/PredictAndBuild_30, you specify carry_on_directory= /Users/Documents/PredictAndBuild_30/PredictAndBuild_30_CarryOn. Then if you also specify carry_on=True, files will be read from this directory and copied to a new carry_on_directory inside the working directory. Normally set automatically by the GUI in the directory containing your job: directory: PredictAndBuild_30, jobname: PredictAndBuild_30 carry_on_directory: PredictAndBuild_30_CarryOn. NOTE: Cannot be the the current directory. NOTE: if you specify a CarryOn directory you must specify carry_on=True if you want it to be used.
- upload_download_directory = /Users/terwill/Downloads/ Directory to use for uploading and downloading files from Colab. Normally use your standard Downloads folder.

• alphafold_model_suffix = AlphaFold_cycle_1.pdb Name of AlphaFold model is <jobname>_<alphafold_model_suffix> when coming from Colab server.

• pae_suffix = PAE_cycle_1.jsn Name of PAE file is <jobname>_<pae_suffix> when coming from Colab server.

• carry_on = False Carry on with a job from where it left off. Read in all files that were created instead of making new ones. To use, run the job you want exactly as you did the first time except set carry_on=True and specify carry_on_directory=xxx where xxx is the carry_on_directory from the run you want to continue. If your previous run was /Users/Documents/PredictAndBuild_30, specify: carry_on_directory= /Users/Documents/PredictAndBuild_30/PredictAndBuild_30_CarryOn. NOTE: if you specify a CarryOn directory you must also specify carry_on=True (False is the default) if you want this to be used.

• prediction_server = *PhenixServer Colab ManualPrediction LocalServer Choose where prediction calculations will be done. Normally use the Phenix server (PhenixServer). As a backup you can use Colab (you will need a Google account.) Use ManualPrediction if you are going to do the prediction off-line and copy the result to the location specified by PredictAndBuild. LocalServer is used internally only.

• run_directly = False Run directly. Used internally only

• max_len_at_server = 1500 Maximum length of sequence at server. Used internally only

• dummy_run = False Write dummy file. Used internally only

• run_server_test = False Run test of server and stop (does nothing else and does not require any inputs)

• copy_of_final_model = None Copy final model here. Used internally and by predict_model. (Also copy map files to same directory). If None, copy to working directory. Use copy_of_final_model=null to not write these files at all.

• copy_of_msa = None Copy final msa here (only if there is just one). Used internally

• copy_of_pae = None Copy final pae here (only if there is just one). Used internally

• predicted_model_prefix = None Predicted models will have this prefix. Used by predict_model.

• job_title = None Job title in PHENIX GUI, not used on command line

• predict_and_buildmax_rmsd_to_predict_together = 2 Two models are considered similar (and not worth predicting separately) if they have an rmsd after superposition of this value or less (A)template_file = None Template model to be used in prediction.get_msa_with_mmseqs2 = True Run the mmseqs2 server from this computer to get MSA if not already available. Alternatives are to supply and MSA or to run the mmseqs2 server from the Phenix server or Colab.combine_rebuilt_and_autobuild = True Combine rebuilt and autobuild models and use if better Rfree (X-ray only).include_side_in_templates = False Include side-chains beyond CB in templates as well as just main and CB (two runs).only_include_side_in_templates = None Include side-chains beyond CB in templates only (one run)use_templates_in_prediction = True Use templates if available (i.e., after first cycle)required_free_r_to_keep = 0.49 Skip refinements if R is worse than this valueautobuild_resolution = 3.5 Worst resolution where autobuild density modification may be useful.autobuild_satisfactory_r_value = 0.25 Skip autobuild if R is this value or betterautobuild_for_xray_density_modification = None Use autobuild rebuilding for density modification. Default is True if resolution is autobuild_resolution or better.autobuild_n_cycle_rebuild_max = 3 Maximum autobuild rebuild cyclesautobuild_n_cycle_build_max = 1 Maximum autobuild build cyclesautobuild_n_cycle_refine = 3 AutoBuild refinement cyclesautobuild_quick = False Run AutoBuild in quick modeautobuild_min_resolution = 2 Minimum resolution for autobuilding. Resolution will be cut off at this value if it is better (smaller number).density_modify_at_start = None Density modify cryo-EM maps before

analysis. Requires two half maps. Default is True if half maps are supplied and no full map is supplied, otherwise False. Works best if maps contain entire structure with boundary (at least 5 A) or are original full-sized maps.density_modify_with_model = None Density modify using rebuilt model each cycle. default is True. Requires two half-maps for cryo-EM. Works best for cryo-EM if maps contain entire structure with boundary (at least 5 A) or are original full-sized maps. Default is True for X-ray and False for cryo_EMvrms_value = None Value of VRMS to use in molecular replacement. This value will be used for all chains if set. Default is to estimate this value for each chain from the average pLDDT value after trimming.large_sequence = 1500 Length of sequence that requires job_size=largepause_after_mr = False Pause after MR and refinement (X-ray only). You can go on after checking your results.pause_after_docking = False Pause after docking and refinement (Cryo-EM only). You can go on after checking your results.stop_after_predict = False Stop after prediction. No map file needed if set.stop_after_mr = False Stop after MR.parallel_predictions = 1 Number of predictions to run at once. If more than one, no feedback is provided until the end. If there are waiting jobs in the queue running more than one at a time does not help (your first job may go near the front of the line but the others stay at the end until the first one finishes).write_prediction_files_and_wait = False Write prediction files in working directory and wait for result to appear. Only used for servers.put_files_for_prediction_here = None Program will put prediction files (pdb, msa) in this directory for the rest server to read (may look like /app/xxx/xxx)files_for_prediction_will_be_transfered_here = None Program will expect prediction files (pdb, msa) from the put_files_for_prediction_here directory will be copied to this directory so the rest server can read them. Your system has to automatically do this transfer.precalculated_msas_dir = /app/results/precalculated_msas Location of precalculated msas (usually on server)precalculated_results_dir = /app/results/precalculated_results Location of precalculated results (usually on server)precalculated_pae_dir = /app/results/precalc...

• max_rmsd_to_predict_together = 2 Two models are considered similar (and not worth predicting separately) if they have an rmsd after superposition of this value or less (A)

• template_file = None Template model to be used in prediction.

• get_msa_with_mmseqs2 = True Run the mmseqs2 server from this computer to get MSA if not already available. Alternatives are to supply and MSA or to run the mmseqs2 server from the Phenix server or Colab.

• combine_rebuilt_and_autobuild = True Combine rebuilt and autobuild models and use if better Rfree (X-ray only).

• include_side_in_templates = False Include side-chains beyond CB in templates as well as just main and CB (two runs).

• only_include_side_in_templates = None Include side-chains beyond CB in templates only (one run)

• use_templates_in_prediction = True Use templates if available (i.e., after first cycle)

• required_free_r_to_keep = 0.49 Skip refinements if R is worse than this value

• autobuild_resolution = 3.5 Worst resolution where autobuild density modification may be useful.

• autobuild_satisfactory_r_value = 0.25 Skip autobuild if R is this value or better

• autobuild_for_xray_density_modification = None Use autobuild rebuilding for density modification. Default is True if resolution is autobuild_resolution or better.

• autobuild_n_cycle_rebuild_max = 3 Maximum autobuild rebuild cycles

• autobuild_n_cycle_build_max = 1 Maximum autobuild build cycles

• autobuild_n_cycle_refine = 3 AutoBuild refinement cycles

• autobuild_quick = False Run AutoBuild in quick mode

• autobuild_min_resolution = 2 Minimum resolution for autobuilding. Resolution will be cut off at this value if it is better (smaller number).

• density_modify_at_start = None Density modify cryo-EM maps before analysis. Requires two half maps. Default is True if half maps are supplied and no full map is supplied, otherwise False. Works best if maps contain entire structure with boundary (at least 5 A) or are original full-sized maps.

• density_modify_with_model = None Density modify using rebuilt model each cycle. default is True. Requires two half-maps for cryo-EM. Works best for cryo-EM if maps contain entire structure with boundary (at least 5 A) or are original full-sized maps. Default is True for X-ray and False for cryo_EM

• vrms_value = None Value of VRMS to use in molecular replacement. This value will be used for all chains if set. Default is to estimate this value for each chain from the average pLDDT value after trimming.

• large_sequence = 1500 Length of sequence that requires job_size=large

• pause_after_mr = False Pause after MR and refinement (X-ray only). You can go on after checking your results.

• pause_after_docking = False Pause after docking and refinement (Cryo-EM only). You can go on after checking your results.

• stop_after_predict = False Stop after prediction. No map file needed if set.

• stop_after_mr = False Stop after MR.

• parallel_predictions = 1 Number of predictions to run at once. If more than one, no feedback is provided until the end. If there are waiting jobs in the queue running more than one at a time does not help (your first job may go near the front of the line but the others stay at the end until the first one finishes).

• write_prediction_files_and_wait = False Write prediction files in working directory and wait for result to appear. Only used for servers.

• put_files_for_prediction_here = None Program will put prediction files (pdb, msa) in this directory for the rest server to read (may look like /app/xxx/xxx)

• files_for_prediction_will_be_transfered_here = None Program will expect prediction files (pdb, msa) from the put_files_for_prediction_here directory will be copied to this directory so the rest server can read them. Your system has to automatically do this transfer.

• precalculated_msas_dir = /app/results/precalculated_msas Location of precalculated msas (usually on server)

• precalculated_results_dir = /app/results/precalculated_results Location of precalculated results (usually on server)

• precalculated_pae_dir = /app/results/precalculated_pae Location of precalculated pae files (usually on server)

• allow_precalculated_result_file = True Allow using precalculated results if inputs are identical

• allow_precalculated_msa_file = True Allow using precalculated msas if inputs are identical

• files_from_prediction_may_be_found_here = None Look for output from prediction here (in case full path is not available)

• rest_server_incoming = None Location on REST server where files should be placed

• max_wait_time = 100000 Maximum wait time for write_prediction_files_and_wait (sec)

• stop_if_internet_not_available = True Stop if attempting to predict models online and internet is not available

• number_of_models = 5 Number of models to generate for each chain. Normal way to set the value of random_seed_iterations. If a good plDDT is found, no more models are created, where good is good_enough_plddt (typically 80)

• use_supplied_random_seed_in_prediction = False Use random seed to set up prediction (default is to use a standard random seed).

• update_unique_sequences = True Update unique sequences based on MR or docking results

• minimum_residues = 20 Minimum residues in a chain

• maximum_residues = 10000 Maximum residues in a sequence that can be used for prediction

• break_into_chunks_if_length_is = 1500 If a sequence is less than maximum_residues but at least break_into_chunks_if_length_is, break it into chunks of length chunk_size with overlap of overlap_size, then reassemble using overlap window of overlap_match_size.

• chunk_size = 600 If a sequence is less than maximum_residues but at least break_into_chunks_if_length_is, break it into chunks of length chunk_size with overlap of overlap_size, then reassemble using overlap window of overlap_match_size.

• overlap_size = 200 If a sequence is less than maximum_residues but at least break_into_chunks_if_length_is, break it into chunks of length chunk_size with overlap of overlap_size, then reassemble using overlap window of overlap_match_size.

• overlap_match_size = 50 If a sequence is less than maximum_residues but at least break_into_chunks_if_length_is, break it into chunks of length chunk_size with overlap of overlap_size, then reassemble using overlap window of overlap_match_size.

• morph_reassembled_model = True If break_into_chunks_if_length_is is applied, after reassembling identify close domains that were in different chunks and repredict them to get their relationships. Then adjust the reassembly to minimize overlaps and match pair relationships. NOTE: This reduces overlaps and can improve domain relationships but can yield distortions of the predicted model.

• minimum_cycles = 2 Minimum cycles to carry out

• use_input_scaffold_if_supplied = True If input scaffold is provided, use it throughout (all cycles)

• default_maxit_dir = /app/results/rest/ Default place to look for maxit if not supplied. This directory should contain the whole maxit-v11.100-prod-src

• skip_copying_files = False Skip copying output files to GUI directory

• refinementmacro_cycles = 3 Refinement macro_cycles

• macro_cycles = 3 Refinement macro_cycles

• rest_serverurl = None The URL for the Phenix REST server Normally set automaticallyurl_type = *prediction ai Type of server url (prediction or ai). Normally set automaticallyport = None The port for interacting with the Phenix REST server Normally set automaticallytoken = None Authentication token for accessing the Phenix REST server. Normally set automaticallytimeout = 5 Time to keep trying to connect to a serverquick_check_interval = 1 Time in seconds between job status checkscheck_interval = 60 Time in seconds between job status checksmax_tries = 10 Number of tries to get result from servermax_tries_on_availability = 2 Number of tries to see if server is upstop_if_server_not_available = True Stop if server is not available (wrong url or down)job_size = small *medium large Size of job (small, medium, large)requires_gpu = True Job requires GPUrunning_server_test = False This is a test jobverbose = False Verbose output

• url = None The URL for the Phenix REST server Normally set automatically

• url_type = *prediction ai Type of server url (prediction or ai). Normally set automatically

• port = None The port for interacting with the Phenix REST server Normally set automatically

• token = None Authentication token for accessing the Phenix REST server. Normally set automatically

• timeout = 5 Time to keep trying to connect to a server

• quick_check_interval = 1 Time in seconds between job status checks

• check_interval = 60 Time in seconds between job status checks

• max_tries = 10 Number of tries to get result from server

• max_tries_on_availability = 2 Number of tries to see if server is up

• stop_if_server_not_available = True Stop if server is not available (wrong url or down)

• job_size = small *medium large Size of job (small, medium, large)

• requires_gpu = True Job requires GPU

• running_server_test = False This is a test job

• verbose = False Verbose output

• input_filesseq_file = None Input sequence file. Required if no models supplied. Used to create predicted models. Format should be Fasta or simple sequences separated by blank lines. One sequence per chain to be generated. Must match input models if both are supplied. (Fasta format or sequences separated by blank lines)xray_data_file = None Input X-ray data file (MTZ format).xray_data_label = None Label specifying which data column to use from xray_data_filexray_test_flag_label = None Label specifying which test flag column (if any) to use from xray_data_filetruncate_models_to_match_sequences = True If input sequences start after or end before supplied predicted models, trim the predictedd models to match sequencesdensity_select = None If set, trim map to region containing molecule (non-zero region) before use. Recommended for cryo-EM maps that have a large map that is mostly unused. Default is True if map is full-size and False if not.predicted_model = None One or more input models in one or more files (normally predicted models) to be placed. Normally should start at residue 1 and match sequence file exactly.b_value_field_is = *plddt rmsd b_value The B-factor field in predicted models can be pLDDT (confidence, 0-1 or 0-100) or rmsd (A) or a B-factor. Only applies to protein chains (always B-factor for non-protein).msa_list = None Multiple sequence alignment (MSA) file. Format is a3m only. First sequence in an MSA file must match a sequence in the sequence file exactly.models_are_already_placed = False You can specify that all models (predicted_model or model) are already placed (docked, placed in unit cell). No docking or MR is done if so. Equivalent to specifying a scaffold_model with the same contents as the model.model_copies_list = None List of number of copies to find of each model in predicted_model. Default is 1 (must specify all or none of them) . Specify all together like model_copies_list=1,2,1,1processed_model_file = None Processed model file (e.g., from phenix.process_predicted_model). If supplied, skip the process_predicted_model step and use this file. This model is expected to have one chain for each domain and actual B-values in the B-value field. It is expected that all poorly-predicted residues have been removed.docked_model_file = None Docked model (e.g., output of dock_processed_model). Not available in predict_and_build GUI. If supplied, use this file as the docked model. (Skip process_predicted_model and docking steps This model is expected to have a single chain with gaps for parts of the model that are not accurately known from the prediction. NOTE: You still need to supply the predicted model as the input model for this procedure in addition to this docked model.morphed_model_file = None Morphed model (e.g., full model morphed to match output of dock_processed_model). If supplied, skip the process_predicted_model and docking steps and use this file as the morphed model. This model is expected to have a single chain with no gaps. NOTE: You do not need to supply the predicted model as the input model for this procedure in addition to this morphed model.previous_model_file = None Previous model (from a previous run or external). Must match sequence of working model exactly. Used as a source of possible model information and as a

hypothesis for docking of working model. If docked_model_file is also supplied, previous_model is used only as source of model information.scaffold_model = None Scaffold model. If supplied, used as target for docking of each chain in predicted_model or model. Must be similar in sequence to models to be docked. If supplied, model_copies_list is ignored and docking is done by superposition onto this structure instead of docking into density or MR. If the scaffold model does not have all chains represented, they are added in by docking.scaffold_minimum_chain_cc = 0.35 Chains in scaffold model that have scaffold_minimum_cc are not re-docked, but those with lower CC are.base_scaffold_...

• seq_file = None Input sequence file. Required if no models supplied. Used to create predicted models. Format should be Fasta or simple sequences separated by blank lines. One sequence per chain to be generated. Must match input models if both are supplied. (Fasta format or sequences separated by blank lines)

• xray_data_file = None Input X-ray data file (MTZ format).

• xray_data_label = None Label specifying which data column to use from xray_data_file

• xray_test_flag_label = None Label specifying which test flag column (if any) to use from xray_data_file

• truncate_models_to_match_sequences = True If input sequences start after or end before supplied predicted models, trim the predictedd models to match sequences

• density_select = None If set, trim map to region containing molecule (non-zero region) before use. Recommended for cryo-EM maps that have a large map that is mostly unused. Default is True if map is full-size and False if not.

• predicted_model = None One or more input models in one or more files (normally predicted models) to be placed. Normally should start at residue 1 and match sequence file exactly.

• b_value_field_is = *plddt rmsd b_value The B-factor field in predicted models can be pLDDT (confidence, 0-1 or 0-100) or rmsd (A) or a B-factor. Only applies to protein chains (always B-factor for non-protein).

• msa_list = None Multiple sequence alignment (MSA) file. Format is a3m only. First sequence in an MSA file must match a sequence in the sequence file exactly.

• models_are_already_placed = False You can specify that all models (predicted_model or model) are already placed (docked, placed in unit cell). No docking or MR is done if so. Equivalent to specifying a scaffold_model with the same contents as the model.

• model_copies_list = None List of number of copies to find of each model in predicted_model. Default is 1 (must specify all or none of them) . Specify all together like model_copies_list=1,2,1,1

• processed_model_file = None Processed model file (e.g., from phenix.process_predicted_model). If supplied, skip the process_predicted_model step and use this file. This model is expected to have one chain for each domain and actual B-values in the B-value field. It is expected that all poorly-predicted residues have been removed.

• docked_model_file = None Docked model (e.g., output of dock_processed_model). Not available in predict_and_build GUI. If supplied, use this file as the docked model. (Skip process_predicted_model and docking steps This model is expected to have a single chain with gaps for parts of the model that are not accurately known from the prediction. NOTE: You still need to supply the predicted model as the input model for this procedure in addition to this docked model.

• morphed_model_file = None Morphed model (e.g., full model morphed to match output of dock_processed_model). If supplied, skip the process_predicted_model and docking steps and use this file as the morphed model. This model is expected to have a single chain with no gaps. NOTE: You do not need to supply the predicted model as the input model for this procedure in addition to this morphed

model.

• previous_model_file = None Previous model (from a previous run or external). Must match sequence of working model exactly. Used as a source of possible model information and as a hypothesis for docking of working model. If docked_model_file is also supplied, previous_model is used only as source of model information.

• scaffold_model = None Scaffold model. If supplied, used as target for docking of each chain in predicted_model or model. Must be similar in sequence to models to be docked. If supplied, model_copies_list is ignored and docking is done by superposition onto this structure instead of docking into density or MR. If the scaffold model does not have all chains represented, they are added in by docking.

• scaffold_minimum_chain_cc = 0.35 Chains in scaffold model that have scaffold_minimum_cc are not re-docked, but those with lower CC are.

• base_scaffold_minimum_chain_cc = -1 Value of scaffold_minimum_chain_cc in cases where scaffold is known to be good.

• fragments_model_file = None Fragments model (e.g., map_to_model.pdb). Used as a source of possible model information and as a hypothesis for docking of working model. If docked_model_file is also supplied, fragments_model is used only as source of model information.

• model_to_rebuild_file = None Model to be rebuilt. Must be already in place (already docked). Must match supplied sequences exactly.

• symmetry_file = None Symmetry file (.ncs_spec format or MATRX records) with reconstruction symmetry. Used in identification of unique part of map. NOTE: symmetry file applies to map file in original position. NOTE 2: only proper symmetry (point-group, helical) is allowed NOTE 3: Only applies to cryo-EM maps

• pae_file = None Optional input json file with matrix of inter-residue estimated errors (pae file)

• distance_model_file = None Distance_model_file. A PDB or mmCIF file containing the model corresponding to the PAE matrix. Only needed if weight_by_ca_ca_distances is True.

• search_model_copies = None Used internally only

• search_model = None used internally only

• map_modelfull_map = None Input map file (for cryo-EM). This can be a boxed or masked map showing just the molecule to dock (best) or a full map with symmetry. If your map has symmetry be sure to set asymmetric_map = False. If you have a map with symmetry you can supply a symmetry file if you want. Otherwise symmetry will be automatically determined.half_map = None Input half map files. Usually supply one full map or 2 half mapsmodel = None Input predicted model (e.g., AlphaFold model). Assumed to have pLDDT values in B-value field (or RMSD values). May have multiple chains. Normally use predicted_model instead.

• full_map = None Input map file (for cryo-EM). This can be a boxed or masked map showing just the molecule to dock (best) or a full map with symmetry. If your map has symmetry be sure to set asymmetric_map = False. If you have a map with symmetry you can supply a symmetry file if you want. Otherwise symmetry will be automatically determined.

• half_map = None Input half map files. Usually supply one full map or 2 half maps

• model = None Input predicted model (e.g., AlphaFold model). Assumed to have pLDDT values in B-value field (or RMSD values). May have multiple chains. Normally use predicted_model instead.

• output_filesoutput_model_prefix = None Output files with superposed models will begin with this prefixoutput_seq_file = None Sequence file (possibly edited) written to this file by PredictAndBuildpdb_out = None Used internally onlytemp_dir = None Temporary directory. Default is dock_and_build_xx where xx creates new directory

• output_model_prefix = None Output files with superposed models will begin with this prefix

• output_seq_file = None Sequence file (possibly edited) written to this file by PredictAndBuild

• pdb_out = None Used internally only

• temp_dir = None Temporary directory. Default is dock_and_build_xx where xx creates new directory

• crystal_inforesolution = None High-resolution limit for main search. This can be lower resolution than the data. The search is quicker at lower resolution. If your model is poor, try 2-3 A lower resolution than your data (i.e, if your data is 2.5 A, try 5 A).scattering_table = *None n_gaussian wk1995 it1992 electron neutron Choice of scattering table for structure factor calculations. Default for X-ray is n_gaussian, for cryoEM is electron.wrapping = None You can specify whether the map is wrapped (can map values outside bounds to inside with cell translations).asymmetric_map = None Specifies that this is an asymmetric map and no symmetry is to be supplied or found. Alternative to supplying a symmetry file or symmetry. Applies to cryo-EM reconstructions only.solvent_content = None Solvent fraction (content) of the cell. You can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.pdb70_text = None Database of PDB entries to be used as templates (pdb70 file). Normally used internally onlysequence = None Old-style sequence string (alternative to sequence file).chain_type = *PROTEIN DNA RNA Chain typemsa = None MSA as text. Supply MSA as text or as an msa_file. Used internallytemplates_as_string = None Templates as string (single PDB file with one or more chains)space_group = None Space group (normally read from the data file, applies to X-ray)space_group_alternatives = *HAND ALL NONE Space group alternatives for MR: ALL: all space groups in point group HAND: enantiomer and listed space group LIST: list supplied (not available) NONE: only supplied space groupunit_cell = None Unit Cell (normally read from the data file, applies to X-ray)unique_sequencesFile names and data labels.sequence = "Enter or edit sequence" copies = 1 Copies of this sequencelabel = ""

• resolution = None High-resolution limit for main search. This can be lower resolution than the data. The search is quicker at lower resolution. If your model is poor, try 2-3 A lower resolution than your data (i.e, if your data is 2.5 A, try 5 A).

• scattering_table = *None n_gaussian wk1995 it1992 electron neutron Choice of scattering table for structure factor calculations. Default for X-ray is n_gaussian, for cryoEM is electron.

• wrapping = None You can specify whether the map is wrapped (can map values outside bounds to inside with cell translations).

• asymmetric_map = None Specifies that this is an asymmetric map and no symmetry is to be supplied or found. Alternative to supplying a symmetry file or symmetry. Applies to cryo-EM reconstructions only.

• solvent_content = None Solvent fraction (content) of the cell. You can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.

• pdb70_text = None Database of PDB entries to be used as templates (pdb70 file). Normally used internally only

• sequence = None Old-style sequence string (alternative to sequence file).

• chain_type = *PROTEIN DNA RNA Chain type

• msa = None MSA as text. Supply MSA as text or as an msa_file. Used internally

• templates_as_string = None Templates as string (single PDB file with one or more chains)

• space_group = None Space group (normally read from the data file, applies to X-ray)

• space_group_alternatives = *HAND ALL NONE Space group alternatives for MR: ALL: all space groups in point group HAND: enantiomer and listed space group LIST: list supplied (not available) NONE: only supplied space group

• unit_cell = None Unit Cell (normally read from the data file, applies to X-ray)

• unique_sequencesFile names and data labels.sequence = "Enter or edit sequence" copies = 1 Copies of this sequencelabel = ""

• sequence = "Enter or edit sequence"

• copies = 1 Copies of this sequence

• label = ""

• iterationcycles = None Cycles of prediction and rebuilding (default is 10 for Thorough, 3 for Standard and 1 for Quickcycle = 1 Iteration cycle

• cycles = None Cycles of prediction and rebuilding (default is 10 for Thorough, 3 for Standard and 1 for Quick

• cycle = 1 Iteration cycle

• process_predicted_modelremove_low_confidence_residues = True Remove low-confidence residues (based on minimum plddt or maximum_rmsd, whichever is specified)continuous_chain = False When removing low-confidence residues, only trim from endssplit_model_by_compact_regions = True Split model into compact regions after removing low-confidence residues.maximum_domains = 3 Maximum domains to obtain. You can use this to merge the closest domains at the end of splitting the model. Make it bigger (and optionally make domain_size smaller) to get more domains. If model is processed in chunks, maximum_domains will apply to each chunk.domain_size = 15 Approximate size of domains to be found (A units). This is the resolution that will be used to make a domain map. If you are getting too many domains, try making domain_size bigger (maximum is 70 A).adjust_domain_size = True If more that maximum_domains are initially found, increase domain_size in increments of 5 A and take the value that gives the smallest number of domains, but at least maximum_domains.minimum_domain_length = None Minimum length of a domain to keep (reject at end if smaller). Default is 10 if no pae matrix or alt_pae_params=False and 20 for pae_matrix with alt_pae_params=Truemaximum_fraction_close = 0.3 Maximum fraction of CA in one domain close to one in another before merging themminimum_sequential_residues = None Minimum length of a short segment to keep (reject at end). Default is 5 if no pae matrix or alt_pae_params=False and 4 for pae_matrix with alt_pae_params=Trueminimum_remainder_sequence_length = 15 used to choose whether the sequence of a removed segment is written to the remainder sequence file.b_value_field_is = *plddt rmsd b_value The B-factor field in predicted models can be pLDDT (confidence, 0-1 or 0-100) or rmsd (A) or a B-factorinput_plddt_is_fractional = None You can specify if the input plddt values (in B-factor field) are fractional (0-1) or not (0-100). By default if all values are between 0 and 1 it is fractional.minimum_plddt = None If low-confidence residues are removed, the cutoff is defined by minimum_plddt or maximum_rmsd, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of input_plddt_is_fractional.maximum_rmsd = 1.5 If low-confidence residues are removed, the cutoff is defined by minimum_plddt or maximum_rmsd, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of input_plddt_is_fractional.default_maximum_rmsd = 1.5 Default value of maximum_rmsd, used if maximum_rmsd is not setsubtract_minimum_b = False If set, subtract the lowest B-value from all B-values just before writing out the final files. Does not affect the cutoff for removing low-

confidence residues.pae_power = None If PAE matrix (predicted alignment error matrix) is supplied, each edge in the graph will be weighted proportional to (1/pae**pae_power). Use this to try and get the number of domains that you want (try 1, 0.5, 1.5, 2). Default is 1 alt_pae_params=False and 2 for alt_pae_params=Truepae_cutoff = None If PAE matrix (predicted alignment error matrix) is supplied, graph edges will only be created for residue pairs with pae<pae_cutoff. Default is 5 alt_pae_params=False and 4 for alt_pae_params=Truepae_graph_resolution = None If PAE matrix (predicted alignment error matrix) is supplied, pae_graph_resolution regulates how aggressively the clustering algorithm is. Smaller values lead to larger clusters. Value should be larger than zero, and values larger than 5 are unlikely to be useful, Default is 0.5 alt_pae_params=False and 4 for alt_pae_params=Truealt_pae_params = False If PAE matrix is supplied, use alternative set of defaults (minimum_domain_length=20 minimum_sequential_residues=10 pae_power=2 pae_cutoff=4 pae_graph_resolutio...

• remove_low_confidence_residues = True Remove low-confidence residues (based on minimum plddt or maximum_rmsd, whichever is specified)

• continuous_chain = False When removing low-confidence residues, only trim from ends

• split_model_by_compact_regions = True Split model into compact regions after removing low-confidence residues.

• maximum_domains = 3 Maximum domains to obtain. You can use this to merge the closest domains at the end of splitting the model. Make it bigger (and optionally make domain_size smaller) to get more domains. If model is processed in chunks, maximum_domains will apply to each chunk.

• domain_size = 15 Approximate size of domains to be found (A units). This is the resolution that will be used to make a domain map. If you are getting too many domains, try making domain_size bigger (maximum is 70 A).

• adjust_domain_size = True If more that maximum_domains are initially found, increase domain_size in increments of 5 A and take the value that gives the smallest number of domains, but at least maximum_domains.

• minimum_domain_length = None Minimum length of a domain to keep (reject at end if smaller). Default is 10 if no pae matrix or alt_pae_params=False and 20 for pae_matrix with alt_pae_params=True

• maximum_fraction_close = 0.3 Maximum fraction of CA in one domain close to one in another before merging them

• minimum_sequential_residues = None Minimum length of a short segment to keep (reject at end). Default is 5 if no pae matrix or alt_pae_params=False and 4 for pae_matrix with alt_pae_params=True

• minimum_remainder_sequence_length = 15 used to choose whether the sequence of a removed segment is written to the remainder sequence file.

• b_value_field_is = *plddt rmsd b_value The B-factor field in predicted models can be pLDDT (confidence, 0-1 or 0-100) or rmsd (A) or a B-factor

• input_plddt_is_fractional = None You can specify if the input plddt values (in B-factor field) are fractional (0-1) or not (0-100). By default if all values are between 0 and 1 it is fractional.

• minimum_plddt = None If low-confidence residues are removed, the cutoff is defined by minimum_plddt or maximum_rmsd, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of input_plddt_is_fractional.

• maximum_rmsd = 1.5 If low-confidence residues are removed, the cutoff is defined by minimum_plddt or maximum_rmsd, whichever is defined (you cannot define both). A minimum plddt of 0.70 corresponds

to a maximum rmsd of 1.5. Minimum plddt values are fractional or not depending on the value of input_plddt_is_fractional.

• default_maximum_rmsd = 1.5 Default value of maximum_rmsd, used if maximum_rmsd is not set

• subtract_minimum_b = False If set, subtract the lowest B-value from all B-values just before writing out the final files. Does not affect the cutoff for removing low- confidence residues.

• pae_power = None If PAE matrix (predicted alignment error matrix) is supplied, each edge in the graph will be weighted proportional to (1/pae**pae_power). Use this to try and get the number of domains that you want (try 1, 0.5, 1.5, 2). Default is 1 alt_pae_params=False and 2 for alt_pae_params=True

• pae_cutoff = None If PAE matrix (predicted alignment error matrix) is supplied, graph edges will only be created for residue pairs with pae<pae_cutoff. Default is 5 alt_pae_params=False and 4 for alt_pae_params=True

• pae_graph_resolution = None If PAE matrix (predicted alignment error matrix) is supplied, pae_graph_resolution regulates how aggressively the clustering algorithm is. Smaller values lead to larger clusters. Value should be larger than zero, and values larger than 5 are unlikely to be useful, Default is 0.5 alt_pae_params=False and 4 for alt_pae_params=True

• alt_pae_params = False If PAE matrix is supplied, use alternative set of defaults (minimum_domain_length=20 minimum_sequential_residues=10 pae_power=2 pae_cutoff=4 pae_graph_resolution=4). Standard parameters (alt_pae_params=False) are: (minimum_domain_length=10 minimum_sequential_residues=5 pae_power=1 pae_cutoff=5 pae_graph_resolution=0.5).

• weight_by_ca_ca_distance = False Adjust the edge weighting for each residue pair according to the distance between CA residues. If this is True, then distance_model can be provided, otherwise supplied model will be used. See also distance_power

• distance_power = 1 If weight_by_ca_ca_distance is True, then edge weights will be multiplied by 1/distance**distance_power.

• stop_if_no_residues_obtained = True Raise Sorry and stop if processing yields no residues

• keep_all_if_no_residues_obtained = False Keep everything if processing yields no residues

• vrms_from_rmsd_intercept = 0.25 Estimate of vrms (error in model) from pLDDT will be based on vrms_from_rmsd_intercept + vrms_from_rmsd_slope * pLDDT where mean pLDDT of non-low_confidence_residues is used.

• vrms_from_rmsd_slope = 1.0 Estimate of vrms (error in model) from pLDDT will be based on vrms_from_rmsd_intercept + vrms_from_rmsd_slope * pLDDT where mean pLDDT of non-low_confidence_residues is used.

• break_into_chunks_if_length_is = 1500 If a sequence is at least break_into_chunks_if_length_is, break it into chunks of length chunk_size with overlap of overlap_size for domain identification using split_model_by_compact_regions without a pae matrix

• chunk_size = 600 If a sequence is at least break_into_chunks_if_length_is, break it into chunks of length chunk_size with overlap of overlap_size for domain identification using split_model_by_compact_regions without a pae_matrix

• overlap_size = 200 If a sequence is at least break_into_chunks_if_length_is, break it into chunks of length chunk_size with overlap of overlap_size for domain identification using split_model_by_compact_regions without a pae_matrix

• searchdock_chains_individually = False Dock each chain (identified by chain_id field) individually. This is useful if the chains might have different orientations in the map compared to the search model, such as in a search model that is a predicted model. Normally used along with create_unique_chain_at_end=True for predicted models to put the model back together.create_unique_chain_at_end = None Take all docked chains and try to create one chain that has no duplicate residue numbers and that has distances between ends of fragments consistent with the number of residues between them. If symmetry has been applied, create N copies representing that symmetry. Default is True if dock_chains_individually = True and False otherwise.minimum_cc_to_keep_domain = 0.2 Do not keep domains in create_unique_chain_at_end if cc is less than minimum_cc_to_keep_domain.weight_sequential_fragments_by_distance = None Try to dock a chain in a way that minimizes the distance between its first residue and the previously-docked residue with the highest residue number less than this, and similarly for the last residue and the next available placed residue. Default is True if create_unique_chain_at_end is set. Normally use only if you have a model with a single chain and you are docking pieces of that chain.choose_better_of_individual_and_group_docking = None Dock entire search model (normally one only) and also by chain...pick better-fitting of the two for each chainlow_res_search = True Try to fit by searching at low resolution firstdock_with_mr = True Try using MR to dock firstssm_search = None Try to fit by searching for secondary structure. Default is False for dock_in_map and True for dock_and_rebuild if dock_with_mr is not used.refine_cycles = 3 rigid-body refinement cyclesssm_search_min_cc = 0.30 Stop ssm search if this cc achievedbackup_resolution_cutoff_searches = 2 If initial fitting does not work, try up to this many times increasing resolution by backup_resolution_ratio each timebackup_resolution_ratio = 1.67 If initial fitting does not work, try up to backup_resolution_cutoff_searches times increasing the resolution by backup_resolution_ratio each timeresolution_radius_scale = 0.5 Resolution for low-res search will be resolution_radius_scale times the radius of gyration of the search model.align_moments = False Try to fit by aligning moments of inertia if size of density region and molecule are similarmax_radius_ratio = 2. Radius of gyration of molecule and density must be within max_radius_ratio of each otherradius_scale = 1.5 Mask for density and atoms will be radius_scale times the radius of gyration of modeluse_symmetry = True If search_model_copies or search_map_copies are the same for each model and greater than one and allow_symmetry is set, use symmetry in the map to place all copies after the first one.skip_if_low_cc = True Skip solution before rigid-body refinement if map-model CC is less than half of min_cc.rigid_body_refinement = True Run rigid-body refinement on final modelrigid_body_refinement_single_unit = True Run rigid-body refinement with just one unit (do not break up into chains)rigid_body_refinement_split_method = *chain_id segid When splitting up molecule for rigid-body refinement (if rigid_body_refinement_single_unit=False), use either chain_id or segid to split up moleculerigid_body_refinement_resolution = None Run rigid-body refinement at this resolution if specifiedappend_to_fixed_model = True Append placed search model to fixed model (if any) after searchmin_cc = 0.4 If quick run, stop if minimum CC is achieved in local search. Also always skip if starting CC_mask is less than 1/4 min_cc.run_in_boxes = True Run on sub-boxes and combine at the endtarget_box_size = 60 Try to get boxes about this big on a side (grid units)target_boxes = None Try to get this many boxes. Default is nproc unless this makes box size much smaller than target_box_sizebox_to_run = None Run only this boxbox_overlap_scale = 1 box overlap (ov...

• dock_chains_individually = False Dock each chain (identified by chain_id field) individually. This is useful if the chains might have different orientations in the map compared to the search model, such as in a search model that is a predicted model. Normally used along with create_unique_chain_at_end=True for predicted models to put the model back together.

• create_unique_chain_at_end = None Take all docked chains and try to create one chain that has no duplicate residue numbers and that has distances between ends of fragments consistent with the number

of residues between them. If symmetry has been applied, create N copies representing that symmetry. Default is True if dock_chains_individually = True and False otherwise.

• minimum_cc_to_keep_domain = 0.2 Do not keep domains in create_unique_chain_at_end if cc is less than minimum_cc_to_keep_domain.

• weight_sequential_fragments_by_distance = None Try to dock a chain in a way that minimizes the distance between its first residue and the previously-docked residue with the highest residue number less than this, and similarly for the last residue and the next available placed residue. Default is True if create_unique_chain_at_end is set. Normally use only if you have a model with a single chain and you are docking pieces of that chain.

• choose_better_of_individual_and_group_docking = None Dock entire search model (normally one only) and also by chain...pick better-fitting of the two for each chain

• low_res_search = True Try to fit by searching at low resolution first

• dock_with_mr = True Try using MR to dock first

• ssm_search = None Try to fit by searching for secondary structure. Default is False for dock_in_map and True for dock_and_rebuild if dock_with_mr is not used.

• refine_cycles = 3 rigid-body refinement cycles

• ssm_search_min_cc = 0.30 Stop ssm search if this cc achieved

• backup_resolution_cutoff_searches = 2 If initial fitting does not work, try up to this many times increasing resolution by backup_resolution_ratio each time

• backup_resolution_ratio = 1.67 If initial fitting does not work, try up to backup_resolution_cutoff_searches times increasing the resolution by backup_resolution_ratio each time

• resolution_radius_scale = 0.5 Resolution for low-res search will be resolution_radius_scale times the radius of gyration of the search model.

• align_moments = False Try to fit by aligning moments of inertia if size of density region and molecule are similar

• max_radius_ratio = 2. Radius of gyration of molecule and density must be within max_radius_ratio of each other

• radius_scale = 1.5 Mask for density and atoms will be radius_scale times the radius of gyration of model

• use_symmetry = True If search_model_copies or search_map_copies are the same for each model and greater than one and allow_symmetry is set, use symmetry in the map to place all copies after the first one.

• skip_if_low_cc = True Skip solution before rigid-body refinement if map-model CC is less than half of min_cc.

• rigid_body_refinement = True Run rigid-body refinement on final model

• rigid_body_refinement_single_unit = True Run rigid-body refinement with just one unit (do not break up into chains)

• rigid_body_refinement_split_method = *chain_id segid When splitting up molecule for rigid-body refinement (if rigid_body_refinement_single_unit=False), use either chain_id or segid to split up molecule

• rigid_body_refinement_resolution = None Run rigid-body refinement at this resolution if specified

• append_to_fixed_model = True Append placed search model to fixed model (if any) after search

• min_cc = 0.4 If quick run, stop if minimum CC is achieved in local search. Also always skip if starting CC_mask is less than 1/4 min_cc.

• run_in_boxes = True Run on sub-boxes and combine at the end

• target_box_size = 60 Try to get boxes about this big on a side (grid units)

• target_boxes = None Try to get this many boxes. Default is nproc unless this makes box size much smaller than target_box_size

• box_to_run = None Run only this box

• box_overlap_scale = 1 box overlap (overlap of boxes) will be box_overlap_scale times the density radius

• edge_ratio = 10 box edge box_overlap times edge_ratio

• density_radius = None Radius for density to be cut out and compared. Default is 6 times the resolution.

• model_radius = 3 Radius for removing density near fixed_model

• zero_value = 0 Value to set map in regions overlapping fixed model

• density_peaks = 20 Number of NCS-related peaks of density to check

• delta_phi = 20 Angular spacing of search

• max_rot = None Maximum rotations to try

• rotz_only = None Rotate only around Z

• single_positions_to_try = 10 Number of offset positions to try in optimizing orientation. Positions along the chain are selected as centers and a local fit near each position is carried out. The resulting offsets relative to the original placement are used to optimize the overall orientation and position.

• max_position_shift_frac = 0.05 Maximum fractional positional shift in single_positions run

• min_relative_cc = 0.67 Minimum local CC relative to original CC to keep a local search. This is a way to reject local searches that are completely wrong.

• sieve_fit = None Use sieve_fit fraction of single positions in fitting. If None, use all

• ncs_copies_max = None Maximum number of matching models to write. If more than one they will be written as MODEL records in the output PDB file. You can get them individually afterwards with phenix.pdbtools placed_model.pdb keep="model 1" etc.

• start_rot = None Three numbers rotx, rotz, rotx defining the starting rotation of the search model. Normally used along with delta_phi=1000 or max_rot=1 to generate exactly one defined rotation.

• search_center = None Optional coordinates in search model for centering search. Note this is different from target_search_center which is the location xyz in the map to look.

• search_center_selection = None Optional selection defining coordinates in search model for centering search.

• target_search_center = None Optional coordinates in reference map where search_center should be approximately located after superimposing maps. Used to eliminate possible superpositions that place the search center elsewhere. Overlap scores decreased based on distance to target_search_center/density_radius.

• model_search_position = None Optional coordinates (usually part of search model) for matching to target_search_position after transformation. These can be specified in addition to target_search_center. Can have multiple model_search_position and target_search_position pairs by specifying each multiple times in order. NOTE: Not compatible with map search

• target_search_position = None Target positions for model_search_position after transformation.

• search_position_radius = None Radius for comparison of target_search_position and transformed search_position values. Default is density_radius. If specified, must be a single value or the same number of values as entries in model_search_position and target_search_position

• rot_id_n = None Number of rotation groups. Along with rot_id_group, allows defining groups of rotations to be carried out in one run. .short_caption = Number of rotation groups

• rot_id_group = None rotation group to include. See rot_id_n.

• map_box = True Run map_box to extract useful part of map before search

• fix_search_position = False You can choose to not move your model center of mass to the origin by fixing the search position

• search_box_size = None You can choose the size of the search box for local searches. Normally set by default corresponding to 3 times density_radius

• lower_bounds = None You can select a part of your map for analysis with lower_bounds and upper_bounds.

• upper_bounds = None You can select a part of your map for analysis with lower_bounds and upper_bounds.

• keep_search_order = None Keep search order as input

• remove_water = False Remove waters and other hetero atoms from input files

• predictionprediction_method = *alphafold Prediction method to usetemplate_search_method = *mmseqs2 structure_search Method to identify templates from PDB and generate MSAs. If structure_search is set, you can specify what PDB databases to use (same as in phenix.structure_search). If structure_search is set, you must supply your own MSA file with the keyword upload_msa_file=True (structure_search does not generate MSAs).starting_alphafold_model = None Starting AlphaFold model. You can supply an AlphaFold model and skip the initial AlphaFold step. This is equivalent to setting up an output_directory with just an AlphaFold model named as the expected first AlphaFold model and specifying carry_on=True.input_directory = ColabInputs Input directory containing density map. The map filename must start with the same characters as the jobname (only including characters before the first underscore). If you are supplying an MSA file it goes here as well.output_directory = ColabOutputs Output directory. Copy outputs to output_directory. Used to restart with carry on.save_outputs_in_google_drive = False If run on Colab, copy outputs to output_directory in Google drivecontent_dir = None Content directory. Default is working directorymaxit_path = None Path to maxit (pdb to cif) converter. Optional.data_dir = /mnt Data directory (location of AlphaFold parameters)upload_file_with_jobname_resolution_sequence_lines = None Upload a file with a set of jobs (Colab only). Each line in the file is a jobname, resolution, and sequencemaximum_cycles = 10 Maximum cycles to carry outcycle_rmsd_to_resolution_ratio = 0.25 Stop iteration if rmsd between subsequent AlphaFold models is less than cycle_rmsd_to_resolution_ratio times the resolution for two cycles in a rowsignificant_increase_in_residues = 5 Significant increase in residues in modelpassword = None Phenix download password (Colab only). The password used to download Phenix at your institution. Updated weekly, so you may need to request a new one frequently.version = dev-4502 Version of Phenix to run (Colab only)query_sequence = None Sequenceresolution = None Resolution of map (A). Internal use only. Normally set instead crystal_info.resolution.jobname = None Name of this job. The first characters before any underscore must be unique and will define the first characters of the corresponding map file in the input directory. The job name will normally also be the name of the working directory. Used internally onlyuse_msa = True Use multiple sequence alignments at some pointskip_all_msa_after_first_cycle = False Skip multiple sequence alignments after first cycleinclude_templates_from_pdb = True Include templates from PDB. Note special behavior when

running predict_and_build: applies only on first cycle, and if PhenixServer or Colab used prediction will be run with and without templates and PDB and highest plddt model will be keptmaximum_templates_from_pdb = 20 Maximum templates from PDB to includerelease_date = None release date for templates from pdb (only use up to this date. Format is 2020-05-14.upload_msa_file = False Supply MSA directly (.a3m format). You can supply the MSA as a file in your input_directory and it will be used instead of using mmseqs2 to generate an MSA. Your file name must end in .a3m. The format is two lines per sequence, the first starts with a greater-than sign and is ignored, the second is a sequence of letters or minus signs. All sequence lines must have the same length. The first sequence must be the target.upload_manual_templates = False Supply templates for AlphaFold prediction. Used in the same way as templates from the PDB unless uploaded_templates_are_map_to_model is set. May be .cif or .pdb files. Templates must start with characters matching the first characters in the jobname (before the first underscore), the remainder of the file name can be anything but just end in .cif or .pdb. The files must be in the input_directory or be uploaded (in Colab only).uploaded_tem...

• prediction_method = *alphafold Prediction method to use

• template_search_method = *mmseqs2 structure_search Method to identify templates from PDB and generate MSAs. If structure_search is set, you can specify what PDB databases to use (same as in phenix.structure_search). If structure_search is set, you must supply your own MSA file with the keyword upload_msa_file=True (structure_search does not generate MSAs).

• starting_alphafold_model = None Starting AlphaFold model. You can supply an AlphaFold model and skip the initial AlphaFold step. This is equivalent to setting up an output_directory with just an AlphaFold model named as the expected first AlphaFold model and specifying carry_on=True.

• input_directory = ColabInputs Input directory containing density map. The map filename must start with the same characters as the jobname (only including characters before the first underscore). If you are supplying an MSA file it goes here as well.

• output_directory = ColabOutputs Output directory. Copy outputs to output_directory. Used to restart with carry on.

• save_outputs_in_google_drive = False If run on Colab, copy outputs to output_directory in Google drive

• content_dir = None Content directory. Default is working directory

• maxit_path = None Path to maxit (pdb to cif) converter. Optional.

• data_dir = /mnt Data directory (location of AlphaFold parameters)

• upload_file_with_jobname_resolution_sequence_lines = None Upload a file with a set of jobs (Colab only). Each line in the file is a jobname, resolution, and sequence

• maximum_cycles = 10 Maximum cycles to carry out

• cycle_rmsd_to_resolution_ratio = 0.25 Stop iteration if rmsd between subsequent AlphaFold models is less than cycle_rmsd_to_resolution_ratio times the resolution for two cycles in a row

• significant_increase_in_residues = 5 Significant increase in residues in model

• password = None Phenix download password (Colab only). The password used to download Phenix at your institution. Updated weekly, so you may need to request a new one frequently.

• version = dev-4502 Version of Phenix to run (Colab only)

• query_sequence = None Sequence

• resolution = None Resolution of map (A). Internal use only. Normally set instead crystal_info.resolution.

• jobname = None Name of this job. The first characters before any underscore must be unique and will define the first characters of the corresponding map file in the input directory. The job name will normally

also be the name of the working directory. Used internally only

• use_msa = True Use multiple sequence alignments at some point

• skip_all_msa_after_first_cycle = False Skip multiple sequence alignments after first cycle

• include_templates_from_pdb = True Include templates from PDB. Note special behavior when running predict_and_build: applies only on first cycle, and if PhenixServer or Colab used prediction will be run with and without templates and PDB and highest plddt model will be kept

• maximum_templates_from_pdb = 20 Maximum templates from PDB to include

• release_date = None release date for templates from pdb (only use up to this date. Format is 2020-05-14.

• upload_msa_file = False Supply MSA directly (.a3m format). You can supply the MSA as a file in your input_directory and it will be used instead of using mmseqs2 to generate an MSA. Your file name must end in .a3m. The format is two lines per sequence, the first starts with a greater-than sign and is ignored, the second is a sequence of letters or minus signs. All sequence lines must have the same length. The first sequence must be the target.

• upload_manual_templates = False Supply templates for AlphaFold prediction. Used in the same way as templates from the PDB unless uploaded_templates_are_map_to_model is set. May be .cif or .pdb files. Templates must start with characters matching the first characters in the jobname (before the first underscore), the remainder of the file name can be anything but just end in .cif or .pdb. The files must be in the input_directory or be uploaded (in Colab only).

• uploaded_templates_are_map_to_model = False The manual templates are models that may or may not have the sequence of the alphafold models. These are used only as suggestions for placement of the main chain.

• upload_maps = True Use maps (required to be True)

• random_seed = 7231771 Random seed. Used internally

• random_seed_iterations = 1 Random seed iterations of AlphaFold in first cycle. The model with the highest plDDT will be used. If running predict_and_build and include_templates_from_pdb is True this many iterations will be carried out with and without templates . In predict_and_build and predict_chain set number_of_models instead.

• minimum_random_seed_iterations = 1 Random seed iterations of AlphaFold after first cycle. The model with the highest plDDT will be used

• big_improvement = 10 How much improvement in plDDT is worth going through all randomization cycles

• good_enough_plddt = 80 Value of plDDT that is good enough to not make any more models

• nproc = 4 Number of processors to use. Internal use only. Normally set instead control.nproc

• debug = False Debugging run (print traceback error messages)

• get_msa_only = False Just get MSA and save it on server, no prediction. Does not return the MSA

• carry_on = False Carry on from where previous run ended. Used (usually in Colab) to go on after a crash or timeout. Requires that files are saved in the output directory

• cif_dir = None Location of templates (normally set automatically)

• template_hit_list = None List of templates (normally set automatically)

• jobnames = None List of jobnames (normally set automatically)

• resolutions = None List of resolutions (normally set automatically)

• include_templates_from_pdb_list = None List of include_templates_from_pdb ((normally set automatically)

• include_side_in_templates_list = None List of include_side_in_templates_list ((normally set automatically)

• map_filename_dict = None Map filename dict (used internally only)

• msa_filename_dict = None MSA filename dict (used internally only)

• cif_filename_dict = None CIF filename dict (used internally only)

• query_sequences = None List of query sequences (one per jobname) (used internally only)

• manual_templates_uploaded = None List of manual templates (used internally only)

• upload_dir = None Upload directory (used internally only)

• working_directory = None Working directory (used internally only)

• maps_uploaded = None List of maps (used internally only)

• msas_uploaded = None List of msas (used internally only)

• num_models = 1 Number of models (used internally only)

• homooligomer = 1 Number of copies (used internally only)

• cycle = None Cycle number (internal use only)

• host_url = https://api.colabfold.com Host url for mmseqs

• use_env = None Use env (internal use only)

• use_custom_msa = None Use custom msa (internal use only)

• use_templates = None Use templates (internal use only)

• template_paths = None Template paths (internal use only)

• mtm_file_name = None map_to_model file name (internal use only)

• cycle_model_file_name = None cycle_model_file_name (internal use only)

• previous_final_model_name = None previous_final_model_name (internal use only)

• msa = None MSA (internal use only)

• msa_is_msa_object = None MSA info (internal use only)

• deletion_matrix = None Deletion matrix (internal use only)

• structure_search_paramsstructure_searchpdb_file = None Enter a PDB file name sequence = None Optional Fasta sequence file. Only needed for a quick sequence search against RCSB without a PDB. output_prefix = 'output' Provide an output prefix if needed blastpath = None Enter path to blastall executable sequence_only = False Do a Blast search against PDBaa sequence instead of doing a Ramanchandran-based structure searchstructure_only = False Do only a Ramanchandran-based structure search. db_used = 'rcsb' structure database used in search. rcsb, scop95, or AF2db = 'rcsb' Database used in search. rcsb, scop95, or AF2.get_ligand = False Use get_ligand=True to retrive ligands.get_ramacode_only = False Generate Rama code for input pdb/cif only. This is for developers only.get_xml_only = False Get BLAST XML output returned as a string object. No coordinate superposition will be performed. Developers only.use_pdb100aa = False Use PDB100 sequence database for sequence search.use_custom_db = False Use custom database specified by custom_db_files/custom_db_dir.custom_db_dir = None The directory of pdb/cif files to make custom database. Default is current directorycustom_db_files = None Filenames of the pdb/cif files seperated by spaces for database. If none specified, all pdb/cif in the custom_db_dir will be collectedatom_selection =

'all' Choose part of the pdb used in the search (default=all). for example: chain B, resseq 113:219, ... etc. get_pdb = 10 get_pdb=N will collect and superpose the top N homologous pdbs. Use get_pdb=0 to disable this option.deposited_before = 0 Specify the latest year of matching structures to be considered for scoring. Pdbs deposited after this year will be discarded.deposited_after = 0 Specify the earliest year of matching structures to be considered for scoring. Pdbs deposited before this year will be discarded.batch_size = 0 Process the pdbs in batch of <batch_size> until <min_match> hits are identified or until all <get_pdb> pdbs are processedmin_match = 0 Finish structure_search when <min_match> matches are found. Usually uses with <trim_ends> to exit the search once find suitable pdbs.keep_all_pdb = False Keep all the PDB files, including full PDB, PDB_Chain and superposed PDB_Chain. Default is False which will keep only superposed PDB_Chain files in the directory specified in the output message.trim_ends = False Remove terminal residues of hit pdbs extending beyond those of the the target pdb.write_pdb = True Set to False if no output pdb file is needed. Sometimes useful if use Structure_Search within another program and only want to pass pdb objects.write_results = True Set to False if no output results/log files is needed. Useful when calling Structure_Search within another program and only want to pass pdb objects.trim_hit_pdb = False Remove extra domains, extended loops, and unfit portions of hit pdbs after superposed to the target pdb.pickle_hits = False Pickle blast hit results from xml output.coot_display = False (default) Display output pdb files in coot.ask_coot = True prompt for coot display optiosPDB_MIRRORDIR = None Enter the top directory of local RCSB PDB mirror. The program will try to retrieve PDBs and/or structure factors from this mirror first. Note this assumes the directory trees under it follows those in RCSB -- pdb files as 'pdb####.ent.gz' in PDB_MIRRORDIR/data/structures/divided/pdb directory. If you use PDB's rsync script, this variable would be the same as the $MIRRORDIR set in the scriptPDB_MIRROR_MMCIF = None Enter the parent directory of the mmcif files in the local PDB mirror. MMCIFs will be retrieved from subdirectory ## where ## are the second and third letters in the PDB id. This keyword should be $PDB_MIRRORDIR/data/structures/divided/mmcif directory. PDB_MIRROR_PDB = None Enter the parent directory of the PDB files in the local PDB mirror. PDBs will be retrieved from subdirectory ## where ## are the second and...

• structure_searchpdb_file = None Enter a PDB file name sequence = None Optional Fasta sequence file. Only needed for a quick sequence search against RCSB without a PDB. output_prefix = 'output' Provide an output prefix if needed blastpath = None Enter path to blastall executable sequence_only = False Do a Blast search against PDBaa sequence instead of doing a Ramanchandran-based structure searchstructure_only = False Do only a Ramanchandran-based structure search. db_used = 'rcsb' structure database used in search. rcsb, scop95, or AF2db = 'rcsb' Database used in search. rcsb, scop95, or AF2.get_ligand = False Use get_ligand=True to retrive ligands.get_ramacode_only = False Generate Rama code for input pdb/cif only. This is for developers only.get_xml_only = False Get BLAST XML output returned as a string object. No coordinate superposition will be performed. Developers only.use_pdb100aa = False Use PDB100 sequence database for sequence search.use_custom_db = False Use custom database specified by custom_db_files/custom_db_dir.custom_db_dir = None The directory of pdb/cif files to make custom database. Default is current directorycustom_db_files = None Filenames of the pdb/cif files seperated by spaces for database. If none specified, all pdb/cif in the custom_db_dir will be collectedatom_selection = 'all' Choose part of the pdb used in the search (default=all). for example: chain B, resseq 113:219, ... etc. get_pdb = 10 get_pdb=N will collect and superpose the top N homologous pdbs. Use get_pdb=0 to disable this option.deposited_before = 0 Specify the latest year of matching structures to be considered for scoring. Pdbs deposited after this year will be discarded.deposited_after = 0 Specify the earliest year of matching structures to be considered for scoring. Pdbs deposited before this year will be discarded.batch_size = 0 Process the pdbs in batch of <batch_size> until <min_match> hits are identified or until all <get_pdb> pdbs are processedmin_match = 0 Finish structure_search when <min_match> matches are found. Usually uses with <trim_ends> to

exit the search once find suitable pdbs.keep_all_pdb = False Keep all the PDB files, including full PDB, PDB_Chain and superposed PDB_Chain. Default is False which will keep only superposed PDB_Chain files in the directory specified in the output message.trim_ends = False Remove terminal residues of hit pdbs extending beyond those of the the target pdb.write_pdb = True Set to False if no output pdb file is needed. Sometimes useful if use Structure_Search within another program and only want to pass pdb objects.write_results = True Set to False if no output results/log files is needed. Useful when calling Structure_Search within another program and only want to pass pdb objects.trim_hit_pdb = False Remove extra domains, extended loops, and unfit portions of hit pdbs after superposed to the target pdb.pickle_hits = False Pickle blast hit results from xml output.coot_display = False (default) Display output pdb files in coot.ask_coot = True prompt for coot display optiosPDB_MIRRORDIR = None Enter the top directory of local RCSB PDB mirror. The program will try to retrieve PDBs and/or structure factors from this mirror first. Note this assumes the directory trees under it follows those in RCSB -- pdb files as 'pdb####.ent.gz' in PDB_MIRRORDIR/data/structures/divided/pdb directory. If you use PDB's rsync script, this variable would be the same as the $MIRRORDIR set in the scriptPDB_MIRROR_MMCIF = None Enter the parent directory of the mmcif files in the local PDB mirror. MMCIFs will be retrieved from subdirectory ## where ## are the second and third letters in the PDB id. This keyword should be $PDB_MIRRORDIR/data/structures/divided/mmcif directory. PDB_MIRROR_PDB = None Enter the parent directory of the PDB files in the local PDB mirror. PDBs will be retrieved from subdirectory ## where ## are the second and third letters in the ...

• pdb_file = None Enter a PDB file name

• sequence = None Optional Fasta sequence file. Only needed for a quick sequence search against RCSB without a PDB.

• output_prefix = 'output' Provide an output prefix if needed

• blastpath = None Enter path to blastall executable

• sequence_only = False Do a Blast search against PDBaa sequence instead of doing a Ramanchandran-based structure search

• structure_only = False Do only a Ramanchandran-based structure search.

• db_used = 'rcsb' structure database used in search. rcsb, scop95, or AF2

• db = 'rcsb' Database used in search. rcsb, scop95, or AF2.

• get_ligand = False Use get_ligand=True to retrive ligands.

• get_ramacode_only = False Generate Rama code for input pdb/cif only. This is for developers only.

• get_xml_only = False Get BLAST XML output returned as a string object. No coordinate superposition will be performed. Developers only.

• use_pdb100aa = False Use PDB100 sequence database for sequence search.

• use_custom_db = False Use custom database specified by custom_db_files/custom_db_dir.

• custom_db_dir = None The directory of pdb/cif files to make custom database. Default is current directory

• custom_db_files = None Filenames of the pdb/cif files seperated by spaces for database. If none specified, all pdb/cif in the custom_db_dir will be collected

• atom_selection = 'all' Choose part of the pdb used in the search (default=all). for example: chain B, resseq 113:219, ... etc.

• get_pdb = 10 get_pdb=N will collect and superpose the top N homologous pdbs. Use get_pdb=0 to disable this option.

• deposited_before = 0 Specify the latest year of matching structures to be considered for scoring. Pdbs deposited after this year will be discarded.

• deposited_after = 0 Specify the earliest year of matching structures to be considered for scoring. Pdbs deposited before this year will be discarded.

• batch_size = 0 Process the pdbs in batch of <batch_size> until <min_match> hits are identified or until all <get_pdb> pdbs are processed

• min_match = 0 Finish structure_search when <min_match> matches are found. Usually uses with <trim_ends> to exit the search once find suitable pdbs.

• keep_all_pdb = False Keep all the PDB files, including full PDB, PDB_Chain and superposed PDB_Chain. Default is False which will keep only superposed PDB_Chain files in the directory specified in the output message.

• trim_ends = False Remove terminal residues of hit pdbs extending beyond those of the the target pdb.

• write_pdb = True Set to False if no output pdb file is needed. Sometimes useful if use Structure_Search within another program and only want to pass pdb objects.

• write_results = True Set to False if no output results/log files is needed. Useful when calling Structure_Search within another program and only want to pass pdb objects.

• trim_hit_pdb = False Remove extra domains, extended loops, and unfit portions of hit pdbs after superposed to the target pdb.

• pickle_hits = False Pickle blast hit results from xml output.

• coot_display = False (default) Display output pdb files in coot.

• ask_coot = True prompt for coot display optios

• PDB_MIRRORDIR = None Enter the top directory of local RCSB PDB mirror. The program will try to retrieve PDBs and/or structure factors from this mirror first. Note this assumes the directory trees under it follows those in RCSB -- pdb files as 'pdb####.ent.gz' in PDB_MIRRORDIR/data/structures/divided/pdb directory. If you use PDB's rsync script, this variable would be the same as the $MIRRORDIR set in the script

• PDB_MIRROR_MMCIF = None Enter the parent directory of the mmcif files in the local PDB mirror. MMCIFs will be retrieved from subdirectory ## where ## are the second and third letters in the PDB id. This keyword should be $PDB_MIRRORDIR/data/structures/divided/mmcif directory.

• PDB_MIRROR_PDB = None Enter the parent directory of the PDB files in the local PDB mirror. PDBs will be retrieved from subdirectory ## where ## are the second and third letters in the PDB id. This keyword should be $PDB_MIRRORDIR/data/structures/divided/pdb directory. We recommend setting PDB_MIRRORDIR and it will take care of both PDB_MIRROR_PDB and others together. However, users may choose to specify PDB_MIRROR_PDB directly

• PDB_MIRROR_STRUCTURE_FACTORS = None Enter the parent directory of the PDB files in the local PDB mirror. structure factors s will be retrieved from subdirectory ## where ## are the second and third letters in the PDB id. This keyword should be the same as the $PDB_MIRRORDIR/data/structures/divided/structure_factors directory. We recommend setting PDB_MIRRORDIR and it will take care of both PDB_MIRROR_PDB and DB_MIRROR_STRUCTURE_FACTORS together. However, users may choose to specify PDB_MIRROR_STRUCTURE_FACTORS directly

• local_pdb_dir = None Enter the path directly to your local PDB repository.

• verbose = False verbose output

• debug = False debugging output

• job_title = None Job title in PHENIX GUI, not used on command line

• guiGUI-specific parameter required for output directoryoutput_dir = None

• output_dir = None

• buildrebuild_strategy = Thorough Standard Quick Rebuilding strategy. Standard is up to 3 cycles, refining and replacing poorly-fitting loops in predicted models each cycle, using autobuild for density modification (if Xray). Quick is refinement only, one cycle. Thorough is up to 10 cycles, extensive rebuilding.refine_only = None Refine only, no rebuilding steps (set automatically with Quick)refine_only_resolution = 3.5 Default cutoff for using Refine onlyrun_fit_loops = True Run standard fit_loopsrun_iterative_morph = True Run iterative morphingrun_trace_loops_through_density = True Run loop fitting with trace_through_density algorithmrun_refine = True Refine morphed model. Required for run_fit_loops or run_trace_loops_through_densityrun_iterative_resolution_refine = True Refine morphed model with iterative resolution methodrun_extend = True Extend ends of morphed modelextract_unique = True Extract unique part of map. Applies after density_select (if set).use_symmetry_in_extract_unique = True Use symmetry from symmetry file (if available) when extracting unique part of map.acceptable_docking_cc = 0.5 Acceptable docking CC for a chainminimum_docking_cc = 0.15 Minimum docking CC for a chainminimum_cutoff = 0.1 Minimum cutoff for estimating density in better parts of modelreasonable_cc_ratio = 0.80 Acceptable CC (ratio) for a chain relative to average. Used to make sure a chain entirely in very bad density is not keptreasonable_cc_diff = 0.15 Acceptable CC (difference) for a chain relative to average. Used to make sure a chain entirely in very bad density is not keptshift_field_distance = None Shift field characteristic distance (default = 10 A) for morphingcc_sd_ratio = 3. Keep residues with CC at least within cc_sd_ratio of the mean CC for good residuescc_sd_ratio_end = 2. Keep residues with CC at least within cc_sd_ratio_end of the mean CC for good residues (applies to end of fragments)cc_sd_ratio_ok = 2. Residues with with CC at least within cc_sd_ratio_ok of the mean CC for good residues are ok (do not delete on the basis of plddt)max_gap_ratio = 3. Allow CA-CA distance to be up to max_gap_ratio times expectedmaximum_connectivity_deviation = 15 Maximum connectivity deviation. Reject a solution with bigger deviation than this plus 2 * resolution.keep_fraction_of_best = 0.5 Acceptable CC to keep as ratio to best found. Applies if best found is at least acceptable_docking_cc.keep_maximum_entries = 10 Maximum dock positions to keeprmsd_for_similar_placement = None RMSD value indicating that two placements are similar. default is resolution of maprigid_body_refine_cycles = 1 Refinement cycles for rigid-body refinementoverlap_ca_ca_distance = 3 Overlap distance for CA-CA atoms (or P-P)ca_ca_distance = 3.8 CA-CA distance (or P-P)allowed_fraction_overlapping = 0.10 Allowed fraction overlappingmaximum_combinations = 100000 Maximum combinations of placements to considerproceed_with_any_symmetry = False Run even with symmetry that is not point-group or helical. Not recommended as symmetry may not work properlybox_cushion = 20 Size of buffer around model when boxingrefine_cycles = 3 Refinement cyclesloop_refine_cycles = 5 Refinement cycles for loopsloop_backup_residues = 3 Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial gapresidues_to_trim = 5 Residues to trim on each end of all trimmed fragmentsfind_ncs_from_model = True Find NCS (symmetry) from working models and apply in density modification. Also turns on finding NCS in any autobuild density modification (including density-based search)allow_split_more_than_one_chain_for_mr = False Allow splitting chains into domains for mr (X-ray only) even if there are multiple chains. Normally only split if a single chain as reassembly may not work with split multiple chains.acceptable_cc_ratio = 0.8 Keep segments with CC at least equal to average of base segments times minimum_cc_ratio minus difference between segment confide...

• rebuild_strategy = Thorough Standard Quick Rebuilding strategy. Standard is up to 3 cycles, refining and replacing poorly-fitting loops in predicted models each cycle, using autobuild for density modification (if Xray). Quick is refinement only, one cycle. Thorough is up to 10 cycles, extensive rebuilding.

• refine_only = None Refine only, no rebuilding steps (set automatically with Quick)

• refine_only_resolution = 3.5 Default cutoff for using Refine only

• run_fit_loops = True Run standard fit_loops

• run_iterative_morph = True Run iterative morphing

• run_trace_loops_through_density = True Run loop fitting with trace_through_density algorithm

• run_refine = True Refine morphed model. Required for run_fit_loops or run_trace_loops_through_density

• run_iterative_resolution_refine = True Refine morphed model with iterative resolution method

• run_extend = True Extend ends of morphed model

• extract_unique = True Extract unique part of map. Applies after density_select (if set).

• use_symmetry_in_extract_unique = True Use symmetry from symmetry file (if available) when extracting unique part of map.

• acceptable_docking_cc = 0.5 Acceptable docking CC for a chain

• minimum_docking_cc = 0.15 Minimum docking CC for a chain

• minimum_cutoff = 0.1 Minimum cutoff for estimating density in better parts of model

• reasonable_cc_ratio = 0.80 Acceptable CC (ratio) for a chain relative to average. Used to make sure a chain entirely in very bad density is not kept

• reasonable_cc_diff = 0.15 Acceptable CC (difference) for a chain relative to average. Used to make sure a chain entirely in very bad density is not kept

• shift_field_distance = None Shift field characteristic distance (default = 10 A) for morphing

• cc_sd_ratio = 3. Keep residues with CC at least within cc_sd_ratio of the mean CC for good residues

• cc_sd_ratio_end = 2. Keep residues with CC at least within cc_sd_ratio_end of the mean CC for good residues (applies to end of fragments)

• cc_sd_ratio_ok = 2. Residues with with CC at least within cc_sd_ratio_ok of the mean CC for good residues are ok (do not delete on the basis of plddt)

• max_gap_ratio = 3. Allow CA-CA distance to be up to max_gap_ratio times expected

• maximum_connectivity_deviation = 15 Maximum connectivity deviation. Reject a solution with bigger deviation than this plus 2 * resolution.

• keep_fraction_of_best = 0.5 Acceptable CC to keep as ratio to best found. Applies if best found is at least acceptable_docking_cc.

• keep_maximum_entries = 10 Maximum dock positions to keep

• rmsd_for_similar_placement = None RMSD value indicating that two placements are similar. default is resolution of map

• rigid_body_refine_cycles = 1 Refinement cycles for rigid-body refinement

• overlap_ca_ca_distance = 3 Overlap distance for CA-CA atoms (or P-P)

• ca_ca_distance = 3.8 CA-CA distance (or P-P)

• allowed_fraction_overlapping = 0.10 Allowed fraction overlapping

• maximum_combinations = 100000 Maximum combinations of placements to consider

• proceed_with_any_symmetry = False Run even with symmetry that is not point-group or helical. Not recommended as symmetry may not work properly

• box_cushion = 20 Size of buffer around model when boxing

• refine_cycles = 3 Refinement cycles

• loop_refine_cycles = 5 Refinement cycles for loops

• loop_backup_residues = 3 Number of tries removing one residue at a time from each end of existing ends of loop if no loop is found with initial gap

• residues_to_trim = 5 Residues to trim on each end of all trimmed fragments

• find_ncs_from_model = True Find NCS (symmetry) from working models and apply in density modification. Also turns on finding NCS in any autobuild density modification (including density-based search)

• allow_split_more_than_one_chain_for_mr = False Allow splitting chains into domains for mr (X-ray only) even if there are multiple chains. Normally only split if a single chain as reassembly may not work with split multiple chains.

• acceptable_cc_ratio = 0.8 Keep segments with CC at least equal to average of base segments times minimum_cc_ratio minus difference between segment confidence (plDDT) and confidence cutoff (typically 0.7)

• low_res_if_multiple_solutions = 3.5 Try phaser MR at increasing lower resolutions up to this value if multiple solutions are found

• delta_low_res = 0.5 Resolution increment for low_res_if_multiple_solutions

• controlstop_after_dock = None Stop after docking stepstop_after_morph = False Stop after morphing stepread_files = False Read existing output files and use them if presentwrite_files = True Write output filesnproc = 1 Number of processors to use on your local machineignore_symmetry_conflicts = True You can ignore the symmetry information (CRYST1) from coordinate files. This may be necessary if your model has been placed in a box with box_map for example.random_seed = 171731 Random seedmax_dirs = 1000 Maximum number of directories (dock_and_build_xxx)verbose = False Verbose outputquick = True Run quickly

• stop_after_dock = None Stop after docking step

• stop_after_morph = False Stop after morphing step

• read_files = False Read existing output files and use them if present

• write_files = True Write output files

• nproc = 1 Number of processors to use on your local machine

• ignore_symmetry_conflicts = True You can ignore the symmetry information (CRYST1) from coordinate files. This may be necessary if your model has been placed in a box with box_map for example.

• random_seed = 171731 Random seed

• max_dirs = 1000 Maximum number of directories (dock_and_build_xxx)

• verbose = False Verbose output

• quick = True Run quickly

• guiGUI-specific parameter required for output directoryoutput_dir = None

• output_dir = None

Rapid model-building with trace_and_build

trace_and_build.html

Author(s)

- trace_and_build: Tom Terwilliger

Purpose

The routine trace_and_build is a tool for rapid protein model-building.

Usage

How trace_and_build works:

The trace_and_build tool traces the path of a polypeptide chain by working from high to low density and following the highest density path that does not yield branching. It then finds CB positions and builds an atomic model.

Using trace_and_build:

Normally you will access the functionality of trace_and_build by running the Phenix map_to_model tool in the Phenix GUI. However you can run it directly as well (there is no GUI for trace_and_build). Usuall you will run trace_and_build on the unique part of a cryo-EM map. Your main options are the resolution, whether to try a quick run (quick=True) or a more thorough run (quick=False), and how many segments to try and build.

Input map file: Usually you should supply trace_and_build with a sharpened map that represents the unique part of your structure. You can use phenix.map_sharpening to sharpen your map. If your map has symmetry you can use phenix.map_box with extract_unique=true to extract this part of the map.

Resolution: Specify the resolution of your map (usually the resolution defined by your half-dataset Fourier shell correlation

Sequence file: Supply a sequence file with the sequence or sequences of the molecule(s) to be built. If more than one they will simply be put together at this point.

Segments to build: you can choose to build just one or a few of the longest segments that trace_and_build can find (max_segments=3), or everything (max_segments=None)

Quick run: You can try to run quickly (quick=true) or more thoroughly (quick=False). One difference is that with quick=True, the number of segments to build is normally limited to the number of residues in your sequence file divided by 50, while with quick=False, it is unlimited. The other is that with quick=False, after building the model an attempt to fix insertions and deletions will be made (fix_insertions_deletions=True). You can set these parameters separately as well.

Input model: You can supply an input fixed model and trace_and_build will use it as a potential interpretation of the tracing of part of the map. It will not be used as is, rather CA positions will be extracted and used in later interpretation steps. This fixed model will take the place of the find_helices_strands step that is otherwise carried out.

Procedure used by trace_and_build

The procedure used by trace_and_build has several steps:

If no model is supplied, an initial fixed model is created by searching for regular secondary structure in the map. Then the tool find_helices_strands is used to analyze the new map to find regular secondary structure. Optionally both directions of each segment of the fixed model can be kept (allow_reverse=True).

The core of trace_and_build is to find, extend, and connect segments of high density in the map. The way this is done is a lot like the way a person would examine the path of a chain in a map, starting from a region of clear (high) density, following the chain until it ends, then lowering the contour level until a path is visible and following that one.

Initial segments of density are identified using the model that is supplied or found with find_helices_strands. New segments are identified from extended regions of high density, working down from high to low density in the map. Connections and extensions are also made working down from high to low density. Chain tracings are only kept if they do not have branching.

Once the path(s) of the polypeptide chain are identified, likely positions of CB atoms are identified from the presence of side-chain density along the traced path. The positions of CA atoms are then guessed and refined with the tool phenix.refine_ca_model which adjusts the number of CA atoms and their positions to match the likely CB positions, the chain tracing, and expected CA-CA distances.

Once a CA-only model is created, an attempt to correct the model using CA positions in the fixed model (input directly or from find_helices_strands). In this step the CA positions in segments in the fixed model are matched with those of the CA-only model, and if they overlap, the CA positions from the fixed model are used.

An all-atom model is generated from each CA-only model using Pulchra . If allow_reverse is set, then each possible direction of each segment is considered. Each of these possibilities is refined and scored based on map-model correlation (CC), agreement of side-chain density with the sequence, and H-bonding in the model.

The highest-scoring model is written out so that it superimposes on the input map.

Examples

Standard run of trace_and_build:

You can use trace_and_build to build a model based on a cryo-EM map:

```
phenix.trace_and_build my_map.mrc resolution=2.8 my_seq.dat
```

Using trace_and_build to evaluate forward and reverse versions of a model:

You can use trace_and_build to work on one or more segments, checking both directions:

```
phenix.trace_and_build my_map.mrc resolution=2.8 my_seq.dat my_fragment.pdb \
find_chains=False extend_chains=False connect_chains=False allow_reverse=True
```

This will read in your fragment(s), create forward and reverse versions, score them both, and try to build the better one (but without extending it or building any new model). You can set verbose=True to see more details of the scoring if you like. If one direction is clearly better than the other, only it will be kept. If you want to keep both, set the parameter convincing_delta_score to a big number or None (take everything).

Possible Problems

Specific limitations and problems:

Literature

Fast procedure for reconstruction of full-atom protein models from reduced representations. P. Rotkiewicz, and J. Skolnick. J Comput Chem 29, 1460-5 (2008). Automated side-chain model building and sequence assignment by template matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 45-9 (2003). Rapid model building of alpha-helices in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 268-75 (2010). Rapid model building of beta-sheets in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 276-84 (2010).

- Fast procedure for reconstruction of full-atom protein models from reduced representations. P. Rotkiewicz, and J. Skolnick. J Comput Chem 29, 1460-5 (2008).
- Automated side-chain model building and sequence assignment by template matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 45-9 (2003).
- Rapid model building of alpha-helices in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 268-75 (2010).
- Rapid model building of beta-sheets in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 276-84 (2010).

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command lineinput_filesmap_file = None File with CCP4-style map. May have origin in any location.model_file = None Input PDB file with fragments to be considered as starting points for building. Used in the same way as the model generated with find_helices_strands is used. If allow_reverse is set, try both directions of each fragment. If connect_chains is set, try connecting pairs of fragments. If extend_chains is set, try extending each end of each fragment.seq_file = None Sequence file (Fasta format or sequences separated by blank lines)marker_model_file = None Input PDB file with marker atoms from trace_chaintrace_model_file = None Input PDB file with dummy CA atoms marking path of chain at about 0.5-1 A intervalsinput_scoring_file = None Input .pkl file with scoring information (can have more than one)fixed_model = None same as model_file.placement_pickle_file = None Read placements from this fileoutput_filespdb_out = trace_and_build.pdb Model built with trace_and_buildoutput_scoring_file = None Output .pkl file with scoring informationoutput_model_file = None Output PDB file with possible CA/CB positionsoutput_marker_model_file = None Output PDB file with marker atoms from trace_chainoutput_placement_pickle_file = None Write placements to this filetemp_dir = None Temporary directory. Default is trace_and_build_xx where xx creates new directorycrystal_inforesolution = None High-resolution limit for map analysis.scattering_table = n_gaussian wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.chain_type = *PROTEIN Chain type. Must be PROTEINca_ca_distance = 3.8 CA-CA distancesolvent_content = None Solvent fraction of the cell. If this is density cut out from a bigger cell, you can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.solvent_content_iterations = 3 Iterations of solvent fraction estimationuse_mask_if_present = True If map is masked, use the mask as solvent contentsequence = None Sequenceswrapping = False For cryo-EM maps, wrapping should be offorigin_cart = None Origin (cartesian coordinates, overrides value based on map)strategyget_path_length_only = False Just find adjusted path

length and return itfind_chains = True Try to create new chains in build_all_loopsextend_chains = True Try to extend chains in build_all_loopsconnect_chains = True Try to connect chains in build_all_loopslength_variants_to_try = 5 You can try several lengths of fragments (number of CA). Not compatible with allow_reverse=Falseallow_reverse = True If two chains are opposite direction but connected, reverse the one with lower score (usually shorter) and connect them. Also consider both directions of fragments from model_file.morph_ca_only = False Just morph chain and return trace through themtrace_outside_model = False Trace outside model. After initial tracing, mask out region found and look for additional segmentscorrect_segments = True Correct segments. Try to fix errors using fixed model as a templatechains_from_fixed_model = True Use fixed model to mark initial chain positions.rerun_with_avail_seq = False Rerun sequence alignment with parts of the sequence already used removed.mask_side_chains = False Mask side chains in existing model in build_all_loopssame_chain_rmsd_max = 1.0 RMSD between two CA models of same length to consider them the samelocal_trace = True Run trace in local regionssplit_and_join = True Split fragments at low density and then rejoin. Criterion is sd_ratio_pare_modeltest_threshold_fixed_segments = False Test thresholds when creating fixed segmentsrequire_ends_in_region = False Add ends of chain to region if necessarytry_alternate_joins = False try joining each end of pair of fragments, not just the firstskip_path_if_missing = True Skip path if points are missingsplit_at_low_d...

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesmap_file = None File with CCP4-style map. May have origin in any location.model_file = None Input PDB file with fragments to be considered as starting points for building. Used in the same way as the model generated with find_helices_strands is used. If allow_reverse is set, try both directions of each fragment. If connect_chains is set, try connecting pairs of fragments. If extend_chains is set, try extending each end of each fragment.seq_file = None Sequence file (Fasta format or sequences separated by blank lines)marker_model_file = None Input PDB file with marker atoms from trace_chaintrace_model_file = None Input PDB file with dummy CA atoms marking path of chain at about 0.5-1 A intervalsinput_scoring_file = None Input .pkl file with scoring information (can have more than one)fixed_model = None same as model_file.placement_pickle_file = None Read placements from this file
- map_file = None File with CCP4-style map. May have origin in any location.
- model_file = None Input PDB file with fragments to be considered as starting points for building. Used in the same way as the model generated with find_helices_strands is used. If allow_reverse is set, try both directions of each fragment. If connect_chains is set, try connecting pairs of fragments. If extend_chains is set, try extending each end of each fragment.
- seq_file = None Sequence file (Fasta format or sequences separated by blank lines)
- marker_model_file = None Input PDB file with marker atoms from trace_chain
- trace_model_file = None Input PDB file with dummy CA atoms marking path of chain at about 0.5-1 A intervals
- input_scoring_file = None Input .pkl file with scoring information (can have more than one)
- fixed_model = None same as model_file.
- placement_pickle_file = None Read placements from this file
- output_filespdb_out = trace_and_build.pdb Model built with trace_and_buildoutput_scoring_file = None Output .pkl file with scoring informationoutput_model_file = None Output PDB file with possible CA/CB positionsoutput_marker_model_file = None Output PDB file with marker atoms from trace_chainoutput_placement_pickle_file = None Write placements to this filetemp_dir = None Temporary directory. Default is trace_and_build_xx where xx creates new directory

• pdb_out = trace_and_build.pdb Model built with trace_and_build

• output_scoring_file = None Output .pkl file with scoring information

• output_model_file = None Output PDB file with possible CA/CB positions

• output_marker_model_file = None Output PDB file with marker atoms from trace_chain

• output_placement_pickle_file = None Write placements to this file

• temp_dir = None Temporary directory. Default is trace_and_build_xx where xx creates new directory

• crystal_inforesolution = None High-resolution limit for map analysis.scattering_table = n_gaussian wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.chain_type = *PROTEIN Chain type. Must be PROTEINca_ca_distance = 3.8 CA-CA distancesolvent_content = None Solvent fraction of the cell. If this is density cut out from a bigger cell, you can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.solvent_content_iterations = 3 Iterations of solvent fraction estimationuse_mask_if_present = True If map is masked, use the mask as solvent contentsequence = None Sequenceswrapping = False For cryo-EM maps, wrapping should be offorigin_cart = None Origin (cartesian coordinates, overrides value based on map)

• resolution = None High-resolution limit for map analysis.

• scattering_table = n_gaussian wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.

• chain_type = *PROTEIN Chain type. Must be PROTEIN

• ca_ca_distance = 3.8 CA-CA distance

• solvent_content = None Solvent fraction of the cell. If this is density cut out from a bigger cell, you can specify the fraction of the volume of this cell that is taken up by the macromolecule. Normally set automatically. Values go from 0 to 1.

• solvent_content_iterations = 3 Iterations of solvent fraction estimation

• use_mask_if_present = True If map is masked, use the mask as solvent content

• sequence = None Sequences

• wrapping = False For cryo-EM maps, wrapping should be off

• origin_cart = None Origin (cartesian coordinates, overrides value based on map)

• strategyget_path_length_only = False Just find adjusted path length and return itfind_chains = True Try to create new chains in build_all_loopsextend_chains = True Try to extend chains in build_all_loopsconnect_chains = True Try to connect chains in build_all_loopslength_variants_to_try = 5 You can try several lengths of fragments (number of CA). Not compatible with allow_reverse=Falseallow_reverse = True If two chains are opposite direction but connected, reverse the one with lower score (usually shorter) and connect them. Also consider both directions of fragments from model_file.morph_ca_only = False Just morph chain and return trace through themtrace_outside_model = False Trace outside model. After initial tracing, mask out region found and look for additional segmentscorrect_segments = True Correct segments. Try to fix errors using fixed model as a templatechains_from_fixed_model = True Use fixed model to mark initial chain positions.rerun_with_avail_seq = False Rerun sequence alignment with parts of the sequence already used removed.mask_side_chains = False Mask side chains in existing model in build_all_loopssame_chain_rmsd_max = 1.0 RMSD between two CA models of same length to consider them the samelocal_trace = True Run trace in local regionssplit_and_join = True Split fragments at low density and then rejoin. Criterion is sd_ratio_pare_modeltest_threshold_fixed_segments = False Test

thresholds when creating fixed segmentsrequire_ends_in_region = False Add ends of chain to region if necessarytry_alternate_joins = False try joining each end of pair of fragments, not just the firstskip_path_if_missing = True Skip path if points are missingsplit_at_low_density = True Split at low density points in split_and_joinrejoin_split_fragments = True Specifically rejoin the split fragments leaving out poor junctionstrim_ends_and_join = False Trim ends in split_and_joinkeep_main_chain_path = False Keep path of main chain when creating fixed fragments. Helps incorporate fixed model into final model, but may decrease length of fragments created (not used by default in sequence_from_map).high_density_from_model = False Create high density in map along path of main chain atoms in fixed model. Also sets keep_main_chain_path. Helps incorporate fixed model into final model, but may decrease length of fragments created (not used by default in sequence_from_map).max_grid_spacing = 1.4 Try to resample map with spacing of about target_grid_spacing if it is greater than max_grid_spacingtarget_grid_spacing = 1.2 Try to resample map with spacing of about target_grid_spacing if it is greater than max_grid_spacingmin_grid_spacing = 0.8 Try to resample map with spacing of about target_min_grid_spacing if it is less than min_grid_spacingtarget_min_grid_spacing = 1.0 Try to resample map with spacing of about target_min_grid_spacing if it is less than min_grid_spacingpare_model = False Pare back model from ends if not convincing. Minimum value is mean minus sd_ratio_pare_model times sd.remove_clashes = True Remove clashing side chains in sequence_from_maptrim_ends_extra_points = 1 Extra points tried in each direction deciding where to trim ends in split_and_joinmin_sd_ratio = 0.1 Minimum ratio of SD to meansd_ratio_pare_model = 2.5 Lower limit of density after pare_model or in split_and_join is mean minus sd_ratio_pare_model times sdbox_buffer = 5 Box buffer. Box will be at least the size of fragments plus this buffer in each direction (grid units)box_size = 150 150 150 You can specify the size of the boxes to use (grid units) when finding chainsbox_overlap_ratio = 3 If increasing the box size by 1/box_overlap_ratio reduces the number of boxes, do this.vary_sharpening = None Variable sharpening values. Zero is always added. For example 75 -50 -100 will sharpen by B = 75 then blur by b = 50 and b = 100. Increases residues built but may decrease accuracymaximum_duplication = 0.5 Maximum duplication in fragmentstarget_n_overlap = 10 You can specify the t...

• get_path_length_only = False Just find adjusted path length and return it

• find_chains = True Try to create new chains in build_all_loops

• extend_chains = True Try to extend chains in build_all_loops

• connect_chains = True Try to connect chains in build_all_loops

• length_variants_to_try = 5 You can try several lengths of fragments (number of CA). Not compatible with allow_reverse=False

• allow_reverse = True If two chains are opposite direction but connected, reverse the one with lower score (usually shorter) and connect them. Also consider both directions of fragments from model_file.

• morph_ca_only = False Just morph chain and return trace through them

• trace_outside_model = False Trace outside model. After initial tracing, mask out region found and look for additional segments

• correct_segments = True Correct segments. Try to fix errors using fixed model as a template

• chains_from_fixed_model = True Use fixed model to mark initial chain positions.

• rerun_with_avail_seq = False Rerun sequence alignment with parts of the sequence already used removed.

• mask_side_chains = False Mask side chains in existing model in build_all_loops

• same_chain_rmsd_max = 1.0 RMSD between two CA models of same length to consider them the same

• local_trace = True Run trace in local regions

• split_and_join = True Split fragments at low density and then rejoin. Criterion is sd_ratio_pare_model

• test_threshold_fixed_segments = False Test thresholds when creating fixed segments

• require_ends_in_region = False Add ends of chain to region if necessary

• try_alternate_joins = False try joining each end of pair of fragments, not just the first

• skip_path_if_missing = True Skip path if points are missing

• split_at_low_density = True Split at low density points in split_and_join

• rejoin_split_fragments = True Specifically rejoin the split fragments leaving out poor junctions

• trim_ends_and_join = False Trim ends in split_and_join

• keep_main_chain_path = False Keep path of main chain when creating fixed fragments. Helps incorporate fixed model into final model, but may decrease length of fragments created (not used by default in sequence_from_map).

• high_density_from_model = False Create high density in map along path of main chain atoms in fixed model. Also sets keep_main_chain_path. Helps incorporate fixed model into final model, but may decrease length of fragments created (not used by default in sequence_from_map).

• max_grid_spacing = 1.4 Try to resample map with spacing of about target_grid_spacing if it is greater than max_grid_spacing

• target_grid_spacing = 1.2 Try to resample map with spacing of about target_grid_spacing if it is greater than max_grid_spacing

• min_grid_spacing = 0.8 Try to resample map with spacing of about target_min_grid_spacing if it is less than min_grid_spacing

• target_min_grid_spacing = 1.0 Try to resample map with spacing of about target_min_grid_spacing if it is less than min_grid_spacing

• pare_model = False Pare back model from ends if not convincing. Minimum value is mean minus sd_ratio_pare_model times sd.

• remove_clashes = True Remove clashing side chains in sequence_from_map

• trim_ends_extra_points = 1 Extra points tried in each direction deciding where to trim ends in split_and_join

• min_sd_ratio = 0.1 Minimum ratio of SD to mean

• sd_ratio_pare_model = 2.5 Lower limit of density after pare_model or in split_and_join is mean minus sd_ratio_pare_model times sd

• box_buffer = 5 Box buffer. Box will be at least the size of fragments plus this buffer in each direction (grid units)

• box_size = 150 150 150 You can specify the size of the boxes to use (grid units) when finding chains

• box_overlap_ratio = 3 If increasing the box size by 1/box_overlap_ratio reduces the number of boxes, do this.

• vary_sharpening = None Variable sharpening values. Zero is always added. For example 75 -50 -100 will sharpen by B = 75 then blur by b = 50 and b = 100. Increases residues built but may decrease accuracy

• maximum_duplication = 0.5 Maximum duplication in fragments

• target_n_overlap = 10 You can specify the targeted overlap of boxes

• ends_only = True Keep track only of the very ends of chains. Ignore direction. Just 2 atoms for each chain (one at each end).

• minimum_new_chain_length = 15 Minimum length to try building a new chain

• similar_threshold_ratio = 0.1 Similar threshold ratio. Thresholds differing by this fraction of the difference between maximim and minimum thresholds are considered similar.

• max_merge_cycles = 100 Maximum cycles of merging fragments

• minimum_extension_length = 10 Minimum length to try extending a chain

• minimum_extension_improvement = 2 Minimum improvement to keep a trial extension

• weight_path_by_density = True Weight path by density to trace through high density. Scale on distance is exp(path_density_weight times log(density-density_min)/(density max - density min))

• weight_fixed_segments_by_density = False Weight fixed segments path by density

• final_connect_chains = True Connect chains at very end after merging fragments. Uses full map

• path_density_weight = 2 Weight on having high density along path of trace.

• min_weight_ratio = 0.0001 Minimum scale on distance in trace through high density

• target_trim_length = 7.5 Trim back chains this much before trying to recombine.

• max_branch_length = 10 Maximum branch (not main path) to keep a segment If a branch is longer than max_branch_length or longer than max_fractional_branch_length times the main path, reject

• max_fract_non_contiguous = 0.25 Maximum fractional non-contiguous density

• max_fractional_branch_length = 0.5 Maximum branch (not main path) to keep a segment. If a branch is longer than max_branch_length or longer than max_fractional_branch_length times the main path, reject

• matching_end_dist = 3 Maximum distance between end atoms to consider them matching for purposes of excluding duplicate ends

• sharing_end_dist = 12 Maximum distance between end atoms to consider them matching for purposes of linking them. Can be larger than matching because the end points may not be at the ends of the chain.

• create_loop_maps = None Find connections between fragments in input model and write out small maps with just the density for the connection and the associated fragments. Default is True.

• max_points_per_region_ratio = 1.5 If set, max_points_per_region will be reset to this value times the current actual maximum points in any region, up to maximum of max_points_per_region_ever

• max_points_per_region_ever = 12000 Maximum number of points in a region ever. See max_points_per_region_ratio and max_points_per_region

• max_points_per_region = 4000 Maximum number of points in a region. Will be ignored if more than this. (Limiting this can prevent very long connections from being made, but reduces possibility that a connection that is totally wrong is made).

• max_new_chains = 1 Maximum number of new chains to obtain in one pass

• regions_for_new_chains = 3 Maximum number of top regions to examine for new chain info

• max_loops = 10 Maximum number of loops to write out in create_loop_maps

• half_window = 2 Half window (grid points) for examining shape of density

• max_loop_iterations = 999 Maximum number of iterations to look for loops

• spacing = 1 Spacing of points for a trace

• intervals = 10 Number of threshold values to try in create_loop_maps

• mean_density_ratio = 2 Guess of mean density at coordinates of atoms is mean density in map plus SD of density in map, corrected for solvent content, times mean_density_ratio

• min_sd_to_mean = 0.20 Limit SD of density at atoms to at least this value times the mean value.

• residues_to_have_middle = 10 Length of a chain to have a middle that can be masked out.

• min_points_in_region = 10 Minimum points in region to be considered as a new chain

• max_points_in_region = 3000 Maximum points in region to be considered as a new chain

• sd_density_ratio = 0.5 Guess of SD of density at coordinates of atoms is SD of density in map, corrected for solvent content, times sd_density_ratio

• threshold = None Density threshold for following density.

• threshold_low = None Density threshold low value for following density.

• threshold_high = None Density threshold high value for following density.

• save_threshold_info = False Use threshold info from initial analysis throughout

• threshold_from_model = False Use density at coordinates in model to estimate thresholds

• test_threshold = False Test thresholds when making connections and use maximum that will connect

• sd_ratio_intervals = 1 SD ratio intervals. Density is traced starting from highest and going to lowest in sd_ratio_intervals. XXX not used

• sd_ratio_create = 1 SD ratio for creating new chains . Default is same as sd_ratio.

• sd_ratio = 2. Lower limit of density to consider is sd_ratio below mean of density at C/N/CA atoms in current model in create_loop_maps. Applies when connecting segments. A high value may result in incorrect connections.

• min_fixed_model_segment_length = 4 Minimum length of segment in fixed_model

• mean_ratio = 0.25 Lower limit of density to consider is mean_ratio times mean of density at C/N/CA atoms in current model in create_loop_maps after subtracting mean of map

• minimum_fraction_intervals_to_try = 0.67 If at least this fraction of intervals have been tried in trace, terminate if recent fraction of long branches is too high.

• retry_long_branches = None If a long branch is found, try to retrace chain

• max_fraction_long_branches = 0.4 Terminate trace if recent fraction of long branches is high

• skip_remainder_on_non_completion = True Skip remainder at this threshold if too many failures

• n_tries_furthest_length = 3 Number of tries to include in looking for longest path

• recent_fraction_length = 8 Number of tries to include in recent fraction of long branches

• expand_size = 1 Expansion of mask when cutting out loop density

• build_segments = True Build segments

• first_cycle_for_fixing = None First macro-cycle where insertions/deletions should be tested

• score_cc_mask_offset = 0.5 Value of CC_mask that just starts to give a positive score. Typical values are 0 or 0.5

• trace_and_buildmin_segment_length = 15 Minimum residues in a segment to keep (after joining and insertion).max_segments = None Maximum number of segments to keep (after joining and insertion). Default is take all with min_segment_length or more residuesmin_segments = 1 Minimum number of

segments to keep (after joining and insertion) before skipping if segment length is too shortfix_insertions_deletions = True Retrace chains and use sequence alignment to fix insertions and deletions.convincing_delta_score = 10. Convincing delta score for forward vs reverse directions that nearly always means the higher-scoring one is correct.find_helices_strands = True Find helices/strands before tracing chain if no starting model is suppliedminimum_length_angstroms_helices_strands = 12 Minimum length (A CA start - CA end) for helix/strand to keeptrim_chains = True Try to trim chains if turns are too tight (i-j-k with dist(i, k) < min_dist_ik)min_dist_ik = 4.5 Minimum i-k distance for CA i j k.fill_gaps = True Try to fill short gaps in input modeltolerance_residue_distance = 2 Tolerance for residue-residue distance (typically 4.0)dot_min = -0.2 Minimum cosine of angle between suquential CA-CA directions.minimum_length = 2 Minimum length between CA-CA positionsmax_gap_residues = 2 Maximum number of residues to try and fill in gaps. Must be 1 or 2max_trim_ends = 1 Maximum number of residues to trim from ends before gap fillingmax_gap_dist = 3 Maximum distance to span in gapminimum_relative_density = 0.50 Minimum density at marker sites, relative to mean at coordinates of input model, to keep. Also minimum along path between a marker site and nearest main_chain trace. Also minimum ratio of density at ends of a connection and in gap to be filled.get_marker_atoms_by_segment = True Get marker atoms by segment. Only applies if map size is at least minimum_size_marker_atoms_by_segmentminimum_size_marker_atoms_by_segment = 3375000 Use get_marker_atoms_by_segment if it is set and map size is at least this big (typically 150x150x150).morph_segments = True Morph segments to fit density before looking for side chainsmorphing_iterations = 3 Number of iterations of morphingpoints_per_atom = 3 Number of points between atoms in tracing path of main chainshift_radius = 2 Marker atoms within shift_radius of a main-chain atom are used to identify shift of main-chain in morphingminimum_sites = 4 Minimum nearby sites for shifting main chain in morphingsmoothing_window = 5 Smoothing window for shifts of main-chain in morphingmain_chain_radius = 1.5 Approximate radius of tube of density for main-chain. Marked points within this radius of an atom in the main-chain are considered main-chain, those outside are side chains and other chainsside_chain_radius = 3.0 Consider marked points between main_chain_radius and side_chain_radius as likely side-chain markers. Best to use value of 3 or less or neighboring side chains will interfere.mask_atoms_radius = 2.5 Radius to use when masking around already-built model atomsdelta_atoms_radius = 2. Increase in mask_atoms_radius arounds atoms at ends of already-built model.non_expanded_atoms_radius = None Atoms radius for creating mask showing main-chain. Should be slightly larger than the grid used so that a point on a grid point will have 6 adjacent grid points within this radius. Also should be comparable to N-CA-C distances. Default = max(1.0, grid+0.01)exclude_residues = 3 Residues to exclude at each end of chain in maskingshell_radius = 0.5 Divide region between main_chain_radius and side_chain_radius into shells of thickness shell_radiuscluster_width_ratio = 0.2 Maximum cluster width relative to CA-CA distanceminimum_vector = 1.0 Minimum length of CB vector to includepruning_ratio = 3 Minimum distance between cluster centers relative to cluster widthn_short_min = 20 N short min. Length of group of CA to optimize together (min). Default in trace_and_build is None, in map-to-model is 20 (quick) or 50 (not quick).n_short_max =...

- min_segment_length = 15 Minimum residues in a segment to keep (after joining and insertion).
- max_segments = None Maximum number of segments to keep (after joining and insertion). Default is take all with min_segment_length or more residues
- min_segments = 1 Minimum number of segments to keep (after joining and insertion) before skipping if segment length is too short
- fix_insertions_deletions = True Retrace chains and use sequence alignment to fix insertions and deletions.

• convincing_delta_score = 10. Convincing delta score for forward vs reverse directions that nearly always means the higher-scoring one is correct.

• find_helices_strands = True Find helices/strands before tracing chain if no starting model is supplied

• minimum_length_angstroms_helices_strands = 12 Minimum length (A CA start - CA end) for helix/strand to keep

• trim_chains = True Try to trim chains if turns are too tight (i-j-k with dist(i, k) < min_dist_ik)

• min_dist_ik = 4.5 Minimum i-k distance for CA i j k.

• fill_gaps = True Try to fill short gaps in input model

• tolerance_residue_distance = 2 Tolerance for residue-residue distance (typically 4.0)

• dot_min = -0.2 Minimum cosine of angle between suquential CA-CA directions.

• minimum_length = 2 Minimum length between CA-CA positions

• max_gap_residues = 2 Maximum number of residues to try and fill in gaps. Must be 1 or 2

• max_trim_ends = 1 Maximum number of residues to trim from ends before gap filling

• max_gap_dist = 3 Maximum distance to span in gap

• minimum_relative_density = 0.50 Minimum density at marker sites, relative to mean at coordinates of input model, to keep. Also minimum along path between a marker site and nearest main_chain trace. Also minimum ratio of density at ends of a connection and in gap to be filled.

• get_marker_atoms_by_segment = True Get marker atoms by segment. Only applies if map size is at least minimum_size_marker_atoms_by_segment

• minimum_size_marker_atoms_by_segment = 3375000 Use get_marker_atoms_by_segment if it is set and map size is at least this big (typically 150x150x150).

• morph_segments = True Morph segments to fit density before looking for side chains

• morphing_iterations = 3 Number of iterations of morphing

• points_per_atom = 3 Number of points between atoms in tracing path of main chain

• shift_radius = 2 Marker atoms within shift_radius of a main-chain atom are used to identify shift of main-chain in morphing

• minimum_sites = 4 Minimum nearby sites for shifting main chain in morphing

• smoothing_window = 5 Smoothing window for shifts of main-chain in morphing

• main_chain_radius = 1.5 Approximate radius of tube of density for main-chain. Marked points within this radius of an atom in the main-chain are considered main-chain, those outside are side chains and other chains

• side_chain_radius = 3.0 Consider marked points between main_chain_radius and side_chain_radius as likely side-chain markers. Best to use value of 3 or less or neighboring side chains will interfere.

• mask_atoms_radius = 2.5 Radius to use when masking around already-built model atoms

• delta_atoms_radius = 2. Increase in mask_atoms_radius arounds atoms at ends of already-built model.

• non_expanded_atoms_radius = None Atoms radius for creating mask showing main-chain. Should be slightly larger than the grid used so that a point on a grid point will have 6 adjacent grid points within this radius. Also should be comparable to N-CA-C distances. Default = max(1.0, grid+0.01)

• exclude_residues = 3 Residues to exclude at each end of chain in masking

• shell_radius = 0.5 Divide region between main_chain_radius and side_chain_radius into shells of thickness shell_radius

• cluster_width_ratio = 0.2 Maximum cluster width relative to CA-CA distance

• minimum_vector = 1.0 Minimum length of CB vector to include

• pruning_ratio = 3 Minimum distance between cluster centers relative to cluster width

• n_short_min = 20 N short min. Length of group of CA to optimize together (min). Default in trace_and_build is None, in map-to-model is 20 (quick) or 50 (not quick).

• n_short_max = 200 N short max. Length of group of CA to optimize together (max)

• n_short_delta = 40 N short delta. Length of group of CA to optimize together (delta)

• ca_ca_distance_tol = 1.0 CA-CA tolerance in scoring a set of possible CA atoms

• ca_ca_distance_tol_curves = 1.0 CA-CA tolerance in curves

• ca_ca_distance_curves = 3.0 CA-CA distance for residues in curves

• residues_defining_curves = 3 Number of residues in a row checked for defining a segment that curves.

• sites_to_delete = 3 Sites to delete and then try to rebuild as a group with 1 extra reside

• add_group = True Delete sites_to_delete and then rebuild as a group with 1 extra residue

• n_add_group = 10 Cycles for sites_to_delete

• n_tries_factor = 2 Try n_sites times n_tries_factor time to improve trace

• sites_cart_split_size = 1000 Size of sites_cart for finding sites with trace_chain

• weightsweight_vdw = 10 weight on very close CA-CA contactsweight_ca_ca_dist = 1. weight on CA-CA distanceweight_proximity_to_known_position = 0.5 weight on proximity to well-identified CA positionweight_chain_follows_trace = 10. weight on following the trace. Basically makes sure that the main chain does not cut across weak densitydistance_chain_follows_trace = 0.5 Target distance for following the trace.weight_target_length = 0.5 weight on total path length given number of CA positionsweight_density = 1.0 weight on density at mid-points between CA atomsweight_cc_mask_score = 1.0 Weight on map correlation in evaluating chain directionweight_seq_score = 1.0 Weight on sequence-map matching evaluating chain directionweight_x_gly_score = 1.0 Weight on excess Gly and X residues in evaluating chain directionweight_refine_rmsd = 0.0 Weight on rmsd between CA before and after refinement

• weight_vdw = 10 weight on very close CA-CA contacts

• weight_ca_ca_dist = 1. weight on CA-CA distance

• weight_proximity_to_known_position = 0.5 weight on proximity to well-identified CA position

• weight_chain_follows_trace = 10. weight on following the trace. Basically makes sure that the main chain does not cut across weak density

• distance_chain_follows_trace = 0.5 Target distance for following the trace.

• weight_target_length = 0.5 weight on total path length given number of CA positions

• weight_density = 1.0 weight on density at mid-points between CA atoms

• weight_cc_mask_score = 1.0 Weight on map correlation in evaluating chain direction

• weight_seq_score = 1.0 Weight on sequence-map matching evaluating chain direction

• weight_x_gly_score = 1.0 Weight on excess Gly and X residues in evaluating chain direction

• weight_refine_rmsd = 0.0 Weight on rmsd between CA before and after refinement

• trace_chainhelices_strands_cc_min = 0.35 Minimum map CC for helices/strands. Along with combine_models_score_min defines which fragments are tossed. Fragments are kept unless both criteria fail.use_all_trace_chains = False Use all trace_chains possibilities, not just the bestn_overlap = 2 Maximum overlap in trace_chain nonamersn_random_frag = 100 n_sift_nona = 2 Maximum nonamers with a common mid-pointdist_ca_tol_start = 0.40 Minimum tolerance for CA-CA distances. dist_ca_tol_max = 0.6 Maximum deviation of CA-CA distances from targetreject_ca_z_score = 2.5 Reject CA atoms in trace if their density density is more than Z of reject_ca_z_score below the mean and more than fraction reject_ca_fraction below the mean.reject_ca_fraction = 0.5 Reject CA atoms in trace if their density density is more than Z of reject_ca_fraction below the mean and more than fraction reject_ca_fraction below the mean.cc_ratio_min = None Minimum map CC relative to maximum for helices/strands.combine_models_score_min = 3.0 Minimum score for a chain in combine_models (after helices_strands). Score is mostly defined by sqrt(n_atoms)*overall map CC .rho_cut_min = 0.75 Minimum density (rho/sigma) at coordinates of potential CA atoms in trace_chain, after normalization for solvent fraction. For constant actual local rms in a map, the sigma (overall rms) of the map is proportional to the sqrt(1-solvent_fraction). Therefore rho_cut_min is adjusted by sqrt(0.5)/sqrt(1-solvent_fraction) to place it on a constant scale relative to a map with standard local rms.target_angle = 180 Target angle for CA-CA-CA (set to 180 to maximize chain length)rho_cut_min_low = 1. Starting value of rho_cut_min. Applies if rho_cut_min_delta is set (rho_cut_min is ignored in this case)rho_cut_min_high = 5 Ending value of rho_cut_min. Applies if rho_cut_min_delta is setrho_cut_min_delta = None Incremental value of rho_cut_min. If set, rho_cut_min will be ignoredrat_pair_min = 0.5 Minimum ratio of density at midpoint between points to trace chain between themrad_sep_trace = 0.6 Dummy atom separation in trace_chain Usual 0.6 A for thorough run and 0.75 for quick Increased automatically if resolution is greater than 3 A Value of rad_mask_trace in resolve will be rad_sep_trace*2target_p_ratio = 4 Target ratio of atoms to peaks in trace_chaintarget_n_ratio = 1 Target ratio of nonamers to peaks in trace_chainmax_triple_ratio = None Maximum ratio of triples to pairs in trace_chainmax_pent_ratio = None Maximum ratio of pentamers to pairs in trace_chainn_atoms_total_scale = 3 Ratio of estimated atoms in au to standard estimateatom_target_ratio = 1.0 Target ratio of CA to look for to expected atoms in structure Standard is 0.45, quick is 0.35min_end_correl = 0.5 Minimum correlation of direction estimated from two ends to use end matching as criterion for keeping a chainadd_side_chains = True Add in side chains at trace_chain stepuser_end_ratio = 2.0 Ratio of rad_user for ends of input chains to middleuser_end_length = 3 Points in input chains within user_end_length of an end are considered near the endrad_user = 2.5 Radius for input chains in trace_chains. Points within this radius of an input chain will be removed.time_per_volume = 5 How long to try in trace chain (sec per volume of 10000 A**3) Try 2-20 to speed up trace_chain. Has similar effect as n_tries_target_p = 2 n_tries_p_ratio = 5.n_tries_target_p = None How many tries to get target p value in trace_chain n_target_p. n_tries_target_p. Set to 40 for slow. 2 quickn_tries_p_ratio = None Tries for p ratio. n_p_ratio. n_tries_max. Set to 20 for slow, 5 for quick

• helices_strands_cc_min = 0.35 Minimum map CC for helices/strands. Along with combine_models_score_min defines which fragments are tossed. Fragments are kept unless both criteria fail.

• use_all_trace_chains = False Use all trace_chains possibilities, not just the best

• n_overlap = 2 Maximum overlap in trace_chain nonamers

• n_random_frag = 100

• n_sift_nona = 2 Maximum nonamers with a common mid-point

• dist_ca_tol_start = 0.40 Minimum tolerance for CA-CA distances.

• dist_ca_tol_max = 0.6 Maximum deviation of CA-CA distances from target

• reject_ca_z_score = 2.5 Reject CA atoms in trace if their density density is more than Z of reject_ca_z_score below the mean and more than fraction reject_ca_fraction below the mean.

• reject_ca_fraction = 0.5 Reject CA atoms in trace if their density density is more than Z of reject_ca_fraction below the mean and more than fraction reject_ca_fraction below the mean.

• cc_ratio_min = None Minimum map CC relative to maximum for helices/strands.

• combine_models_score_min = 3.0 Minimum score for a chain in combine_models (after helices_strands). Score is mostly defined by sqrt(n_atoms)*overall map CC .

• rho_cut_min = 0.75 Minimum density (rho/sigma) at coordinates of potential CA atoms in trace_chain, after normalization for solvent fraction. For constant actual local rms in a map, the sigma (overall rms) of the map is proportional to the sqrt(1-solvent_fraction). Therefore rho_cut_min is adjusted by sqrt(0.5)/sqrt(1-solvent_fraction) to place it on a constant scale relative to a map with standard local rms.

• target_angle = 180 Target angle for CA-CA-CA (set to 180 to maximize chain length)

• rho_cut_min_low = 1. Starting value of rho_cut_min. Applies if rho_cut_min_delta is set (rho_cut_min is ignored in this case)

• rho_cut_min_high = 5 Ending value of rho_cut_min. Applies if rho_cut_min_delta is set

• rho_cut_min_delta = None Incremental value of rho_cut_min. If set, rho_cut_min will be ignored

• rat_pair_min = 0.5 Minimum ratio of density at midpoint between points to trace chain between them

• rad_sep_trace = 0.6 Dummy atom separation in trace_chain Usual 0.6 A for thorough run and 0.75 for quick Increased automatically if resolution is greater than 3 A Value of rad_mask_trace in resolve will be rad_sep_trace*2

• target_p_ratio = 4 Target ratio of atoms to peaks in trace_chain

• target_n_ratio = 1 Target ratio of nonamers to peaks in trace_chain

• max_triple_ratio = None Maximum ratio of triples to pairs in trace_chain

• max_pent_ratio = None Maximum ratio of pentamers to pairs in trace_chain

• n_atoms_total_scale = 3 Ratio of estimated atoms in au to standard estimate

• atom_target_ratio = 1.0 Target ratio of CA to look for to expected atoms in structure Standard is 0.45, quick is 0.35

• min_end_correl = 0.5 Minimum correlation of direction estimated from two ends to use end matching as criterion for keeping a chain

• add_side_chains = True Add in side chains at trace_chain step

• user_end_ratio = 2.0 Ratio of rad_user for ends of input chains to middle

• user_end_length = 3 Points in input chains within user_end_length of an end are considered near the end

• rad_user = 2.5 Radius for input chains in trace_chains. Points within this radius of an input chain will be removed.

• time_per_volume = 5 How long to try in trace chain (sec per volume of 10000 A**3) Try 2-20 to speed up trace_chain. Has similar effect as n_tries_target_p = 2 n_tries_p_ratio = 5.

• n_tries_target_p = None How many tries to get target p value in trace_chain n_target_p. n_tries_target_p. Set to 40 for slow. 2 quick

• n_tries_p_ratio = None Tries for p ratio. n_p_ratio. n_tries_max. Set to 20 for slow, 5 for quick

• sequencingrandom_sequences = None Number of random sequences of each length to use as baselinepositive_gap_penalty = None Penalty for missing residues is positive_gap_penalty * gap**2negative_gap_penalty = None Gap penalty for extra residues is negative_gap_penalty * gap**2max_gap_length = None Maximum gap length for adjacent alignmentsminimum_crossover_length = None Minimum length of crossoversminimum_alignment_length = None Minimum length of an alignmentminimum_crossover_segment_length = 10 Minimum length of a segment in crossoverstoo_far_crossover = 0.75 fraction of CA-CA distance that is too far to cross over between chainstoo_much_further_crossover = 0.5 fraction of CA-CA distance that is too much further than the last matching CA-CA distance to cross over between chainsscore_by_residue_groups = None Use residue groups in sequence alignment and listing of optimal sequences. Default is use default from sequence_from_mapsplit_using_alignment = False At end, split fragments based on alignment. Default is False

• random_sequences = None Number of random sequences of each length to use as baseline

• positive_gap_penalty = None Penalty for missing residues is positive_gap_penalty * gap**2

• negative_gap_penalty = None Gap penalty for extra residues is negative_gap_penalty * gap**2

• max_gap_length = None Maximum gap length for adjacent alignments

• minimum_crossover_length = None Minimum length of crossovers

• minimum_alignment_length = None Minimum length of an alignment

• minimum_crossover_segment_length = 10 Minimum length of a segment in crossovers

• too_far_crossover = 0.75 fraction of CA-CA distance that is too far to cross over between chains

• too_much_further_crossover = 0.5 fraction of CA-CA distance that is too much further than the last matching CA-CA distance to cross over between chains

• score_by_residue_groups = None Use residue groups in sequence alignment and listing of optimal sequences. Default is use default from sequence_from_map

• split_using_alignment = False At end, split fragments based on alignment. Default is False

• rebuildingrefine = True Refine all-atom models at end of procedurerefine_b = None Refine B-valuesrefine_cycles = None Refinement cycles (except final cycles). Typical is 1 for quick or superquick and 5 for thorough buildinggood_enough_cc = None If all residues have this CC, don't bother rebuilding themminimum_contact_distance = 3 Minimum distance between CA atoms not immediately connectedsplit_with_sequence = None Use sequence to identify sequence register errors. Runs replace_side_chains with reassign_sequence = true and assign_sequence with iterative_assignment = true.rebuild_length_worst = 15 Longest rebuild length to trymax_insert_or_delete = 1 Maximum residues to insert or delete in on rebuild stagetime_per_residue = 1 How long to try in fitting loops (sec/residue)try_as_is = None Try rebuilding without insertions/deletionssplit_loop = None Try split_loop method for fixing insertions/deletionsmask_secondary_structure_in_split_loop = True Mask out mask secondary structure in split looptry_insertions = None Try insertionstry_deletions = None Try deletionsmax_rebuild_cycles = None Maximum rebuilding cycles per macro cyclemacro_cycles = None Macro cycles of rebuilding and refinementstart_rebuild = None Starting residue to rebuild. If specified, this is all that is doneend_rebuild = None Ending residue to rebuild. If specified, this is all that is donerebuild_segment = 1 Segment to rebuild from start_rebuild to end_rebuild. None means rebuild all segments from start_rebuild to end_rebuild.min_z_score = 2.5 Minimum Z-score for keeping a residue in trace-chain stage. (density at N C CA atoms less than mean for all residues minus min_z_score times SD for all residues).min_average_z_score = 2 Minimum average Z-score for keeping a residue in trace-chain stage. (average density at N C CA atoms less than mean for all residues minus min_average_z_score times SD for all residues).

• refine = True Refine all-atom models at end of procedure

• refine_b = None Refine B-values

• refine_cycles = None Refinement cycles (except final cycles). Typical is 1 for quick or superquick and 5 for thorough building

• good_enough_cc = None If all residues have this CC, don't bother rebuilding them

• minimum_contact_distance = 3 Minimum distance between CA atoms not immediately connected

• split_with_sequence = None Use sequence to identify sequence register errors. Runs replace_side_chains with reassign_sequence = true and assign_sequence with iterative_assignment = true.

• rebuild_length_worst = 15 Longest rebuild length to try

• max_insert_or_delete = 1 Maximum residues to insert or delete in on rebuild stage

• time_per_residue = 1 How long to try in fitting loops (sec/residue)

• try_as_is = None Try rebuilding without insertions/deletions

• split_loop = None Try split_loop method for fixing insertions/deletions

• mask_secondary_structure_in_split_loop = True Mask out mask secondary structure in split loop

• try_insertions = None Try insertions

• try_deletions = None Try deletions

• max_rebuild_cycles = None Maximum rebuilding cycles per macro cycle

• macro_cycles = None Macro cycles of rebuilding and refinement

• start_rebuild = None Starting residue to rebuild. If specified, this is all that is done

• end_rebuild = None Ending residue to rebuild. If specified, this is all that is done

• rebuild_segment = 1 Segment to rebuild from start_rebuild to end_rebuild. None means rebuild all segments from start_rebuild to end_rebuild.

• min_z_score = 2.5 Minimum Z-score for keeping a residue in trace-chain stage. (density at N C CA atoms less than mean for all residues minus min_z_score times SD for all residues).

• min_average_z_score = 2 Minimum average Z-score for keeping a residue in trace-chain stage. (average density at N C CA atoms less than mean for all residues minus min_average_z_score times SD for all residues).

• controlmultiprocessing = *multiprocessing sge lsf pbs condor pbspro slurm Choices are multiprocessing (single machine) or queuing systemsqueue_run_command = None run command for queue jobs. For example qsub.nproc = 1 Number of processors to userandom_seed = 171731 Random seedverbose = False Verbose outputwrite_maps = True Verbose map outputskip_temp_dir = None Skip temp_dirtrace_only = False Trace map and stopuse_starting_model_ca_directly = False Use CA from starting model directlyuse_starting_model_ca_as_guide = False Applies if use_starting_model_ca_directly. Only use supplied CA as a guidereturn_trace_fragments = False return trace fragments with trace_only (both must be set)quick = True Quick run. Refine just 1 cycle and look for up to 3 segmentssuperquick = False Very quick runmax_dirs = 1000 Maximum number of directories (trace_and_build_xxx)resolve_size = None Size of resolve to use. coarse_grid = None Use a coarse grid in RESOLVE (saves on memory)em_side_density = None Use EM side chain density

• multiprocessing = *multiprocessing sge lsf pbs condor pbspro slurm Choices are multiprocessing (single machine) or queuing systems

• queue_run_command = None run command for queue jobs. For example qsub.

• nproc = 1 Number of processors to use

• random_seed = 171731 Random seed

• verbose = False Verbose output

• write_maps = True Verbose map output

• skip_temp_dir = None Skip temp_dir

• trace_only = False Trace map and stop

• use_starting_model_ca_directly = False Use CA from starting model directly

• use_starting_model_ca_as_guide = False Applies if use_starting_model_ca_directly. Only use supplied CA as a guide

• return_trace_fragments = False return trace fragments with trace_only (both must be set)

• quick = True Quick run. Refine just 1 cycle and look for up to 3 segments

• superquick = False Very quick run

• max_dirs = 1000 Maximum number of directories (trace_and_build_xxx)

• resolve_size = None Size of resolve to use.

• coarse_grid = None Use a coarse grid in RESOLVE (saves on memory)

• em_side_density = None Use EM side chain density

• guiGUI-specific parameter required for output directoryoutput_dir = None

• output_dir = None